



“十二五”普通高等教育本科国家级规划教材

数据库系统概论

(第6版)

王 珊 杜小勇 陈 红 编著

中国教育出版传媒集团
高等教育出版社



“十二五”普通高等教育本科国家级规划教材

数据库系统概论

(第6版)

王 珊 杜小勇 陈 红 编著

中国教育出版传媒集团
高等教育出版社·北京

内容提要

本书自1983年出版第1版至今,40年间始终紧跟数据库技术发展,适时修订,不断与时俱进,得到广大读者的认可与肯定,为全国数百所高校所采用,并三次荣获国家/教育部优秀教材奖(1988年、2002年、2021年)。本书作者团队以教材为蓝本建设的相应课程先后入选北京市精品课程、国家级精品课程、国家级精品资源共享课程、国家级精品在线开放课程和国家级一流本科课程。

本书系统全面地阐述了数据库系统的基础理论、基本技术和基本方法。全书分为4篇共18章。其中:

第一篇基础篇,包括绪论、关系模型、关系数据库标准语言SQL、数据库安全性和数据库完整性,共5章。

第二篇设计与应用开发篇,包括关系数据理论、数据库设计和数据库编程,共3章。

第三篇系统篇,包括关系数据库存储管理、关系查询处理和查询优化、数据库恢复技术、并发控制和数据库管理系统概述,共5章。

第四篇新技术篇,包括数据库发展概述、大数据管理系统、数据仓库与联机分析处理、内存数据库系统、区块链与数据库,共5章。

本书可作为高等学校计算机科学与技术、软件工程、数据科学与大数据技术、信息系统与信息管理等相关专业数据库课程的教材,也可供从事数据库系统研究、开发和应用的工程技术人员参考。

图书在版编目(CIP)数据

数据库系统概论 / 王珊, 杜小勇, 陈红编著. --6

版. --北京: 高等教育出版社, 2023. 3

ISBN 978-7-04-059125-5

I. ①数… II. ①王… ②杜… ③陈… III. ①数据库系统-概论 IV. ①TP311.13

中国版本图书馆CIP数据核字(2022)第141995号

Shujuku Xitong Gailun

策划编辑 倪文慧

责任编辑 倪文慧

封面设计 李卫青

责任绘图 于博

版式设计 王艳红

责任校对 马鑫蕊

责任印制 刘思涵

出版发行 高等教育出版社

社址 北京市西城区德外大街4号

邮政编码 100120

印刷 唐山市润丰印务有限公司

开本 787mm×1092mm 1/16

印张 31.25

字数 710千字

购书热线 010-58581118

咨询电话 400-810-0598

网 址 <http://www.hep.edu.cn>

<http://www.hep.com.cn>

网上订购 <http://www.hepmall.com.cn>

<http://www.hepmall.com>

<http://www.hepmall.cn>

版 次 1983年4月第1版

2023年3月第6版

印 次 2023年3月第1次印刷

定 价 59.00元

本书如有缺页、倒页、脱页等质量问题,请到所购图书销售部门联系调换

版权所有 侵权必究

物料号 59125-00

作者简介

王 珊



王珊，中国人民大学教授，博士生导师，大数据管理与分析方法研究北京市重点实验室学术委员会主任。曾任中国人民大学信息学院院长、中国计算机学会副理事长、中国计算机学会数据库专业委员会主任、教育部第五届科学技术委员会委员等。

自1981年以来一直从事数据库方向的教学、科研和系统开发工作。1984—1986年应邀赴美国马里兰大学工作，设计和开发可扩展的关系数据库管理系统XDB。回国后主持和承担了国家科技攻关项目、国家高技术研究发展计划（863计划）、北京市科技计划重大项目、国家自然科学基金重点及重大项目、国家核高基科技重大专项等50多项科研项目。在国内外学术期刊上发表论文300余篇，出版数据库方面著作30多部。

40余年来培养博士硕士研究生130余名，主持研制和开发了多个具有自主知识产权的数据库管理系统。科研成果获多项国家级和省部级科技进步奖；编著的教材《数据库系统概论》3次获教育部/国家级优秀教材奖，相应课程入选国家级一流本科课程等；获全国五一劳动奖章以及全国优秀教师、国家教学名师等荣誉称号。

作者简介

杜小勇



杜小勇，中国人民大学教授、杰出学者特聘教授，博士生导师，中国人民大学校长助理、明理书院院长、数据工程与知识工程教育部重点实验室主任、中

国计算机学会大数据专家委员会主任。曾任中国人民大学计算机系主任、信息学院院长，中国计算机学会数据库专业委员会主任等。

长期从事数据库与大数据领域的教学研究工作，曾任863计划数据库重大专项专家组组长、云计算与大数据国家重点研发计划总体专家组专家等。国家级一流本科课程“数据库系统概论”主讲教师，教育部计算机领域本科教育教学改革试点工作计划（101计划）数据库系统课程建设组负责人，教育部首批虚拟教研室建设试点数据库课程虚拟教研室负责人。入选2021年度高校计算机专业优秀教师奖励计划，获北京市高等教育教学成果奖一等奖、国家科技进步奖二等奖、教育部科学技术进步奖一等奖等奖项。

作者简介

陈红



陈红，中国人民大学教授，杰出学者特聘教授，博士生导师。中国人民大学信息学院党委书记兼副院长，数据库与商务智能教育部工程研究中心主任，国

家重点研发计划首席科学家，教育部计算机领域本科教育教学改革试点工作计划（101计划）数据库系统课程建设组负责人，中国计算机学会数据库专业委员会常务委员。

主持国家重点研发计划项目、国家“核高基”重大科技专项课题、863计划项目、国家自然科学基金重点项目等20余项。2005年入选教育部新世纪优秀人才支持计划。国家级一流本科课程“数据库系统概论”主讲教师。获国家科技进步奖二等奖、北京市高等教育教学成果奖一等奖、教育部科学技术进步奖一等奖和二等奖等奖项。

数据库系统 概论

(第6版)

王珊 杜小勇 陈红

- 1 计算机访问<http://abook.hep.com.cn/187537>, 或手机扫描二维码、下载并安装Abook应用。
- 2 注册并登录, 进入“我的课程”。
- 3 输入封底数字课程账号(20位密码, 刮开涂层可见), 或通过Abook应用扫描封底数字课程账号二维码, 完成课程绑定。
- 4 单击“进入课程”按钮, 开始本数字课程的学习。



课程绑定后一年为数字课程使用有效期。受硬件限制, 部分内容无法在手机端显示, 请按提示通过计算机访问学习。

如有使用问题, 请发邮件至abook@hep.com.cn。



扫描二维码
下载 Abook 应用

<http://abook.hep.com.cn/187537>

第6版前言

数据库技术从20世纪60年代产生至今,已跨越一甲子。其间数据库技术发展迅速,成果丰厚,建立了一门以数据建模和数据库管理系统核心技术为主,内容丰富、领域宽广的学科;带动了一个巨大的软件产业——数据库管理系统产品及其相关工具和解决方案。数据库系统已成为现代信息系统不可或缺的基石。

《数据库系统概论》教材始终紧跟数据库技术发展,适时修订,不断保持科学性、先进性和实用性。教材第1版~第5版分别于1983年、1991年、2000年、2006年和2014年出版,并于1988年、2002年、2021年先后荣获“全国高等学校优秀教材奖”“全国普通高等学校优秀教材奖”和“全国优秀教材奖”。

1. 第6版教材架构和主要内容

第6版教材秉承前版一贯风格,结构严谨、层次分明,概念清晰、循序渐进,语言精练、通俗易懂。全书总体架构不变,分为基础篇、应用篇(设计与应用开发篇)、系统篇和新技术篇,共4篇18章。每一篇均设有篇首语,概要介绍本篇的主要内容。

第1章~第12章是基础教程,为本科生必读内容,标题前有*号的部分可作为选讲/选读内容。第13章~第18章是高级教程,供本科生、研究生选读。

本科生必读	1. 绪论 2. 关系模型 3. 关系数据库标准语言 SQL 4. 数据库安全性 5. 数据库完整性	基础篇	第1章实验 实验准备 第3章实验 SQL查询与操纵 第4章实验 安全性控制 第5章实验 完整性控制	数据库管理员和数据库应用系统设计开发人员
	6. 关系数据理论 7. 数据库设计 8. 数据库编程	设计与应用开发篇	第7章实验 数据库设计 第8章实验 数据库编程与大作业	
	9. 关系数据库存储管理 10. 关系查询处理和查询优化 11. 数据库恢复技术 12. 并发控制	系统篇	第10章实验 性能监视与调优 第11章实验 数据库备份与恢复 第12章实验 并发控制	
本科生、研究生选读	* 13. 数据库管理系统概述			
	14. 数据库发展概述 15. 大数据管理系统 16. 数据仓库与联机分析处理 17. 内存数据库系统 18. 区块链与数据库	新技术篇		

第6版在保持系统性的基础上,力求使内容模块化、可裁剪,以满足不同类型、不同层次、不同专业学生的需求,也方便教师在讲授时根据学生情况进行内容选择。我们努力做到既全面讲解数据库的基本概念和关键技术,又注重引导学生了解和掌握数据库方法的核心与精华,使其在掌握具体技术的基础上能够举一反三,创造性地把所学知识应用到计算机相关领域和应用领域,如大数据系统的管理和应用之中。

2. 第6版教材修订的主要内容

为了保持科学性、先进性和实用性,我们在第5版教材基础上对全书内容进行了修改、更新和充实。

在科学性方面,我们在系统篇中增加了第9章关系数据库存储管理,讲解数据库的逻辑与物理组织方式及索引结构。增加这部分内容有助于学生更好地理解关系数据库查询优化的原理,也有助于其在应用开发中更好地进行数据库物理设计。

在先进性方面,我们主要做了如下修订。

① 根据数据库技术发展及数据库应用系统开发实践,对前三篇包含的章节内容进行了更新。例如,

- 对于第1章绪论,在1.2.1小节引入数据建模概念,介绍数据建模过程,引出概念模型和数据模型;增加1.2.7小节数据库领域中不断涌现的数据模型,这是大数据时代带来的特色;增加1.5节数据库系统的体系结构,让学生从多角度认识数据库系统——从数据库系统内部的三级模式结构到数据库系统外部的体系结构。

- 对于第3章关系数据库标准语言SQL,根据SQL标准的发展进行相应更新和扩展。在介绍SQL的同时,进一步讲解关系数据库系统的基本概念,加深学生对前两章中关系数据库有关概念的认识,并使其更加丰富;增加对新的SQL语句的介绍;本章主要示例和SQL语句都以贯穿全书的实例为应用场景。

- 对于第4章数据库安全性和第5章数据库完整性,根据当前数据安全性和完整性的要求对有关内容进行了调整、补充和修改。

- 对于第7章数据库设计,以“高校本科教务管理”信息系统为例介绍数据库设计步骤,并概括了计算机学科中抽象、理论和设计三个学科形态。

- 对于第8章数据库编程,以实际应用任务需求为牵引,从两方面介绍数据库编程技术:一是扩展SQL语言自身的功能,二是在高级语言程序中使用SQL。这是对众多数据库编程技术的分析和归纳,使学生更容易理解数据库编程技术,掌握开发复杂业务逻辑的技术和方法。

② 根据数据库新技术的应用和发展,重新撰写了第四篇(新技术篇)有关章节的内容,为读者进一步学习和研究做必要的准备和铺垫。由于区块链技术已成为重要的研究和应用方向,增加了第18章区块链与数据库。

在实用性方面,我们主要做了如下修订。

① 设计了一个贯穿全书的实例。用一个实例把基础篇、设计与应用开发篇、系统篇中的知识点讲解和设计样例(如SQL语言、E-R图设计、数据库设计、应用系统开发、数据存储与

索引、查询处理等)贯穿起来。

这个实例以教师和学生熟悉的“高校本科教务管理”信息系统为主线,抽象了“学生选课管理”“学生学籍管理”“教师教学管理”三个应用场景。从第1章起就以学生选课管理为主要示例讲解数据库的基本概念、关系数据模型和SQL语句。在第7章讲解E-R图设计,介绍如何在理论指导下抽象出“高校本科教务管理”信息系统的分E-R图,并对分E-R图进行集成和优化。第9章和第10章则以学生选课管理为例,介绍关系数据库存储管理、关系查询处理和查询优化的基本方法。实例内容由简到繁,由浅入深。

本书附录给出了“高校本科教务管理”信息系统完整的E-R图和相应的关系模式。

②在每一章的开始部分增加了导读。导读概要介绍本章讲解的主要内容、学习重点和难点,具有纲举目张的作用,引导学生更好地理解本章的知识点以及知识点之间的有机联系。

③以二维码形式展示知识点讲解视频等数字化内容,作为学生的扩展阅读和学习参考。例如,数据库领域5位图灵奖得主的介绍、文件系统和数据库系统操作比较示例、ALPHA语言例题解析、不同产品DROP TABLE处理策略比较、信息安全标准发展简述、求解关系模式 $R(U,F)$ 所有候选码的方法、若干数据库产品的常用内置函数、B+树索引的查找和维护、查询执行示例、冲突可串行化判断演示等,使教材内容更加丰富多样,更好地辅助学生学习相关内容。

④修订出版配套的教辅用书《数据库系统概论(第6版)习题解析与实验指导》(高等教育出版社出版)。辅导书对教材各章习题做了解析,提供了补充习题;根据教材章节内容安排了必修实验和选修实验,进一步加强了实验教学环节。

3. 第6版教材教学建议

《数据库系统概论》(以下简称《概论》)有众多学校选用,不同类型的学校培养目标不尽相同。讲授的对象不同,讲授的内容也应该有所差别。因此,任课教师应根据学生的实际情况对教材内容进行选择与裁剪,设计适合本校教学要求的数据库课程,做到有的放矢。这里,我们首先介绍数据库系统的知识体系与《概论》教材的知识范畴,在其基础上给出教学内容选择与裁剪建议。

(1) 数据库系统的知识体系与《概论》的知识范畴

数据库系统的知识体系包括基础知识、关系数据库、数据库设计、数据库管理系统和数据库新技术等,按研究范畴可以分为理论、方法、技术和应用4个方面,按学习的深度和广度可以分为使用、管理和设计三个层次,如图1所示。由此可见,数据库系统的内容覆盖了本科生阶段和研究生阶段。数据库使用者、应用系统开发人员、数据库管理人员、数据库管理系统研发人员和数据库研究者等不同类型的人员,应该了解和掌握的数据库技术的侧重点是不一样的。

对照图1,《概论》教材的知识范畴如下:

①基础篇从使用者的角度讲解数据库的基本概念和基本方法,介绍关系数据库的概念和关系数据库标准语言SQL。

②设计与应用开发篇讲解如何在选定的数据库管理系统上设计应用系统的数据库模式,并介绍数据库编程技术。

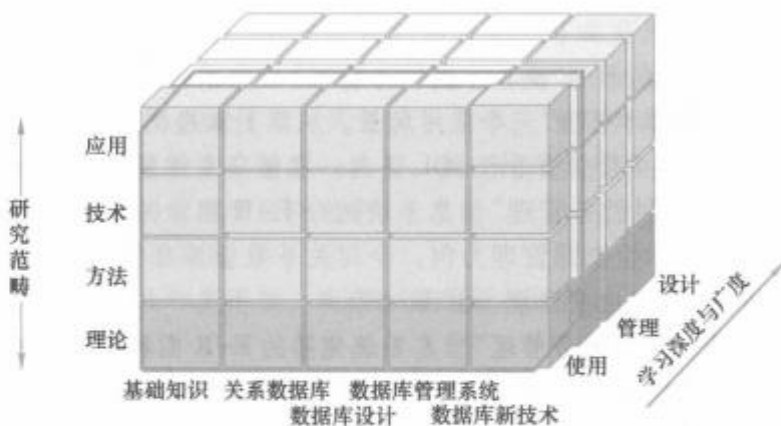


图1 数据库系统知识体系结构示意图

③ 系统篇讲解数据库管理系统中数据的存储管理方法、查询处理和查询优化技术、数据库恢复技术、并发控制。这些技术和方法是数据库管理系统中最基本的技术，其目的是使读者更好地使用数据库，而不是设计开发数据库管理系统。因为研发数据库管理系统需要的技术更多、更深入，属于研究生或本科生高年级的“数据库管理系统实现”课程应讲授的内容。

(2) 如何进行教学内容的选择和裁剪

依据《概论》教材的知识范畴，任课教师可以根据不同的培养目标和课程学时安排首先对教学内容进行宏观选择。

① 基础篇应为必选部分。

② 设计与应用开发篇可以进一步培养学生基于数据库进行应用系统开发的能力，其中数据库设计和数据库编程两部分是需要讲解的。

③ 系统篇讲解的技术和方法对于运行维护数据库管理系统、更高效地使用数据库和优化数据库设计有着重要作用，教师可以酌情选择有关内容进行讲解。

其次对具体章节中的知识点进行选择。主要有以下一些建议。

① 知识点讲解的次序有一定的灵活性。数学有严格的逻辑关联性，讲乘法除法一定要先讲加法减法，而数据库技术则没有这么严格的先后次序。例如，在讲解数据模型时一般按照数据库技术发展进程先讲层次模型、网状模型，再讲关系模型。但是，课时较少时也可以只讲关系模型。

② 知识点讲解的详细程度有一定的灵活性。以数据库完整性约束的讲解为例，可以由浅入深，由点到面逐步展开：第1章提及完整性概念；第2章从数据模型角度讲解关系模型必须满足的完整性约束；第3章讲解如何用SQL语句定义完整性约束；第5章则系统地讲解什么是数据库的完整性，数据库完整性实现的机制，包括完整性约束的定义机制、检查方法和违背完整性约束时应采取的动作等。因此，教师在了解教材关于完整性约束的知识点后，可以对讲解的详细程度进行适当裁剪。

③ 知识点的掌握程度可以“因地制宜”灵活要求。我们在教材每一章的导读中把知识点分为需要了解的、需要牢固掌握的和需要举一反三的。教师可以根据学生情况进行适度调整。

(3) 加强教辅用书的学习参考

《概论》的教辅用书《数据库系统概论(第6版)习题解析与实验指导》对难度较大的习题做了解析,并在实验部分提供了实验报告。这些内容对于学生自然可以起到辅学作用,而对于教师则更值得借鉴参考。

4. 致谢

许多教师和学生参与了第6版教材的资料收集和整理、实例上机验证、动画演示制作、图表修改等工作。

中国人民大学的卢卫教授一直参加每周一次的教材修订工作会,他提出了许多中肯、有益的修改建议,并协助撰写了设计与应用开发篇部分章节。大连海事大学的张俊教授在教材修订过程中与作者同步进行书稿审阅,他细致认真,给出了许多具体的建议,使我们能够及时进行修改。

新技术篇的撰写凝聚了许多教师的智慧和辛劳。中国人民大学陈跃国教授、卢卫教授、柴云鹏教授、覃雄派博士、李翠平教授、张延松博士、陈晋川博士,清华大学黄向东博士, PingCAP 公司首席技术官黄东旭先生和校友廖朝晖等参加了该部分内容的修订工作。

第6版教材中的SQL语句均在金仓数据库管理系统上测试通过,北京人大金仓信息技术股份有限公司高级工程师王建华和冯玉,中国人民大学博士生刘鹏举、张县婷等参加了测试工作。

第6版教材修订过程中,曾在中国计算机学会计算课程改革导教班和中国高校计算机教育MOOC联盟数据库工作组会议期间召开多次座谈会和研讨会,全国数十所高校的百余名教师对教材内容提出了许多中肯的建议。中国计算机学会数据库专业委员会也对本书的修订工作给予了支持,专委会主任西北工业大学李战怀教授,委员北京交通大学王宁教授、东北大学杨晓春教授等都提出了有益建议。第6版教材内容中的许多修订就是来自这些宝贵的建议。

此外,很多高校教师和学生、社会读者在使用本书时通过邮件、课程网站和慕课讨论区对本书内容和习题提出的问题和建议,也是本书修订中非常宝贵的参考资料。

在此,一并向上述人员表示衷心的感谢。

第6版教材由王珊教授、杜小勇教授和陈红教授执笔修改。王珊教授审定了全书。

在修订过程中,作者阅读参考了大量国内外教材、专著、论文和资料,努力跟踪数据库学科的新发展、新技术,并有选择地将其纳入教材,但因学科发展速度快,书中必有许多不足之处,希望学术同人 不吝赐教。

王珊 杜小勇 陈红

2023 兔年春节

第5版前言

为了反映数据库学科的新成果和应用的新方向,适应数据库技术的进展,保持本书的先进性、科学性和实用性,我们对本书第4版进行了修订。本书第1版、第2版、第3版和第4版分别于1983年、1991年、2000年和2006年出版。第5版是“十二五”普通高等教育本科国家级规划教材。

本书分为4篇16章,如下表所示。第1章至第11章(表中序号1~11)是本科专业的基本教程(书中有*号的部分除外),第12章至第16章(表中序号12~16)是高级教程。

本科生必读	1. 绪论 2. 关系数据库 3. 关系数据库标准语言 SQL 4. 数据库安全性 5. 数据库完整性	基础篇	实验准备 实验1 数据库定义与操作语言 实验2 安全性语言 实验3 完整性语言 实验4 触发器	DBA 和数据库应用系统设计开发人员
	6. 关系数据理论 7. 数据库设计 8. 数据库编程	设计与应用开发篇	实验5 数据库设计 实验6 存储过程 实验7 数据库应用开发 实验8 数据库设计与应用开发大作业	
	9. 关系查询处理和查询优化 10. 数据库恢复技术 11. 并发控制	系统篇	实验9 数据库监视与性能优化 实验10 数据库恢复技术 实验11 并发控制	
本科生、研究生选读	12. 数据库管理系统			
	13. 数据库技术发展概述 14. 大数据管理 15. 内存数据库系统 16. 数据仓库与联机分析处理技术	新技术篇		

第5版主要修改的内容包括:

① 在基础篇中保持重点讲解关系数据库系统的传统,对SQL的内容根据标准的发展做了相应更新。随着数据的安全性、完整性越来越重要,对数据库安全性和完整性的内容进行了补充修改。

② 在设计与应用开发篇中把原来第4版第1章的E-R图设计移到了第7章7.3概念结构

设计一节中,成为概念结构设计的重要知识点。作为选读,增加了扩展E-R图的内容。修改补充了第8章数据库编程中PL/SQL、存储过程和函数、ODBC、OLE DB、JDBC等概念和方法。

③在系统篇的第11章并发控制中增加了三级封锁协议的内容,并在11.8其他并发控制机制一节(作为选读内容)概要介绍了多版本并发控制(MVCC)技术。

④在新技术篇中修改了第13章数据库技术发展概述和第16章数据仓库与联机分析处理技术的内容,增加了反映数据管理最新发展的重要技术,如大数据管理、内存数据库系统等章节,限于篇幅删去了第4版中分布式数据库系统、对象关系数据库系统和XML数据库。

⑤提供了作者编写的配套教辅用书《数据库系统概论(第5版)习题解析与实验指导》(高等教育出版社出版)。其中根据教材章节内容安排了必修实验和选修实验,进一步加强了实验教学环节;对本书各章习题做了解析,还增加了补充习题。

在高等教育出版社易课程网站和精品课程教学网站上均给出了本书配套的教学资源,包括电子教案、教学视频、实验要求及部分实验报告示例、补充习题及参考答案等,供读者学习参考。这些实验均使用国产金仓数据库管理系统Kingbase ES作为实验平台。该系统可以从北京人大金仓信息技术股份有限公司的网站免费下载。

本书内容全面丰富,教师可以针对不同专业和不同类别的学生挑选书中不同章节的内容进行讲解。

全书由王珊教授执笔。大连海事大学张俊教授详细审阅了书稿,提出了许多有益的意见。中国人民大学陈红教授和杜小勇教授根据讲授本书的实际体会,对内容和实验提出了许多中肯有益的修改建议并审阅了书稿。陈红教授、杜小勇教授、张孝副教授、文继荣教授、李翠平教授、中国调查与数据中心(中国人民大学)张延松博士、教育部数据工程与知识工程重点实验室(中国人民大学)陈跃国博士等协助修改和撰写了部分内容,在此向他们表示衷心感谢。

北京人大金仓信息技术股份有限公司任永杰博士、冯玉博士、李海华博士和冷建全高工等提供了金仓数据库产品的技术资料,研究生周宁南和孟庆钟等参与了部分资料的收集工作,在此向他们表示衷心感谢。

还要感谢广大读者、教师和学生在使用本书时通过邮件、课程网站对本书内容和习题提出的问题和建议,这是本书修订中非常宝贵的参考资料。

在本书的修订过程中,作者阅读参考了大量国内外教材、专著、论文和资料,努力跟踪数据库学科的新发展、新技术,有选择地把它纳入教材中,但因学科发展太快,书中必有许多不足之处,希望学术同仁不吝赐教。

王 珊

2014 马年春节

第4版前言

数据库技术的发展十分迅速,为适应数据库技术的进展,《数据库系统概论》每隔几年就修订一次,以反映学科的新成果和应用的新方向,保持本书的先进性、科学性和实用性。

本书第1版、第2版和第3版分别于1983年、1991年、2000年出版。第4版是普通高等教育“十五”国家级规划教材与国家精品课程建设的成果。全书分为4篇17章,如下表所示。第1章至第11章是本科专业的基本教程(书中有*号的部分除外),第12章至第17章是高级教程。

本科生 必读	1. 绪论 2. 关系数据库 3. 关系数据库标准语言SQL 4. 数据库安全性 5. 数据库完整性	基础篇	实验1 认识DBMS 实验2 交互式SQL 实验3 数据控制(完全性部分) 实验4 数据控制(完整性部分)	DBA 和数据库 应用系统 设计开发 人员
	6. 关系数据理论 7. 数据库设计 8. 数据库编程	设计与应用 开发篇	课堂大作业《数据库设计与应用开发》 实验5 通过嵌入式SQL访问数据库 实验6 使用PL/SQL编写存储过程访问数据库 实验7 通过ODBC访问数据库 实验8 通过JDBC访问数据库	
	9. 关系查询处理和查询优化 10. 数据库恢复技术 11. 并发控制	系统篇	实验9 查询优化	
本科生、 研究生 选读	12. 数据库管理系统			
	13. 数据库技术新发展 14. 分布式数据库系统 15. 对象关系数据库系统 16. XML数据库 17. 数据仓库与联机分析 处理技术	新技术篇		

第4版主要修改的内容包括:

① 在基础篇中继续加强关系数据库系统的讲解,特别是SQL的内容紧跟标准的发展,更新了数据库安全性和完整性的部分内容。

② 在系统篇中重写了关系查询处理和查询优化一章,内容做了适度的加宽加深。

③ 在设计与应用开发篇中增加数据库编程一章,讲解了应用开发所需要的ODBC、JDBC

等概念和方法。

④在新技术篇中修改了数据库技术新发展一章的内容，添加了反映数据库最新发展的重要技术，如对象关系数据库系统、XML数据库、数据仓库和联机分析处理等章节。限于篇幅删去了第3版中并行数据库一章。

⑤最为关键的是第4版提供了实验环境和实验指导，进一步加强实验和课程设计等教学环节。根据教材章节的内容安排了9个实验和1个大作业。在我们的教学网站上给出了本书每个实验的详细要求和部分实验的报告示例，供读者学习参考。这些实验均使用国产金仓数据库管理系统 Kingbase ES(Kingbase Enterprise Server)作为实验平台，本书中所讲解的 SQL 例子都在 Kingbase ES 上运行通过。该系统可以从人大金仓公司的网站上免费下载。

本书内容全面丰富，教师可以针对不同专业和不同类别的学生，挑选本书中不同章节的内容进行讲解。

本书由王珊教授执笔。中国科学技术大学研究生院罗晓沛教授、北京工商大学姜同强副教授详细审阅了书稿，并提出了许多有益的意见，陈红教授、杜小勇教授等根据他们讲授本书的实际体会，对内容和实验提出了修改建议。在此向他们表示衷心的感谢。

硕士研究生龚玮薇在金仓数据库 Kingbase ES 平台上运行通过了书中所有 SQL 语句，并完成了作业中的上机实验。王秋月博士，博士研究生张俊、彭朝晖、栾华，硕士研究生郑肇万、陈瑞文等参与了部分资料的收集整理工作。在此向他们表示感谢。

在修订本书的过程中，我参考了大量国内外教材、专著、论文和资料，努力跟踪数据库学科的新发展、新技术，有选择地把它们纳入教材中来。书中也包含了我 20 多年来教学中的经验体会和研究开发成果。但由于学科发展太快，本人才疏学浅，必有许多不足之处，希望学术同人人不吝赐教。

王 珊

2005 年岁末于中国人民大学

第3版前言

数据库技术从20世纪60年代中期产生到今天仅仅30多年的历史,已经历了三代演变,造就了C. W. Bachman、E. F. Codd和James Gray三位图灵奖得主;发展了以数据建模和DBMS核心技术为主,内容丰富的一门学科;带动了一个巨大的软件产业——DBMS产品及其相关工具和解决方案。30多年成就辉煌。

数据库技术是计算机科学技术中发展最快的领域之一,也是应用最广的技术之一,它已成为计算机信息系统与应用系统的核心技术和重要基础。本书系统地阐述数据库系统的理论、技术和方法,是《数据库系统概论》的第3版。第一版1983年出版(1988年获国家级优秀教材奖),第2版1991年出版。

针对数据库技术的发展和我国应用水平的提高,我们对第2版做了较大的调整、修改和增删,但原书的基本宗旨和风格不变,保持讲述数据库的基本概念、基本理论和基本技术为主的特点。第3版主要的修改是:

① 根据我国实际情况,网状、层次数据库系统已很少使用,因此把它们删去了,有关的主要概念放在第1章数据模型中介绍。

② 进一步加强了关系数据库系统的讲解,特别是SQL语言的介绍,以适应当前广泛使用关系数据库系统的需要。

③ 把第2版第8章数据库保护中的安全性、完整性、并发控制和恢复4节扩展为系统篇中的4章,内容做了适度的加宽和加深。随着大型数据库系统的普遍使用,这些知识和技术是运行和维护数据库系统必不可少的。本书从使用和管理角度讲解这些知识而不是讨论实现这些功能的内部技术。

④ 为了反映数据库技术的发展,增加了新技术篇。第12章全面介绍了数据库发展的总体轮廓,从数据模型、新技术内容、应用领域三个方面阐述新一代数据库系统及其相互关系,并选择当前较重要的新技术在后面三章中介绍,它们是面向对象数据库系统、分布式数据库系统和并行数据库系统。

⑤ 每章后配有小结、习题及阅读参考文献,并对许多文献做了简要的注释,以便读者进一步参考。

此外,为了辅助教学和加强上机实习,我们编写了与本书配套的《数据库系统概论学习指导与习题解答》,该书附有光盘。人大金仓信息技术有限公司开发了“人大金仓数据库管理系统Kingbase ES”,该系统的工作组版可以从人大金仓公司的网站免费下载,作为你的实验平台。

全书分为4篇共15章。第1章至第10章是计算机软件专业本科生的基本教程(书中有*

号的部分除外), 第11章至第15章是高级教程。

下图给出了本书各章之间的联系和读者对象示意。

本科生	1. 绪论 2. 关系数据库 3. 关系数据库标准语言SQL 4. 关系系统及其查询优化 5. 关系数据理论	基础篇	DBA和数据库系统设计人员
	6. 数据库设计	设计篇	
	7. 数据库恢复技术 8. 并发控制 9. 数据库安全性 10. 数据库完整性 11. 数据库管理系统	系统篇	
	12. 数据库技术新发展 13. 面向对象数据库系统 14. 分布式数据库系统 15. 并行数据库	新技术篇	
本科生、研究生选读			

本书内容丰富, 讲授时可根据学生及专业情况酌情取舍。例如, 对于计算机专业本科学士生, 第2章2.5关系演算、第5章5.4模式分解可适当压缩; 新技术篇中的章节, 教师可以选择部分内容进行讲解。

本书由王珊教授执笔, 萨师煊教授审定, 陈红副教授和研究生曹会萍、王静等参与了内容讨论和书稿校阅工作。

中国科学技术大学研究生院邵佩英教授和北京大学杨冬青教授详细审阅了全稿并提出了许多有益的意见, 在此向她们表示衷心的感谢。

我们在编写本书的过程中, 努力跟踪数据库学科的新发展、新技术, 把它们纳入教材中来, 力求反映当代新技术, 以保持本书的先进性和实用性。但由于学识浅陋, 见闻不广, 必有许多不足之处, 希望学术同人 不吝赐教。

萨师煊 王珊

1999年仲夏于中国人民大学

第2版前言

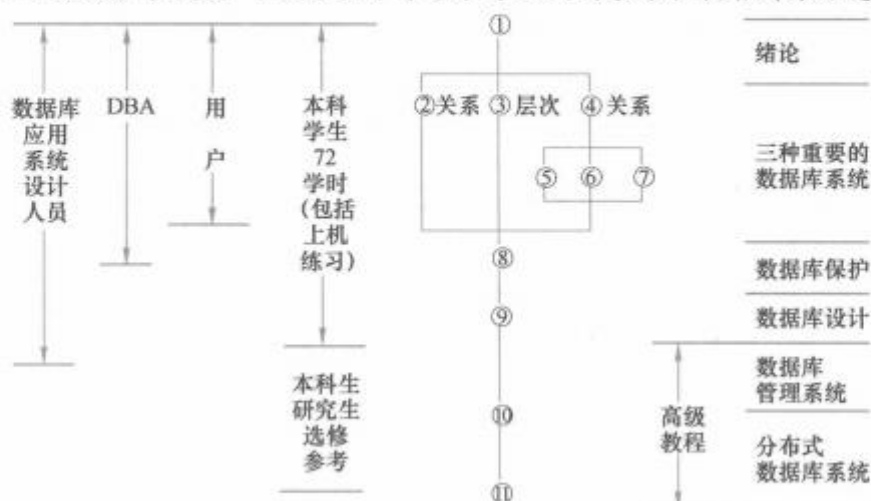
《数据库系统概论》第1版出版于1983年,距今已6年。在这6年中,不仅数据库技术有了很大进展,而且国内计算机专业的学生和技术人员的水平也有显著提高。因此在第2版中,我们针对这些情况对原书从结构到内容做了较大的调整、修改和增删,但基本宗旨和风格不变,仍以国家教育委员会颁布的《数据库系统概论教学大纲》作为本书编写的基本依据,保持讲述数据库的基本概念、基本理论和基本技术为主的特点。

全书分为两大部分,共11章。第一部分包括第1章至第9章,是计算机软件专业本科生的基本教程。第二部分包括第10章、第11章,是高级教程。

第1章绪论,概述了数据管理的进展、数据模型和数据库系统构成的一般概念。第2章至第7章介绍三种重要的数据库系统:网状数据库 DBTG 系统、层次数据库 IMS 和关系数据库系统。鉴于关系数据库具有许多优点并已在应用中日趋成熟,我们把重点放在这一部分,进一步充实了关系数据库的内容,共计有4章(第4章至第7章)。第4章概述关系模型的基本概念、关系代数和关系演算,第5章详细介绍关系数据库标准语言 SQL,第6章讨论关系系统及其查询优化,第7章讲述关系数据理论。

第8章数据库保护(包括数据库的安全性、完整性、并发控制和恢复)和第9章数据库设计都做了较大的变动,充实了内容,增强了实用性。

第二部分是新增的。包括第10章数据库管理系统和第11章分布式数据库系统。这是为了加强读者对 DBMS 的了解,适应对“分布处理”日益普遍的需要,引导读者从学习本书开始向某些数据库的重要新领域过渡。下图给出了本书各章之间的联系和读者对象示意。



第1版中有附录I文件组织与附录II IMAGE数据库系统。前者是基础性内容,由于读者水平的提高已无存在的必要;后者原作为实习课的参考内容,鉴于近几年引进多种商品化的关系数据库系统且应用甚为广泛,因此我们已另行编写了一本《实用数据库系统汇编》作为实习教材(高等教育出版社1990年出版),其中介绍了5种关系系统和IMAGE系统。因此本书删去了第1版中的附录。

本书由王珊执笔,萨师煊审定,杜小勇、李曦老师和研究生唐元昌、武志文、任永杰等参与了内容讨论和校阅工作。中国科学技术大学研究生院罗晓沛教授、清华大学俞盘祥副教授和北京大学杨冬青副教授详细审阅了全稿并提出了许多有益的意见,在此向他们致以衷心的感谢。我们在改编过程中,尽可能引入新的观点和方法,力求能反映当代技术水平。但由于学识浅陋,见闻不广,必有许多不足之处,希望同行指正。

萨师煊 王珊

1989年12月

于中国人民大学数据工程与知识工程研究所

第1版前言

数据库是数据管理的最新技术，是计算机科学的重要分支。十余年来，数据库管理系统已从专用的应用程序包发展成为通用的系统软件。由于数据库具有数据结构化、最低冗余度、较高的程序与数据独立性、易于扩充、易于编制应用程序等优点，较大的信息系统都是建立在数据库设计之上的。因此，不仅大型计算机及中小型计算机，甚至微型机都配有数据库管理系统。目前，对数据库各种模型的研究以及理论上的探讨都还在蓬勃开展，其应用也从一般管理扩大到计算机辅助设计、人工智能以及科技计算等领域。国外高等学校计算机科学系、计算机应用与信息系统系等都开设有数据库系统方面的课程。今年来，我国在计算机科学教育中也对数据库予以应有的重视。1983年教育部部属高等学校计算机软件专业教学方案将数据库概论列为四年制本科的必修课程，并已于1983年6月通过了教学大纲，我们这本书便是按照该大纲编写的一本教材。

全书共分10章和两个附录。附录I的“文件组织”系学习数据库的预备知识，但已散见于数据结构等先行课中，列在这里，仅供参考用，一般不必作为正式讲授内容。第1、2、3章为有关数据库的一般概念；第4至8章介绍三种重要数据库模型。其中第8章的公理系统与模式分解的大部分内容已超出大纲范围，讲授时可酌情取舍。第9章为数据库控制，第10章为数据库设计。附录II的“IMAGE 3000数据库系统”介绍了如何在HP-3000计算机的IMAGE 3000数据库管理系统上建立数据库和使用数据库。对于能够用到HP-3000计算机的单位，这一部分可作为实习教材；对于不能用上HP-3000计算机的单位则可作为参考，借以从中了解如何在一个实际的数据库管理系统上建立一个数据库应用系统。本书是编者根据多年来进行数据库讲演及在高等学校为研究生、本科生开设数据库系统概论课程的经验，并在多次编写的内部交流讲义的基础上修改而成的，由于我们水平不高，而数据库作为计算机科学中的一个新兴分支发展又非常迅速，因此，在本书中存在的问题一定不少，希望读者提出批评意见。

中国科学技术大学研究生院罗晓沛同志，清华大学计算机科学与工程系杨德元同志、俞盘祥同志与沈金发同志详细阅读了全稿，并提出许多有益的意见，在此谨向他们致以衷心的感谢。

本书作为内部交流讲义阶段，有50余所兄弟院校采用作为教本，其中不少同志在使用以后曾以口头或书面形式提出意见，我们也对他们表示感谢。

在讲稿整理过程中，我校信息管理系的刘怡同志，前78级、79级研究生吴鸥琦等同志，82级研究生张秉训、刘伶、杨明伟等同志以及资料室的王小平等同志，都从不同方面分别做了一些工作，对此作者一并表示诚挚的谢意。

萨师煊 王珊

1983年4月于中国人民大学

目 录

第一篇 基 础 篇

第1章 绪论	3	2.1.3 关系数据库	39
1.1 数据库系统概述	4	2.1.4 关系模型的存储结构	39
1.1.1 数据库的4个基本概念	4	2.2 关系操作	40
1.1.2 数据管理技术的产生和发展	6	2.2.1 基本的关系操作	41
1.2 数据模型	12	2.2.2 关系数据语言的分类	41
1.2.1 数据建模	12	2.3 关系的完整性	42
1.2.2 概念模型	13	2.3.1 实体完整性	42
1.2.3 数据模型的三要素	14	2.3.2 参照完整性	43
1.2.4 层次模型	15	2.3.3 用户定义的完整性	45
1.2.5 网状模型	18	2.4 关系代数	45
1.2.6 关系模型	20	2.4.1 传统的集合运算	46
1.2.7 数据库领域中不断涌现的数据模型	23	2.4.2 专门的关系运算	46
1.3 数据库系统的三级模式结构	23	*2.5 关系演算	53
1.3.1 数据库系统中模式的概念	24	*2.5.1 元组关系演算语言 ALPHA	53
1.3.2 数据库系统的三级模式结构	24	*2.5.2 域关系演算语言 QBE	58
1.3.3 数据库的两级映像与数据独立性	25	本章小结	64
1.4 数据库系统的组成	27	习题2	64
*1.5 数据库系统的体系结构	28	参考文献2	66
本章小结	30	第3章 关系数据库标准语言 SQL	67
习题1	30	3.1 SQL 概述	67
第1章实验 实验准备	31	3.1.1 SQL 的产生与发展	67
参考文献1	31	3.1.2 SQL 的特点	68
第2章 关系模型	33	3.1.3 SQL 的基本概念	70
2.1 关系模型的数据结构及形式化定义	34	3.2 数据定义	71
2.1.1 关系	34	3.2.1 模式的定义与删除	72
2.1.2 关系模式	37	3.2.2 基本表的定义、删除与修改	73
		3.2.3 索引的建立与删除	78
		3.2.4 数据字典	80

3.3 数据查询	80	4.2.6 强制存取控制方法	141
3.3.1 单表查询	81	4.3 视图机制	142
3.3.2 连接查询	91	4.4 审计	143
3.3.3 嵌套查询	96	4.5 数据加密	145
3.3.4 集合查询	105	4.6 其他安全性保护	147
3.3.5 基于派生表的查询	107	本章小结	148
3.4 数据更新	108	习题4	148
3.4.1 插入数据	108	第4章实验 安全性控制	149
3.4.2 修改数据	109	参考文献4	149
3.4.3 删除数据	110	第5章 数据库完整性	151
3.5 空值的处理	111	5.1 数据库完整性概述	151
3.6 视图	114	5.2 实体完整性	152
3.6.1 定义视图	114	5.2.1 定义实体完整性	152
3.6.2 查询视图	117	5.2.2 实体完整性检查和违约处理	153
3.6.3 更新视图	119	5.3 参照完整性	155
3.6.4 视图的作用	121	5.3.1 定义参照完整性	155
本章小结	122	5.3.2 参照完整性检查和违约处理	155
习题3	123	5.4 用户定义的完整性	157
第3章实验 SQL 查询与操纵	124	5.4.1 属性上的约束	158
参考文献3	124	5.4.2 元组上的约束	159
第4章 数据库安全性	127	5.5 完整性约束命名子句	160
4.1 数据库安全性概述	127	* 5.6 域的完整性限制	161
4.1.1 数据库的不安全因素	128	5.7 触发器	162
4.1.2 安全标准简介	128	5.7.1 定义触发器	162
4.2 数据库安全性控制	131	5.7.2 执行触发器	165
4.2.1 用户身份鉴别	132	5.7.3 删除触发器	165
4.2.2 存取控制	133	本章小结	166
4.2.3 自主存取控制方法	134	习题5	166
4.2.4 授予与收回对数据的操作权限	135	第5章实验 完整性控制	166
4.2.5 数据库角色	139	参考文献5	167
第二篇 设计与应用开发篇			
第6章 关系数据理论	171	6.2.2 码	176
6.1 问题的提出	171	6.2.3 范式	176
6.2 规范化	174	6.2.4 2NF	177
6.2.1 函数依赖	174	6.2.5 3NF	178

6.2.6 BCNF	179	7.5 物理结构设计	232
6.2.7 多值依赖	180	7.5.1 数据库物理结构设计的内容和方法	232
6.2.8 4NF	183	7.5.2 选择关系模式存取方法	233
6.3 数据依赖的公理系统	185	7.5.3 确定数据库的存储结构	235
6.4 保持函数依赖的模式分解	189	7.5.4 评价数据库的物理结构	235
*6.5 无损连接的模式分解	191	7.6 数据库的实施和维护	236
6.5.1 无损连接的模式分解定义	191	7.6.1 数据的载入和应用程序的编码与调试	236
6.5.2 无损连接的模式分解与保持函数依赖的模式分解之间的关系	193	7.6.2 数据库的试运行	236
6.5.3 既无损连接又保持函数依赖的模式分解算法	193	7.6.3 数据库的运行和维护	237
6.5.4 无损连接的模式分解算法	194	本章小结	238
本章小结	195	习题 7	238
习题 6	196	第 7 章实验 数据库设计	239
参考文献 6	197	参考文献 7	240
第 7 章 数据库设计	201	第 8 章 数据库编程	241
7.1 数据库设计概述	201	8.1 概述	241
7.1.1 数据库设计的特点	202	8.1.1 SQL 表达能力的限制	241
7.1.2 数据库设计的方法	203	8.1.2 扩展 SQL 的功能	244
7.1.3 数据库设计的基本步骤	204	8.1.3 通过高级语言实现复杂应用	248
7.1.4 数据库设计过程中的各级模式	207	8.2 过程化 SQL	250
7.2 需求分析	207	8.2.1 过程化 SQL 的块结构	250
7.2.1 需求分析的任务	207	8.2.2 变量和常量的定义	251
7.2.2 需求分析的方法	208	8.2.3 流程控制	251
7.2.3 数据字典	209	8.2.4 游标的定义与使用	253
7.3 概念结构设计	211	8.2.5 存储过程	254
7.3.1 概念模型	211	8.2.6 存储函数	256
7.3.2 E-R 模型	211	8.3 JDBC 编程	257
*7.3.3 扩展的 E-R 模型	215	8.3.1 JDBC 工作原理概述	257
*7.3.4 用 UML 中的类图表示 E-R 图	218	8.3.2 JDBC APIs 基础	259
7.3.5 用 E-R 图进行概念结构设计	220	8.3.3 使用 JDBC 操纵数据库的工作流程	259
7.4 逻辑结构设计	228	*8.4 基于 MVC 框架的数据库应用开发	266
7.4.1 E-R 图向关系模型的转换	229	本章小结	270
7.4.2 数据模型的优化	230		
7.4.3 设计用户模式	231		

习题 8	270	参考文献 8	271
第 8 章实验 数据库编程与大作业	271		

第三篇 系 统 篇

第 9 章 关系数据库存储管理	275	* 10.5 查询计划的执行	315
9.1 数据组织	275	本章小结	315
9.1.1 数据库的逻辑组织方式与物理 组织方式	276	习题 10	316
9.1.2 记录表示	277	第 10 章实验 性能监视与调优	317
9.1.3 块的组织	279	参考文献 10	317
9.1.4 关系表的组织	281	第 11 章 数据库恢复技术	319
9.2 索引结构	283	11.1 事务的基本概念	319
9.2.1 顺序表索引	283	11.2 数据库恢复概述	321
9.2.2 辅助索引	286	11.3 故障的种类	321
9.2.3 B+树索引	288	11.4 恢复的实现技术	323
9.2.4 哈希索引	291	11.4.1 数据转储	323
9.2.5 位图索引	294	11.4.2 登记日志文件	324
本章小结	297	11.5 恢复策略	326
习题 9	297	11.6 具有检查点的恢复技术	327
参考文献 9	298	11.7 数据库镜像	329
第 10 章 关系查询处理和查询优化	301	本章小结	330
10.1 关系数据库管理系统的查询 处理	301	习题 11	330
10.1.1 查询处理步骤	301	第 11 章实验 数据库备份与恢复	331
10.1.2 实现查询操作的算法	302	参考文献 11	331
10.2 关系数据库管理系统的查询 优化	306	第 12 章 并发控制	333
10.2.1 查询优化概述	306	12.1 并发控制概述	334
10.2.2 一个实例	307	12.2 事务的隔离级别	336
10.3 代数优化	309	12.3 封锁	337
10.3.1 关系代数表达式等价变换规则	309	12.4 封锁协议	338
10.3.2 语法树的启发式优化	310	12.5 活锁和死锁	340
10.4 物理优化	312	12.5.1 活锁	340
10.4.1 基于启发式规则的优化	312	12.5.2 死锁	340
10.4.2 基于代价估算的优化	313	12.6 并发调度的可串行性	342
		12.6.1 可串行化调度	343
		12.6.2 冲突可串行化调度	343
		12.7 两段锁协议	344

12.8 封锁的粒度	346	13.2 数据库管理系统的系统结构	356
12.8.1 多粒度封锁	346	13.2.1 数据库管理系统的层次结构	356
12.8.2 意向锁	347	13.2.2 关系数据库管理系统的运行 过程示例	357
* 12.9 其他并发控制机制	348	13.3 语言处理层	359
12.9.1 多版本并发控制方法	349	13.4 数据存取层	361
12.9.2 改进的多版本并发控制 方法	350	13.4.1 数据存取层的系统结构	361
本章小结	351	13.4.2 数据存取层的功能子系统	362
习题 12	351	13.5 缓冲区管理	365
第 12 章实验 并发控制	352	13.6 数据库的物理组织	366
参考文献 12	353	本章小结	368
* 第 13 章 数据库管理系统概述	355	习题 13	368
13.1 数据库管理系统的基本功能	355	参考文献 13	369

第四篇 新技术篇

第 14 章 数据库发展概述	373	参考文献 14	388
14.1 数据库系统发展概述	373	第 15 章 大数据管理系统	391
14.2 数据库系统发展简史	375	15.1 大数据概述	391
14.2.1 第一代数据库系统	375	15.1.1 什么是大数据	391
14.2.2 第二代数据库系统	376	15.1.2 大数据的特征	392
14.2.3 关系数据库系统的扩展	377	15.2 大数据管理系统	395
14.2.4 大数据时代的数据库系统	379	15.2.1 键值对数据库	396
14.3 计算机技术对数据库系统 发展的支撑作用	381	15.2.2 文档数据库	398
14.3.1 高速网络与分布式数据库	381	15.2.3 图数据库	401
14.3.2 新硬件与内存数据库	383	15.2.4 时序数据库	405
14.3.3 人工智能与数据库技术	384	本章小结	408
14.4 数据库发展展望	385	习题 15	408
14.4.1 多数据模型并存	385	参考文献 15	409
14.4.2 新硬件驱动	386	第 16 章 数据仓库与联机分析处理	411
14.4.3 云原生数据库	386	16.1 数据仓库技术	411
14.4.4 支持混合事务分析处理型应用	387	16.2 联机分析处理技术	414
14.4.5 面向人工智能的数据管理技术	387	16.3 混合事务分析处理技术	416
本章小结	387	16.4 大数据时代的新型数据仓库	418
习题 14	388	本章小结	421
		习题 16	422

参考文献 16	422	参考文献 17	441
第 17 章 内存数据库系统	425	第 18 章 区块链与数据库	443
17.1 内存数据库概述	425	18.1 区块链的概念与工作原理	443
17.2 新硬件技术推动内存数据库 技术发展	427	18.1.1 区块链的定义	444
17.2.1 多核处理器	427	18.1.2 区块的数据结构	445
17.2.2 图形处理器	428	18.1.3 区块链的工作机制	447
17.2.3 新型非易失性内存	429	18.1.4 区块链适用的应用场景	449
17.3 内存数据库的若干关键实现 技术	429	18.2 区块链的发展进程	450
17.3.1 数据存储	430	18.2.1 区块链发展的三个阶段	450
17.3.2 查询处理及优化	431	18.2.2 公有链、联盟链及私有链	451
17.3.3 并发与恢复	434	18.3 区块链系统的技术架构与关键 技术	453
17.4 内存数据库的几种实现方案	435	18.3.1 区块链系统的技术架构	453
17.4.1 混合的内存加速引擎	435	18.3.2 区块链技术的难点	455
17.4.2 独立的内存数据库系统	437	18.4 区块链与数据库	456
17.4.3 GPU 数据库	437	18.4.1 区块链与数据库的对比	456
17.5 内存数据库前沿技术与展望	438	18.4.2 区块链与数据库的融合	458
本章小结	440	本章小结	459
习题 17	441	习题 18	460
		参考文献 18	460
附录 “高校本科教务管理”信息系统的 E-R 图和关系模式	461		

第一篇

基础篇

本篇介绍数据库系统的基本概念和基础知识，这些概念和知识是读者进一步学习后面各个章节以及数据库系统其他课程的基础。

本篇包括5章。

第1章绪论，初步讲解数据库若干基本概念，介绍数据建模和数据模型的概念、数据库系统的三级模式结构和数据库系统的组成等基本知识。

第2章关系模型，系统讲解关系模型的数据结构、关系操作和三类完整性约束等重要概念以及关系代数和关系演算。

第3章关系数据库标准语言SQL，系统而详尽地讲解SQL的数据定义、数据查询和数据更新等主要功能。还讲解了索引的创建、空值的处理、视图的定义和作用等。

第4章数据库安全性，全面讲解实现数据库安全性的技术和方法，包括用户身份鉴别、自主存取控制方法、强制存取控制方法、视图机制、审计、数据存储加密和传输加密等。

第5章数据库完整性，系统而详尽地讲解用SQL实现实体完整性、参照完整性和用户定义的完整性，包括这些完整性约束的定义方法、完整性检查机制和违约处理，并介绍了触发器的作用和使用方法等。

数据库技术产生于20世纪60年代,是数据管理的核心技术。数据库管理系统(database management system, DBMS)是大型复杂的基础软件,也是现代信息系统的核心和基础。

数据库技术在发展过程中不断被证明是从应用需求出发开展理论研究与技术创新,再扎扎实实进行产品开发与广泛应用,从而形成良性循环的典范。数据库技术不仅发展成为一门学科,诞生了5位图灵奖得主(参见二维码内容),而且催生并推动了一个巨大的数据库软件产业。



扩展阅读:5位
图灵奖得主介绍

数据库技术是计算机科学与技术的重要分支,是计算机科学与技术中发展最快的领域之一,也是应用最广的技术之一,它极大地促进了计算机应用向各行各业的渗透。随着互联网的发展,广大用户可以通过互联网应用来访问数据库中的数据,例如通过京东、淘宝订购图书、食品和日用品,通过携程订购机票、火车票和酒店,通过网上银行或手机银行转账、检索和管理自己的账户等。小至个人使用的手机APP、车载导航系统,大到各个学校、银行、医院、商店的信息系统,再大到跨国企业和公司的大型信息管理系统,几乎所有的应用系统都是建立在数据库之上,依靠数据库系统存储、管理、处理和分析各类数据。数据库已成为每个人生活中不可或缺的部分。今天大数据应用、云计算技术的迅猛发展,更加凸显出数据库技术的重要性。因此,无论是信息系统的开发者,还是信息系统的运维管理者,或者是信息系统的最终使用者,都须具备一定的数据库基础知识。

本章导读

本章采用“盲人摸象”的方式勾画出数据库系统的轮廓,初步介绍数据库的若干基本概念与发展历史、数据建模与数据模型、数据库系统的模式结构和数据库系统组成等基本知识。目的是让读者了解本课程学什么,以及学习数据库系统的必要性和重要性。

本章首先介绍数据库4个最基本的概念,即数据、数据库、数据库管理系统和数据库系统;讲解数据管理技术的产生和发展过程,从中可以了解数据库系统和文件系统的区别以及数据库系统的优点。接着讲解数据建模的概念、概念模型、数据模型的基本要素,以及层次模型、网状模型、关系模型的基本知识,讨论数据库系统的三级模式结构和两级映像。最后介绍

数据库系统的组成以及数据库系统的体系结构。

学习本章应把注意力放在掌握基本概念和基本知识方面,为进一步学习后面的章节打好基础。**本章的难点是读者要在短时间内接触数据库领域诸多基本概念**,有些概念对于刚步入数据库领域的读者来说会感到比较抽象,不易理解。但不要紧,随着学习的循序渐进,后续章节将继续深入讲解这些概念,使读者逐步掌握和理解。

1.1 数据库系统概述

在系统地介绍数据库的基本概念之前,这里首先介绍一些数据库最常用的术语和基本概念。

1.1.1 数据库的4个基本概念

数据、数据库、数据库管理系统和数据库系统,是与数据库技术密切相关的4个基本概念。

1. 数据

数据(data)是数据库中存储的基本对象。对于数据,大多数人的第一个反应就是数字,例如93、1 000、99.5、-330.86、¥6 880、\$726等。其实数字只是最常见的一种数据,是对数据的一种传统和狭义的理解。广义的理解认为数据的种类很多,例如文本(text)、图形(graph)、图像(image)、音频(audio)、视频(video)、互联网上的博客、微信中的聊天记录、学生的档案记录、个人的网购记录、医院的患者病历等,都是数据。

为了认识世界,交流信息,人们需要描述事物。可以对数据做如下定义:描述事物的符号记录称为数据。描述事物的符号可以是数字,也可以是文字、图形、图像、音频、视频等。数据有多种表现形式,它们都可以经过数字化后存入计算机。

在现代计算机系统中数据的概念是广义的。早期的计算机系统主要用于科学计算,处理的数据是数值型数据,如整数、实数、浮点数等。现在的计算机存储和处理的对象十分广泛,用于表示这些对象的数据也随之变得越来越复杂。

数据的表现形式还不能完全表达其内容,需要经过解释,数据和关于数据的解释是不可分的。例如,93是一个数据,它可以是一个学生某门课程的考试成绩,也可以是某个人的体重,还可以是计算机科学与技术专业2018级的学生人数。数据的解释是指对数据含义的说明,数据的含义称为数据的语义,数据与其语义是不可分的。

在日常生活中,人们可以直接用自然语言(如汉语)描述事物。例如,可以这样描述某校一个计算机科学与技术专业学生的基本情况:学号为20180002、姓名为刘晨的女生,1999年9月1日出生,计算机科学与技术专业。在计算机中则常用如下的形式描述:

(20180002, 刘晨, 女, 1999-9-1, 计算机科学与技术)

即把学生的学号、姓名、性别、出生日期、主修专业等组织在一起,构成一条记录。这里的学

生记录就是描述学生的数据,这样的数据是有结构的。记录是计算机中表示和存储数据的一种格式或一种方法。

2. 数据库

数据库(database, DB),顾名思义,是存放数据的仓库。只不过这个仓库是在计算机存储设备上,而且数据是按一定的格式存放的。

人们采集一个应用所需要的大量数据之后,应将其保存起来以供进一步加工处理,并从中抽取有用信息。在数据采集手段越来越方便的今天,数据量急剧增加,过去人们把数据存放在文件柜里,现在存储在数据库中是最佳选择。借助数据库技术保存和管理大量复杂的数据,可以充分地利用这些宝贵的信息资源。

所谓数据库,就是长期存储在计算机内有组织、可共享的大量数据的集合。数据库中的数据按一定的数据模型组织、描述和存储,具有较小的数据冗余(data redundancy)、较高的数据独立性(data independency)和可扩展性(scalability),并可为各种用户共享。数据库的这些特性将在1.1.2小节继续讲解。

3. 数据库管理系统

了解数据和数据库的初步概念之后,下一个问题就是如何科学地组织和存储这些数据,以及如何高效地处理和维护数据。完成该任务的即为数据库管理系统。

数据库管理系统是位于用户与操作系统之间的数据管理软件。它和操作系统一样是计算机的基础软件,也是一类大型复杂的软件系统。它的主要功能包括以下几个方面。

① 数据定义功能。数据库管理系统提供数据定义语言(data definition language, DDL),用户通过它可以方便地对存储在数据库中的数据对象的组成与结构进行定义。

② 数据组织、存储和管理功能。数据库管理系统要分类组织、存储和管理各种数据,包括数据字典、用户数据、数据存取路径等。要确定以何种文件结构和存取方式在存储器上组织这些数据,以及如何实现数据之间的联系。数据组织和存储的基本目标是提高存储空间利用率和方便存取,可提供多种存取方法(如索引查找、哈希查找、顺序查找等)来提高存取效率。

③ 数据操纵功能。数据库管理系统还提供数据操纵语言(data manipulation language, DML),用户可以使用它操纵数据,实现对数据库的基本操作,如查询、插入、删除和修改等。

④ 数据库的事务管理和运行管理功能。数据库在建立、运行和维护时由数据库管理系统统一管理和控制,以保证事务的正确运行、数据的安全性、完整性、多用户对数据的并发使用,以及发生故障后的系统恢复。

⑤ 数据库的建立和维护功能。数据库的建立和维护功能包括数据库初始数据的输入和转换功能,数据库的转储和恢复功能,数据库的重组、性能监视和数据分析等功能。这些功能通常是由一些实用程序或管理工具完成的。

⑥ 其他功能。其他功能包括数据库管理系统与网络中其他软件系统的通信功能、一个数

数据库管理系统与另一个数据库管理系统或文件系统的数据转换功能、异构数据库之间的互访和互操作功能等。

4. 数据库系统

数据库系统(database system, DBS)是指引入数据库后的计算机系统,一般是指由数据库、数据库管理系统(及其应用开发工具)、应用系统和数据库管理员(database administrator, DBA)组成的存储、管理、处理和维护数据的系统。应当指出的是,数据库的建立、使用和维护等工作只靠一个数据库管理系统远远不够,还要有专门的人员来完成,这些人被称为数据库管理员。

数据库系统可以用图 1.1 表示。其中数据库提供数据的存储功能,数据库管理系统提供数据的组织、存取、管理和维护等基础功能,应用系统根据应用需求使用数据库,数据库管理员负责数据库管理系统的运行。图 1.2 是引入数据库管理系统后计算机系统的层次结构。

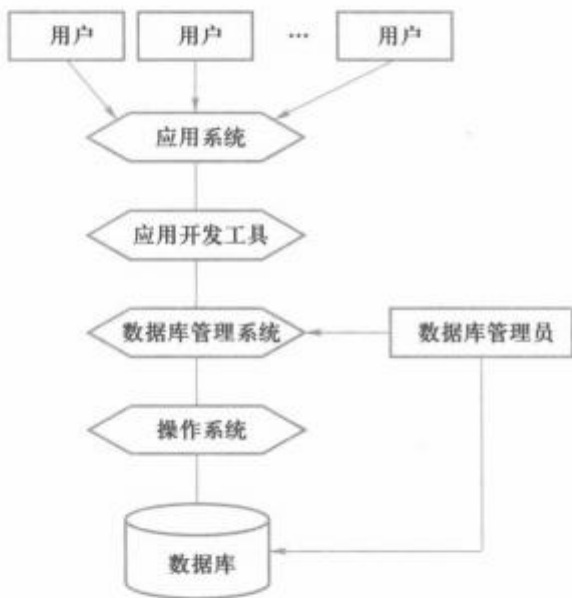


图 1.1 数据库系统



图 1.2 引入数据库管理系统后计算机系统的层次结构

在不引起混淆的情况下,人们常常把数据库系统简称为数据库。

1.1.2 数据管理技术的产生和发展

数据库技术是为满足数据管理任务的需要而产生的。数据管理是指对数据进行分类、组织、编码、存储、检索和维护,它是数据处理的中心问题。而数据处理是指对各种数据进行收集、存储、加工和传播的一系列活动的总和。

在应用需求的推动下,在计算机软硬件发展的基础上,数据管理技术经历了人工管理、文件系统、数据库系统三个阶段。这三个阶段的比较如表 1.1 所示。

表 1.1 数据管理技术三个阶段的比较

对比项	人工管理阶段	文件系统阶段	数据库系统阶段
应用领域	科学计算	科学计算、数据管理	大规模数据管理
主要硬件	无直接存取存储设备	磁盘、磁鼓	大容量磁盘、磁盘阵列
主要软件	没有操作系统, 没有管理数据的专门软件	有文件系统	有数据库管理系统
数据处理方式	批处理	联机实时处理、批处理	联机实时处理、分布处理、批处理
数据管理者	人(程序员)	文件系统	数据库管理系统
数据面向对象	某一应用程序	某一应用	现实世界(部门、企业、跨国组织等)
数据共享程度	不共享, 冗余度极高	共享性弱, 冗余度高	共享性强, 冗余度低且易扩充
数据独立性	不独立, 完全依赖应用程序	独立性弱	具有较强的物理独立性和一定的逻辑独立性
数据结构化	无结构	记录内有结构、整体无结构	整体结构化, 用数据模型描述
数据控制能力	应用程序自己控制	应用程序自己控制	由数据库管理系统提供数据安全性、完整性、并发控制和数据库恢复功能

1. 人工管理阶段

20 世纪 50 年代中期以前, 计算机主要用于科学计算。当时的硬件状况是外存只有纸带、卡片、磁带, 没有磁盘等直接存取存储设备; 软件状况是没有操作系统, 没有管理数据的专门软件; 数据处理方式是批处理。

人工管理数据具有如下特点:

① 数据不保存。当时的计算机主要用于科学计算, 一般不需要长期保存数据, 只是在计算时将数据输入, 用完就撤走。不仅对用户数据如此处置, 对系统软件有时也是这样处置。

② 应用程序管理数据。数据需要由程序员在应用程序中设计、说明(定义)和管理, 没有相应的软件系统负责数据的管理工作。应用程序不仅要规定数据的逻辑结构, 而且要规定数据的物理结构, 包括存储结构、存取方法、输入方式等。因此程序员的负担很重。

③ 数据不共享。数据是面向应用程序的, 一组数据只能对应一个程序。当多个应用程序涉及某些相同的数据时必须各自定义, 无法互相利用、互相参照, 因此程序与程序之间有大量的冗余数据。

④ 数据不具有独立性。数据独立性是指数据与应用程序相互独立, 即数据的结构发生变化后, 应用程序不必做相应的修改。

在人工管理阶段, 数据由程序员在应用程序中定义, 数据结构一旦发生变化, 就需要修改应用程序, 数据完全依赖于应用程序, 称之为数据缺乏独立性, 这就加重了程序员的负担。

在人工管理阶段, 应用程序与数据(表现为数据集)之间的一一对应关系可用图 1.3 表示。

2. 文件系统阶段

20 世纪 50 年代后期~20 世纪 60 年代中期, 这时硬件方面已有磁盘、磁鼓等直接存取存储设备; 软件方面表现为操作系统中已有专门的数据管理软件, 一般称为文件系统; 处理方式不仅能够进行批处理, 而且能够进行联机实时处理。

用文件系统管理数据具有如下特点:

① 数据可以长期保存。由于计算机主要用于数据处理, 一般将数据长期保留在外存上以反复进行查询、修改、插入和删除等操作。

② 数据由专门的软件进行管理, 文件系统把数据组织成相互独立的数据文件, 利用“按文件名访问, 按记录进行存取”的管理技术, 提供了对文件进行打开与关闭、对记录进行读取和写入等存取方式。

文件系统实现了记录内的结构性。但是, 文件系统仍存在以下缺点:

① 数据共享性弱, 冗余度高。在文件系统中, 一个(或一组)文件基本上对应于一个应用程序, 即文件仍然是面向应用的。当不同的应用程序具有部分相同的数据时也必须建立各自的文件, 而不能共享相同的数据, 因此数据的冗余度高, 浪费存储空间。同时, 相同数据的重复存储与各自管理容易造成数据的不一致性, 给数据的修改和维护带来困难。

② 数据独立性弱。文件系统文件是为某一特定应用服务的, 文件的逻辑结构针对具体的应用设计和优化, 因此要想对文件中的数据再增加一些新的应用会很困难。而且, 当数据的逻辑结构改变时, 应用程序中文件结构的定义必须修改, 应用程序中对数据的使用也要改变, 因此数据依赖于应用程序, 缺乏独立性。可见, 文件系统仍然是一个不具有弹性的无整体结构的数据集合, 即文件之间是孤立的, 不能反映现实世界事物之间的内在联系。

在文件系统阶段, 应用程序与数据(表现为文件组)之间的关系如图 1.4 所示。



图 1.3 人工管理阶段应用程序与数据之间的——对应关系

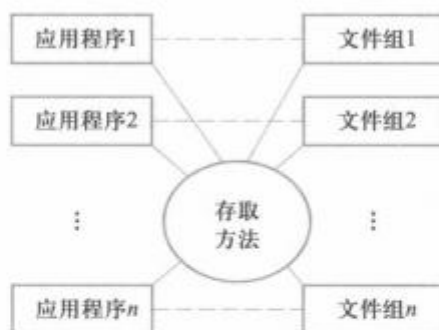


图 1.4 文件系统阶段应用程序与数据之间的对应关系

3. 数据库系统阶段

20 世纪 60 年代后期以来, 计算机管理的对象规模越来越大, 应用范围越来越广泛, 数据量急剧增长。同时, 多种应用、多种语言互相交错地共享数据集的要求越来越强烈。

这时硬件方面已有大容量磁盘和磁盘阵列，硬件价格下降；软件方面则表现为软件价格上升，编制和维护系统软件及应用程序所需的成本相对增加；在处理方式上更多地应用联机实时处理，并开始提出和考虑分布式处理。在这种背景下，以文件系统作为数据管理手段已不能满足应用的需求，于是为解决多用户、多应用共享数据的需求，使数据为尽可能多的应用服务，数据库技术便应运而生，出现了统一管理数据的专门软件系统——数据库管理系统。

数据库系统阶段的数据具有如下特点。

(1) 整体数据的结构化

数据库系统阶段实现了整体数据的结构化，这是数据库的主要特征之一，也是数据库系统阶段与文件系统阶段的本质区别。

在文件系统中，文件中的记录具有结构，但是记录的结构和记录之间的联系被固化在程序中，需要由程序员加以维护。这种工作模式既加重了程序员的负担，又不利于结构的变动。二维码所示内容通过一个例子来比较文件系统操作与数据库系统操作的差异，从中可看到数据库系统的优点。



扩展阅读：文件系统与数据库系统操作比较示例

所谓整体数据的结构化，是指数据库中的数据不再是仅仅针对某一个应用，而是面向整个组织或企业的多种应用需求；不仅数据本身是结构化的，而且整体数据也是结构化的，即数据之间具有联系。也就是说，不仅要考虑某个应用的数据结构，还要考虑整个组织的数据结构。

例如，一个学校的信息系统不仅要考虑教务处的课程管理、学生选课管理、成绩管理，还要考虑学生处的学生学籍管理、研究生院的研究生管理、人事处的教师人事管理、科研处的科研管理等。因此，学校信息系统中的学生数据就要面向各个部门的应用，而不仅仅是面向教务处的一个学生选课应用。图 1.5 为“高校本科教务管理”信息系统^①中与学生有关的数据结构。



图 1.5 “高校本科教务管理”信息系统中与学生有关的数据结构

^① 本书用一个实例——“高校本科教务管理”信息系统将数据库系统知识点讲解和设计样例贯穿起来，随着知识点讲解的深入，该实例的内容也将根据实际情况不断丰富和变化。相关介绍详见附录。

图 1.5 所示的数据组织方式为各部门的应用提供了必要的记录,使整体数据结构化了。这就要求在描述数据时不仅要描述数据本身,还要描述数据之间的联系。

记录的结构和记录之间的联系由数据库管理系统维护,从而减轻了程序员的负担,提高了工作效率。

在数据库系统中,不仅整体数据是结构化的,而且存取数据的方式也很灵活,可以存取数据库中的某一个或一组数据项、一条记录或一组记录;而在文件系统中,数据的存取单位是记录,粒度不能细到数据项。

(2) 数据的共享性强、冗余度低且易于扩充

数据库系统从整体角度看待和描述数据,数据不再是面向某个应用,而是面向整个系统。因此,数据可以被多个用户或多个应用通过不同的接口、不同的编程语言共享使用。这种较强的数据共享性可以大大降低数据的冗余度,节省存储空间,还能避免数据的不相容性与不一致性。

所谓数据的不一致性,是指同一数据不同副本的值不一样。采用人工管理或文件系统管理时,由于数据被重复存储,不同的应用使用和修改不同的副本就很容易造成数据的不一致;而在数据库中数据共享则减少了由数据冗余造成的不一致现象。

数据库数据的共享性强还使其易于增加新的应用,易于扩充,这也是数据库系统“弹性大”的原因。我们可以选取整体数据的各种子集用于不同的应用系统,当应用需求改变或增加时,只要重新选取不同的子集或加上一部分数据即可满足新的需求。

(3) 数据的独立性强

数据的独立性强是数据库数据的一个显著优点。数据独立性已成为数据库领域的一个常用术语和重要概念,其目标是应用程序与数据(定义)相分离。数据的独立性包括数据的物理独立性和数据的逻辑独立性。

数据的物理独立性,是指用户的应用程序与数据库中数据的物理存储是相互独立的。也就是说,数据在数据库中怎样存储是由数据库管理系统管理的,用户程序不需要了解,这样当数据的物理存储改变时应用程序不用改变。

数据的逻辑独立性,是指用户的应用程序与数据库的逻辑结构是相互独立的。也就是说,数据的逻辑结构改变时用户程序也可以不变。

数据的独立性是由数据库管理系统提供的两级映像功能来保证的,相关内容将在 1.3.3 小节进行讨论。

数据的独立性把数据的定义从应用程序中分离出去,而存取数据的方法又由数据库管理系统负责提供,从而简化了应用程序的编制,大大减少了应用程序的维护和修改工作。

(4) 数据由数据库管理系统统一管理和控制

数据库数据的共享将会带来数据库的安全隐患,且因这种共享是并发性(concurrency)的,即多个用户可以同时存取数据库中的数据,甚至可以同时存取数据库中的同一个数据,这又会带来不同用户间相互干扰的隐患。另外,数据库中数据的正确性与一致性也必须得到保障。为

此,数据库管理系统还必须提供以下几方面的数据管理功能。

① 数据的安全性(security)保护。数据的安全性是指保护数据以防不合法使用造成数据泄露和破坏。每个用户只能依照规定对某些数据按指定方式进行使用和处理,相关内容将在第4章进行讲解。

② 数据的完整性(integrity)检查。数据的完整性是指数据的正确性、有效性和相容性。完整性检查将数据控制在有效的范围内,并保证数据之间满足一定的关系,相关内容将在第5章进行讲解。

③ 数据的并发控制。当多个用户的并发进程同时存取、修改数据库时,可能会因相互干扰而得到错误的结果或使数据库的完整性遭到破坏。因此,必须对多用户的并发操作加以控制和协调,以保证一个用户事务的执行不受其他事务的干扰,从而避免造成数据的不一致性。相关内容将在第12章进行讲解。

④ 数据库的恢复(recovery)。计算机系统的硬件/软件故障、操作失误以及蓄意破坏等会影响数据库中数据的正确性,甚至造成数据库部分或全部数据的丢失。数据库管理系统必须具有将数据库从错误状态恢复到某一已知的正确状态(亦称为完整状态或一致状态)的功能,即数据库的恢复功能。相关内容将在第11章进行讲解。

在数据库系统阶段,应用程序与数据(表现为数据库)之间的对应关系可用图1.6表示。



图 1.6 数据库系统阶段应用程序与数据之间的对应关系

综上所述,数据库是长期存储在计算机内的有组织、可共享的大量数据的集合。它可以供各种用户共享,具有最小的冗余度和较强的数据独立性。数据库管理系统在数据库建立、运维时对数据库进行统一控制,以保证数据的完整性和安全性,并在多用户同时使用数据库时进行并发控制,在发生故障后对数据库进行恢复。

数据库系统的出现,使信息系统从以加工数据的程序为中心转向以共享的数据库为中心的新阶段,即以软件为中心向以数据为中心的计算机平台的迁移。这样既便于数据的集中管理,又能简化应用系统的研制和维护,提高了数据的利用率和决策的可靠性。

数据库技术的发展经历了20世纪60年代的网状数据库、层次数据库,20世纪70年代的关系数据库,以及20世纪80年代的以面向对象模型为主要特征的数据库系统。随着大数据应

用的迅猛发展,又陆续出现了众多新型的数据模型和数据库管理系统。

数据库技术与网络通信技术、面向对象程序设计技术、并行/分布式计算技术、云计算技术、人工智能技术、新硬件技术等互相渗透、互相结合,成为当前数据库技术发展的主要特征。我们将在新技术篇的第14章中介绍数据库技术的最新发展。

1.2 数据模型

数据库技术是计算机领域中发展最快的技术之一,其发展是沿着数据模型的主线推进的。模型,特别是具体模型对人们来说并不陌生。一张地图、一组建筑设计沙盘、一架精致的航模飞机都是具体的模型,一眼望去就会使人联想到真实生活中的事物。模型是对现实世界中某个对象特征的模拟和抽象。例如,航模飞机是对生活中飞机的一种模拟和抽象,它可以模拟飞机的起飞、飞行和降落,并抽象了飞机的基本特征——机头、机身、机翼、机尾等。

数据模型(data model)也是一种模型,它是对现实世界数据特征的抽象。也就是说,数据模型是用来描述数据、组织数据和对数据进行操作的。

由于计算机不可能直接处理现实世界中的具体事物,所以人们必须事先把具体事物转换成计算机能够处理的数据,也就是首先要数字化,把现实世界中具体的人、物、活动等用数据模型这个工具来抽象、表示和处理。通俗地讲,数据模型就是现实世界的模拟。

现有的数据库系统均是基于某种数据模型的,数据模型是数据库系统的核心和基础。因此,了解数据模型的基本概念是学习数据库的基础。

1.2.1 数据建模

把现实世界中的具体事物抽象、组织为某一数据库管理系统支持的数据模型,这个过程称为**数据建模(data modeling)**。

在数据库系统中,数据建模过程通常分为两步进行。

1. 建立概念模型

首先将现实世界抽象为信息世界。也就是把现实世界中的客观对象抽象为某一种信息结构,这种信息结构并不依赖于具体的计算机系统,不是某一个数据库管理系统支持的数据模型,而是概念级的模型,因此称为**概念模型(conceptual model)**。

概念模型是按用户的观点来对数据建模,主要用于数据库设计。从现实世界到概念模型的建模任务由数据库设计人员完成,也可以通过数据库设计工具辅助设计人员完成。

2. 将概念模型转换为数据模型

接下来将信息世界转换为机器世界,即把概念模型转换为计算机上某一数据库管理系统支持的数据模型。

数据模型是按计算机系统的观点对数据建模,是数据库管理系统支持的,用于数据库管理系统的实现。从概念模型到数据模型的转换由数据库设计人员完成,也可以通过数据库设计工

具辅助设计人员完成。

上述抽象过程如图 1.7 所示。



图 1.7 现实世界中客观对象的抽象过程

下面首先介绍概念模型，然后讲解数据模型的基本要素，并介绍几种不同的数据模型。

1.2.2 概念模型

由图 1.7 可以看出，概念模型实际上是现实世界到机器世界的一个中间层次。

概念模型用于信息世界的建模，是现实世界到信息世界的第一层抽象，是数据库设计人员进行数据库设计的有力工具，也是数据库设计人员和用户之间进行交流的语言。因此，概念模型一方面应具有较强的语义表达能力，能够方便、直接地表达应用中的各种语义知识，另一方面还应简单、清晰、易于用户理解。

1. 信息世界中的基本概念

信息世界主要涉及以下一些概念。

① 实体(entity)。客观存在并可相互区别的事物称为实体。实体可以是具体的人、事、物，也可以是抽象的概念或联系。例如，一个职工、一个学生、一个部门、一门课等都是实体。

② 属性(attribute)。实体所具有的某一特性称为属性。一个实体可以由若干个属性来刻画。例如，“学生”实体可以由“学号”“姓名”“性别”“出生日期”“主修专业”等属性组成，属性组合(20180003, 王敏, 女, 2001-8-1, 计算机科学与技术)即表征了一个学生。

③ 码(key)。唯一标识实体的属性集称为码。例如，“学号”是“学生”实体的码。

④ 实体类型(entity type)。具有相同属性的实体必然具有共同的特征和性质。用实体名及其属性名集合来抽象和刻画同类实体，称为实体类型，或称实体型。例如，学生(学号, 姓名, 性别, 出生日期, 主修专业)就是一个实体型。

⑤ 实体集(entity set)。同一类型实体的集合称为实体集。例如，全体学生就是一个实

体集。

⑥ 联系 (relationship)。在现实世界中,事物内部以及事物之间是有联系的,这些联系在信息世界中反映为实体(型)内部的联系和实体(型)之间的联系。实体内部的联系通常是指组成实体的各属性之间的联系,实体之间的联系通常是指不同实体集之间的联系。

实体之间的联系有一对一、一对多和多对多等多种类型(具体内容将在第7章7.3.2小节E-R模型中进行介绍)。

2. 概念模型的一种表示方法: 实体-联系模型

概念模型是对信息世界建模,所以概念模型应该能够方便、准确地表示上述信息世界中的常用概念。概念模型的表示方法很多,其中最为常用的是P. P. S. Chen于1976年提出的实体-联系模型(entity-relationship model),简称E-R模型。该方法用E-R图(E-R diagram)来描述现实世界的概念模型。图1.8是学生选课E-R图示例。

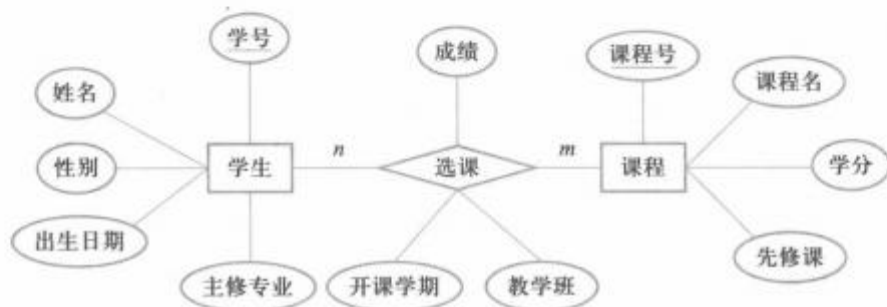


图 1.8 学生选课 E-R 图示例

图1.8抽象了学校中的“学生”和“课程”两个客观事物:“学生”实体和“课程”实体;还抽象了现实世界中事物之间的联系:一门课程可以有多个学生选修,一个学生可以选修多门课程,用“课程”实体与“学生”实体的多对多($m:n$)联系来描述。

有关如何认识和分析现实世界,从中抽取实体、实体的属性和实体之间的联系,建立概念模型,画出E-R图等方法,将在第7章数据库设计中详细讲解。

1.2.3 数据模型的三要素

一般地讲,数据模型是严格定义的一组概念的集合。这些概念精确地描述了系统的静态特性、动态特性和完整性约束(integrity constraint)。因此数据模型通常由数据结构、数据操纵和完整性约束三部分组成,称为数据模型的三要素。

1. 数据结构

数据结构描述数据库的组成对象以及对象之间的联系。也就是说,数据结构描述的内容有两类:一类是与对象的类型、内容、性质有关的,如网状模型中的数据项、记录,关系模型中的域、属性、关系等;一类是与数据之间的联系有关的对象,如网状模型中用系(set)表示记录之间的联系。

数据结构是刻画一个数据模型性质的最重要的方面。因此,在数据库系统中人们通常按照其数据结构的类型来命名数据模型。例如,层次结构、网状结构和关系结构的数据模型分别命名为层次数据模型、网状数据模型和关系数据模型,简称为层次模型、网状模型和关系模型。

总之,数据结构是所描述的对象类型的集合,是对系统静态特性的描述。

2. 数据操纵

数据操纵是指对数据库中各种对象(型)的实例(值)允许执行的操作的集合,包括操作及有关的操作规则。

数据库主要有查询和更新(包括插入、删除、修改)两大类操作。数据模型必须定义这些操作的确切含义、操作符号、操作规则(如优先级)以及实现操作的语言。

数据操纵是对系统动态特性的描述。

3. 完整性约束

完整性约束是一组完整性规则。完整性规则是给定的数据模型中的数据及其联系所具有的制约和依存规则,用以限定符合数据模型的数据库状态以及状态的变化,以保证数据的正确、有效和相容。

数据模型应该反映和规定其必须遵守的基本和通用的完整性约束。例如,在关系模型中,任何关系必须满足实体完整性和参照完整性。本书第2章关系模型、第5章数据库完整性将详细讨论这两类完整性约束。

注意,这里讲的数据模型都是逻辑上的,它们都能用某种语言描述,使数据库管理系统能够理解。被数据库管理系统支持的数据视图,也是用户可以用这种语言描述和处理的数据结构。

这些数据模型将以一定的组织方式存储在数据库管理系统中,是数据模型在数据库管理系统内部的物理存储结构,人们称之为数据的物理模型。

物理模型描述数据在数据库管理系统内部的数据组织和存取方法的实现,包括存储结构和索引结构的物理实现。例如,记录的表示、记录如何在一个物理块中组织存储、有哪些索引结构、索引的组织方式和查找方法等。常用的索引有顺序表索引、B+树索引和哈希索引等,本书第9章关系数据库存储管理将详细讲解。

数据结构、数据操纵和完整性约束这三方面内容完整地描述了一个数据模型,其中数据结构是刻画模型性质的最基本的内容。为了使读者对数据模型有一个初步认识,下面简要介绍层次模型(hierarchical model)、网状模型(network model)和关系模型(relational model)。

1.2.4 层次模型

在层次模型中实体用记录表示,实体的属性对应记录的字段(或数据项)。这样可以将实体之间的联系转换成记录之间的两两联系。

在层次模型中,数据结构的单位是基本层次联系。所谓基本层次联系是指两个记录及其之

间的一对多(包括一对一)联系,如图 1.9 所示。

图中 R_i 位于联系 L_{ij} 的始点,称为双亲(parent)结点, R_j 位于联系 L_{ij} 的终点,称为子女(child)结点。

层次模型是数据库系统中最早出现的数据模型,层次数据库系统采用层次模型作为数据的组织方式。层次数据库系统的典型代表是 IBM 公司的 IMS(information management system),这是 1968 年 IBM 公司推出的第一个大型商用数据库管理系统,曾经得到广泛的使用。

层次模型用树形结构表示各类实体以及实体间的联系。现实世界中许多实体之间的联系本来就呈现出一种很自然的层次关系,如行政机构、家族关系等。

1. 层次模型的数据结构

在数据库中,满足下面两个条件的基本层次联系的集合为层次模型:

- ① 有且只有一个结点没有双亲结点,这个结点称为根结点。
- ② 根以外的其他结点有且只有一个双亲结点。

在层次模型中,每个结点表示一个记录类型,记录类型之间的联系用结点之间的连线(有向边)表示,这种联系是双亲结点与子女结点之间的一对多联系。这就使得层次数据库系统只能处理一对多的实体联系。

每个记录类型可包含若干字段,这里记录类型描述的是实体型,字段描述实体的属性。各个记录类型及其字段都必须命名。各个记录类型、同一记录类型中的各个字段不能同名。每个记录类型可以定义一个排序字段,也称为码字段,如果定义该排序字段的值是唯一的,则它能唯一地标识一个记录值。

一个层次模型在理论上可以包含任意有限个记录类型和字段,但任何实际的系统都会因为存储容量或实现复杂度而限制模型中包含的记录类型和字段的数量。

在层次模型中,同一双亲结点的子女结点称为兄弟(twin 或 sibling)结点,没有子女结点的结点称为叶(leaf)结点。图 1.10 给出了一个层次模型示例,其中, R_1 为根结点; R_2 和 R_3 为兄弟结点,且是 R_1 的子女结点; R_4 和 R_5 为兄弟结点,且是 R_2 的子女结点; R_3 、 R_4 和 R_5 为叶结点。

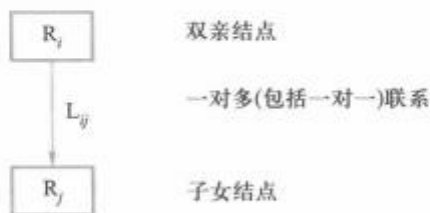


图 1.9 基本层次联系



图 1.10 一个层次模型示例

从图 1.10 中可以看到层次模型像一棵倒立的树,每个结点的双亲结点都是唯一的。

层次模型的一个基本的特点是,任何一个给定的记录值只能按其层次路径查看,没有一个

子女记录值能够脱离双亲记录值而独立存在。

图 1.11 是“高校本科教务管理”信息系统中“学生学籍管理”子系统的层次模型示意图。该层次模型有 4 个记录类型，其中记录类型“学院”是根结点，由三个字段(学院编号、学院名、建院时间)组成，它有两个子女结点“系”和“学生”。记录类型“系”是“学院”的子女结点，同时又是“教师”的双亲结点，它由 4 个字段(系编号、系名、联系人、联系方式)组成。记录类型“学生”由 5 个字段(学号、姓名、性别、出生日期、主修专业)组成。记录类型“教师”由 4 个字段(职工号、姓名、职称、出生日期)组成。“学生”与“教师”是叶结点，它们没有子女结点。由“学院”到“系”、由“系”到“教师”、由“学院”到“学生”均是一对多的联系。



图 1.11 “学生学籍管理”子系统的层次模型示意图

图 1.12 是图 1.11 所示层次模型对应的一个值。该值是由“信息学院”一个记录值及其所有后代记录值组成的一棵树。其中，“信息学院”有三个系记录值和三个学生记录值，“计算机系”有 4 个教师记录值，“数学系”有两个教师记录值。

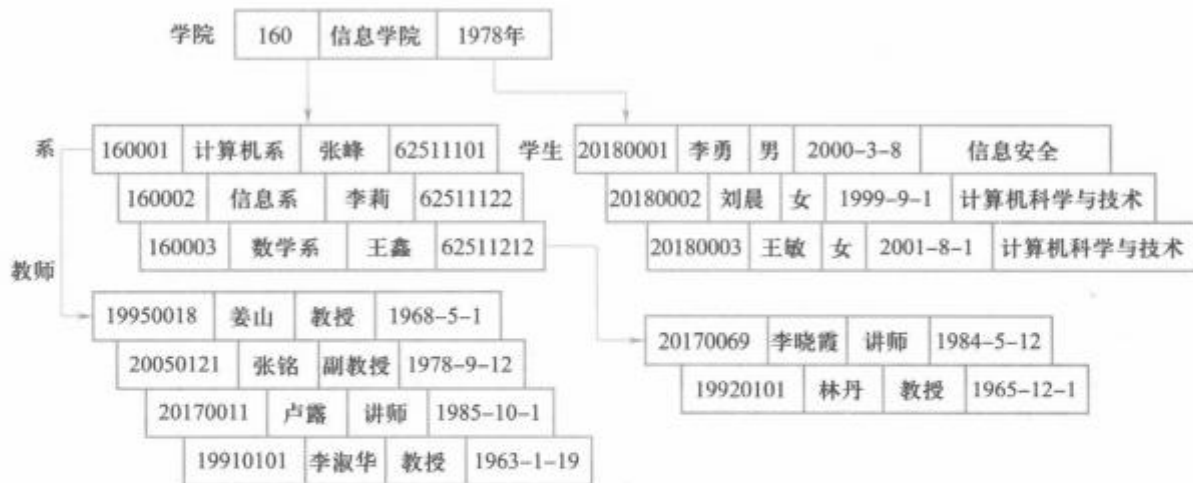


图 1.12 “学生学籍管理”子系统层次模型的一个值

2. 层次模型的数据操纵与完整性约束

层次模型的数据操纵主要有查询、插入、删除和更新操作。在进行插入、删除和更新操作

时要满足层次模型的完整性约束。

进行插入操作时,如果没有相应的双亲结点值就不能插入其子女结点值。例如在图 1.12 所示的层次数据库中,若新调入一名教师,但尚未分配到某个系,这时就不能将新教师插入数据库。

进行删除操作时,如果删除双亲结点值,则相应的子女结点值也将被同时删除。例如在图 1.12 所示的层次数据库中,若删除“数学系”,则“数学系”所有教师的数据将全部丢失。

3. 层次模型的优缺点

层次模型的优点主要有:

① 层次模型的数据结构比较简单清晰。

② 层次数据库的查询效率高。层次模型中记录之间的联系用有向边表示,这种联系在数据库管理系统中常用指针来实现,因此这种联系也就是记录之间的存取路径。当要存取某个结点的记录值时,数据库管理系统沿着这条路径很快就能找到该记录值,所以层次数据库的性能优良,存取效率高。

③ 层次数据模型提供了良好的完整性约束支持。

层次模型的缺点主要有:

① 现实世界中很多联系是非层次性的,如结点之间具有多对多联系,不适合用层次模型表示。

② 如果一个结点具有多个双亲结点,用层次模型表示这类联系就很笨拙,只能通过引入冗余数据(易产生不一致性)或创建非自然的数据结构(引入虚拟结点)来解决,对插入和删除操作的限制比较多,因此应用程序的编写比较复杂。

③ 查询子女结点必须通过双亲结点。

④ 由于结构严密,层次命令趋于程序化。

可见,用层次模型对具有一对多层次联系的部门描述非常自然、直观,容易理解。这是层次数据库的突出优点。

1.2.5 网状模型

在现实世界中事物之间的联系更多为非层次关系,用层次模型表示非树形结构很不直接,而网状模型则可以克服这一弊端。

网状数据库系统采用网状模型作为数据的组织方式,其典型代表是 DBTG 系统,亦称 CODASYL 系统。这是 20 世纪 70 年代数据系统语言会议(conference on data system language, CODASYL)下属的数据库任务组(data base task group, DBTG)提出的一个系统方案。DBTG 系统虽然不是实际的数据库系统软件,但是它的基本概念、方法和技术具有普遍意义,对于网状数据库系统的研制和发展产生了重大的影响。后来不少系统都采用 DBTG 系统或者简化的 DBTG 系统模型,如 Cullinet Software 公司的 IDMS、Univac 公司的 DMS-1100、Honeywell 公司的 IDS/2、HP 公司的 IMAGE 等。

1. 网状模型的数据结构

在数据库中,把满足以下两个条件的基本层次联系集合称为网状模型:

- ① 允许一个以上的结点无双亲结点。
- ② 一个结点可以有多个的双亲结点。

网状模型是一种比层次模型更具普遍性的结构。它去掉了层次模型的两个限制,允许多个结点没有双亲结点,且允许结点有多个双亲结点;此外它还允许两个结点之间有多种联系(称之为复合联系)。因此,网状模型可以更直接地描述现实世界,而层次模型实际上是网状模型的一个特例。

与层次模型一样,网状模型中每个结点表示一个记录类型(实体型),每个记录类型可包含若干字段(实体的属性),结点间的连线表示记录类型(实体型)之间一对多的联系。

从网状模型的定义可以看出,层次模型中子女结点与双亲结点的联系是唯一的,而在网状模型中这种联系可以不唯一。因此要为每个联系命名,并指出与该联系有关的双亲结点和子女结点。例如,图 1.13(a)中 R_3 有两个双亲结点 R_1 和 R_2 ,因此把 R_1 与 R_3 之间的联系命名为 L_1 , R_2 与 R_3 之间的联系命名为 L_2 。图 1.13(a)(b)(c)都是网状模型的例子。

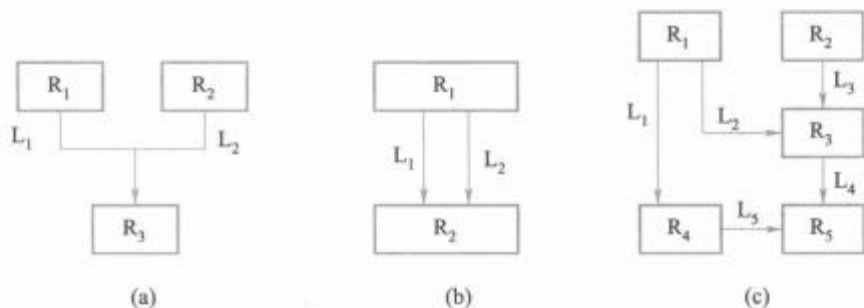


图 1.13 网状模型示例

下面以“学生选课”子系统为例,观察网状数据库是怎样组织数据的。

按照常规语义,一个学生可以选修若干门课程,某一课程可以被多个学生选修,因此学生与课程之间是多对多的联系。因为 DBTG 系统模型不能表示记录之间多对多的联系,为此引进一个“学生选课”的连接记录,它由 5 个字段(学号、课程号、成绩、开课学期,教学班)组成,表示某个学生在某个学期的某个教学班选修了某一门课程及其成绩。这样,“学生选课”子系统共包含 3 个记录类型“学生”“课程”和“学生选课”(如图 1.14 所示)。

每个学生可以选修多门课程,显然对“学生”记录类型中的一个值,“学生选课”记录类型中可以有多个值与之联系,而“学生选课”记录类型中的一个值,只能与“学生”记录类型中的一个值联系。“学生”与“学生选课”之间的联系是一对多的联系,联系名为“S-SC”。同样,“课程”与“学生选课”之间的联系也是一对多的联系,联系名为“C-SC”。

2. 网状模型的数据操纵与完整性约束

一般来说,网状模型没有层次模型那样严格的完整性约束,但具体的网状数据库系统对数



图 1.14 “学生选课”子系统的网状模型

据操纵都加了一些限制，提供了一定的完整性约束。

例如，DBTG 系统在模式数据定义语言中提供了定义 DBTG 数据库完整性的若干概念和语句，主要有：

① 支持记录类型中“码”的概念，码即唯一标识记录类型的数据项的集合。例如，图 1.14 的“学生”记录类型中“学号”是码，因此数据库中不允许“学号”出现重复值。

② 保证一个联系中双亲结点和子女结点之间是一对多的联系。

③ 可以支持双亲结点和子女结点之间的某些约束条件。例如，有些子女结点值要求双亲结点值存在才能插入，双亲结点值删除时也连同删除。例如图 1.14 中“学生选课”记录类型就应该满足这种约束条件，学生选课的记录值必须是数据库中存在的某一学生选修存在的某一门课的记录。DBTG 系统提供了“属籍类别”(membership class)的概念来描述这类约束条件。

3. 网状模型的优缺点

网状模型的优点主要有：

① 能够更为直接地描述现实世界，如一个结点可以有多个双亲结点，结点之间可以有多种联系。

② 具有良好的性能，存取效率较高。

网状模型的缺点主要有：

① 结构比较复杂，而且随着应用环境的扩大数据库的结构变得越来越复杂，不利于最终用户掌握。

② 数据定义语言和数据操纵语言比较复杂，并且要嵌入某一种高级语言(如 COBOL、C)中，用户不容易掌握和使用。

③ 由于记录类型之间的联系是通过存取路径实现的，应用程序在访问数据时必须选择适当的存取路径，因此用户必须了解系统结构的细节，加重了编写应用程序的负担。

1.2.6 关系模型

关系模型是最重要的一种数据模型。关系数据库系统采用关系模型作为数据的组织方式。

1970 年，美国 IBM 公司 San Jose 研究室的研究员 Edgar F. Codd 首次提出了数据库系统的关系模型，开创了数据库关系方法和关系数据理论的研究，为关系数据库技术奠定了理论基础。Edgar F. Codd 也因该项杰出工作于 1981 年获得图灵奖。

20 世纪 80 年代以来, 计算机厂商新推出的数据库管理系统几乎都支持关系模型, 非关系数据库系统的产品也大都加上了关系接口。因此本书的重点将放在关系数据库上, 后面有关章节将详细介绍关系数据库。

1. 关系模型的数据结构

关系模型与以往的模型不同, 它建立在严格的数学概念基础之上。其严格的定义将在第 2 章给出, 这里仅结合实例进行简单勾画。从用户观点看, 关系模型由一组关系组成, 每个关系的数据结构是一张规范化的二维表。例如, 表 1.2 所示的学生表即为一个典型的关系模型的数据结构。下面以该表为例, 介绍关系模型中的一些术语。

表 1.2 关系模型的数据结构示例: 学生表

学号 Sno	姓名 Sname	性别 Ssex	出生日期 Sbirthdate	主修专业 Smajor
20180001	李勇	男	2000-3-8	信息安全
20180002	刘晨	女	1999-9-1	计算机科学与技术
20180003	王敏	女	2001-8-1	计算机科学与技术
20180004	张立	男	2000-1-8	计算机科学与技术
20180005	陈新奇	男	2001-11-1	信息管理与信息系统
20180006	赵明	男	2000-6-12	数据科学与大数据技术
20180007	王佳佳	女	2001-12-7	数据科学与大数据技术

① 关系 (relation): 一个关系对应通常说的一张二维表, 如表 1.2 所示。

② 元组 (tuple): 表中的一行即为一个元组。

③ 属性: 表中的一列即为一个属性, 每列的名称即为属性名。如表 1.2 中共有 5 列, 对应 5 个属性 (学号, 姓名, 性别, 出生日期, 主修专业)。

④ 码: 又称为码键或键, 是表中的某一个属性或一组属性, 其值可以唯一确定一个元组。如表 1.2 中的属性“学号”可以唯一确定一个学生, 该属性也就成为本关系的码。

⑤ 域 (domain): 在表 1.2 中, 域表示某一属性的取值范围。例如, 属性“性别”的域是 (男, 女), “主修专业”的域是学生所在学院所有专业名称的集合。

⑥ 分量 (component): 元组中的一个属性值。例如, 表 1.2 中的“李勇”即为元组 (20180001, 李勇, 男, 2000-3-8, 信息安全) 的一个分量。

⑦ 关系模式: 对关系的描述, 一般表示为: 关系名 (属性 1, 属性 2, ..., 属性 n)。例如, 表 1.2 的关系可描述为: 学生 (学号, 姓名, 性别, 出生日期, 主修专业)。

关系模型要求关系必须是规范化 (normalization) 的, 即要求关系必须满足一定的规范条件, 这些规范条件中最基本的一条就是, 关系的每一个分量必须是一个不可分的数据项。也就是说, 不允许表中还有表。例如, 表 1.3 中“联系方式”是可分的数据项, 即“联系方式”又分

为“手机号”“Email”“微信号”三个数据项，所以表 1.3 不符合关系模型要求。

表 1.3 非规范化的表示例：表中有表

学号	姓名	性别	出生日期	主修专业	联系方式		
					手机号	Email	微信号
20180001	李勇	男	2000-3-8	信息安全	18301200745	liyong@qq.com	liyong@ruc
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮

可以把关系术语和现实生活中的表格所使用的术语做一个粗略对比，如表 1.4 所示。

表 1.4 关系术语与现实生活中表格使用的术语对比

关系术语	现实生活中表格的术语
关系名	表名
关系模式	表头(表格的描述)
关系	(一张)二维表
元组	记录或行
属性	列
属性名	列名
属性值	列值
分量	一条记录中的一个列值
非规范关系	表中有表(大表中嵌有小表)

2. 关系模型的数据操纵与完整性约束

关系模型的数据操纵主要包括查询、插入、删除和更新数据，这些操作必须满足关系的完整性约束。关系的完整性约束包括实体完整性、参照完整性和用户定义的完整性三大类，其具体含义将在后续内容中介绍。

关系模型中的数据操纵是集合操作，操作对象和操作结果都是关系，即若干元组的集合；而层次模型和网状模型中的数据操纵则是单记录操作。另一方面，关系模型把存取路径向用户隐蔽起来，用户只要指出“干什么”或“找什么”，不必详细说明“怎么干”或“怎么找”，从而大大地增强了数据的独立性，提高了用户生产率。

3. 关系模型的优缺点

关系模型具有下列优点：

① 关系模型建立在严格的数学概念基础上。

② 关系模型的概念单一。无论实体还是实体之间的联系都用关系来表示，对数据的检索和更新结果也是关系，所以其数据结构简单、清晰，用户易懂易用。关系模型标准语言 SQL

简洁易用,推动了关系数据库的广泛应用。

③ 关系模型的存取路径对用户隐蔽。它具有更强的数据独立性、更好的安全保密性,也简化了程序员的工作和数据库开发建立的工作。

关系模型的上述优点使其在诞生之后发展迅速,深受用户的喜爱。

当然,关系模型也有缺点,例如,由于存取路径对用户是隐蔽的,其查询效率往往不如层次模型和网状模型。为了提高性能,关系数据库管理系统必须对用户的查询请求进行优化,因此增加了开发关系数据库管理系统的难度。不过用户不必考虑这些系统内部的优化技术细节,数据库管理系统的查询优化器会自动选择优化的查询执行计划,为用户提供较高的查询性能。

1.2.7 数据库领域中不断涌现的数据模型

数据模型是数据库系统的核心,数据模型按照数据库系统的发展进程可分为层次模型、网状模型、关系模型、面向对象数据模型(object-oriented data model)、对象关系数据模型(object relational data model)、半结构化的XML(semi-structured extensible markup language)数据模型等。

随着大数据技术的发展和数据采集手段的便利多样,产生了众多需要管理和处理的不同类型的数据,如图数据、多媒体数据、网页数据等。为了管理这些数据,满足大数据应用的实际需要,人们研究和提出了多种新型数据结构和存储结构,并称其为新型数据模型。这些新型数据模型主要有键值(key value, KV)对数据模型、文档数据模型、图数据模型、时序数据模型、时空数据模型,流数据模型、多媒体数据模型等。

需要指出的是,相比于关系模型,这些模型按照数据模型应具备的三个基本要素(数据结构、数据操纵和完整性约束)来衡量是不严格的。例如,对这些数据的完整性约束就需要进一步研究和发展。

这些新型的数据结构或数据模型将在新技术篇中介绍部分内容。

数据模型小结

以上介绍了数据模型的初步概念,在数据库系统中针对不同的应用目的和对象采用了不同的数据模型,可以按照数据建模的过程将这些模型划分为概念模型和数据模型两个层次。概念模型是按用户的观点对数据和信息建模。数据模型是按计算机系统的观点对数据建模。数据模型是对现实世界客观对象的抽象,“抽象”是十分重要的概念,希望读者初步了解数据抽象的概念和方法。

不同的数据模型实际上提供了模型化数据和信息的不同工具。它们都应尽量满足三方面要求:一是能比较真实地模拟现实世界,二是容易为人所理解,三是便于在计算机上实现。

1.3 数据库系统的三级模式结构

考察数据库系统的结构可以有多种层次或角度。从数据库管理系统角度来看,数据库系统通常采用模式、外模式和内模式三级模式结构。这是数据库系统内部的体系结构。

1.3.1 数据库系统中模式的概念

在数据模型中有“类型”(或称型)和“值”的概念。型是指对某一类数据的结构和属性的说明,值是型的一个具体赋值。例如,学生(学号,姓名,性别,出生日期,主修专业)表示“学生”记录类型,而(20180003,王敏,女,2001-8-1,计算机科学与技术)则是该记录类型的一个记录值。

模式(schema)也称逻辑模式,是数据库中全体数据的逻辑结构和特征的描述,它仅仅涉及型的描述,不涉及具体的值。模式的一个具体值称为模式的一个实例(instance)。同一个模式可以有很多实例。

例如,在“学生选课”数据库模式中包含“学生”记录、“课程”记录和“学生选课”记录。现有一个具体的“学生选课”数据库实例,该实例包含了2020年学校中所有学生的记录(如果某校有10 000个学生,则有10 000条学生记录)、学校开设的所有课程的记录和所有学生选课的记录。

“2019年学生选课”数据库实例与“2020年学生选课”数据库实例是不同的。实际上“2020年学生选课”数据库实例也会随时间变化,因为在该年度可能会有学生退学或转专业等情况发生。不同时刻“学生选课”数据库实例是不同的和变化的,不变的是“学生选课”数据库模式。

模式是相对稳定的,而实例是相对变动的,因为数据库中的数据是在不断更新的。模式反映的是数据的结构及其联系,而实例反映的是数据库某一时刻的状态。

1.3.2 数据库系统的三级模式结构

虽然实际的数据库管理系统产品种类很多,它们支持不同的数据模型,使用不同的数据库语言,建立在不同的操作系统之上,数据的存储结构也各不相同,但它们在体系结构上通常都具有相同的特征,即采用三级模式结构并提供两级映像功能。

数据库系统的三级模式结构是指数据库系统是由模式、外模式和内模式三级构成,如图1.15所示。

1. 模式

模式是所有用户的公共数据视图。它是数据库系统模式结构的中间层,不涉及数据的物理存储细节和硬件环境,且与具体的应用程序、所使用的应用开发工具及高级程序设计语言无关。

模式实际上是数据库数据在逻辑级上的视图。一个数据库对应一个模式。数据库模式以某一种数据模型为基础,综合考虑了所有用户的需求,并将这些需求有机地结合成一个逻辑整体。定义模式时不仅要定义数据的逻辑结构,例如数据记录由哪些数据项构成,数据项的名称、类型、取值范围等,而且要定义数据之间的联系,以及与数据有关的安全性、完整性要求。

数据库管理系统提供模式数据定义语言来严格地定义模式。



图 1.15 数据库系统的三级模式结构

2. 外模式 (external schema)

外模式也称子模式 (subschema) 或用户模式, 它是数据库用户 (包括应用程序员和最终用户) 能够看见和使用的局部数据的逻辑结构和特征的描述。外模式是数据库用户的数据视图, 是与某一应用有关的数据的逻辑表示。

一个数据库可以有多个外模式。由于它是各个用户的数据视图, 如果不同的用户在应用需求、看待数据的方式、对数据保密的要求等方面存在差异, 则其外模式描述就是不同的。即使模式中的同一数据, 在不同的外模式中的结构、类型、长度、保密级别等都可能不同。此外, 同一外模式也可以为某一用户的多个应用系统所使用, 但一个应用程序只能使用一个外模式。

外模式是保证数据库安全性的一个有力措施。每个用户只能看见和访问所对应的外模式中的数据, 数据库中的其余数据是不可见的。

数据库管理系统提供外模式数据定义语言来严格地定义外模式。

3. 内模式 (internal schema)

内模式也称物理模式 (physical schema) 或存储模式 (storage schema), 一个数据库只有一个内模式。它是对数据物理结构和存储方式的描述, 是数据在数据库内部的组织方式。例如, 记录采用哪种方式存储, 是堆存储还是按照某个 (些) 属性值的升 (降) 序存储, 或按照属性值聚簇存储; 索引采用什么方式组织, 是 $B+$ 树索引还是哈希索引; 数据是否压缩存储, 是否加密; 数据的存储记录结构有何规定, 如采用定长结构或变长结构、一个记录不能跨物理页存储, 等等。

1.3.3 数据库的两级映像与数据独立性

数据库系统的三级模式是数据的三个抽象级别, 它把数据的具体组织留给数据库管理系统

管理,使用户能逻辑、抽象地处理数据,而不必关心数据在计算机中的具体表示方式与存储方式。为了能够在系统内部实现这三个抽象层次的联系和转换,数据库管理系统在这三级模式之间提供了两级映像:外模式/模式映像和模式/内模式映像。

正是这两级映像保证了数据库系统中的数据能够具有较强的逻辑独立性和物理独立性。

1. 外模式/模式映像

前已提及,模式描述的是数据的全局逻辑结构,外模式描述的是数据的局部逻辑结构。对于同一个模式,可以有任意多个外模式。对于每一个外模式,数据库系统都有一个外模式/模式映像来定义该外模式与模式之间的对应关系。这些映像定义通常包含在各自外模式的描述中。

当模式改变时(如增加新的关系、新的属性,改变属性的数据类型等),由数据库管理员对各个外模式/模式的映像做相应改变,可以使外模式保持不变。应用程序是依据数据的外模式编写的,因而应用程序不必修改,保证了数据与程序的逻辑独立性,简称数据的逻辑独立性。

2. 模式/内模式映像

数据库只有一个模式,也只有一个内模式,所以模式/内模式映像是唯一的,它定义了数据全局逻辑结构与存储结构之间的对应关系。例如,说明逻辑记录和字段在内部是如何表示的。该映像定义通常包含在模式描述中。当数据库的存储结构改变时(如选用了另一种存储结构),由数据库管理员对模式/内模式映像做相应改变,可以使模式保持不变,因而应用程序也不必改变,保证了数据与程序的物理独立性,简称数据的物理独立性。

数据库系统的三级模式结构小结

在数据库的三级模式结构,其中,模式(即全局逻辑结构)是数据库的核心与关键,它独立于数据库的其他层次。因此设计数据库模式结构时应首先确定数据库的逻辑模式。

内模式依赖于数据库的全局逻辑结构,但独立于数据库的用户视图(即外模式),也独立于具体的存储设备。它将全局逻辑结构中所定义的数据结构及其联系按照一定的物理存储策略进行组织,以达到较好的时间与空间效率。

外模式面向具体的应用程序,它定义在逻辑模式之上,但独立于存储模式和存储设备。当应用需求发生较大变化,相应的外模式不能满足其视图要求时,该外模式就得做相应改动。所以设计外模式时应充分考虑到应用的扩充性。

特定的应用程序是在外模式描述的数据结构上编制的,它依赖于特定的外模式,与数据库的模式和存储结构独立。不同的应用程序有时可以共用同一个外模式。数据库的两级映像保证了数据库外模式的稳定性,从而从底层保证了应用程序的稳定性,除非应用需求本身发生变化,否则应用程序一般不需修改。数据与程序之间的独立性,使得数据的定义和描述可以从应用程序中分离出去。另外,数据的组织和存取交由数据库管理系统负责,简化了应用程序的编制,大大减少了应用程序的开发和维护成本。

1.4 数据库系统的组成

在 1.1 节中已经讲到,数据库系统是指引入数据库后的计算机系统,一般由数据库、数据库管理系统(及其应用开发工具)、应用系统和数据库管理员组成(参见图 1.1)。本节从支撑数据库系统的硬件平台、软件平台及人员三方面对数据库系统的组成进行介绍。

1. 硬件平台

数据库存放在计算机存储设备中。从数据库技术的发展来看,硬件平台中存储器与处理器技术的升级推动了数据库技术从磁盘数据库到内存数据库的技术升级。特别是随着大数据应用的日益广泛,海量存储设备与高速处理器的硬件特性成为新型数据库存储引擎和查询引擎设计的重要因素。

2. 软件平台

支撑数据库系统的软件平台主要包括操作系统、数据库管理系统、开发应用系统的高级语言及其编译系统、各种各样的应用开发工具,以及为特定应用背景开发的数据库应用系统等。它们共同为数据库系统的开发和应用提供了良好的软件生态环境。

3. 人员

开发、管理和使用数据库系统的人员主要包括数据库管理员、系统分析员和数据库设计员、应用程序员和最终用户。不同角色的人员各司其职,涉及不同的数据抽象级别,具有不同的数据视图,如图 1.16 所示。

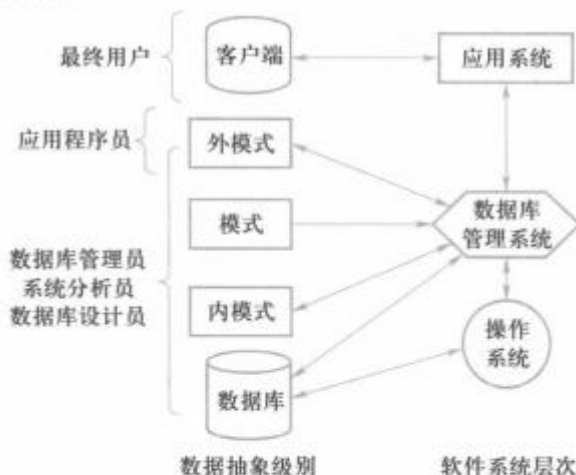


图 1.16 数据库系统各种人员的数据视图

(1) 数据库管理员 (DBA)

其主要职责包括:

① 设计与定义数据库。数据库管理员必须参与数据库设计的全过程,与最终用户、应用

程序员、系统分析员密切结合,设计概念模式、数据库模式以及各个应用的外模式;还要熟悉所使用的数据库管理系统产品,决定数据库的存储结构和存取策略,设计数据库的内模式。

② 帮助最终用户使用数据库系统。数据库管理员要担负起培训最终用户的责任,并负责解答最终用户日常使用数据库系统时遇到的问题。

③ 负责数据库系统的运维工作。数据库管理员负责监视数据库系统的运行情况,及时处理运行过程中出现的问题,控制不同用户访问数据库的权限,收集数据库的审计信息,保证数据库的安全性和完整性。

④ 改进和重组数据库系统,调优数据库系统的性能。数据库管理员负责监视、分析数据库系统的性能,包括空间利用率和处理效率。虽然在系统设计时已经充分考虑了性能要求,但性能的好坏只能通过实际运行结果来检验,所以数据库管理员必须对运行状况进行记录、统计和分析,并根据实际应用环境不断改进数据库设计,例如根据实际情况修改某些系统参数和环境变量值来改善系统性能。另一方面,数据库运行过程中会不断地插入、删除、修改数据,这样在一段时间后必然会影响数据的物理布局,导致系统性能下降,因此数据库管理员要定期或按一定的策略对数据库进行重组。

⑤ 转储与恢复数据库。为了减少硬件、软件或人为故障对数据库系统的破坏,数据库管理员必须定义和实施适当的后援和恢复策略。例如,周期性地转储数据、维护日志文件等。一旦系统发生故障,数据库管理员必须能够在最短时间内把数据库恢复到某一正确状态,并且尽可能不影响或少影响计算机系统其他部分的正常运行。

⑥ 重构数据库。当用户的应用需求增加或改变时,数据库管理员需要对数据库进行较大的改造,包括修改内模式或模式,即重新构造数据库。

(2) 系统分析员和数据库设计员

其主要职责为应用系统的需求分析与规范说明,进行总体设计。因此他们必须与用户及数据库管理员相结合,进行数据库各级模式的设计,确定系统的软硬件配置。

(3) 应用程序员

其主要职责为以外模式为基础开发应用系统,编制具体的应用程序。数据库的两级映像保证了他们不必考虑数据的存储细节。

(4) 最终用户

其主要职责为具体操作应用系统,通过应用系统客户端的用户界面使用数据库以完成业务活动。

* 1.5 数据库系统的体系结构

根据计算机的系统结构,从数据库最终用户角度来看,数据库系统可分为集中式数据库系统、客户-服务器(浏览器/应用服务器/数据库服务器)数据库系统、并行数据库系统、分布式数据库系统和云计算环境下的数据库系统(云数据库系统)等。这是数据库系统外部的体系

结构。

1. 集中式数据库系统

集中式数据库系统的数据库管理系统、数据库和应用程序都在一台计算机上。在小型机和大型机上的集中式数据库系统一般是多用户系统,即多个用户通过各自的终端运行不同的应用系统,共享数据库。微型计算机上的数据库系统一般是单用户的。

2. 客户-服务器数据库系统

在客户-服务器数据库系统中,数据库管理系统、数据库驻留在服务器上,而应用程序放置在客户机上(微型计算机或工作站),客户机和服务器通过网络进行通信。在这种结构中,客户机负责提供业务数据处理流程和应用程序界面,当要存取数据库中的数据时就向服务器发出请求,服务器接收客户机的请求后进行处理,并将客户要求的数据返回给客户机。

随着互联网技术的应用,客户-服务器两层结构已经发展为三层或多层结构。三层结构一般是指浏览器/应用服务器/数据库服务器结构。用户界面采用统一的浏览器方式,应用服务器上安装应用系统或应用模块,数据库服务器上安装数据库管理系统和数据库。两层或三层结构对数据库管理系统的功能进行了合理的分配,减轻了数据库服务器的负担,从而使服务器有更多的能力完成事务处理和数据访问控制,支持更多的用户,提高系统的性能。

3. 并行数据库系统

并行数据库系统是在并行计算机上运行的具有并行处理能力的数据库系统,是数据库技术与并行计算技术相结合的产物。并行计算机系统有共享内存型、共享磁盘型、非共享型以及混合型等。并行计算技术利用多处理机并行处理产生的规模效益来提高系统的整体性能。并行数据库系统发挥了多处理机的优势,采用并行查询处理技术和并行数据分布与管理技术,具有高性能、高可用性、高扩展性等优点。

4. 分布式数据库系统

分布式数据库系统是指数据库中的数据在逻辑上是一个整体,但物理地分布在计算机网络的不同节点上。网络中的每个节点独立处理本地数据库中的数据(称为场地自治),执行局部应用;也可以执行全局应用,即通过网络通信系统同时存取和处理多个节点上数据库的数据。

分布式数据库系统适用于企业部门分布的组织结构,可以降低费用,提高系统的可靠性和可用性,具有良好的可扩展性。

5. 云数据库系统

近年来大数据应用飞速发展,数据量激增,用户对性能的要求越来越高,并发用户数在峰值和低谷时差距明显,这使得企业自己运营和维护数据库的成本越来越高。如果按峰值配置设备,平时会造成很大的浪费,但如果按低谷值配置设备,又会出现无法应对峰值的情形。于是伴随云计算(cloud computing)技术的发展,云数据库系统应运而生。

云数据库系统把数据库部署或虚拟化在云计算环境下,通过计算机网络以服务的形式提供数据库的功能,包括数据存储、数据更新、查询处理、事务管理等。早期的云数据库主要运行在单个节点上,该节点有多处理器、大内存和大磁盘容量。现在的云数据库是运行在集群上的

并行数据库系统，能够较好地进行动态伸缩、按需分配计算资源和存储资源。但是，云数据库存储的安全可信、隐私保护等问题不可忽视，亟待研究解决。

本章小结

本章概述了数据库的基本概念，并通过对数据管理技术进展情况的介绍阐述了数据库技术产生和发展的背景，说明了数据库系统的优点。

数据模型是数据库系统的核心和基础。本章简要介绍了概念模型、组成数据模型的三要素（数据结构、数据操纵、完整性约束）和三种数据模型（层次模型、网状模型、关系模型）。

本章介绍了数据库系统的三级模式和两级映像，这样的系统架构保证了数据库系统能够具有较强的逻辑独立性和物理独立性。

本章还介绍了数据库系统的组成，使读者了解数据库系统不仅是一个计算机系统，而且是一个人机系统（man-machine system）。最后根据计算机的系统结构，本章简要介绍了集中式数据库系统、客户-服务器（浏览器/应用服务器/数据库服务器）数据库系统、并行数据库系统、分布式数据库系统和云数据库系统等。

习题 1

1. 试述数据、数据库、数据库管理系统、数据库系统的概念。
2. 试述文件系统与数据库系统之间的区别和联系。
3. 分别举出适合用文件系统的应用例子，以及适合用数据库系统的应用例子。
4. 试述数据库系统的特点。
5. 数据库管理系统的主要功能有哪些？
6. 什么是概念模型？试述概念模型的作用。
7. 定义并解释概念模型中以下术语：
实体，实体型，实体集，实体之间的联系
8. 试述数据模型的概念、作用及其包含的三个要素。
9. 试述层次模型的概念，举出三个层次模型的实例。
10. 试述网状模型的概念，举出三个网状模型的实例。
11. 试述网状数据库、层次数据库的优缺点。
12. 试述关系模型的概念，定义并解释以下术语：
关系，属性，域，元组，码，分量，关系模式
13. 试述关系模型的优缺点。
14. 试述数据库系统的三级模式结构，并说明这种结构的优点是什么。
15. 试述数据与程序的物理独立性和逻辑独立性。为什么数据库系统具有较强的数据与程序的独立性？
16. 试述数据库系统的组成。

第1章实验 实验准备

实验准备主要包括关系数据库管理系统选择、实验环境配置和实验数据准备等工作。

推荐使用国产的金仓数据库管理系统,例如金仓数据库 KingbaseES。也可以使用任何一种关系数据库管理系统产品或开源数据库管理系统。

建议学生亲自安装选用的关系数据库管理系统产品,观察数据库安装过程,记录和理解安装配置参数,熟悉数据库管理系统的客户端图形管理工具。

我们在《数据库系统概论(第6版)习题解析与实验指导》(以下简称《概论辅导书》)中详细介绍了各章实验内容与实验目的,并提供有实验报告样例等,供读者上机实验参考。

参考文献 1

[1] BRODIE M L, SCHMIDT J W. Final report of the ANSI/X3/SPARC DBS-SG relational database task group [J]. ACM SIGMOD Record, 1982, 12(4):1-62.

[2] ANSI/X3/SPARC Study Group on Data Base Management Systems[R]. Interim Report, FDT-ACM SIGMOD bulletin, 1975, 7(2).

文献[1][2]介绍了 ANSI/X3/SPARC 及数据库系统三级模式结构。

[3] 萨师煊,王珊.数据库系统概论[M].2版.北京:高等教育出版社,1991.

文献[3]中第2、3章比较详细地介绍了层次数据库系统和网状数据库系统的基本概念和一般原理。

[4] BACHMAN C W, WILLIAMS S B. A general purpose programming system for random access memories[C]. Proceedings of the Fall Joint Computer Conference, AFIPS'64, 1964: 411-422.

文献[4]描述的是 Charles Bachman 在第一个商业数据库管理系统的开发期间进行的网状数据模型早期研究工作。

[5] BACHMAN C W. Data structure diagrams[J]. ACM SIGMIS Database: the DATABASE for Advances in Information Systems, 1969, 1(2): 4-10.

文献[5]最早提出了用数据结构简图表示数据之间联系的思想。

[6] BACHMAN C W. The Programmer as Navigator[J]. CACM-Communications of the ACM, 1973, 16(11): 635-638.

Bachman 的成就使他于 1973 年荣获了 ACM 最高荣誉——图灵奖。他在图灵奖演说中把数据库看作基本资源,把程序设计者看作数据库中的领航员。

[7] BACHMAN C W. The data structure set model[C]. Proceedings of the 1974 ACM SIGMOD Workshop on Data Description, Access and Control: Data Structure Set Versus Relation, 1974, 2.

在 1974 年支持和反对关系模型研究的讨论中, Bachman 持反对态度。

[8] Data Base Task Group Report to the CODASYL Programming Language Committee[J], ACM SIGMIS Database: the DATABASE for Advances in Information Systems, 1971, 2(2).

CODASYL 的数据库任务组(DBTG)对网状数据库系统进行了系统研究,在本报告中提出了模式数据定义语言、子模式数据定义语言和嵌入 COBOL 的数据操纵语言,以后又发表了多篇报告。

本章知识点讲解微视频:



数据库发展概述



数据库的4个基本概念



数据建模与数据模型



数据库的三级模式结构

关系模型应用数学方法来处理数据库中的数据。1962年 CODASYL 发表的“信息代数”最早提出将这类方法用于数据处理,1968年 David Child 在 IBM 7090 机上实现了集合论数据结构,但系统、严格地提出关系模型的是 Edgar F. Codd。

1970年,Edgar F. Codd 在美国计算机学会 (ACM) 会刊上发表了题为 *A Relational Model of Data for Large Shared Data Banks* 的论文,开创了数据库系统的新纪元。1983年,该论文被 ACM 列为 1958 年以来的 1/4 世纪中具有里程碑意义的 25 篇研究论文之一。此后,Edgar F. Codd 又陆续发表了多篇论文,奠定了关系数据库的理论基础。

20 世纪 70 年代末,关系方法的理论研究和软件系统的研制紧密结合,均取得了丰硕的成果。例如,IBM 公司的 San Jose 实验室在 IBM 370 系列机上研制的关系数据库实验系统 System R 历时 6 年获得成功。1981 年,IBM 公司宣布具有 System R 全部特征的新的数据库软件产品 SQL/DS 问世。

与 System R 同期,1973 年美国加州大学伯克利分校的 Michael Stonebraker 和 Eugene Wong 研制了 INGRES 关系数据库实验系统,并由 INGRES 公司发展成为 INGRES 数据库产品。在 INGRES 的基础上,20 世纪 80 年代中期 PostgreSQL 系统发布并成功开放其源代码。

在此期间 Oracle 公司成立,1978 年发布了 Oracle 1.0,以后短短十几年中 Oracle 产品不断成熟,Oracle 公司也逐渐发展成为著名的数据库软件及服务供应商。

此后数据库领域又涌现出许多优秀的关系数据库产品,关系数据库系统逐渐从实验室走入社会,得到了广泛的应用和快速的发展。

本章导读

本章基于第一章对关系模型及其基本术语的介绍,进一步深入讲解关系模型的有关内容。

本章按照数据模型的三要素,介绍关系模型的数据结构及形式化定义、关系操作和关系数据语言的大致分类、关系的三类完整性约束。希望读者能牢固掌握关系模型三个组成部分的内涵,为学习 SQL 和数据库安全性与完整性打下基础。

本章还讲解了关系代数和关系演算。关系代数用对关系的运算来表达查询要求,关系代数

表达式是关系数据库管理系统查询优化中代数优化的依据,读者应掌握关系代数表达查询的方法。关系演算是以数理逻辑中的谓词演算为基础,用谓词来表达查询要求。本章讲解了元组关系演算语言 ALPHA 和域关系演算语言 QBE,希望读者基本掌握关系演算语言的概念和使用方法,并通过习题加强练习。

2.1 关系模型的数据结构及形式化定义

本书第1章1.2.6小节非形式化地介绍了关系模型的基本概念。由于关系模型是建立在集合代数基础上的,本节从集合论角度给出关系数据结构的形式化定义,以及关系模式的形式化表示,并简要介绍关系数据库和关系模型的存储结构。

2.1.1 关系

关系模型的数据结构非常简单,只包含单一的数据结构——关系。在用户看来,关系模型中数据的逻辑结构是一张二维表。

关系模型的数据结构虽然简单却能够表达丰富的语义,描述现实世界的实体以及实体间的各种联系。也就是说,在关系模型中,现实世界的实体以及实体间的各种联系均用单一的结构类型,即关系来表示。

1. 域

定义 2.1 域是一组具有相同数据类型的值的集合。

例如,自然数、整数、实数、长度小于 25 B 的变长字符串集合、 $\{0,1\}$ 、 $\{\text{男},\text{女}\}$ 、 $0\sim 100$ 之间的正整数等,都可以是域。

2. 笛卡儿积

笛卡儿积(Cartesian product)是域上的一种集合运算。

定义 2.2 给定一组域 $D_1, D_2, \dots, D_n$, 允许其中某些域是相同的, $D_1, D_2, \dots, D_n$ 的笛卡儿积为

$$D_1 \times D_2 \times \dots \times D_n = \{(d_1, d_2, \dots, d_n) \mid d_i \in D_i, i=1, 2, \dots, n\}$$

其中,每一个元素 $(d_1, d_2, \dots, d_n)$ 叫作一个 n 元组(n -tuple),简称元组。元组中的每一个值 d_i 叫作一个分量。

一个域允许的不同取值个数称为这个域的基数(cardinal number)。

若 $D_i (i=1, 2, \dots, n)$ 为有限集,其基数为 $m_i (i=1, 2, \dots, n)$, 则 $D_1 \times D_2 \times \dots \times D_n$ 的基数 M 为

$$M = \prod_{i=1}^n m_i$$

笛卡儿积可表示为一张二维表。表中的每一行对应一个元组,表中的每一列的值来自一个域。例如,给出三个域:

$$D_1 = \text{导师 (SUPERVISOR)} = \{\text{张清玫, 刘逸}\}$$

$$D_2 = \text{专业 (MAJOR)} = \{\text{计算机科学与技术, 信息管理与信息系统}\}$$

$$D_3 = \text{研究生 (POSTGRADUATE)} = \{\text{李勇, 刘晨, 王敏}\}$$

则 D_1 、 D_2 、 D_3 的笛卡儿积如下:

$$D_1 \times D_2 \times D_3 = \{(\text{张清玫, 计算机科学与技术, 李勇}), (\text{张清玫, 计算机科学与技术, 刘晨}),$$

$$(\text{张清玫, 计算机科学与技术, 王敏}), (\text{张清玫, 信息管理与信息系统, 李勇}),$$

$$(\text{张清玫, 信息管理与信息系统, 刘晨}), (\text{张清玫, 信息管理与信息系统, 王敏}),$$

$$(\text{刘逸, 计算机科学与技术, 李勇}), (\text{刘逸, 计算机科学与技术, 刘晨}),$$

$$(\text{刘逸, 计算机科学与技术, 王敏}), (\text{刘逸, 信息管理与信息系统, 李勇}),$$

$$(\text{刘逸, 信息管理与信息系统, 刘晨}), (\text{刘逸, 信息管理与信息系统, 王敏})\}$$

其中,“(张清玫, 计算机科学与技术, 李勇)”“(张清玫, 计算机科学与技术, 刘晨)”等都是元组,“张清玫”“计算机科学与技术”“李勇”“刘晨”等都是元组的分量。

该笛卡儿积的基数为 $2 \times 2 \times 3 = 12$ 。也就是说, $D_1 \times D_2 \times D_3$ 共有 $2 \times 2 \times 3 = 12$ 个元组。这 12 个元组可列成一张二维表, 如表 2.1 所示。

表 2.1 笛卡儿积示例

导师 SUPERVISOR	专业 MAJOR	研究生 POSTGRADUATE
张清玫	计算机科学与技术	李勇
张清玫	计算机科学与技术	刘晨
张清玫	计算机科学与技术	王敏
张清玫	信息管理与信息系统	李勇
张清玫	信息管理与信息系统	刘晨
张清玫	信息管理与信息系统	王敏
刘逸	计算机科学与技术	李勇
刘逸	计算机科学与技术	刘晨
刘逸	计算机科学与技术	王敏
刘逸	信息管理与信息系统	李勇
刘逸	信息管理与信息系统	刘晨
刘逸	信息管理与信息系统	王敏

3. 关系

一般来说, 在关系模型中 $D_1, D_2, \dots, D_n$ 的笛卡儿积是没有实际语义的, 只有它的某个真子集才有实际含义。

例如,可以发现表 2.1 的笛卡儿积中许多元组是没有意义的。因为在学校中一个专业方向有多个导师,而一个导师只在一个专业方向带研究生;一个导师可以带多个研究生,而一个研究生只有一个导师,学习某一个专业。因此,表 2.1 中的一个子集才是有意义的,可以表示导师与研究生的关系。把该关系取名为 SMP,如表 2.2 所示,李勇和刘晨是计算机科学与技术专业张清玫老师的研究生,王敏是信息管理与信息系统专业刘逸老师的研究生。

表 2.2 导师-研究生关系 SMP

导师 SUPERVISOR	专业 MAJOR	研究生 POSTGRADUATE
张清玫	计算机科学与技术	李勇
张清玫	计算机科学与技术	刘晨
刘逸	信息管理与信息系统	王敏

关系 SMP 包含的属性为 SUPERVISOR、MAJOR 和 POSTGRADUATE,则其关系模式可以表示为

SMP(SUPERVISOR, MAJOR, POSTGRADUATE)

下面给出关系及关系模式的形式化定义和相关概念。

定义 2.3 给定一组域 $D_1, D_2, \dots, D_n$, 允许其中某些域是相同的, $D_1, D_2, \dots, D_n$ 的笛卡儿积 $D_1 \times D_2 \times \dots \times D_n$ 的子集称为这组域上的关系, 表示为

$$R(D_1, D_2, \dots, D_n)$$

这里 R 表示关系名, n 是关系的目或度(degree)。

当 $n=1$ 时, 称该关系为一元关系(unary relation)或单元关系、单目关系。

当 $n=2$ 时, 称该关系为二元关系(binary relation)或二目关系。

关系中的每个元素是关系中的元组, 通常用 t 表示。

关系是笛卡儿积的有限子集, 所以关系是一张二维表, 表的每一行对应一个元组, 表的每一列对应一个域。由于域可以相同, 为了加以区分, 必须对每一列起一个名字, 称为属性。 n 目关系必有 n 个属性。

例如, SMP(SUPERVISOR, MAJOR, POSTGRADUATE)有三个属性, 是一个三目关系。

(张清玫, 计算机科学与技术, 李勇)、(张清玫, 计算机科学与技术, 刘晨)和(刘逸, 信息管理与信息系统, 王敏)是 SMP 关系的三个元组。

关系可以有三种类型: 基本关系(通常又称为基本表或基表)、查询结果和视图。其中, 基本关系是实际存在的表, 它是实际存储数据的逻辑表示; 查询结果是查询执行产生的结果对应的临时表; 视图是由基本表或其他视图导出的虚表, 不存储实际数据。

按照定义 2.2, 关系可以是一个无限集合。例如, 某个域是整数, 而整数是个无限集合。此外, 在数学上组成笛卡儿积的域不满足交换律, 所以按照数学定义, $(d_1, d_2, \dots, d_n) \neq (d_2,$

$d_1, \dots, d_n$)。因此,当关系作为数据模型的数据结构时,需要给予如下的限定和扩充。

① 无限关系在数据库系统中是无意义的。因此,限定关系模型中的关系必须是有限集合。

② 通过为关系的每个列附加一个属性名的方法取消关系属性的有序性,使得列的次序可以任意交换:

$$(d_1, d_2, \dots, d_i, d_j, \dots, d_n) = (d_1, d_2, \dots, d_j, d_i, \dots, d_n) \quad (i, j = 1, 2, \dots, n)$$

因此,基本关系具有以下 6 条性质。

① 列是同质的(homogeneous),即每一列中的分量是同一类型的数据,来自同一个域。

② 不同的列可出自同一个域,称其中的每一列为一个属性,不同的属性要给予不同的属性名。例如,在上面的例子中,也可以只给出两个域:

人(PERSON) = {张清玫,刘逸,李勇,刘晨,王敏}

专业(MAJOR) = {计算机科学与技术,信息管理与信息系统}

SMP 关系的导师属性和研究生属性都是从 PERSON 域中取值。为了避免混淆,必须给这两个属性取不同的属性名,而不能直接使用域名。因此,定义导师属性名为 SUPERVISOR,研究生属性名为 POSTGRADUATE。

③ 列的顺序无所谓,即列的次序可以任意交换。由于列顺序是无关紧要的,因此在许多实际的关系数据库产品中增加新属性时,永远是将其插至最后一列。

④ 任意两个元组的码不能取相同的值。

⑤ 行的顺序无所谓,即行的次序可以任意交换。

⑥ 分量必须取原子值,即每一个分量都必须是不可分的数据项。

关系模型要求关系必须满足一定的规范条件,其中最基本的一条就是元组的每一个分量必须是一个不可分的数据项。仍以导师-研究生关系为例,表 2.3 虽然很好地表达了两之间的一对多联系,即一个导师可以指导多个研究生,但是由于属性“POSTGRADUATE”中分量取了两个值“PG1”“PG2”,不符合规范化的要求,因此这样的关系在关系数据库中是不允许的。直观地描述,表 2.3 中还有一个小表,不符合规范化的条件。

表 2.3 非规范化关系

SUPERVISOR	MAJOR	POSTGRADUATE	
		PG1	PG2
张清玫	计算机科学与技术	李勇	刘晨
刘逸	信息管理与信息系统	王敏	

小表

规范化的关系简称为范式(normal form, NF),将在第 6 章关系数据理论中做进一步讲解。

2.1.2 关系模式

第 1 章 1.3.1 小节已经简单介绍了数据库系统中模式的概念。强调了在数据库中要区分数

据模型的型和值。在关系模型中,关系是元组的集合,关系是值;关系模式是对关系的描述,关系模式是型。

那么关系模式需要描述关系的哪些方面呢?

首先,关系模式必须描述关系元组集合的结构,即它由哪些属性构成,这些属性来自哪些域,以及属性与域之间的映像关系。

其次,关系模式要描述关系的完整性约束。现实世界随着时间在不断地变化,因而在不同的时刻关系模式所描述的关系也会不断变化。例如,在“学生”关系中,2020年某校的在校生与2019年就不同。但是,现实世界的许多已有事实和规则限定了关系模式所有可能的关系必须满足一定的完整性约束条件。这些约束条件或者通过对属性取值范围进行限定来描述,或者通过属性与属性之间的相互关联和相互约束反映出来。

定义 2.4 关系的描述称为关系模式(relation schema)。它可以形式化地表示为

$$R(U, D, \text{DOM}, F)$$

其中 R 为关系名, U 为组成该关系的属性的属性名集合, D 为 U 中属性所来自的域, DOM 为属性向域的映像集合, F 为属性间数据依赖关系的集合。

属性间的数据依赖将在第6章讨论,本章中关系模式仅涉及关系名、属性名、域名、属性向域的映像4部分,即 $R(U, D, \text{DOM})$ 。

例如,在上面例子中,由于导师和研究生出自同一个域——人:

$$\text{DOM}(\text{SUPERVISOR}) = \text{DOM}(\text{POSTGRADUATE}) = \text{PERSON}$$

所以要取不同的属性名,并在模式中定义属性向域的映像,即说明它们分别出自哪个域。

关系模式通常简记为

$$R(U)$$

或

$$R(A_1, A_2, \dots, A_n)$$

其中 R 为关系名, $A_1, A_2, \dots, A_n$ 为属性名;而域名及属性向域的映像则常直接说明为属性的类型、长度。

若关系模式中的某一个属性或一组属性的值能唯一地标识一个元组,而它的真子集不能唯一地标识一个元组,则称该属性或属性组为候选码(candidate key)。

若一个关系有多个候选码,则选定其中一个为主码(primary key),或称主键。

例如,在导师-研究生关系 $\text{SMP}(\text{SUPERVISOR}, \text{MAJOR}, \text{POSTGRADUATE})$ 中,假设研究生不会重名(这在实际生活中可能做不到,这里只是为了举例方便),则 POSTGRADUATE 属性的每一个值都唯一地标识了一个元组,因此可以作为 SMP 关系的主码,用在其名称下加下划线表示。

候选码的诸属性称为主属性(prime attribute)。不包含在任何候选码中的属性称为非主属性(non prime attribute)或非码属性(non-key attribute)。

在最简单的情况下, 候选码只包含一个属性。在最极端的情况下, 关系模式的所有属性是这个关系模式的候选码, 称为全码(all-key)。

关系是关系模式在某一时刻的状态或内容。关系模式是静态的、稳定的, 而关系是动态的、随时间不断变化的, 这是因为关系操作在不断地更新着数据库中的数据。例如, “学生”关系模式在不同的学年是相同的, 而“学生”关系是不同的。在实际工作中, 人们常常把关系模式和关系都笼统地称为关系, 这可以根据上下文加以区别, 希望读者注意。

2.1.3 关系数据库

支持关系模型的数据库系统称为关系数据库系统。在关系模型中, 实体以及实体间的联系都是用关系来表示的。例如, “学生”实体、“课程”实体、学生与课程之间选修课程的多对多联系都可以分别用一个关系模式来描述。

“学生”关系模式: Student(Sno, Sname, Ssex, Sbirthdate, Smajor), 包括学号、姓名、性别、出生日期和主修专业等属性。

“课程”关系模式: Course(Cno, Cname, Ccredit, Cpno), 包括课程号、课程名、学分、先修课(直接先修课)等属性。

“学生选课”关系模式: SC(Sno, Cno, Grade, Semester, Teachingclass), 包括学号、课程号、成绩、开课学期、教学班等属性。

在一个关系数据库中, 某一时刻所有关系模式对应的关系的集合构成一个关系数据库。例如如图 2.1 就是一个“学生选课”数据库示例, 该数据库包含 3 个关系。

关系数据库也有类型和值之分。关系数据库的类型就是关系数据库中所有关系模式的集合, 是对关系数据库的描述, 通常称为关系数据库模式。关系数据库的值是这些关系模式在某一时刻对应的关系的集合, 通常称为关系数据库。

2.1.4 关系模型的存储结构

关系模型是关系数据的逻辑结构, 用关系数据定义语言描述, 例如第 3 章将介绍的关系数据库标准语言 SQL。

支持关系模型的关系数据库管理系统(relational database management system, RDBMS)将以一定的组织方式存储和管理数据, 即设计和实现关系模型的存储结构, 这是关系数据库管理系统的重要职责之一。

例如, 有的关系数据库管理系统中一个表对应一个操作系统文件, 将物理数据组织的许多任务交给操作系统完成; 有的关系数据库管理系统则从操作系统那里申请若干个大的文件, 自己划分文件空间, 组织表、索引等存储结构并进行存储管理。

本书第 9 章将专门讲解关系数据库存储管理, 重点介绍基于磁盘的数据库的组织与存储。

Student

学号 Sno	姓名 Sname	性别 Ssex	出生日期 Sbirthdate	主修专业 Smajor
20180001	李勇	男	2000-3-8	信息安全
20180002	刘晨	女	1999-9-1	计算机科学与技术
20180003	王敏	女	2001-8-1	计算机科学与技术
20180004	张立	男	2000-1-8	计算机科学与技术
20180005	陈新奇	男	2001-11-1	信息管理与信息系统
20180006	赵明	男	2000-6-12	数据科学与大数据技术
20180007	王佳佳	女	2001-12-7	数据科学与大数据技术

(a)

Course

课程号 Cno	课程名 Cname	学分 Ccredit	先修课 Cpno
81001	程序设计基础与C语言	4	
81002	数据结构	4	81001
81003	数据库系统概论	4	81002
81004	信息系统概论	4	81003
81005	操作系统	4	81001
81006	Python语言	3	81002
81007	离散数学	4	
81008	大数据技术概论	4	81003

(b)

SC

学号 Sno	课程号 Cno	成绩 Grade	开课学期 Semester	教学班 Teachingclass
20180001	81001	85	20192	81001-01
20180001	81002	96	20201	81002-01
20180001	81003	87	20202	81003-01
20180002	81001	80	20192	81001-02
20180002	81002	98	20201	81002-01
20180002	81003	71	20202	81003-02
20180003	81001	81	20192	81001-01
20180003	81002	76	20201	81002-02
20180004	81001	56	20192	81001-02
20180004	81002	97	20201	81002-02
20180005	81003	68	20202	81003-01

(c)

图 2.1 “学生选课”数据库示例

2.2 关系操作

关系模型给出了关系操作能力的说明，但不关系数据库管理系统语言做具体的语法要求，也就是说不同的关系数据库管理系统可以定义和开发不同的语言来实现关系操作。

2.2.1 基本的关系操作

关系模型中常用的关系操作包括查询(query)操作和更新操作两大部分,而更新操作又可分为插入(insert)、删除(delete)、修改(update)等操作。

关系的查询表达能力很强,因此查询操作是关系操作中最主要的部分。查询操作又可进一步分为选择(select)、投影(project)、连接(join)、除(divide)、并(union)、差(difference)、交(intersection)、笛卡儿积等操作。其中选择、投影、并、差、笛卡儿积是5种基本操作,其他操作可以用基本操作来定义和导出,就像乘法可以用加法来定义和导出一样。

关系操作的特点是集合操作方式,即操作的对象和结果都是集合。这种操作方式也称为成组数据处理(set-at-a-time processing),即一次一个集合的操作方式。相应地,层次模型和网状模型的数据操作方式则为一次一个记录(record-at-a-time)的方式。这里强调一下,关系操作的所有输入和输出均是关系,包括关系操作的中间结果也是关系。

2.2.2 关系数据语言的分类

早期的关系操作能力通常用代数方式或逻辑方式来表示,分别称为关系代数(relational algebra)和关系演算(relational calculus)。关系代数用对关系的运算来表达查询要求,关系演算则用谓词来表达查询要求。关系演算又可按谓词变元的基本对象是元组变量还是域变量分为元组关系演算和域关系演算。一个关系数据语言能够表示关系代数可以表示的查询,称为具有完备的表达能力,简称关系完备性。已经证明关系代数、元组关系演算和域关系演算三种关系数据语言在表达能力上是等价的,都具有完备的表达能力。

关系代数、元组关系演算和域关系演算均是抽象的查询语言,这些抽象的语言与具体的关系数据库管理系统中实现的实际语言并不完全一样。但它们能用作评估实际系统中查询语言能力的标准或基础。实际的查询语言除了提供关系代数或关系演算的功能外,还提供了许多附加功能,例如聚集函数(aggregation function)、关系赋值、算术运算等,使得目前实际数据库系统中查询语言的功能十分强大。

此外,还有一种结构化查询语言(structured query language, SQL)。SQL不仅具有丰富的查询功能,而且具有数据定义和数据控制功能,是集数据查询语言(data query language, DQL)、数据定义语言(DDL)、数据操纵语言(DML)和数据控制语言(data control language, DCL)于一体的关系数据语言。它充分体现了关系数据语言的特点和优点,自20世纪80年代起成为关系数据库的标准语言。

综合来看,关系数据语言可以分为以下几类:

关系数据语言	{	关系代数(如 ISBL)	
		{	元组关系演算语言(如 ALPHA、QUEL)
			域关系演算语言(如 QBE)
	结构化查询语言(SQL), 具有关系代数和关系演算双重特点		

特别地, SQL 是一种高度非过程化的语言, 用户不必请求数据库管理员为其建立特殊的存取路径, 存取路径的选择由关系数据库管理系统的优化机制来完成。例如, 在一个存储有千百万条记录的关系中查找符合条件的某一条记录或某一些记录, 从原理上讲可以有多种查找方法。例如, 可以顺序扫描这个关系, 也可以通过某一种索引来查找。不同的查找路径(也称为存取路径)的效率是不同的, 有的完成某一个查询可能很快, 有的可能极慢。关系数据库管理系统研究和开发了查询优化方法, 系统可以自动选择较优的存取路径, 以提高查询效率。

2.3 关系的完整性

关系模型的完整性约束是对关系的某种约束条件。也就是说关系的值随着时间变化时应该满足一些约束条件, 这些约束条件实际上是现实世界的要求。任何关系在任何时刻都要满足这些语义约束。

关系模型中有三类完整性约束: 实体完整性(entity integrity)、参照完整性(referential integrity)和用户定义的完整性(user-defined integrity)。其中实体完整性和参照完整性是关系模型必须满足的完整性约束, 被称作是关系的两个不变性, 应该由关系系统自动支持。用户定义的完整性是应用领域需要遵循的约束条件, 体现了具体领域中的语义约束。

2.3.1 实体完整性

关系数据库中每个元组应该是可区分的, 是唯一的。这样的约束条件用实体完整性来保证。

规则 2.1 实体完整性约束 若属性(指一个或一组属性) A 是基本关系 R 的主属性, 则 A 不能取空值(null value)。所谓空值就是“不知道”或“不存在”或“无意义”的值。有关空值的处理将在第3章 3.5 节空值的处理中详细讲解。

例如, 学生(学号, 姓名, 性别, 出生日期, 主修专业)关系中“学号”为主码并用下划线标识, 则学号取值唯一, 且不能取空值。

按照实体完整性约束的规定, 如果主码由若干属性组成, 则所有这些属性都不能取空值。例如, 学生选课(学号, 课程号, 成绩, 开课学期, 教学班)关系中, “学号, 课程号”为主码, 则“学号”和“课程号”两个属性都不能取空值。

对于实体完整性约束说明如下:

① 实体完整性约束是针对基本关系而言的。一个基本表通常对应现实世界的一个实体集。例如, “学生”关系对应学生的集合。

② 现实世界中的实体是可区分的, 即它们具有某种唯一性标识。例如, 每个学生都是独立的个体, 是不一样的。

③ 相应地, 关系模型中以主码作为唯一性标识。

④ 主码中的属性不能取空值, 如果取了空值, 就说明存在某个不可标识的实体, 即存在

不可区分的实体,这与②相矛盾。因此这个规则称为实体完整性。

2.3.2 参照完整性

现实世界中的实体之间往往存在某种联系,在关系模型中实体及实体间的联系都是用关系来描述的,这样就自然存在着关系与关系间的引用。下面先来看三个示例。

[例 2.1]“学生”实体和“专业”实体可以用下面的关系模式来表示。其中,“专业”实体有 2 个候选码“专业编号”和“专业名”,可以选其中一个为主码,即“专业名”。

学生(学号,姓名,性别,出生日期,主修专业)

专业(专业名,专业编号) /* 专业名、专业编号都是候选码,取专业名为主码 */

这两个关系之间存在着属性的引用,即“学生”关系引用了“专业”关系的主码“专业名”。显然,学生关系中的“主修专业”值必须是确实存在的专业名,即“专业”关系中有该专业的记录。也就是说,“学生”关系中“主修专业”属性的取值需要参照“专业”关系中“专业名”属性的取值。

[例 2.2]“学生”“课程”关系以及两者之间的多对多联系可以用如下三个关系模式表示。

学生(学号,姓名,性别,出生日期,主修专业)

课程(课程号,课程名,学分,先修课)

学生选课(学号,课程号,成绩,开课学期,教学班)

这三个关系模式之间也存在着属性之间的引用,即“学生选课”关系引用了“学生”关系的主码“学号”和“课程”关系的主码“课程号”。同样,“学生选课”关系中的“学号”值必须是确实存在的学生的学号,即“学生”关系中有该学生的记录;“学生选课”关系中的“课程号”值也必须是确实存在的课程的编号,即“课程”关系中有该课程的记录。换句话说,“学生选课”关系中某些属性的取值需要参照其他关系的属性取值。

不仅两个或两个以上的关系间可能存在引用关系,同一关系的内部属性间也可能存在引用关系。

[例 2.3]在课程(课程号,课程名,学分,先修课)关系中,“课程号”属性是主码,“先修课”属性表示选修该门课程之前需要完成先修课程的课程号。它引用了本关系的“课程号”属性,即“课程号”必须是确实存在的课程的编号。

这三个例子说明关系与关系之间、同一关系内部属性之间存在相互引用、相互约束的情况。下面先引入外码的概念,然后给出表达关系之间相互引用约束的参照完整性的定义。

定义 2.5 设 F 是基本关系 R 的一个或一组属性,但不是关系 R 的码, K_s 是基本关系 S 的主码。如果 F 与 K_s 相对应,则称 F 是 R 的外码(foreign key),并称基本关系 R 为参照关系(referencing relation),基本关系 S 为被参照关系(referenced relation)或目标关系(target relation)。关系 R 和 S 不一定是不同的关系。如图 2.2 所示。

显然,目标关系 S 的主码 K_s 和参照关系 R 的外码 F 必须定义在同一个(或同一组)域上。

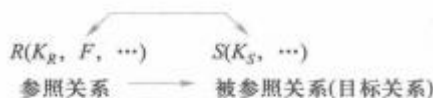


图 2.2 参照关系与被参照关系示意图

在例 2.1 中,“学生”关系的“主修专业”属性与“专业”关系的主码“专业名”相对应,因此“主修专业”属性是“学生”关系的外码。这里“专业”关系是被参照关系,“学生”关系为参照关系。如图 2.3(a)所示。

在例 2.2 中,“学生选课”关系的“学号”属性与“学生”关系的主码“学号”相对应,“学生选课”关系的“课程号”属性与“课程”关系的主码“课程号”相对应,因此“学号”和“课程号”是“学生选课”关系的外码。这里“学生”关系和“课程”关系均为被参照关系,“学生选课”关系为参照关系。如图 2.3(b)所示。

在例 2.3“课程”关系中,“先修课”属性与本身的主码“课程号”属性相对应,因此“先修课”是外码。这里,“课程”关系既是参照关系也是被参照关系。如图 2.3(c)所示。



图 2.3 关系的参照图

需要指出的是,外码并不一定要与相应的主码同名,如例 2.1 中“专业”关系的主码为“专业名”,外码为“主修专业”;例 2.3 中“课程”关系的主码为“课程号”,外码为“先修课”。

在实际应用中为了便于识别,当外码与相应的主码属于不同关系时,往往给它们取相同的名字,例如“学生选课”关系中的外码“课程号”与“课程”关系中的主码取相同属性名。

参照完整性约束就是定义外码与主码之间的引用规则。

规则 2.2 参照完整性约束 若属性(或属性组) F 是基本关系 R 的外码,它与基本关系 S 的主码 K_S 相对应(基本关系 R 和 S 不一定是不同的关系),则对于 R 中每个元组在 F 上的值必须:

- ① 或者取值(F 的每个属性值均为空值)。
- ② 或者等于 S 中某个元组的主码值。

例如,对于例 2.1,“学生”关系中每个元组的“主修专业”属性只能取下面两类值:

- ① 空值,表示该学生尚未选择主修专业。
- ② 非空值,这时该值必须是“专业”关系中某个元组的“专业名”值,表示该学生不可能选

一个不存在的专业。即被参照关系“专业”中一定存在一个元组，它的主码值等于该参照关系“学生”中的外码值。

对于例 2.2，按照参照完整性约束，“学号”和“课程号”属性也可以取两类值：空值或目标关系中已经存在的值。但由于这两个属性是“学生选课”关系中的主属性，按照实体完整性约束，它们均不能取空值，所以“学生选课”关系中的“学号”和“课程号”属性实际上只能取相应被参照关系中已经存在的主码值。

在参照完整性约束中， R 与 S 可以是同一个关系。

例如对于例 2.3，按照参照完整性约束，“先修课”属性值可以取两类值：

- ① 空值，表示该门课程不存在先修课。
- ② 非空值，这时该值必须是本关系中某个元组的课程号。

2.3.3 用户定义的完整性

任何关系数据库系统都应该支持实体完整性和参照完整性，这是关系模型所要求的。除此之外，不同的关系数据库根据其应用场景不同，往往还需要一些特殊的约束条件。用户定义的完整性就是针对某一具体关系数据库的约束条件，它反映某一具体应用所涉及的数据必须满足的语义要求。例如，某个属性必须取唯一值，某个非主属性不能取空值等。例如，在例 2.1 的“学生”关系中，若要求学生必须有姓名，则可以定义“姓名”属性不能取空值；“学生选课”关系中“成绩”的取值范围可以定义为 0~100 等。

关系模型应提供定义和检验这类完整性约束的机制，以使用统一的系统的方法处理它们，而不需要由应用程序承担这一功能。

2.4 关系代数

关系代数是一种抽象的查询语言，它用对关系的运算来表达查询。

任何一种运算都是将一定的运算符作用于一定的运算对象上，得到预期的运算结果。所以运算对象、运算符、运算结果是运算的三大要素。

关系代数的运算对象是关系，运算结果亦为关系。关系代数用到的运算符包括两类：集合运算符和专门的关系运算符，如表 2.4 所示。

按运算符的不同，关系代数的运算可分为传统的集合运算和专门的关系运算两类。其中，传统的集合运算将关系看成元组的集合，其运算是从关系的“水平”方向，即行的角度来进行；专门的关系运算不仅涉及行，而且涉及列。比较运

表 2.4 关系代数运算符

运算符		含义
集合运算符	$\cup$	并
	$-$	差
	$\cap$	交
	$\times$	笛卡儿积
专门的关系运算符	σ	选择
	Π	投影
	$\bowtie$	连接
	$\div$	除

算符和逻辑运算符用于辅助专门的关系运算符进行操作。

2.4.1 传统的集合运算

传统的集合运算是二目运算,包括并、差、交、笛卡儿积4种运算。

设关系 R 和关系 S 具有相同的目 n (即两个关系都有 n 个属性),且相应的属性取自同一个域, t 是元组变量($t \in R$, 表示 t 是 R 的一个元组)。

可以定义并、差、交、笛卡儿积运算如下。

1. 并

关系 R 与关系 S 的并记作

$$R \cup S = \{t \mid t \in R \vee t \in S\}$$

其结果仍为 n 目关系,由属于 R 或属于 S 的元组组成。

2. 差

关系 R 与关系 S 的差记作

$$R - S = \{t \mid t \in R \wedge t \notin S\}$$

其结果关系仍为 n 目关系,由属于 R 而不属于 S 的所有元组组成。

3. 交

关系 R 与关系 S 的交记作

$$R \cap S = \{t \mid t \in R \wedge t \in S\}$$

其结果关系仍为 n 目关系,由既属于 R 又属于 S 的元组组成。关系的交可以用差来表示,即 $R \cap S = R - (R - S)$ 。

4. 笛卡儿积

这里的笛卡儿积严格地讲应该是广义笛卡儿积,因为这里笛卡儿积的元素是元组。

两个分别为 n 目和 m 目的关系 R 和 S ,其笛卡儿积是一个 $n+m$ 列的元组的集合。元组的前 n 列是关系 R 的一个元组,后 m 列是关系 S 的一个元组。若 R 有 k_1 个元组, S 有 k_2 个元组,则关系 R 和关系 S 的笛卡儿积有 $k_1 \times k_2$ 个元组。记作

$$R \times S = \{\widehat{t_i t_j} \mid t_i \in R \wedge t_j \in S\}$$

图 2.4 为关系 R 、 S 以及两者之间的传统集合运算示例。

2.4.2 专门的关系运算

专门的关系运算包括选择、投影、连接、除等运算。为了叙述上的方便,这里先引入几个记号。

① 设关系模式为 $R(A_1, A_2, \dots, A_n)$, 它的一个关系设为 R 。 $t \in R$, 表示 t 是 R 的一个元组。 $T[A_i]$ 则表示元组 t 中相应于属性 A_i 的一个分量。

② 若 $A = \{A_{i1}, A_{i2}, \dots, A_{ik}\}$, 其中 $A_{i1}, A_{i2}, \dots, A_{ik}$ 是 $A_1, A_2, \dots, A_n$ 的一部分, 则称 A 为属性列或属性组。 $T[A] = (T[A_{i1}], T[A_{i2}], \dots, T[A_{ik}])$ 表示元组 t 在属性列 A 上诸分量的集合, $\bar{A}$ 则

R			S			$R \cup S$		
A	B	C	A	B	C	A	B	C
a_1	b_1	c_1	a_1	b_2	c_2	a_1	b_1	c_1
a_1	b_2	c_2	a_1	b_3	c_2	a_1	b_2	c_2
a_2	b_2	c_1	a_2	b_2	c_1	a_2	b_2	c_1
a_1	b_3	c_2				a_1	b_3	c_2

(a) (b) (c)

$R \cap S$			$R \times S$					
A	B	C	$R.A$	$R.B$	$R.C$	$S.A$	$S.B$	$S.C$
a_1	b_2	c_2	a_1	b_1	c_1	a_1	b_2	c_2
a_2	b_2	c_1	a_1	b_1	c_1	a_1	b_3	c_2
			a_1	b_1	c_1	a_2	b_2	c_1
			a_1	b_2	c_2	a_1	b_2	c_2
			a_1	b_2	c_2	a_1	b_3	c_2
			a_1	b_2	c_2	a_2	b_2	c_1
			a_2	b_2	c_1	a_1	b_2	c_2
			a_2	b_2	c_1	a_1	b_3	c_2
			a_2	b_2	c_1	a_2	b_2	c_1

(d) (e) (f)

$R - S$		
A	B	C
a_1	b_1	c_1

图 2.4 关系 R 、 S 以及两者之间的传统集合运算举例

表示 $\{A_1, A_2, \dots, A_n\}$ 中去掉 $\{A_{i1}, A_{i2}, \dots, A_{ik}\}$ 后剩余的属性列。

③ R 为 n 目关系, S 为 m 目关系。 $t_r \in R$, $t_s \in S$, $\widehat{t_r t_s}$ 称为元组的连接(concatenation)或元组的串接。它是一个 $n+m$ 列的元组, 前 n 个分量为 R 中的一个 n 元组, 后 m 个分量为 S 中的一个 m 元组。

④ 给定一个关系 $R(X, Z)$, X 和 Z 为属性列。当 $t[X] = x$ 时, x 在 R 中的象集(images set)定义为

$$Z_x = \{t[Z] \mid t \in R, t[X] = x\}$$

它表示 R 中属性列 X 上值为 x 的诸元组在 Z 上分量的集合。

例如, 图 2.5 中,

$$x_1 \text{ 在 } R \text{ 中的象集 } Z_{x_1} = \{Z_1, Z_2, Z_3\}$$

$$x_2 \text{ 在 } R \text{ 中的象集 } Z_{x_2} = \{Z_2, Z_3\}$$

$$x_3 \text{ 在 } R \text{ 中的象集 } Z_{x_3} = \{Z_1, Z_3\}$$

R	
x_1	Z_1
x_1	Z_2
x_1	Z_3
x_2	Z_2
x_2	Z_3
x_3	Z_1
x_3	Z_3

图 2.5 象集举例

下面给出这些专门的关系运算的定义。

1. 选择(selection)

选择又称为限制(restriction)。它是在关系 R 中选择满足给定条件的诸元组, 记作

$$\sigma_F(R) = \{t | t \in R \wedge F(t) = \text{'真'}\}$$

其中 F 表示选择条件, 它是一个逻辑表达式, 取逻辑值“真”或“假”。

逻辑表达式 F 的基本形式为

$$X_1 \theta Y_1$$

其中 θ 表示比较运算符, 它可以是 $>$, $\geq$, $<$, $\leq$, $=$ 或 $<>$ 。 X_1 , Y_1 是属性名, 或为常量, 或为简单函数; 属性名也可以用其序号来代替。在基本的选择条件上可以进一步进行逻辑运算, 即进行求非($\neg$)、与($\wedge$)、或($\vee$)运算。条件表达式中的运算符如表 2.5 所示。

表 2.5 条件表达式中的运算符

运算符		含义
比较运算符	$>$	大于
	$\geq$	大于或等于
	$<$	小于
	$\leq$	小于或等于
	$=$	等于
	$<>$	不等于
逻辑运算符	$\neg$	非
	$\wedge$	与
	$\vee$	或

选择运算实际上是从关系 R 中选取使逻辑表达式 F 为真的元组。这是从行的角度进行的运算。

下面对 2.1.3 小节中图 2.1 的“学生选课”数据库中关系 Student、Course 和 SC 进行运算。

[例 2.4] 查询信息安全专业的全体学生。

$$\sigma_{\text{Smajor} = \text{'信息安全'}}(\text{Student})$$

结果如图 2.6(a) 所示。

[例 2.5] 查询 2001 年之后(包含 2001 年)出生的学生。

$$\sigma_{\text{Shirhdate} \geq 2001-1-1}(\text{Student})$$

结果如图 2.6(b) 所示。

2. 投影(projection)

关系 R 上的投影是从 R 中选择若干属性列组成新的关系。记作

$$\pi_A(R) = \{t[A] | t \in R\}$$

其中 A 为 R 中的属性列。

投影操作是从列的角度进行的运算。

Sno	Sname	Ssex	Sbirthdate	Smajor
20180001	李勇	男	2000-3-8	信息安全

(a) 查询信息安全专业的全体学生

Sno	Sname	Ssex	Sbirthdate	Smajor
20180003	王敏	女	2001-8-1	计算机科学与技术
20180005	陈新奇	男	2001-11-1	信息管理与信息系统
20180007	王佳佳	女	2001-12-7	数据科学与大数据技术

(b) 查询2001年之后（包含2001年）出生的学生

图 2.6 选择运算示例

[例 2.6] 查询学生的学号和主修专业，即求关系 Student 在“学号”和“主修专业”两个属性上的投影。

$$\Pi_{\text{Sno, Smajor}}(\text{Student})$$

结果如图 2.7(a) 所示。

投影不仅取消了原关系中的某些属性列，还可能取消某些元组，因为取消了某些属性列后就可能出现重复行，应取消这些完全相同的行。

[例 2.7] 查询关系 Student 中的学生都主修了哪些专业，即查询关系 Student 在“主修专业”属性上的投影。

$$\Pi_{\text{Smajor}}(\text{Student})$$

结果如图 2.7(b) 所示。关系 Student 原来有 7 个元组，而投影结果取消了重复的元组，因此最终结果只有 4 个元组。

学号 Sno	专业 Smajor
20180001	信息安全
20180002	计算机科学与技术
20180003	计算机科学与技术
20180004	计算机科学与技术
20180005	信息管理与信息系统
20180006	数据科学与大数据技术
20180007	数据科学与大数据技术

(a) 查询学生的学号和主修专业

专业 Smajor
信息安全
计算机科学与技术
信息管理与信息系统
数据科学与大数据技术

(b) 查询学生主修的专业

图 2.7 投影运算示例

3. 连接(join)

连接也称为 θ 连接，指从两个关系的笛卡儿积中选取其属性间满足一定条件的元组。记作

$$R \bowtie_{A \theta B} S = \{ \widehat{t_r t_s} \mid t_r \in R \wedge t_s \in S \wedge t_r[A] \theta t_s[B] \}$$

其中, A 和 B 分别为关系 R 和 S 上列数相等且可比的属性列, θ 是比较运算符。具体来说, 连接运算是从笛卡儿积 $R \times S$ 中选取关系 R 在属性列 A 上的值与关系 S 在属性列 B 上的值满足比较关系 θ 的元组。

连接运算中有两种最为重要也最为常用的连接, 一种是等值连接(equijoin), 另一种是自然连接(natural join)。

θ 为“=”的连接运算称为等值连接。它是从关系 R 与 S 的广义笛卡儿积中选取 A 、 B 属性值相等的那些元组, 即

$$R \bowtie_{A=B} S = \{ \widehat{t_r t_s} \mid t_r \in R \wedge t_s \in S \wedge t_r[A] = t_s[B] \}$$

自然连接是一种特殊的等值连接。它要求两个关系中进行比较的分量必须是同名的属性列, 并且在结果中把重复的属性列去掉。即若 R 和 S 中具有相同的属性列 B , U 为 R 和 S 的全体属性集合, 则自然连接可记作

$$R \bowtie S = \{ \widehat{t_r t_s} [U-B] \mid t_r \in R \wedge t_s \in S \wedge t_r[B] = t_s[B] \}$$

一般的连接操作是从行的角度进行运算, 但自然连接还需要取消重复属性列, 所以是同时从行和列的角度进行运算。

[例 2.8] 对于图 2.8(a)(b)所示的关系 R 和 S , 图 2.8(c)(d)(e)分别给出了非等值连接 $R \bowtie_{C < E} S$ 、等值连接 $R \bowtie_{R.B=S.B} S$ 和自然连接 $R \bowtie S$ 的结果。

R			S		$R \bowtie_{C < E} S$				
A	B	C	B	E	A	R.B	C	S.B	E
a_1	b_1	5	b_1	3	a_1	b_1	5	b_2	7
a_1	b_2	6	b_2	7	a_1	b_1	5	b_3	10
a_2	b_3	8	b_3	10	a_1	b_2	6	b_2	7
a_2	b_4	12	b_3	2	a_1	b_2	6	b_3	10
			b_5	2	a_2	b_3	8	b_3	10

(a) 关系R

(b) 关系S

(c) 非等值连接

$R \bowtie_{R.B=S.B} S$					$R \bowtie S$			
A	R.B	C	S.B	E	A	B	C	E
a_1	b_1	5	b_1	3	a_1	b_1	5	3
a_1	b_2	6	b_2	7	a_1	b_2	6	7
a_2	b_3	8	b_3	10	a_2	b_3	8	10
a_2	b_3	8	b_3	2	a_2	b_3	8	2

(d) 等值连接

(e) 自然连接

图 2.8 连接运算举例

两个关系 R 和 S 在做自然连接时, 选择两个关系在公共属性上值相等的元组构成新的关系。此时, 关系 R 中某些元组有可能在 S 中不存在公共属性上值相等的元组, 从而造成 R 中这些元组在操作时被舍弃了; 同样, S 中某些元组也可能被舍弃。这些被舍弃的元组称为悬浮元组(dangling tuple)。例如, 在图 2.8(e) 的自然连接中, R 中的第 4 个元组、 S 中的第 5 个元组都是被舍弃的悬浮元组。

如果把悬浮元组也保存在结果关系中, 而在其他属性上填空值(NULL), 那么这种连接就叫作外连接(outer join), 记作 $R \bowtie S$; 如果只保留左边关系 R 中的悬浮元组就叫作左外连接(left outer join 或 left join), 记作 $R \ltimes S$; 如果只保留右边关系 S 中的悬浮元组就叫作右外连接(right outer join 或 right join), 记作 $R \rtimes S$ 。

例如, 图 2.9(a) 是图 2.8 中关系 R 和关系 S 的外连接, 图 2.9(b) 是左外连接, 图 2.9(c) 是右外连接。

A	B	C	E
a_1	b_1	5	3
a_1	b_2	6	7
a_2	b_3	8	10
a_2	b_3	8	2
a_2	b_4	12	NULL
NULL	b_5	NULL	2

(a) 外连接

A	B	C	E
a_1	b_1	5	3
a_1	b_2	6	7
a_2	b_3	8	10
a_2	b_3	8	2
a_2	b_4	12	NULL

(b) 左外连接

A	B	C	E
a_1	b_1	5	3
a_1	b_2	6	7
a_2	b_3	8	10
a_2	b_3	8	2
NULL	b_5	NULL	2

(c) 右外连接

图 2.9 外连接运算举例

4. 除 (division)

设关系 R 除以关系 S 的结果为关系 T , 则 T 包含所有在 R 但不在 S 中的属性及其值, 且 T 的元组与 S 的元组的所有组合都在 R 中。

下面用象集来定义除法。

给定关系 $R(X, Y)$ 和 $S(Y, Z)$, 其中 X, Y, Z 为属性列。 R 中的 Y 与 S 中的 Y 可以有不同的属性名, 但必须出自相同的域。

R 与 S 的除运算得到一个新的关系 $P(X)$, P 是 R 中满足下列条件的元组在 X 属性列上的投影: 元组在 X 上分量值 x 的象集 Y_x 包含 S 在 Y 上投影的集合。记作

$$R \div S = \{t_r[X] \mid t_r \in R \wedge \Pi_Y(S) \subseteq Y_x\}$$

其中 Y_x 为 x 在 R 中的象集, $x = t_r[X]$ 。

除操作是同时从行和列角度进行运算。

[例 2.9] 设关系 R, S 分别为图 2.10 中的(a)和(b), $R \div S$ 的结果为图 2.10(c)。

在关系 R 中, A 可以取 4 个值 $\{a_1, a_2, a_3, a_4\}$ 。其中:

$$a_1 \text{ 的象集为 } \{(b_1, c_2), (b_2, c_3), (b_2, c_1)\}$$

a_2 的象集为 $\{(b_3, c_7), (b_2, c_3)\}$

a_3 的象集为 $\{(b_4, c_6)\}$

a_4 的象集为 $\{(b_6, c_6)\}$

S 在 (B, C) 上的投影为 $\{(b_1, c_2), (b_2, c_1), (b_2, c_3)\}$ 。

显然只有 a_1 的象集 $(B, C)_{a_1}$ 包含了 S 在 (B, C) 属性列上的投影, 所以

$$R \div S = \{a_1\}$$

R			S		
A	B	C	B	C	D
a_1	b_1	c_2	b_1	c_2	d_1
a_2	b_3	c_7	b_2	c_1	d_1
a_3	b_4	c_6	b_2	c_3	d_2
a_1	b_2	c_3			
a_4	b_6	c_6			
a_2	b_2	c_3			
a_1	b_2	c_1			

(a)

R÷S		
A		
a_1		

(c)

图 2.10 除运算举例

图 2.11 为例 2.9 的除运算过程示意。

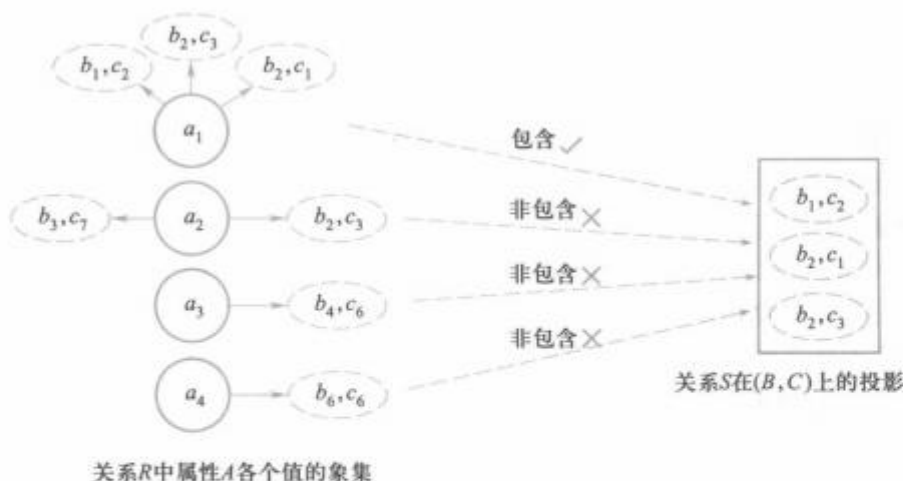


图 2.11 例 2.9 的除运算过程示意图

下面仍以 2.1.3 小节图 2.1 “学生选课”数据库为例, 给出几个综合应用多种关系代数运算进行查询的例子。

[例 2.10] 查询至少选修 81001 号课程和 81003 号课程的学生的学号。

首先建立一个临时关系 K :

K	
Cno	
81001	
81003	

然后求:

$$\Pi_{Sno, Cno}(SC) \div K = \{20180001, 20180002\}$$

求解过程与例 2.9 类似, 先对关系 SC 在 (Sno, Cno) 属性上投影, 然后逐一求出每一学生 (Sno) 的象集, 并依次检查这些象集是否包含 K 。

[例 2.11] 查询选修了 81002 号课程的学生的学号。

$$\Pi_{Sno}(\sigma_{Cno='81002'}(SC)) = \{20180001, 20180002, 20180003\}$$

[例 2.12] 查询至少选修了一门其直接先修课为 81003 号课程的学生的姓名。

$$\Pi_{Sname}(\sigma_{Cpno='81003'}(Course)) \bowtie SC \bowtie \Pi_{Sno, Sname}(Student)$$

或

$$\Pi_{Sname}(\Pi_{Sno}(\sigma_{Cpno='81003'}(Course)) \bowtie SC) \bowtie \Pi_{Sno, Sname}(Student)$$

[例 2.13] 查询选修了全部课程的学生的学号和姓名。

$$\Pi_{Sno, Cno}(SC) \div \Pi_{Cno}(Course) \bowtie \Pi_{Sno, Sname}(Student)$$

关系代数小结

本节介绍了 8 种关系代数运算, 其中并、差、笛卡儿积、选择和投影这 5 种运算为基本的运算。其他三种运算, 即交、连接和除, 均可以用这 5 种基本运算来表达。引进它们并不增加语言的运算能力, 但可以简化表达。

关系代数中, 这些运算经有限次复合而形成的表达式称为关系代数表达式。

此外, 还有扩展的关系代数, 如关系的重新命名、查询结果的去重操作、分组操作、排序操作和聚集函数等, 这里就不介绍了。

* 2.5 关系演算

关系演算是以数理逻辑中的谓词演算为基础的。按谓词变元不同, 关系演算可分为元组关系演算和域关系演算。本节通过元组关系演算语言 ALPHA 和域关系演算语言 QBE 来介绍关系演算的基本概念和查询方法。

* 2.5.1 元组关系演算语言 ALPHA

元组关系演算以元组变量作为谓词变元的基本对象。一种典型的元组关系演算语言是 Edgar F. Codd 提出的 ALPHA。这一语言虽然没有实际实现, 但关系数据库管理系统 INGRES 最初所用的 QUEL 语言就是参照 ALPHA 研制的, 与其十分类似。

ALPHA 语言主要有 GET、PUT、HOLD、UPDATE、DELETE、DROP 这 6 条语句。其语句的基本格式如下。

操作语句 工作空间名(表达式): 操作条件

其中,“表达式”用于指定语句的操作对象,它可以是关系名或(和)属性名,一条语句可以同时操作多个关系或多个属性。“操作条件”是一个逻辑表达式,用于将操作结果限定在满足条件的元组中,操作条件可以为空。除此之外,还可以在基本格式的基础上加上排序要求以及指定返回元组的条数等。

1. 检索操作

检索操作用 GET 语句实现。

(1) 简单检索(即不带条件的检索)

[例 2.14] 查询所有被选修的课程的课程号。

```
GET W(SC. Cno)
```

其中,“W”为工作空间名。这里条件为空,表示没有限定条件。

[例 2.15] 查询所有学生的数据。

```
GET W(Student)
```

(2) 限定的检索(即带条件的检索)

[例 2.16] 查询计算机科学与技术专业 2000 年之前出生的学生的学号和出生日期。

```
GET W(Student. Sno, Student. Sbirthdate): Student. Smajor='计算机科学与技术'  
      ^ Student. Sbirthdate<'2000-1-1'
```

(3) 带排序的检索

[例 2.17] 查询计算机科学与技术专业学生的学号和出生日期,结果按出生日期降序排序。

```
GET W(Student. Sno, Student. Sbirthdate): StudentSmajo='计算机科学与技术'  
      DOWN Student. Sbirthdate
```

其中,“DOWN”表示降序排序。

(4) 指定返回元组的条数的检索

[例 2.18] 取出一个信息安全专业学生的学号。

```
GET W(1)(Student. Sno): Student. Smajor='信息安全'
```

其中,“W”后括号中的数量就是指定的返回元组的个数。

[例 2.19] 查询选修了 81003 号课程、成绩在前三名的学生的学号及其成绩。

```
GET W(3)(SC. Sno, SC. Grade): SC. Cno='81003' DOWN SC. Grade
```

(5) 用元组变量的检索

前面已讲到,元组关系演算是以元组变量作为谓词变元的基本对象。元组变量是在某一关

系范围内变化的, 所以也称为范围变量(range variable), 一个关系可以设多个元组变量。

元组变量主要有两方面的用途:

① 简化关系名。如果关系名很长, 使用起来感到不方便, 则可以设一个较短名字的元组变量来代替关系名。

② 操作条件中使用量词时必须用元组变量。

[例 2.20] 查询信息安全专业学生的姓名。

```
RANGE Student X
GET W(X.Sname):X.Smajor='信息安全'
```

ALPHA 语言用 RANGE 来说明元组变量。本例中 X 是关系 Student 上的元组变量, 用途是简化关系名, 即用 X 代表 Student。

(6) 用存在量词(existential quantifier)的检索

例 2.21、例 2.22、例 2.23 中的元组变量都是为存在量词而设的。其中例 2.23 需要对两个关系使用存在量词, 所以设了两个元组变量。

[例 2.21] 查询选修了 81002 号课程的学生的姓名。

```
RANGE SC X
GET W(Student.Sname): $\exists X(X.Sno=Student.Sno \wedge X.Cno='81002')$ 
```

[例 2.22] 查询选修了直接先修课程是 81002 号课程的学生的学号。

```
RANGE Course CX
GET W(SC.Sno): $\exists CX(CX.Cno=SC.Cno \wedge CX.Cpno='81002')$ 
```

[例 2.23] 查询至少选修一门其先修课为 81002 号课程的学生的姓名。

```
RANGE Course CX
SC SCX
GET W(Student.Sname): $\exists SCX(SCX.Sno=Student.Sno \wedge$ 
 $\exists CX(CX.Cno=SCX.Cno \wedge CX.Cpno='81002'))$ 
```

在本例中的元组关系演算公式可以变换为前束范式(prenex normal form)的形式:

```
RANGE Course CX
SC SCX
GET W(Student.Sname): $\exists SCX \exists CX(SCX.Sno=Student.Sno \wedge$ 
 $CX.Cno=SCX.Cno \wedge CX.Pcno='81002')$ 
```

(7) 带有多个关系的表达式的检索

上面所举的各个例子中, 虽然查询时可能会涉及多个关系, 即公式中可能涉及多个关系, 但查询结果表达式中只有一个关系。实际上表达式中是可以有多个关系的。

[例 2.24] 查询成绩为 90 分以上的学生姓名与课程名称。

本查询所要求的结果是学生姓名和课程名称, 分别在 Student 和 Course 两个关系中。

RANGE SC SCX

GET W(Student, Sname, Course, Cname) : $\exists$ SCX(SCX, Grade $\geq 90 \wedge$
SCX. Sno = Student. Sno $\wedge$ Course. Cno = SCX. Cno)

(8) 用全称量词(universal quantifier)的检索

[例 2.25] 查询不选 81004 号课程的学生姓名。

RANGE SC SCX

GET W(Student, Sname) : $\forall$ SCX(SCX, Sno $\neq$ Student. Sno $\vee$ SCX. Cno $\neq$ '81004')

本例也可以用存在量词来表示:

RANGE SC SCX

GET W(Student, Sname) : $\neg \exists$ SCX(SCX, Sno = Student. Sno $\wedge$ SCX. Cno = '81004')

(9) 用两种量词的检索

[例 2.26] 查询选修了全部课程的学生姓名。

RANGE Course CX

SC SCX

GET W(Student, Sname) : $\forall$ CX $\exists$ SCX(SCX, Sno = Student. Sno $\wedge$ SCX. Cno = CX. Cno)

(10) 用蕴涵(implication)的检索

[例 2.27] 查询至少选修了 20180003 号学生所选课程的学生学号。

本例题的求解思路是, 对 Course 中的所有课程依次检查每一门课程, 看 20180003 是否选修了该课程, 如果选修了, 则再看某一个学生是否也选修了该门课。如果对于 20180003 所选的每门课程该学生都选修了, 则该学生为满足要求的学生。把所有这样的学生全都找出来即完成了本题。

RANGE Course CX

SC SCX /* 注意, 这里 SC 设了两个元组变量 */

SC SCY

GET W(Student, Sno) : $\forall$ CX($\exists$ SCX (SCX, Sno = '20180003' $\wedge$ SCX. Cno = CX. Cno)
 $\Rightarrow \exists$ SCY(SCY, Sno = Student. Sno $\wedge$ SCY. Cno = CX. Cno))

(11) 聚集函数

用户在使用查询语言时经常要做一些简单的计算, 例如要计算符合某一查询要求的元组数, 计算某个关系中所有元组在某属性上的值的总和或平均值等。为了方便用户, 关系数据语言中建立了有关这类运算的标准函数库供用户选用。这类函数通常称为聚集函数或内置函数(built-in function)。关系演算中提供了 COUNT、TOTAL、MAX、MIN、AVG 等聚集函数, 其含义如表 2.6 所示。

表 2.6 关系演算中的聚集函数

函数名	功能
COUNT	对元组计数
TOTAL	求总和
MAX	求最大值
MIN	求最小值
AVG	求平均值

[例 2.28] 查询学生所选的专业的数目。

```
GET W( COUNT( Student. Smajor ) )
```

COUNT 函数在计数时会自动排除重复值。

[例 2.29] 查询 2020 年第二学期选修了 81003 号课程的学生的平均成绩。

```
GET W( AVG( SC. Grade ) ); SC. Cno = '81003' ^ SC. Semester = '20202'
```

2. 更新操作

(1) 修改操作

修改操作用 UPDATE 语句实现。其步骤是：

- ① 用 HOLD 语句将要修改的元组从数据库读到工作空间中。
- ② 用宿主语言修改工作空间中元组的属性值。
- ③ 用 UPDATE 语句将修改后的元组送回数据库。

需要注意的是，单纯检索数据使用 GET 语句即可，但为修改数据而读元组时必须使用 HOLD 语句，HOLD 语句是带并发控制的 GET 语句。有关并发控制的概念将在第 12 章并发控制中详细介绍。

[例 2.30] 把学号为 20180004 的学生从计算机科学与技术专业转到信息管理与信息系统专业。

```
HOLD W( Student. Sno, Student Smajor ); Student. Sno = '20180004'
/* 从 Student 关系中读出学号为 20180004 的学生的数据 */
MOVE '信息管理与信息系统' TO W. Smajor /* 用宿主语言进行修改 */
UPDATE W /* 把修改后的元组送回 Student 关系 */
```

在该例中用 HOLD 语句来读 20180004 号学生的数据，而不是用 GET 语句。

如果修改操作涉及两个关系，就要执行两次 HOLD-MOVE-UPDATE 操作序列。

在 ALPHA 语言中，修改关系主码的操作是不允许的，例如不能用 UPDATE 语句将学号“20180004”改为“20180009”。如果需要修改主码值，只能先用删除操作删除该元组，然后再把具有新主码值的元组插入关系。

(2) 插入操作

插入操作用 PUT 语句实现。其步骤是：

- ① 用宿主语言在工作空间中建立新元组。
- ② 用 PUT 语句将该元组插入指定的关系。

[例 2.31] 学校新开设了一门 2 学分的课程“计算机组织与结构”，其课程号为 81009，直接先行课为 81005 号课程。插入该课程元组。

```
MOVE '81009' TO W. Cno
MOVE '计算机组织与结构' TO W. Cname
```

```

MOVE '2' TO W. Ccredit
MOVE '81005' TO W. Cpno
PUT W( Course)      /* 把 W 中的元组插入指定关系 Course */

```

PUT 语句只对一个关系操作,也就是说表达式必须为单个关系名。

(3) 删除

删除操作用 DELETE 语句实现。其步骤为:

- ① 用 HOLD 语句把要删除的元组从数据库读到工作空间中。
- ② 用 DELETE 语句删除该元组。

[例 2.32] 20180007 号学生因故退学,删除该学生元组。

```

HOLD W( Student) : Student. Sno = '20180007'
DELETE W

```

[例 2.33] 将学号 20180001 改为 20180010。

```

HOLD W( Student) : Student. Sno = '20180001'
DELETE W
MOVE '20180010' TO W. Sno
MOVE '李勇' TO W. Sname
MOVE '男' TO W. Ssex
MOVE '2000-3-8' TO W. Sbirthdate
MOVE '信息安全' TO W. Smajor
PUT W( Student)

```

[例 2.34] 删除全部学生。

```

HOLD W( Student)
DELETE W

```

由于 SC 关系与 Student 关系之间具有参照关系,为保证参照完整性,删除 Student 中的元组时要相应地删除 SC 中的元组(手工删除或由数据库管理系统自动删除)。

```

HOLD W( SC)
DELETE W

```

ALPHA 语言例题解析可参见二维码内容。



扩展阅读: ALPHA
语言例题解析

* 2.5.2 域关系演算语言 QBE

关系演算的另一种形式是域关系演算。域关系演算以元组变量的分量(即域变量)作为谓词变元的基本对象。1975 年由 M. M. Zloof 提出的 QBE 就是一个很有特色的域关系演算语言,

该语言于 1978 年在 IBM 370 上得以实现。

QBE 是 Query By Example(即通过例子进行查询)的简称,它最突出的特点是操作方式。它是一种高度非过程化的基于屏幕表格的查询语言,用户通过终端屏幕编辑程序,以填写表格的方式构造查询要求,而查询结果也是以表格形式显示,因此非常直观、易学易用。

QBE 用示例元素来表示查询结果可能的情况,示例元素实质上就是域变量。QBE 的操作框架如图 2.12 所示。

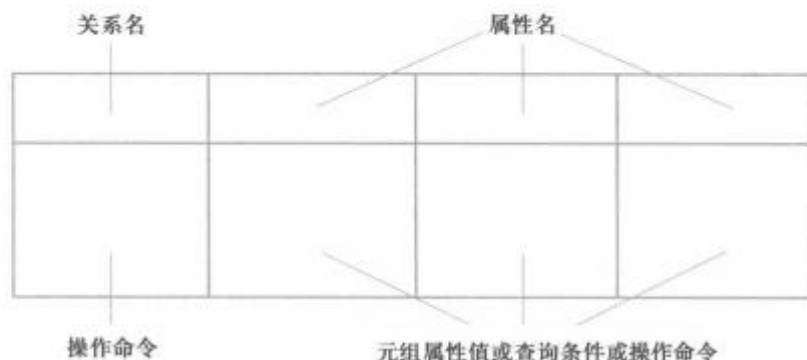


图 2.12 QBE 的操作框架

下面仍以 2.1.3 小节图 2.1 所示的“学生选课”关系数据库为例,说明 QBE 的用法。

1. 检索操作

(1) 简单查询

[例 2.35]求信息安全专业全体学生的姓名。

操作步骤如下:

- ① 用户提出要求。
- ② 屏幕显示如下空白表格。

- ③ 用户在最左边一栏输入关系名 Student。

Student					

- ④ 系统显示该关系的属性名。

Student	Sno	Sname	Ssex	Sbirthdate	Smajor

⑤ 用户在上面构造查询要求。

Student	Sno	Sname	Ssex	Sbirthdate	Smajor
		P. <u>T</u>			信息安全

这里“T”是示例元素，即域变量。QBE 要求示例元素下面一定要加下划线。“信息安全”是查询条件，不用加下划线。“P.”是操作符，表示打印(print)，实际上是显示。

查询条件中可以使用比较运算符>、≥、<、≤、=和≠，其中=可以省略。

示例元素是这个域中可能的一个值，它不必是查询结果中的元素。比如若求信息系的学生，只要给出任意的一个学生名即可，而不必真是信息系的某个学生名。

对于例 2.35，可构造查询要求如下：

Student	Sno	Sname	Ssex	Sbirthdate	Smajor
		P. 刘晨			信息安全

这里的查询条件是 Smajor='信息安全'，其中“=”被省略。

⑥ 屏幕显示查询结果。

Student	Sno	Sname	Ssex	Sbirthdate	Smajor
		李勇			信息安全

即根据用户的查询要求找出信息系的学生姓名。

[例 2.36] 查询全体学生的全部数据。

Student	Sno	Sname	Ssex	Sbirthdate	Smajor
	P. <u>20180002</u>	P. 刘晨	P. <u>女</u>	P. <u>1999-9-1</u>	P. 计算机科学与技术

显示全部数据也可以简单地把 P. 操作符作用在关系名上。因此本查询也可以简单地表示如下：

Student	Sno	Sname	Ssex	Sbirthdate	Smajor
P.					

(2) 条件查询

[例 2.37] 求出生日期晚于 1999-9-1 的学生的学号。

Student	Sno	Sname	Ssex	Sbirthdate	Smajor
	P. <u>20180002</u>			>1999-9-1	

[例 2.38] 求计算机科学与技术专业出生日期晚于 1999-9-1 的学生的学号。

本查询的条件是 Smajor='计算机科学与技术'和 Sbirthdate>'1999-9-1'两个条件的“与”。在 QBE 中，表示两个条件的“与”有两种方法：

① 把两个条件写在同一行上。

Student	Sno	Sname	Ssex	Sbirthdate	Smajor
	P. 20180002			>1999-9-1	计算机科学与技术

② 把两个条件写在不同行上，但使用相同的示例元素值。

Student	Sno	Sname	Ssex	Sbirthdate	Smajor
	P. 20180002				计算机科学与技术
	P. 20180002			>1999-9-1	

[例 2.39] 查询计算机科学与技术专业或者出生日期晚于 1999-9-1 的学生的学号。

本查询的条件是 Smajor='计算机科学与技术' 和 Sbirthdate>'1999-9-1' 两个条件的“或”。在 QBE 中把两个条件写在不同行上，并且使用不同的示例元素值，即表示条件的“或”。

Student	Sno	Sname	Ssex	Sbirthdate	Smajor
	P. 20180002				计算机科学与技术
	P. 20180003			>1999-9-1	

对于多行条件的查询，先输入哪一行是任意的，不影响查询结果。这就允许用户以不同的思考方式进行查询，十分灵活、自由。

[例 2.40] 查询既选修了 81001 号课程又选修了 81002 号课程的学生的学号。

本查询条件是在一个属性中的“与”关系，它只能用“与”条件的方法②表示，即写两行，但示例元素相同。

SC	Sno	Cno	Grade	Semester	Teachingclass
	P. 20180002	81001			
	P. 20180002	81002			

[例 2.41] 查询选修 81001 号课程的学生的姓名。

本查询涉及 SC 和 Student 两个关系。在 QBE 中实现这种查询的方法是通过相同的连接属性值把多个关系连接起来。这里示例元素 Sno 是连接属性，其值在两个表中要相同。

Student	Sno	Sname	Ssex	Sbirthdate	Smajor
	20180002	P. 李勇			

SC	Sno	Cno	Grade	Semester	Teachingclass
	20180002	81001			

[例 2.42] 查询未选修 81001 号课程的学生的姓名。

这里的查询条件中用到逻辑“非”。在 QBE 中表示逻辑“非”的方法是将运算符“¬”写在关系名下面。

Student	Sno	Sname	Ssex	Sbirthdate	Smajor
	<u>2018002</u>	P. 李勇			
	<u>2018002</u>	P. 李勇			

SC	Sno	Cno	Grade	Semester	Teachingclass
┐	<u>2018002</u>	81001			
┐	<u>2018002</u>				

这个查询就是显示学号示例为“2018002”的学生的姓名，而该学生选修 81001 号课程的情况为假或者什么课程都没有选修。

[例 2.43] 查询有两个人以上选修的课程的课程号。

本查询是在一个表内连接。这个查询就是要显示这样的课程号“81001”，它不仅被学生“2018002”选修，而且也被另一个学生“┐ 2018002”选修了。

SC	Sno	Cno	Grade	Semester	Teachingclass
	<u>2018002</u>	P. <u>81001</u>			
	┐ <u>2018002</u>	<u>81001</u>			

(3) 聚集函数

为了方便用户，QBE 提供了一些聚集函数，主要包括 CNT、SUM、AVG、MAX、MIN 等，其含义如表 2.7 所示。

表 2.7 QBE 中的聚集函数

函数名	功能
CNT	对元组计数
SUM	求总和
AVG	求平均值
MAX	求最大值
MIN	求最小值

[例 2.44] 查询信息安全专业学生的总人数。

Student	Sno	Sname	Ssex	Sbirthdate	Smajor
	P. <u>CNT</u>			.	信息安全

(4) 对查询结果排序

对查询结果按某个属性值的升序排序，只需在相应列中填入“AO.”，按降序排序则填“DO.”。如果按多列排序，用“AO(*i*).”或“DO(*i*).”表示，其中 *i* 为排序的优先级，*i* 值越小，优先级越高。

[例 2.45] 查询全体男生的姓名，要求查询结果按专业名升序排序，对同一专业的学生按出生日期降序排序。

Student	Sno	Sname	Ssex	Sbirthdate	Smajor
		P. 李勇	男	DO(2).	AO(1).

2. 更新操作

(1) 修改操作

修改操作符为“U.”。QBE 不允许修改关系的主码，如果需要修改某个元组的主码，只能先删除该元组，然后再插入新主码的元组。

[例 2.46] 把学号为 20180001 的学生的出生日期改为“2001-3-8”。

这是一个简单修改操作，不包含算术表达式，因此可以有两种表示方法：

① 将操作符“U.”放在值上。

Student	Sno	Sname	Ssex	Sbirthdate	Smajor
	20180001			U. 2001-3-8	

② 将操作符“U.”放在关系上。

Student	Sno	Sname	Ssex	Sbirthdate	Smajor
U.	20180001			2001-3-8	

这里，码“20180001”标明要修改的元组。“U.”标明是修改后的新值。由于主码是不能修改的，所以即使在第二种写法中，系统也不会混淆要修改的属性值。

[例 2.47] 将学号为 20180004 的学生在 2019 年第 2 学期选修 81001 号课程的成绩增加 6 分。

这个修改操作涉及表达式，所以只能将操作符“U.”放在关系上。

SC	Sno	Cno	Grade	Semester	Teachingclass
	20180004	81001	56	20192	
U.	20180004		56+6		

[例 2.48] 将 2019 年第 2 学期选修 81001 号课程的学生们的成绩都增加 5 分。

SC	Sno	Cno	Grade	Semester	Teachingclass
	<u>20180001</u>	81001	<u>70</u>	20192	
U.	<u>20180001</u>		<u>70+5</u>	20192	

(2) 插入操作

插入操作符为“I.”。新插入的元组必须具有主码值，其他属性值可以为空。

[例 2.49] 把计算机科学与技术专业的学生“高大卫，男，学号 20190006，出生日期 2001-6-28”存入数据库。

Student	Sno	Sname	Ssex	Sbirthdate	Smajor
I.	20190006	高大卫	男	2001-6-28	计算机科学与技术

(3) 删除操作

删除操作符为“D.”。

[例 2.50] 删除学号为 20180006 的学生。

Student	Sno	Sname	Ssex	Sbirthdate	Smajor
D.	20180006				

由于 SC 关系与 Student 关系之间具有参照关系, 为保证参照完整性, 删除学号为 20180006 的学生后, 通常还应删除该学生选修的全部课程。

SC	Sno	Cno	Grade	Semester	Teachingclass
D.	20180006				

本章小结

关系模型是关系数据库的核心概念和基础, 关系数据库系统是目前使用最广泛的数据库系统。20 世纪 70 年代以后开发的数据库管理系统产品几乎都是基于关系模型的。在数据库发展的历史上, 最重要的创新成果之一就是关系模型。

关系模型与层次模型、网状模型最重要的区别是, 关系模型只有“表”这一种数据结构, 而层次模型、网状模型还有其他数据结构, 以及对这些数据结构的操作。关系模型的数据结构简单清晰, 用户容易理解, 又具有坚实的数学基础。



扩展阅读: 元组
关系演算表达式

本章系统地讲解了关系模型的重要概念, 包括关系模型的数据结构、关系操作以及关系的三类完整性约束; 介绍了用代数方式和逻辑方式来表达的关系语言, 即关系代数、元组关系演算和域关系演算。在关系演算中介绍了元组关系演算语言 ALPHA 和域关系演算语言 QBE。有关元组关系演算表达式的内容, 感兴趣的读者可参见二维码介绍。

习 题 2

1. 试述关系模型的三个组成部分。
2. 简述关系数据语言的特点和分类。
3. 定义并理解下列术语, 说明它们之间的联系与区别:
 - ① 域, 笛卡儿积, 关系, 元组, 属性;
 - ② 主码, 全码, 候选码, 外码, 主属性、非主属性;
 - ③ 关系模式, 关系, 关系数据库。
4. 举例说明关系模式和关系的区别。
5. 试述关系模型的完整性约束。在参照完整性中, 什么情况下外码属性的值可以为空值?
6. 设有一个 SPJ 数据库, 包括 4 个关系模式 S、P、J 和 SPJ。

```
S(SNO, SNAME, STATUS, CITY);
P(PNO, PNAME, COLOR, WEIGHT);
```

J(JNO,JNAME,CITY);
SPJ(SNO,PNO,JNO,QTY)。

供应商表 S 由供应商代码(SNO)、供应商姓名(SNAME)、供应商状态(STATUS)、供应商所在城市(CITY)组成。

零件表 P 由零件代码(PNO)、零件名(PNAME)、颜色(COLOR)、重量(WEIGHT)组成。

工程项目表 J 由工程项目代码(JNO)、工程项目名(JNAME)、工程项目所在城市(CITY)组成。

供应情况表 SPJ 由供应商代码(SNO)、零件代码(PNO)、工程项目代码(JNO)、供应数量(QTY)组成,表示某供应商供应某种零件给某工程项目的数量为 QTY。

今有若干数据如图 2.13 所示。

S

SNO	SNAME	STATUS	CITY
S1	精益	20	天津
S2	盛锡	10	北京
S3	东方红	30	北京
S4	丰泰盛	20	天津
S5	为民	30	上海

P

PNO	PNAME	COLOR	WEIGHT
P1	螺母	红	12
P2	螺栓	绿	17
P3	螺丝刀	蓝	14
P4	螺丝刀	红	14
P5	凸轮	蓝	40
P6	齿轮	红	30

J

JNO	JNAME	CITY
J1	三建	北京
J2	一汽	长春
J3	弹簧厂	天津
J4	造船厂	天津
J5	机车厂	唐山
J6	无线电厂	常州
J7	半导体厂	南京

SPJ

SNO	PNO	JNO	QTY
S1	P1	J1	200
S1	P1	J3	100
S1	P1	J4	700
S1	P2	J2	100
S2	P3	J1	400
S2	P3	J2	200
S2	P3	J4	500
S2	P3	J5	400
S2	P5	J1	400
S2	P5	J2	100
S3	P1	J1	200
S3	P3	J1	200
S4	P5	J1	100
S4	P6	J3	300
S4	P6	J4	200
S5	P2	J4	100
S5	P3	J1	200
S5	P6	J2	200
S5	P6	J4	500

图 2.13 SPJ 数据库数据

试用关系代数、元组关系演算语言 ALPHA 和域关系演算语言 QBE 完成如下查询:

- ① 求供应工程 J1 零件的供应商代码 SNO。
- ② 求供应工程 J1 零件 P1 的供应商代码 SNO。
- ③ 求供应工程 J1 零件为红色的供应商代码 SNO。
- ④ 求没有使用天津供应商生产的红色零件的工程号 JNO。

- ⑤ 求至少使用了与供应商 S1 所供应的全部零件相同零件号的工程号 JNO。
 7. 试述等值连接与自然连接的区别和联系。
 8. 关系代数的基本运算有哪些? 如何用这些基本运算来表示其他运算?

参考文献 2

[1] CODD E F. A relational model of data for large shared data banks[J]. CACM-Communications of the ACM, 1970, 13(6): 377-387.

1970 年, Edgar F. Codd 在文献[1]中首先提出了关系数据模型, 以后 Codd 又提出了关系代数和关系演算的概念, 以及函数依赖的概念, 1972 年提出了关系的第一、第二、第三范式, 1974 年提出了 BC(Boyce-Codd) 范式, 为关系数据库系统奠定了理论基础。由于对关系数据库理论的突出贡献, 1981 年 Codd 获得了图灵奖。

[2] CODD E F. A data base sublanguage founded on the relational calculus[C]. Proceedings of the ACM SIGMOD Workshop on Data Description, Access and Control, 1971: 35-68.

[3] CODD E F. Relational completeness of database sublanguages in data base systems[R]. Courant Computer Science Symposia Series, 1972(6).

[4] CODD E F. Further normalization of the data base relational model[M]. Data Base Systems. Prentice-Hall, 1972.

[5] ULLMAN J D. Principles of database systems[M]. 2nd ed. Computer Science Press, 1982.

Ullman 在文献[5]中给出了关系代数、元组关系演算和域关系演算的等价性证明。

[6] ZLOOF M M. Query By Example[C]. Proceeding of the National Computer Conference and Exposition(AFIPS'75), 1975: 431-438.

[7] LACROIX M, PIROTTE A. Domain-oriented relational language. Proceedings of the 3rd International Conference on Very Large Data Bases[C], 1977: 370-378.

[8] LACROIX M, PIROTTE A. ILL: an English structured query language for relational data bases[J]. ACM SIGART Bulletin, 1977, 61: 61-63.

域关系演算的思想出现在文献[7]描述的 QBE 语言中, 形式化定义由 Lacroix 和 Pirotte 在文献[8]中给出。文献[8]给出了一个域关系演算语言 ILL。

[9] STONEBRAKER M, WONG E, KREPS P, et al. The design and implementation of INGRES[J]. ACM Transactions on Database Systems, 1976, 1(3): 189-222.

[10] DATE C J. Referential integrity[C]. Proceedings of the 7th International Conference on Very Large Data Bases, 1981, 7: 2-12.

[11] DATE C J. Why relations should have exactly one primary key[R]. ANSI Database Committee (X3H2) Working Paper X3H2-84-118, 1984.

本章知识点讲解微视频:



关系模型



关系代数



关系演算

除法运算

第3章

关系数据库标准语言 SQL

结构化查询语言(SQL)是关系数据库的标准语言,包括数据查询、数据库模式创建、数据库数据的增删改、数据库安全性和完整性定义与控制等一系列功能。

本章导读

本章详细介绍 SQL 的基本功能,并进一步讲述关系数据库的基本概念,关系数据库管理系统(RDBMS)对数据库三级模式的支持。

本章首先讲述了 SQL 的产生与发展、SQL 的特点,非过程化 SQL 和过程化语言的区别。读者可以从中进一步了解关系数据库技术和关系数据库管理系统产品的发展过程,从而体会关系数据库系统能够减轻用户负担,提高用户开发数据库应用系统生产率的优点。然后通过实例详细讲解 SQL 的功能、语法和使用要点。

SQL 功能丰富、简洁易懂,不过根据需求正确无误地写出 SQL 语句并不容易,特别是使用 SQL 完成复杂查询更需要认真练习才能掌握。

希望读者通过上机实际操作掌握 SQL 的运用,包括数据表定义、数据的查询、插入、删除、更新,以及索引和视图的创建和删除等操作。

本章的难点是正确写出 SQL 语句并完成各种查询,学习方法就是反复训练,通过具体的数据库产品上进行实际练习,才能举一反三牢固掌握。

3.1 SQL 概述

自 SQL 成为数据库国际标准语言以来,几乎所有数据库厂商都采用了 SQL,而且许多软件厂商也纷纷推出与 SQL 接口的软件,使不同数据库系统之间、数据库系统与其他软件系统之间的互操作有了共同的基础。其意义重大,因此有人把确立 SQL 为关系数据库标准语言及其后的发展称为是一场革命。

3.1.1 SQL 的产生与发展

SQL 是在 1974 年由 Boyce 和 Chamberlin 提出的,最初的名称是 SEQUEL(structured English

query language), 并在 IBM 公司研制的关系数据库管理系统原型 System R 上实现, 后改名为更容易记忆的 SQL。

SQL 是一种非过程化语言, 用户只要说明需要的数据库内容, 不必说明存取所需数据的具体过程和操作。通俗地讲, 即用户只需提出“要什么”, 无须具体指明“怎么干”, 存取路径选择和具体处理操作均由关系数据库管理系统自动完成。SQL 简单易学, 功能丰富, 深受用户及计算机工业界欢迎。

1986 年 10 月, 美国国家标准学会(American National Standard Institute, ANSI)的数据库委员会 X3H2 批准将 SQL 作为关系数据库语言的美国标准, 同年公布了 SQL 标准文本(简称 SQL 86)。1987 年, 国际标准化组织(International Organization for Standardization, ISO)也通过了这一标准。

SQL 标准从公布以来随数据库技术的发展而不断发展、不断丰富, 如表 3.1 所示。

表 3.1 SQL 标准的发展过程

标准	篇幅(约)/页	发布日期/年	标准	篇幅(约)/页	发布日期/年
SQL 86		1986	SQL 2003	3 600	2003
SQL 89(FIPS 127-1)	120	1989	SQL 2008	3 777	2008
SQL 92(SQL2)	622	1992	SQL 2011	3 817	2011
SQL 99(SQL 3)	1 700	1999	SQL 2016	4 035	2016

SQL 86 和 SQL 89 都是单个文档。

SQL 92 和 SQL 99 已扩展为一系列开放的部分。例如, SQL 92 除了 SQL 基本部分外, 还增加了 SQL 调用接口、SQL 永久存储模块。

SQL 99 则进一步扩展为框架、SQL 基础部分、SQL 调用接口、SQL 永久存储模块、SQL 宿主语言绑定、SQL 外部数据的管理和 SQL 对象语言绑定等多个部分。

SQL 2016 已扩展到 12 个部分, 陆续引入了 XML 类型、Window 函数、TRUNCATE 操作、时序数据以及 JSON(JavaScript object notation)类型等。

可以发现, SQL 标准的内容越来越丰富, 也越来越复杂。目前, 没有一个关系数据库管理系统能够支持 SQL 标准的所有概念和特性, 都只实现了 SQL 标准的一个子集。同时, 许多厂商又对 SQL 基本命令集进行了不同程度的扩充和修改, 支持标准以外的一些功能特性。因此, 在使用具体的关系数据库管理系统时一定要查阅相应产品的用户手册。

3.1.2 SQL 的特点

SQL 之所以能够为用户和业界所接受并成为国际标准, 是因为它是一个综合性、功能极强同时又简洁易学的语言。SQL 集数据定义(data definition)、数据查询(data query)、数据操纵(data manipulation)和数据控制(data control)等功能于一体, 其主要特点包括以下几部分。

1. 功能综合且风格统一

数据库系统的主要功能是通过数据库支持的数据语言来实现的。

层次模型和网状模型的数据语言一般都分为模式数据定义语言(DDL)、外模式数据定义语言、数据存储描述语言(data storage description language, DSDL)、数据操纵语言(DML)等。它们分别用于定义模式、外模式、内模式和进行数据的存取与处置。当用户数据库投入运行后,如果需要修改模式,必须停止现有数据库的运行,转储数据、修改模式并编译后再重装数据库,十分麻烦。

而 SQL 语言则集数据定义语言、数据操纵语言和数据控制语言的功能于一体,语言风格统一,可以独立完成数据库生命周期中的全部活动,包括:

- ① 创建和删除数据库模式。
- ② 创建基本表,创建视图。
- ③ 使用数据库,如查询和增、删、改数据、事务处理等。
- ④ 数据库控制,如安全性控制、完整性控制和并发控制等。
- ⑤ 数据库维护和重构,如修改和删除基本表、数据库备份与恢复等。

这就为数据库应用系统开发提供了良好的环境。例如,用户在数据库投入运行后,还可根据需要随时或逐步创建模式,并不影响数据库整体的运行,从而使系统具有良好的可扩展性。

另外,在关系模型中实体和实体间的联系均用关系表示,这种数据结构的单一性带来了数据操作符的统一性,查找、插入、删除、更新等每一种操作都只需一种操作符,从而克服了层次模型和网状模型由于信息表示方式的多样性带来的操作复杂性。

例如,在 DBTG 网状数据库系统中需要两种插入操作符:STORE 和 CONNECT。其中,STORE 用来把记录存入数据库,CONNECT 用来把记录插入系值(系值是网状数据库中记录之间的一种联系方式),以建立数据之间的联系。

2. 数据操纵高度非过程化

层次模型和网状模型的数据操纵语言是面向过程的语言,用过程化语言完成某项请求必须指定存取路径。而用 SQL 进行数据操纵时只要提出“做什么”,无须指明“怎么做”,因此不需要了解存取路径,存取路径的选择以及 SQL 的操作过程由系统自动完成。这不但大大减轻了用户负担,而且当数据存储结构变化时,数据操纵语句一般不用改变,提高了数据独立性。

3. 面向集合的操作方式

层次模型和网状模型采用的是面向记录的操作方式,操作对象是一条记录。例如查询所有平均成绩在 80 分以上的学生姓名,用户必须一条一条地把满足条件的学生记录找出来(通常要说明具体处理过程,即按照哪条路径、如何循环等)。而 SQL 采用集合操作方式,不仅操作对象、查找结果可以是元组的集合,而且一次插入、删除、更新操作的对象也可以是元组的集合。

4. 以统一的语法结构提供多种使用方式

SQL 既是独立的语言,又是嵌入式语言。作为独立的语言,它能够独立用于联机交互的使用方式,用户可以在终端键盘上直接键入 SQL 命令对数据库进行操作;作为嵌入式语言,SQL 语句能够嵌入高级语言(如 C、C++、Java、Python 等)程序,供程序员设计程序时使用。在两种不同的使用方式下,SQL 的语法结构基本上是一致的。这种以统一的语法结构提供多种不同

使用方式的做法,提高了易用性与方便性。

5. 语言简洁且易学易用

SQL 的功能极强,但由于其设计巧妙,语言十分简洁,即可只用 9 个动词完成核心功能,如表 3.2 所示。SQL 接近英语口语,因此易于学习和使用。

表 3.2 SQL 完成核心功能的 9 个动词

SQL 功能	动词
数据定义	CREATE, DROP, ALTER
数据查询	SELECT
数据操纵	INSERT, UPDATE, DELETE
数据控制	GRANT, REVOKE

3.1.3 SQL 的基本概念

SQL 同样支持数据库系统的三级模式结构,如图 3.1 所示。其中外模式是用户能够看见和使用的数据结构,对应视图(view)和部分基本表(base table),模式对应基本表,内模式对应存储文件(stored file)。

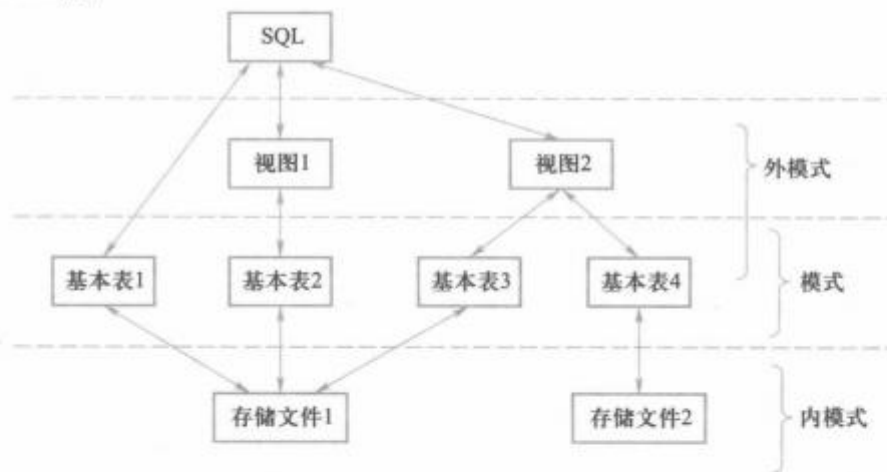


图 3.1 SQL 对关系数据库三级模式的支持

用户可以用 SQL 对基本表和视图进行查询和其他操作。基本表和视图一样,都是关系。

基本表是本身独立存在的表,在关系数据库管理系统中一个关系就对应一个基本表,一个或多个基本表对应一个存储文件。一个表可以带若干索引,索引也存放在存储文件中。

存储文件的逻辑结构和物理结构组成了关系数据库的内模式。存储文件的物理结构是由数据库管理系统设计确定的。

视图是从基本表或其他视图中导出的表。它本身不独立存储在数据库中,即数据库中只存放视图的定义而不存放视图对应的数据。这些数据仍存放在导出视图的基本表中,因此视图是

一个虚表。

用户可以用 SQL 对视图和基本表进行查询。在用户眼中，视图和基本表都是关系，用户可以在视图上再定义视图。

本书从 3.2 节起，将逐一介绍 SQL 语句的功能和语法。为了突出基本概念和基本功能，将会略去许多语法细节。不同关系数据库管理系统产品在实现标准 SQL 时各有差别，与 SQL 标准的符合程度也不相同（一般符合的百分比在 85% 以上）。因此，读者在使用时还应参阅具体产品的用户手册。

3.2 数据定义

本节介绍 SQL 的数据定义功能，包括数据库模式、表、视图和索引的定义等。SQL 的数据定义语句如表 3.3 所示。

表 3.3 SQL 的数据定义语句

操作对象	操作方式		
	创建	删除	修改
数据库模式	CREATE SCHEMA	DROP SCHEMA	* SQL 标准无修改语句
表	CREATE TABLE	DROP TABLE	ALTER TABLE
视图	CREATE VIEW	DROP VIEW	
索引	* CREATE INDEX	* DROP INDEX	* ALTER INDEX

SQL 标准没有提供修改数据库模式定义的语句。用户如果想修改此对象，只能先将它删除然后再重建。SQL 标准也没有提供索引相关的语句，但为了提高查询效率，商用关系数据库管理系统通常都提供了索引机制和相关的语句，如表 3.3 中创建、删除和修改索引等。

在早期的数据库系统中，所有数据库对象都属于一个数据库，也就是说只有一个命名空间。所以在第 1 章图 1.15 中一个数据库只对应一个内模式、一个模式。

当前的关系数据库管理系统提供了层次结构的数据库对象命名机制，如图 3.2 所示。一个关系数据库管理系统的实例可以建多个数据库，一个数据库中又可以建多个模式，一个模式下通常包含多个表、视图和索引等数据库对象。

数据库(有的系统称为目录)



模式



表以及视图、索引等

图 3.2 数据库对象命名机制的层次结构

本节仅介绍如何定义模式、基本表和索引，视图的概念及其定义方法将在 3.6 节专门讨论。

3.2.1 模式的定义与删除

1. 定义模式

在 SQL 中，定义模式语句如下：

```
CREATE SCHEMA [ <模式名> ] AUTHORIZATION <用户名>;
```

如果没有指定<模式名>，那么<模式名>隐含为<用户名>。

要创建模式，调用该命令的用户必须拥有数据库管理员 (DBA) 权限，或者获得了 DBA 授予的 CREATE SCHEMA 权限。

[例 3.1] 为用户 WANG 定义一个“学生选课”模式 S-C-SC。

```
CREATE SCHEMA "S-C-SC" AUTHORIZATION WANG;
```

注意：标识符要么加双引号，要么没有任何引号。这里 "S-C-SC" 是加了双引号的。

[例 3.2] 未指定模式名的模式定义示例。

```
CREATE SCHEMA AUTHORIZATION WANG;
```

该语句没有指定<模式名>，所以<模式名>隐含为用户名“WANG”。

定义模式实际上定义了一个命名空间，在这个空间中可以进一步定义该模式包含的数据库对象，如基本表、视图、索引等。

这些数据库对象可以用表 3.3 中相应的 CREATE 语句来定义。

目前，在 CREATE SCHEMA 中可以接受 CREATE TABLE、CREATE VIEW 和 GRANT 子句。也就是说，用户可以在创建模式的同时在该模式定义中进一步创建基本表、视图，定义授权。即

```
CREATE SCHEMA <模式名> AUTHORIZATION <用户名>
[ <表定义子句> | <视图定义子句> | <授权定义子句>];
```

[例 3.3] 为用户 ZHANG 创建一个模式 Test，并且在其中定义一个表 Tab1。

```
CREATE SCHEMA Test AUTHORIZATION Zhang
CREATE TABLE Tab1 (Col1 SMALLINT,
                    Col2 INT,
                    Col3 CHAR(20),
                    Col4 NUMERIC(10,3),
                    Col5 DECIMAL(5,2)
);
```

2. 删除模式

在 SQL 中，删除模式语句如下：

```
DROP SCHEMA <模式名> <CASCADE | RESTRICT>;
```

其中 CASCADE 和 RESTRICT 两者必选其一。选择了 CASCADE(级联),表示在删除模式的同时把该模式中所有的数据库对象全部删除。选择了 RESTRICT(限制),表示如果该模式中已经定义了数据库对象(如表、视图等),则拒绝该删除语句的执行;只有当该模式中没有任何数据库对象时才能执行 DROP SCHEMA 语句。

[例 3.4] 删除例 3.3 中建立的模式 Test。

```
DROP SCHEMA Test CASCADE;
```

该语句删除了模式 Test,同时也将该模式中已经定义的表 Tab1 删除了。

3.2.2 基本表的定义、删除与修改

1. 定义基本表

创建了一个模式就建立了一个数据库的命名空间,一个框架。在这个空间中首先要定义的是该模式包含的数据库基本表。

SQL 使用 CREATE TABLE 语句定义基本表,其基本格式如下:

```
CREATE TABLE <表名>(<列名><数据类型>[列级完整性约束]
    [,<列名><数据类型>[列级完整性约束]]
    ...
    [,<表级完整性约束>]);
```

建表的同时,通常还可以定义与该表有关的完整性约束,这些完整性约束被存入系统的数据字典。当用户操作表中数据时,由关系数据库管理系统自动检查该操作是否违背这些完整性约束。如果完整性约束涉及该表的多个属性列,则必须定义在表级上,否则既可以定义在列级也可以定义在表级。

本章以“学生选课”数据库为例来讲解 SQL 语句。为此,首先要定义一个“学生选课”模式 S-C-SC,该模式中包含以下三个表。

- ① “学生”表: Student(Sno, Sname, Ssex, Sbirthdate, Smajor)。
- ② “课程”表: Course(Cno, Cname, Ccredit, Cpno)。
- ③ “学生选课”表: SC(Sno, Cno, Grade, Semester, Teachingclass)。

注意:关系的主码加下划线表示。

这三个基本表的定义如下,其示例数据可参见二维码内容(即第 2 章图 2.1)。

[例 3.5] 建立一个“学生”表 Student。

```
CREATE TABLE Student
    (Sno CHAR(8) PRIMARY KEY, /* 列级完整性约束条件,Sno 是主码 */
    Sname VARCHAR(20) UNIQUE, /* Sname 取唯一值 */
    Ssex CHAR(6),
    Sbirthdate Date,
```



示例数据:学生选课
模式 3 个表的示例
数据

```
Smajor VARCHAR(40)
);
```

系统执行该 CREATE TABLE 语句后,就在数据库中建立一个新的空“学生”表 Student,并将该表及有关约束的定义存放在数据字典中。

[例 3.6] 建立一个“课程”表 Course。

```
CREATE TABLE Course
( Cno CHAR(5) PRIMARY KEY,          /* 列级完整性约束,Cno 是主码 */
  Cname VARCHAR(40) NOT NULL,       /* 列级完整性约束,Cname 不能取空值 */
  Ccredit SMALLINT,
  Cpno CHAR(5),
  FOREIGN KEY (Cpno) REFERENCES Course(Cno)
  /* 表级完整性约束,Cpno 是外码,被参照表是 Course,被参照列是 Cno */
);
```

参照表和被参照表可以是同一个表。Cpno 是 Cno 课程的直接先修课,并且只列出一门直接先修课^①。

[例 3.7] 建立“学生选课”表 SC。

```
CREATE TABLE SC
( Sno CHAR(8),
  Cno CHAR(5),
  Grade SMALLINT,                  /* 成绩 */
  Semester CHAR(5),               /* 开课学期 */
  Teachingclass CHAR(8),          /* 学生选修某一门课所在的教学班 */
  PRIMARY KEY(Sno,Cno),
  /* 主码由两个属性构成,必须作为表级完整性进行定义 */
  FOREIGN KEY(Sno) REFERENCES Student(Sno),
  /* 表级完整性约束,Sno 是外码,被参照表是 Student */
  FOREIGN KEY(Cno) REFERENCES Course(Cno)
  /* 表级完整性约束,Cno 是外码,被参照表是 Course */
);
```

2. 数据类型

关系模型中一个很重要的概念是域。每一个属性来自一个域,它的取值必须是域中的值。在 SQL 中域的概念用数据类型来实现。定义表的各个属性时需要指明其数据类型及长度。SQL 标准支持多种数据类型,表 3.4 列出了几种常用的数据类型。要注意,不同的关系数据库

^① 实际情况会复杂些,一门课程可能会有几门直接先修课,我们将在第 7 章数据库设计中进一步讲解。

管理系统中支持的数据类型会有细微差别，读者使用时要参考具体产品的手册。

表 3.4 SQL 标准常用的数据类型

数据类型	含义
CHAR(<i>n</i>), CHARACTER(<i>n</i>)	长度为 <i>n</i> 的定长字符串
VARCHAR(<i>n</i>), CHARACTER VARYING(<i>n</i>)	最大长度为 <i>n</i> 的变长字符串
CLOB	字符串大对象
BLOB	二进制大对象
INT, INTEGER	整数(4 字节), 取值范围是 $[-2\ 147\ 483\ 648, 2\ 147\ 483\ 647]$
SMALLINT	短整数(2 字节), 取值范围是 $[-32\ 768, 32\ 767]$
BIGINT	大整数(8 字节), 取值范围是 $[-2^{63}, 2^{63}-1]$
NUMERIC(<i>p</i> , <i>d</i>)	定点数, 由 <i>p</i> 位数字(不包括符号、小数点)组成, 小数点后面有 <i>d</i> 位数字
DECIMAL(<i>p</i> , <i>d</i>), DEC(<i>p</i> , <i>d</i>)	同 NUMERIC 类似, 但数值精度不受 <i>p</i> 和 <i>d</i> 的限制
REAL	取决于机器精度的单精度浮点数
DOUBLE PRECISION	取决于机器精度的双精度浮点数
FLOAT(<i>n</i>)	可选精度的浮点数, 精度至少为 <i>n</i> 位数字
BOOLEAN	逻辑布尔量
DATE	日期, 包含年、月、日, 格式为 YYYY-MM-DD
TIME	时间, 包含一日的时、分、秒, 格式为 HH:MM:SS
TIMESTAMP	时间戳类型
INTERVAL	时间间隔类型

一个属性选用哪种数据类型要根据实际情况来决定。一般从两个方面来考虑, 即取值范围和要做的运算。例如, 对于学生选修某一门课程的成绩(Grade)属性, 可以采用 CHAR(3)作为数据类型, 但考虑到要在成绩上做算术运算(如求平均成绩), 而 CHAR(*n*)数据类型不能进行算术运算, 所以采用整数作为数据类型。整数又有大整数、整数和短整数三种, 通常课程的成绩采用百分制, 所以选用短整数作为成绩的数据类型。

3. 模式与表

每一个基本表都属于某一个模式, 一个模式包含多个基本表。当定义基本表时一般可以有三种方法定义它所属的模式。例如, 在例 3.1 中定义了一个“学生选课”模式 S-C-SC。现在要在该模式中定义 Student、Course、SC 等基本表。

方法一：在表名中明显地给出模式名。

```
CREATE TABLE "S-C-SC". Student (...); /* Student 所属的模式是 S-C-SC */
CREATE TABLE "S-C-SC". Course (...); /* Course 所属的模式是 S-C-SC */
CREATE TABLE "S-C-SC". SC (...); /* SC 所属的模式是 S-C-SC */
```

方法二：在创建模式语句中同时创建表，如例 3.3 所示。

方法三：设置所属的模式，这样在创建表时表名中不必给出模式名。

当用户创建基本表(其他数据库对象也一样)时若没有指定模式，系统根据搜索路径来确定该对象所属的模式。

搜索路径包含一组模式列表，关系数据库管理系统会使用模式列表中第一个存在的模式作为数据库对象的模式名。若搜索路径中的模式名都不存在，系统将给出错误提示。

使用下面的语句可以显示当前的搜索路径：

```
SHOW SEARCH_PATH;
```

搜索路径的当前默认值是“\$user, PUBLIC”，表示首先搜索与用户名相同的模式名，如果该模式名不存在，则使用 PUBLIC 模式。

DBA 也可以设置搜索路径，例如：

```
SET SEARCH_PATH TO "S-C-SC", PUBLIC;
```

然后，定义基本表：

```
CREATE TABLE Student (...);
```

实际结果是建立了 S-C-SC. Student 基本表。因为关系数据库管理系统发现搜索路径中第一个模式名是 S-C-SC，就把该模式作为基本表 Student 所属的模式。

4. 修改基本表

随着应用环境和应用需求的变化，有时需要修改已建好的基本表。SQL 用 ALTER TABLE 语句修改基本表，其一般语法如下：

```
ALTER TABLE <表名>
[ADD[ COLUMN]<新列名><数据类型>[完整性约束]]
[ADD <表级完整性约束>]
[DROP[ COLUMN]<列名>[CASCADE| RESTRICT]]
[DROP CONSTRAINT<完整性约束名>[ RESTRICT | CASCADE ]]
[RENAME COLUMN <列名> TO <新列名>]
[ALTER COLUMN <列名> TYPE <数据类型>];
```

其中：

① <表名>是要修改的基本表。

② ADD 子句用于增加新列、新的列级完整性约束和新的表级完整性约束。

③ DROP COLUMN 子句用于删除表中的列，如果指定了 CASCADE 短语，则自动删除引用了该列的其他对象，比如视图；如果指定了 RESTRICT 短语，则如果该列被其他对象引用，关系数据库管理系统将拒绝删除该列。

④ DROP CONSTRAINT 子句用于删除指定的完整性约束。

⑤ RENAME COLUMN 子句用于修改列名。

⑥ ALTER COLUMN 子句用于修改列的数据类型。

[例 3.8] 向 Student 表增加“邮箱地址”列 Semail，其数据类型为字符型。

```
ALTER TABLE Student ADD Semail VARCHAR(30);
```

注意：不论基本表中原来是否已有数据，新增加的列一律为空值。

[例 3.9] 将 Student 表中出生日期 Sbirthdate 的数据类型由 DATE 型改为字符型。

```
ALTER TABLE Student ALTER COLUMN Sbirthdate TYPE VARCHAR(20);  
/* DATE 类型占用 19 字节，所以修改为 VARCHAR 类型时长度要大于或等于 19 */
```

[例 3.10] 增加课程名称必须取唯一值的约束条件。

```
ALTER TABLE Course ADD UNIQUE(Cname);
```

5. 删除基本表

当某个基本表不再使用时，可以使用 DROP TABLE 语句将其删除。其一般格式为：

```
DROP TABLE <表名>[RESTRICT | CASCADE];
```

同样，若选择 RESTRICT，则该表的删除有限制条件，即该表不能被其他表的约束所引用（如 CHECK、FOREIGN KEY 等约束），不能有视图，不能有触发器(trigger)，不能有存储过程或函数等。如果存在这些依赖该表的对象，则此表不能被删除。

若选择 CASCADE，则该表的删除没有限制条件。在删除基本表的同时，相关的依赖对象，都可能被一起删除。

该语句的默认选项是 RESTRICT。

[例 3.11] 删除 Student 表，选择 CASCADE。

```
DROP TABLE Student CASCADE;
```

执行该语句后，Student 基本表定义一旦被删除，不仅表中的数据和此表的定义将被删除，而且此表上建立的索引、约束、触发器等对象一般也都将被删除。有的关系数据库管理系统会同时删除在此表上建立的视图。如果欲删除的基本表被其他基本表所引用，则这些表相应的参照完整性约束也可能被级联删除。因此删除基本表的操作一定要格外小心。

[例 3.12] 删除 Student 表，若表上建有视图，选择 RESTRICT 时表不能删除；选择 CAS-

CADE 时可以删除表, 视图也自动被删除。

```
CREATE VIEW CS_Student /* 在 Student 表上建立计算机科学与技术专业的学生视图 */
AS
```

```
SELECT Sno, Sname, Ssex, Sbirthdate, Smajor
```

```
FROM Student
```

```
WHERE Smajor='计算机科学与技术';
```

```
DROP TABLE Student RESTRICT; /* 采用 RESTRICT 方式删除 Student 表 */
```

```
--ERROR: cannot drop table Student because other objects depend on it
```

```
/* 系统返回错误信息, 指出存在依赖该表的对象, 此表不能被删除 */
```

```
DROP TABLE Student CASCADE; /* 采用 CASCADE 方式删除 Student 表 */
```

```
--NOTICE: drop cascades to view CS_Student /* 系统返回提示, 此表上的视图也被删除 */
```

```
SELECT * FROM CS_Student; /* CS_Student 视图不存在 */
```

```
--ERROR: relation "CS_Student" does not exist
```

注意: 不同的数据库产品, 在遵循 SQL 标准的基础上, 其具体实现和处理 DROP TABLE 语句的策略会与标准有差别。所以, 在实际应用中一定要阅读相应的产品手册。



扩展阅读: 不同产品 DROP TABLE 处理策略比较

我们对比了 DROP TABLE 的 SQL 标准与 Kingbase ES、Oracle、MS SQL Server 三种数据库产品的不同处理策略。读者可以扫描二维码进行阅读。

3.2.3 索引的建立与删除

当表的数据量很大时, 查询操作会比较耗时。建立索引是加快查询速度的有效手段。数据库索引类似于图书的目录, 能快速定位到需要查询的内容。用户可以根据应用环境的需要在基本表上建立一个或多个索引, 以提供多种存取路径, 加快查找速度。

数据库索引有多种类型, 常见的索引结构包括顺序表 (sequential list) 索引、B+树索引、哈希索引 (hash index)、位图索引 (bitmap index) 等。顺序表索引是针对按指定属性值升序或降序存储的关系, 在该属性上建立一个顺序索引文件, 索引文件由属性值和相应的元组指针组成。B+树索引是将索引属性组织成 B+树形式, 其叶结点为属性值和相应的元组指针。B+树索引具有动态平衡的优点。哈希索引是建立若干个桶, 将索引属性按照其哈希函数值映射到相应桶中, 桶中存放索引属性值和相应的元组指针。哈希索引具有查找速度快的特点。位图索引是用位向量记录索引属性中可能出现的值, 每个位向量对应一个可能值。第 9 章中将详细介绍索引技术。

索引虽然能够加速数据库查询, 但需要占用一定的存储空间, 当基本表更新时, 索引也要进行相应的维护, 这些都会增加数据库的负担, 因此要根据实际应用的需要有选择地创建索引。

目前 SQL 标准中没有涉及索引,但商用关系数据库管理系统一般都支持索引机制,只是不同的关系数据库管理系统支持的索引类型不尽相同。

一般说来,建立与删除索引由数据库管理员或表的属主(owner,即建立表的人)负责完成。关系数据库管理系统在执行查询时会自动选择是否使用索引或使用哪个索引作为存取路径,用户不必也不能自主地选择索引。索引是关系数据库管理系统的内部实现技术,属于内模式的范畴。

1. 建立索引

在 SQL 语言中,建立索引使用 CREATE INDEX 语句,其一般格式如下:

```
CREATE[ UNIQUE][ CLUSTER] INDEX <索引名>  
ON <表名>(<列名>[<次序>][,<列名>[<次序>]]...);
```

其中,<表名>是要建索引的基本表的名称。索引可以建立在该表的一列或多列上,各列名之间用逗号分隔。每个<列名>后面还可以用<次序>指定索引值的排列次序,可选 ASC(升序)或 DESC(降序),默认值为 ASC。

UNIQUE 表明此索引的每一个索引值只对应唯一的数据记录。

CLUSTER 表示要建立的索引是聚簇索引。一张表上只能建立一个聚簇索引,多个有联系的表可以建立聚簇索引。有关聚簇索引将在第 7 章 7.5.2 小节关系模式存取方法选择中进一步介绍。

[例 3.13]为“学生选课”数据库中的 Student、Course 和 SC 三个表建立索引。其中 Student 表按学生姓名升序建唯一索引, Course 表按课程名升序建唯一索引, SC 表按学号升序和课程号降序建唯一索引(即先按照学号升序,对同一个学号再按课程号降序)。

```
CREATE UNIQUE INDEX Idx_StuSname ON Student(Sname);  
/* 保证了 Sname 取唯一值的约束 */  
CREATE UNIQUE INDEX Idx_CouCname ON Course(Cname);  
/* 加上 Cname 取唯一值的约束 */  
CREATE UNIQUE INDEX Idx_SCCno ON SC(Sno ASC,Cno DESC);
```

2. 修改索引

对于已经建立的索引,如果需要对其重新命名,可以使用 ALTER INDEX 语句。其一般格式如下:

```
ALTER INDEX <旧索引名> RENAME TO <新索引名>;
```

[例 3.14]将 SC 表的 Idx_SCCno 索引名改为 Idx_SCSnoCno。

```
ALTER INDEX Idx_SCCno RENAME TO Idx_SCSnoCno;
```

3. 删除索引

索引一经建立就由系统使用和维护,无须用户干预。建立索引是为了减少查询操作的时间。

间,但如果数据更新(增、删、改)频繁,系统会花费许多时间来维护索引,从而降低了查询效率。这时可以删除一些不必要的索引。

在 SQL 中,删除索引使用 DROP INDEX 语句,其一般格式如下:

DROP INDEX <索引名>;

[例 3.15] 删除 Student 表的 Idx_StuSname 索引。

DROP INDEX Idx_StuSname;

删除索引时,系统会同时从数据字典中删去有关该索引的描述。

3.2.4 数据字典

数据字典是关系数据库管理系统内部的一组系统表,它记录了数据库中所有的定义信息,包括关系模式定义、视图定义、索引定义、完整性约束定义、各类用户对数据库的操作权限、统计信息等。关系数据库管理系统在执行 SQL 的数据定义语句时,实际上就是把这些定义信息存入系统的数据字典以更新其中的相应信息。在进行查询处理和查询优化时,关系数据库管理系统要根据数据字典中的信息执行处理算法和优化算法,因此数据字典是关系数据库管理系统运行的重要依据。

3.3 数据查询

数据查询是数据库的核心操作。SQL 提供了 SELECT 语句进行数据查询,该语句具有灵活的使用方式和丰富的功能。

SELECT 语句的一般格式如下:

```
SELECT[ ALL|DISTINCT]<目标列表表达式>[别名][,<目标列表表达式>[别名]]...  
FROM <表名或视图名>[别名][,<表名或视图名>[别名]]... | (<SELECT 语句>)[AS]<别名>  
[WHERE <条件表达式>]  
[GROUP BY <列名 1>[HAVING <条件表达式>]]  
[ORDER BY <列名 2>[ASC|DESC]]  
[LIMIT <行数 1>[OFFSET <行数 2>]];
```

SELECT 语句的含义是,根据 WHERE 子句的条件表达式从 FROM 子句指定的基本表、视图或派生表中找出满足条件的元组,再按 SELECT 子句中的目标列表表达式选出元组中的属性值形成结果表。

如果有 GROUP BY 子句,则将结果按<列名 1>的值进行分组,该属性列值相等的元组为一个组,通常会在每组中作用聚集函数。如果 GROUP BY 子句带 HAVING 短语,则只有满足指定条件的组才予以输出。

如果有 ORDER BY 子句, 则结果表还要按<列名 2>的值的升序或降序排序。

如果有 LIMIT 子句, 则限制 SELECT 语句查询结果的数量为<行数 1>行, OFFSET <行数 2>表示在计算<行数 1>行前忽略<行数 2>行。OFFSET 子句可省略, 代表不忽略任何行。LIMIT 是和 GROUP BY、ORDER BY 并列的子句。

SELECT 语句既可以完成简单的单表查询, 也可以完成复杂的连接查询和嵌套查询。下面以“学生选课”数据库为例, 说明 SELECT 语句的各种用法。

3.3.1 单表查询

单表查询是指仅涉及一个表的查询。

1. 选择表中的若干列

选择表中的全部或部分列为关系代数的投影运算。

(1) 查询指定列

在很多情况下, 用户只对表中的一部分属性列感兴趣, 这时可以通过在 SELECT 子句的<目标列表表达式>中指定要查询的属性列。

[例 3.16] 查询全体学生的学号与姓名。

```
SELECT Sno,Sname FROM Student;
```

该语句的执行过程可以是这样的: 从 Student 表中取出一个元组, 取出该元组在属性 Sno 和 Sname 上的值, 形成一个新的元组作为输出。对 Student 表中的所有元组做相同的处理, 最后形成一个结果关系作为输出。

[例 3.17] 查询全体学生的姓名、学号、主修专业。

```
SELECT Sname,Sno,Smajor FROM Student;
```

<目标列表表达式>中各个列的先后顺序可以与表中的顺序不一致。用户可以根据应用的需要改变列的显示顺序。本例中先列出姓名, 再列出学号和主修专业。

(2) 查询全部列

将表中的所有属性列都选出来有两种方法, 一种方法就是在 SELECT 关键字后列出所有列名; 如果列的显示顺序与其在基表中的顺序相同, 也可以简单地将<目标列表表达式>指定为“*”。

[例 3.18] 查询全体学生的详细记录。

```
SELECT * FROM Student;
```

等价于

```
SELECT Sno,Sname,Ssex,Sbirthdate,Smajor FROM Student;
```

(3) 查询经过计算的值

SELECT 语句的<目标列表表达式>可以是算术表达式、字符串常量、函数等。

[例 3.19] 查询全体学生的姓名及其年龄。

查询要求中“年龄”不能在学生表中直接查到,需要通过计算得到。

如果使用 KingbaseES 数据库系统,其 SQL 语句如下:

```
SELECT Sname, (extract(year from current_date)-extract(year from Sbirthdate)) "年龄"
FROM Student; /* SELECT 语句的第 2 列是表达式,起了别名“年龄” */
```

extract(year from current_date) 是 Kingbase 提供的内置函数,其功能是获取当前日期的年份,extract(year from Sbirthdate) 为获取表中出生日期的年份。

学生的出生年份是从属性列“出生日期”中计算得到的,不同的数据库管理系统提供的内置函数不同,语法也不尽相同。所以,读者在书写 SQL 语句时一定要参考所使用的产品手册。本书对部分数据库管理系统产品的主要函数进行了整理,放在第 8 章的二维码 8.1 中,供读者参考。

本例查询结果如图 3.3 所示。

注意: CURRENT_DATE 为实际上机实验的日期,读者的实际操作结果与这里的输出结果会有出入。此外,本例计算年龄仅考虑年份,如果以具体出生日期为界限的话就可能有误差了。本章后面的示例也均作此考虑。

用户还可以通过指定别名来改变查询结果的列标题,这对于含算术表达式、常量、函数名的目标列表达式尤为有用。在例 3.19 中就定义了“年龄”别名。

用户也可以在 SELECT 语句中加上说明含义的一个列标题。例如在例 3.20 中加上'Date of Birth:',表明右边一列是出生日期。

[例 3.20] 查询全体学生的姓名、出生日期和主修专业。

```
SELECT Sname,'Date of Birth:',Sbirthdate,Smajor
FROM Student;
```

查询结果如图 3.4 所示。

Sname	年龄
李勇	21
刘晨	22
王敏	20
张立	21
陈新奇	20
赵明	21
王佳佳	20

图 3.3 例 3.19 查询结果

Sname	Date of Birth:	Sbirthdate	Smajor
李勇	Date of Birth:	2000-3-8	信息安全
刘晨	Date of Birth:	1999-9-1	计算机科学与技术
王敏	Date of Birth:	2001-8-1	计算机科学与技术
张立	Date of Birth:	2000-1-8	计算机科学与技术
陈新奇	Date of Birth:	2001-11-1	信息管理与信息系统
赵明	Date of Birth:	2000-6-12	数据科学与大数据技术
王佳佳	Date of Birth:	2001-12-7	数据科学与大数据技术

图 3.4 例 3.20 查询结果

2. 选择表中的若干元组

(1) 消除取值重复的行

两个本来并不完全相同的元组在投影到指定的某些列上后，可能会变成相同的行。可以用 DISTINCT 消除它们。

[例 3.21] 查询选修了课程的学生学号。

```
SELECT Sno
FROM SC;
```

该查询结果里包含了许多重复的行。如想去掉结果表中的重复行，必须指定 DISTINCT：

```
SELECT DISTINCT Sno
FROM SC;
```

查询结果如图 3.5 所示。

如果没有指定 DISTINCT 关键词，则默认值为 ALL，即保留结果表中取值重复的行。

```
SELECT Sno
FROM SC;
```

等价于

```
SELECT ALL Sno
FROM SC;
```

Sno
20180001
20180002
20180003
20180004
20180005

图 3.5 例 3.21 查询结果

(2) 查询满足条件的元组

查询满足指定条件的元组可以通过 WHERE 子句实现。WHERE 子句常用的查询条件如表 3.5 所示。

表 3.5 常用的查询条件

查询条件	谓词
比较	=, >, <, >=, <=, !=, <>, !>, !<; NOT+上述比较运算符
确定范围	BETWEEN AND, NOT BETWEEN AND
确定集合	IN, NOT IN
字符匹配	LIKE, NOT LIKE
空值	IS NULL, IS NOT NULL
多重条件(逻辑运算)	AND, OR, NOT

① 比较大小

用于进行比较的运算符一般包括=(等于), >(大于), <(小于), >=(大于或等于), <=

(小于或等于), != 或 <> (不等于), !> (不大于), !< (不小于)。

[例 3.22] 查询主修计算机科学与技术专业全体学生的姓名。

```
SELECT Sname
FROM Student
WHERE Smajor='计算机科学与技术'; /* 字符串常数要用单引号(英文符号)括起来 */
```

关系数据库管理系统执行该查询的一种可能过程是：对 Student 表进行全表扫描，取出一个元组，检查该元组在 Smajor 列的值是否等于“计算机科学与技术”，如果相等，则取出 Sname 列的值形成一个新的元组输出；否则跳过该元组，取下一个元组。重复该过程，直到处理完 Student 表的所有元组。

如果全校有数万个学生，主修计算机科学与技术专业的学生人数约是全校学生的 9%，则可以在 Student 表的 Smajor 列上建立索引，系统会利用该索引找出 Smajor='计算机科学与技术'的元组，从中取出 Sname 列值形成结果关系。这就避免了对 Student 表的全表扫描，加快了查询速度。

注意：如果全校学生较少，索引查找不一定能提高查询效率，系统仍会使用全表扫描。这由查询优化器按照算法规则或估计执行代价来做出选择。

[例 3.23] 查询 2000 年及 2000 年后出生的所有学生的姓名及其性别。

```
SELECT Sname, Ssex
FROM Student
WHERE extract(year from Sbirthdate) >= 2000;
/* 函数 extract(year from Sbirthdate) 用于从出生日期中抽取年份 */
```

[例 3.24] 查询考试成绩不及格的学生的学号。

```
SELECT DISTINCT Sno
FROM SC
WHERE Grade < 60;
```

这里使用了 DISTINCT 短语，当一个学生有多门课程不及格时，其学号也只列一次。

② 确定范围

谓词 BETWEEN...AND... 和 NOT BETWEEN...AND... 可以用来查找属性值在 (或不在) 指定范围内的元组，其中 BETWEEN 后是范围的下限 (即低值)，AND 后是范围的上限 (即高值)。

[例 3.25] 查询年龄在 20~23 岁 (包括 20 岁和 23 岁) 之间的学生的姓名、出生日期和主修专业。

```
SELECT Sname, Sbirthdate, Smajor
FROM Student
```

```
WHERE extract(year from current_date) - extract(year from Sbirthdate)
      BETWEEN 20 AND 23;
```

[例 3.26] 查询年龄不在 20~23 岁范围内的学生的姓名、出生日期和主修专业。

```
SELECT Sname,Sbirthdate,Smajor
FROM Student
WHERE extract(year from current_date) - extract(year from Sbirthdate)
      NOT BETWEEN 20 AND 23;
```

③ 确定集合

谓词 IN 可以用来查找属性值属于指定集合的元组。

[例 3.27] 查询计算机科学与技术专业和信息安全专业的学生的姓名及性别。

```
SELECT Sname,Ssex
FROM Student
WHERE Smajor IN ( '计算机科学与技术','信息安全' );
```

与 IN 相对的谓词是 NOT IN，用于查找属性值不属于指定集合的元组。

[例 3.28] 查询非计算机科学与技术专业和信息安全专业的学生的姓名和性别。

```
SELECT Sname,Ssex
FROM Student
WHERE Smajor NOT IN ( '计算机科学与技术','信息安全' );
```

④ 字符匹配

谓词 LIKE 可以用来进行字符串的匹配。其一般语法格式如下：

[NOT]LIKE '<匹配串>' [ESCAPE '<换码字符>']

即查找指定的属性列值与<匹配串>相匹配的元组。<匹配串>可以是一个完整的字符串，也可以含有通配符%和_。其中：

a. %(百分号)代表任意长度(长度可以为0)的字符串。例如“a%b”表示以a开头，以b结尾的任意长度的字符串。如acb、addgb、ab等都满足该匹配串。

b. _(下划线)代表任意单个字符。例如“a_b”表示以a开头，以b结尾的长度为3的任意字符串，如acb、afb等都满足该匹配串。

[例 3.29] 查询学号为 20180003 的学生的详细情况。

```
SELECT *
FROM Student
WHERE Sno LIKE '20180003';
```

等价于

```
SELECT *  
FROM Student  
WHERE Sno='20180003';
```

如果 LIKE 后面的匹配串中不含通配符,则可以用“=”(等于)运算符取代 LIKE 谓词,用“!=”或“<>”(不等于)运算符取代 NOT LIKE 谓词。

[例 3.30] 查询所有姓刘的学生的姓名、学号和性别。

```
SELECT Sname,Sno,Ssex  
FROM Student  
WHERE Sname LIKE '刘%';
```

[例 3.31] 查询 2018 级学生的学号和姓名。

```
SELECT Sno,Sname  
FROM Student  
WHERE Sno LIKE '2018%';      /* 学号的数据类型是字符,用字符匹配 */
```

[例 3.32] 查询课程号为 81 开头,最后一位是 6 的课程名称和课程号。

```
SELECT Cname,Cno  
FROM Course  
WHERE Cno LIKE '81__6';      /* 注意课程关系中课程号为固定长度,占 5 个字符大小 */
```

[例 3.33] 查询所有不姓刘的学生的姓名、学号和性别。

```
SELECT Sname,Sno,Ssex  
FROM Student  
WHERE Sname NOT LIKE '刘%';
```

如果用户要查询的字符串本身就含有通配符%或_,这时就要使用 ESCAPE '<换码字符>'短语对通配符进行转义了。

[例 3.34] 查询 DB_Design 课程的课程号和学分。

```
/* 假设已插入了一条数据库设计课程元组,课程名称为 DB_Design */  
SELECT Cno,Ccredit  
FROM Course  
WHERE Cname LIKE 'DB\_Design' ESCAPE '\';
```

“ESCAPE ‘\’”表示“\”为换码字符。这样匹配串中紧跟在“\”后面的字符“_”不再具有通配符的含义,转义为普通的“_”字符。

[例 3.35] 查询以“DB_”开头,且倒数第三个字符为 i 的课程的具体情况。

```
SELECT *
```

```
FROM Course
WHERE Cname LIKE 'DB\__%i\_ ' ESCAPE '\';
```

这里的匹配串为'DB__%i_ ' ESCAPE '\'. 第一个“_”前有换码字符“\”，所以它被转义为普通的“_”字符。而“i”后的两个“_”的前面均没有换码字符“\”，所以它们仍作为通配符。

⑤ 涉及空值的查询

[例 3.36] 某些学生选修课程后没有参加考试，所以有选课记录但没有考试成绩。查询缺少成绩的学生的学号和相应的课程号。

```
SELECT Sno,Cno
FROM SC
WHERE Grade IS NULL;          /* 分数 Grade 是空值 */
```

注意这里的“IS”不能用等号(=)代替。

[例 3.37] 查所有有成绩的学生的学号和选修的课程号。

```
SELECT Sno,Cno
FROM SC
WHERE Grade IS NOT NULL;
```

⑥ 多重条件查询

逻辑运算符 AND 和 OR 可用来连接多个查询条件。AND 的优先级高于 OR，但用户可以用括号改变优先级。

[例 3.38] 查询主修计算机科学与技术专业，在 2000 年(包括 2000 年)以后出生的学生的学号、姓名和性别。

```
SELECT Sno,Sname,Ssex
FROM Student
WHERE Smajor='计算机科学与技术' AND extract(year from Sbirthdate)>=2000;
```

在例 3.27 中的 IN 谓词实际上是多个 OR 运算符的缩写，因此该例中的查询也可以用 OR 运算符写成如下等价形式：

```
SELECT Sname,Ssex
FROM Student
WHERE Smajor='计算机科学与技术' OR Smajor='信息安全';
```

3. ORDER BY 子句

用户可以用 ORDER BY 子句对查询结果按照一个或多个属性列的升序(ASC)或降序(DESC)排列，默认值为升序。

[例 3.39] 查询选修了 81003 号课程的学生的学号及其成绩，查询结果按分数的降序排列。

```
SELECT Sno,Grade
FROM SC
WHERE Cno='81003'
ORDER BY Grade DESC;
```

对于空值,排序时显示的次序由具体系统实现来决定。例如按升序排,含空值的元组最后显示;按降序排,含空值的元组则最先显示。各个系统的实现可以不同,请读者参考产品手册。

[例 3.40] 查询全体学生选修课程情况,查询结果先按照课程号升序排列,同一课程中按成绩降序排列。

```
SELECT *
FROM SC
ORDER BY Cno,Grade DESC;

/* 排序字段的说明默认为升序 ASC,可以省略,而 DESC 需要明确写出来。 */
```

注意: 这里是对查询结果进行排序,不影响基本表中数据的存储状况。

4. 聚集函数

为了进一步方便用户,增强检索功能,SQL 提供了许多聚集函数,主要有:

COUNT(*)	统计元组个数
COUNT([DISTINCT ALL]<列名>)	统计一列中值的个数
SUM([DISTINCT ALL]<列名>)	计算一列值的总和(此列必须是数值型)
AVG([DISTINCT ALL]<列名>)	计算一列值的平均值(此列必须是数值型)
MAX([DISTINCT ALL]<列名>)	求一列值中的最大值
MIN([DISTINCT ALL]<列名>)	求一列值中的最小值

如果指定 DISTINCT 短语,则表示在计算时要取消指定列中的重复值。如果不指定 DISTINCT 短语或指定 ALL 短语(ALL 为默认值),则表示不取消重复值。

当遇到空值时,聚集函数中除 COUNT(*) 函数外,其他函数都跳过空值而只处理非空值。COUNT(*) 是对元组进行计数,某个元组的一个或部分列取空值不影响其统计结果。

[例 3.41] 查询学生总人数。

```
SELECT COUNT(*)
FROM Student;
```

[例 3.42] 查询选修了课程的学生人数。

```
SELECT COUNT(DISTINCT Sno)
FROM SC;
```

学生每选修一门课,在 SC 中都有一条相应的记录。一个学生可选修多门课程,为避免重

复计算学生人数，必须在 COUNT 函数中用 DISTINCT 短语。

[例 3.43] 计算选修 81001 号课程的学生平均成绩。

```
SELECT AVG(Grade)
FROM SC
WHERE Cno='81001';
```

[例 3.44] 查询选修 81001 号课程的学生最高分数。

```
SELECT MAX(Grade)
FROM SC
WHERE Cno='81001';
```

[例 3.45] 查询学号为 20180003 的学生选修课程的总学分数。

```
SELECT SUM(Ccredit)
FROM SC, Course
WHERE Sno='20180003' AND SC.Cno=Course.Cno;
```

注意：WHERE 子句不能直接用聚集函数作为条件表达式。聚集函数只能用于 SELECT 子句和 GROUP BY 子句中的 HAVING 短语。

5. GROUP BY 子句

GROUP BY 子句将查询结果按某一列或多列的值分组，值相等的为一组，如果是多列，则先对第一列的值分组，然后对每一组中的值按照第二列值分组，以此类推。结果集中如果有重复的列组值，则将其合并为一行输出。

例如，若对 A, B 两列进行如下分组：

```
(1,2)
(1,2)
(1,3)
(2,4)
```

那么，最后的分组结果为

```
(1,2)
(1,3)
(2,4)
```

对查询结果分组的目的之一是细化聚集函数的作用对象。如果未对查询结果分组，聚集函数将作用于整个查询结果，如前面的例 3.41~3.45。分组后聚集函数将作用于每一个组，即每组都有一个聚集函数值。

[例 3.46] 求各个课程号及选修该课程的人数。

```
SELECT Cno,COUNT(Sno)
FROM SC
GROUP BY Cno;
```

该语句对查询结果按 Cno 的值分组,所有具有相同 Cno 值的元组为一组,然后对每一组应用聚集函数 COUNT 进行计算,以求得该组的学生人数。

查询结果可能如图 3.6 所示。

如果分组后还要求按一定的条件对这些组进行筛选,最终只输出满足指定条件的组,则可以使用 HAVING 短语指定筛选条件。

[例 3.47] 查询 2019 年第 2 学期选修课程数超过 10 门的学生学号。

Cno	COUNT(Sno)
81001	42
81002	44
81003	44
81004	33
81005	48
81006	45
81007	48
81008	39

图 3.6 例 3.46 查询结果

```
SELECT Sno
FROM SC
WHERE Semester='20192'          /* 先求出 2019 年第 2 学期选课的所有学生 */
GROUP BY Sno                    /* 用 GROUP BY 子句按 Sno 进行分组 */
HAVING COUNT(*) > 10;           /* 用聚集函数 COUNT 对每一组进行计数 */
```

HAVING 短语给出了选择组的条件,只有满足条件(即一个组中元组个数>10,表示此学生 2019 年第 2 学期选修的课超过 10 门)的组才会被选出来。

WHERE 子句与 HAVING 短语的区别在于作用对象不同。WHERE 子句作用于基本表或视图,从中选择满足条件的元组。HAVING 短语作用于组,从中选择满足条件的组。

关系数据库管理系统在处理 SQL 语句时,先处理 WHERE 子句,根据条件选出合格元组,生成一个临时表,本例中选出了 2019 年第 2 学期选课的所有学生,再使用 GROUP BY 子句对临时表按照学号分组,最后用 HAVING 条件中的聚集函数 COUNT 对每一组进行计数,选出 COUNT > 10 的组,输出该组的 Sno。

[例 3.48] 查询平均成绩大于或等于 90 分的学生学号和平均成绩。

```
SELECT Sno,AVG(Grade)
FROM SC
GROUP BY Sno
HAVING AVG(Grade) >= 90;
```

下面的语句是错误的:

```
SELECT Sno,AVG(Grade)
FROM SC
WHERE AVG(Grade) >= 90
GROUP BY Sno;
```

因为 WHERE 子句不能用聚集函数作为条件表达式。

6. LIMIT 子句

LIMIT 子句用于限制 SELECT 语句查询结果的(元组)数量,其一般形式如下:

LIMIT <行数 1>[OFFSET <行数 2>];

语义是取<行数 1>,忽略前<行数 2>行,作为查询结果数据。OFFSET 可以省略,代表不忽略任何行。

实际应用中 LIMIT 子句经常和 ORDER BY 子句一起使用。

[例 3.49] 查询选修数据库系统概论课程且成绩排名在前 10 名的学生的学号。

```
SELECT Sno
FROM SC, Course
WHERE Course. Cname='数据库系统概论' AND SC. Cno=Course. Cno
ORDER BY Grade DESC
LIMIT 10;           /* 取前 10 行数据为查询结果 */
```

[例 3.50] 查询平均成绩排名在第 3~7 名的学生的学号和平均成绩。

```
SELECT Sno,AVG( Grade)
FROM SC
GROUP BY Sno
ORDER BY AVG( Grade) DESC
LIMIT 5 OFFSET 2;  /* 取 5 行数据,忽略前 2 行,之后为查询结果数据 */
```

3.3.2 连接查询

前面的查询都是针对一个表进行的。若一个查询同时涉及两个以上的表,则称之为连接查询。连接查询是关系数据库中最常用的查询,包括等值连接查询、非等值连接查询、自然连接查询、自身连接查询、外连接查询、复合条件连接查询、多表连接查询等。

连接的概念在第 2 章做初步讲解,读者可以复习一下 2.4.2 小节中的有关概念。

1. 等值与非等值连接查询

连接查询的 WHERE 子句中用来连接两个表的条件称为连接条件或连接谓词,其一般格式如下:

[<表名 1>.]<列名 1><比较运算符>[<表名 2>.]<列名 2>

其中,比较运算符主要有=、>、<、>=、<=、!= (或<>)等。

此外,连接谓词还可以使用下面的形式:

[<表名 1>.]<列名 1> BETWEEN [<表名 2>.]<列名 2> AND [<表名 2>.]<列名 3>

比较运算符为“=”的连接查询称为等值连接查询。使用其他比较运算符的连接查询则称为非等值连接查询。

连接谓词中的“列名”称为连接字段。连接条件中的各连接字段类型必须是可比的，但名字不必相同。

[例 3.51] 查询每个学生及其选修课程的情况。

学生情况存放在 Student 表中，学生选课情况存放在 SC 表中，所以本查询实际上涉及 Student 与 SC 两个表。这两个表之间的联系是通过公共属性 Sno 实现的。

```
SELECT Student. *, SC. *
FROM Student, SC
WHERE Student. Sno=SC. Sno    /* 将 Student 与 SC 中同一学生的元组连接起来 */
```

假设 Student 表、SC 表的数据如第 2 章图 2.1 所示，本查询的执行结果如图 3.7 所示。

Student. Sno	Sname	Ssex	Sbirthdate	Smajor	SC.Sno	Cno	Grade	Semester	Teachingclass
20180001	李勇	男	2000-3-8	信息安全	20180001	81001	85	20192	81001-01
20180001	李勇	男	2000-3-8	信息安全	20180001	81002	96	20201	81002-01
20180001	李勇	男	2000-3-8	信息安全	20180001	81003	87	20202	81003-01
20180002	刘晨	女	1999-9-1	计算机科学与技术	20180002	81001	80	20192	81001-02
20180002	刘晨	女	1999-9-1	计算机科学与技术	20180002	81002	98	20201	81002-01
20180002	刘晨	女	1999-9-1	计算机科学与技术	20180002	81003	71	20202	81003-02
...	...	...	...	...	...	...	...	...	...

图 3.7 例 3.51 查询结果示例

说明：为节省篇幅，图 3.7 没有列出查询的全部结果，仅列出了“李勇”和“刘晨”两个学生及其选课情况。

关系数据库管理系统执行该连接操作的一种可能过程是：首先在表 Student 中找到第一个元组，然后从头开始扫描 SC 表，逐一查找与 Student 第一个元组的 Sno 相等的 SC 元组，

找到后就将 Student 中的第一个元组与该元组拼接起来, 形成结果表中的一个元组。SC 全部查找完后, 再找 Student 中第二个元组, 然后再从头开始扫描 SC, 逐一查找满足连接条件的元组, 找到后就将 Student 中的第二个元组与该元组拼接起来, 形成结果表中的一个元组。重复上述操作, 直到 Student 中的全部元组都处理完毕为止(见图 3.8 所示)。这就是嵌套循环连接算法的基本思想。

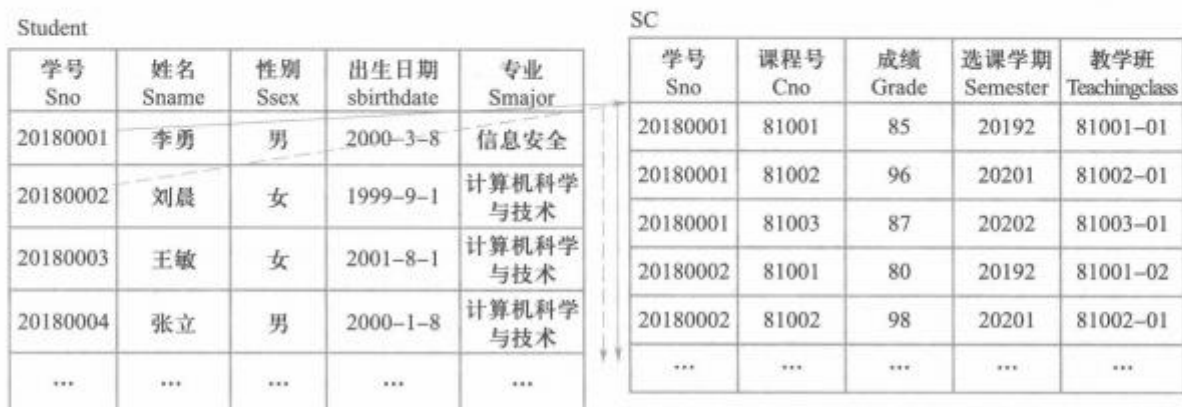


图 3.8 关系数据库管理系统执行连接操作的过程示意图

如果在 SC 表的 Sno 上建立了索引, 就不需要每次都全表扫描 SC 表了, 而是根据 Sno 值通过索引找到相应的 SC 元组。用索引查询 SC 中满足条件的元组一般会比全表扫描快。

2. 自然连接查询

把结果表目标列中重复的属性列去掉的等值连接查询则为自然连接查询, 如例 3.52 中去掉了 SC 表中的 Sno。

[例 3.52] 查询每个学生的学号、姓名、性别、出生日期、主修专业及该学生选修课程的课程号与成绩。

```
SELECT Student.Sno,Sname,Ssex,Sbirthdate,Smajor,Cno,Grade
FROM Student,SC
WHERE Student.Sno=SC.Sno;
```

本例中, 由于 Sname、Ssex、Sbirthdate、Smajor、Cno 和 Grade 属性列在 Student 表与 SC 表中是唯一的, 因此引用时可以去掉表名前缀; 而 Sno 在两个表都出现了, 因此 SELECT 子句和 WHERE 子句在引用时必须加上表名前缀。

3. 复合条件连接查询

使用一条 SQL 语句可以同时完成选择和连接查询, 这时 WHERE 子句是由连接谓词和选择谓词组成的复合条件。WHERE 子句中有多个条件的连接查询, 称为复合条件连接查询。

[例 3.53] 查询选修 81002 号课程且成绩在 90 分以上的所有学生的学号和姓名。

```
SELECT Student.Sno,Sname
```

```

FROM Student,SC
WHERE Student.Sno=SC.Sno AND          /* 连接谓词 */
      SC.Cno='81002' AND SC.Grade>90; /* 其他选择条件 */

```

该查询的一种优化(高效)的执行过程是,先从 SC 中挑选出 Cno='81002'并且 Grade>90 的元组形成一个中间关系,再和 Student 中满足连接条件的元组进行连接得到最终的结果关系。

4. 自身连接查询

连接操作不仅可以在两个表之间进行,也可以是一个表与其自身进行连接,此时的连接查询称为表的**自身连接查询**。

[例 3.54] 查询每一门课的间接先修课(即先修课的先修课)。

先来分析一下,题目要求查询每一门课程的先修课的先修课,在 Course 表中只有每门课的直接先修课的课程号 Cpno,而没有先修课的先修课课程号。要查询每一门课的间接先修课,必须先对一门课找到其直接先修课 Cpno,再按此先修课的课程号查找它的先修课程。这相当于将 Course 表与其自身连接后,取第一个副本的课程号与第二个副本的先修课号作为目标列中的属性。

具体写 SQL 语句时,为清楚起见,可以为 Course 表取两个别名:FIRST 和 SECOND,也可以在考虑问题时就把 Course 表想成是两个完全一样的表,一个是 FIRST 表,另一个是 SECOND 表,然后把这两个表连接。这就是 Course 表与其自身连接。完成该查询的 SQL 语句如下:

```

SELECT FIRST.Cno,SECOND.Cpno
FROM Course FIRST,Course SECOND
WHERE FIRST.Cpno=SECOND.Cno and SECOND.Cpno IS NOT NULL;

```

在 FROM 子句中为 Course 表定义了两个不同的别名,这样就可以分别在 SELECT 子句和 WHERE 子句中的属性名前加这两个别名来进行区分(见图 3.9 所示)。

FIRST表(Course表)

课程号 Cno	课程名 Cname	学分 Ccredit	先修课 Cpno
81001	程序设计基础与C语言	4	
81002	数据结构	4	81001
81003	数据库系统概论	4	81002
81004	信息系统概论	4	81003
81005	操作系统	4	81001
81006	Python语言	3	81002
81007	离散数学	4	
81008	大数据技术概论	4	81003

SECOND表(Course表)

课程号 Cno	课程名 Cname	学分 Ccredit	先修课 Cpno
81001	程序设计基础与C语言	4	
81002	数据结构	4	81001
81003	数据库系统概论	4	81002
81004	信息系统概论	4	81003
81005	操作系统	4	81001
81006	Python语言	3	81002
81007	离散数学	4	
81008	大数据技术概论	4	81003

图 3.9 自身连接查询示例

查询结果如图 3.10 所示。

Cno	Cpno
81003	81001
81004	81002
81006	81001
81008	81002

图 3.10 例 3.54 查询结果

注意：在 Course 表中只列出 Cno 课程的一门直接先修课 Cpno，这样才能保证 Cno 是主码。实际情况会复杂些，一门课程可能会有几门直接先修课。这时就需要重新设计 S-C-SC 模式，有关方法将在第二篇设计与应用开发篇中讲解。

5. 外连接查询

在第 2 章 2.4.2 小节也讲解过外连接的有关概念。

在通常的连接操作中，只有满足连接条件的元组才能作为结果输出。如例 3.51 的结果表中就不会有学号为 20180006 和 20180007 的两个学生的信息，原因在于他们还没有选课，在 SC 表中没有相应的元组，导致 Student 中这些元组在连接时被舍弃了（这些被舍弃的元组称为悬浮元组）。

但是，有时我们想以 Student 表为主体列出每个学生的基本情况及其选课情况，若某个学生没有选课，则只输出其基本情况的数据，而把选课信息填为空值 NULL，这时就需要使用外连接查询。

[例 3.55]

```
SELECT Student.Sno,Sname,Ssex,Sbirthdate,Smajor,Cno,Grade
FROM Student LEFT OUTER JOIN SC ON (Student.Sno=SC.Sno);
```

查询结果如图 3.11 所示。

左外连接列出 FROM 子句中左边关系（如本例 Student）所有的元组，右外连接列出 FROM 子句中右边关系中（如本例 SC）所有的元组。

6. 多表连接查询

连接查询除了可以是两表连接查询、一个表与其自身连接查询外，还可以是两个以上的表进行连接查询，后者通常称为多表连接查询。

[例 3.56] 查询每个学生的学号、姓名、选修的课程名及成绩。

本例查询涉及 3 个表，存放学生学号和姓名的 Student 表、存放学生选课成绩的 SC 表和存放课程名的 Course 表。完成该查询的 SQL 语句如下：

```
SELECT Student.Sno,Sname,Cname,Grade
FROM Student,SC,Course
WHERE Student.Sno=SC.Sno AND SC.Cno=Course.Cno;
```

Student.Sno	Sname	Ssex	Sbirthdate	Smajor	Cno	Grade
20180001	李勇	男	2000-3-8	信息安全	81001	85
20180001	李勇	男	2000-3-8	信息安全	81002	96
20180001	李勇	男	2000-3-8	信息安全	81003	87
20180002	刘晨	女	1999-9-1	计算机科学与技术	81001	80
20180002	刘晨	女	1999-9-1	计算机科学与技术	81002	98
20180002	刘晨	女	1999-9-1	计算机科学与技术	81003	71
20180003	王敏	女	2001-8-1	计算机科学与技术	81001	81
20180003	王敏	女	2001-8-1	计算机科学与技术	81002	76
20180004	张立	男	2000-1-8	计算机科学与技术	81001	56
20180004	张立	男	2000-1-8	计算机科学与技术	81002	97
20180005	陈新奇	男	2001-11-1	信息管理与信息系统	81003	68
20180006	赵明	男	2000-6-12	数据科学与大数据技术	NULL	NULL
20180007	王佳佳	女	2001-12-7	数据科学与大数据技术	NULL	NULL

图 3.11 例 3.55 查询结果

关系数据库管理系统在执行多表连接查询时，通常是先进行两个表的连接查询，再将其结果与第三个表进行连接查询。本例的一种可能的执行方式是，先将 Student 表与 SC 表进行连接查询，得到每个学生的学号、姓名、所选课程号和相应的成绩，然后再将其与 Course 表进行连接查询，得到最终结果。

关系数据库管理系统执行连接查询时，多个表连接的次序会影响执行效率。这是关系数据库管理系统查询优化算法要考虑的问题，即通过代价估算确定实际执行连接的次序。

3.3.3 嵌套查询

在 SQL 中，一个 SELECT-FROM-WHERE 语句称为一个查询块。将一个查询块嵌套在另一个查询块的 WHERE 子句或 HAVING 短语的条件中的查询称为嵌套查询(nested query)。例如，查询选修 81003 号课程的所有学生姓名，其 SQL 语句如下：

```
SELECT Sname           /* 外层查询或父查询 */
FROM Student
WHERE Sno IN
    (SELECT Sno         /* 内层查询或子查询 */
     FROM SC
     WHERE Cno='81003');
```

本例中，下层查询块 SELECT Sno FROM SC WHERE Cno='201803'是嵌套在上层查询块 SE-

LECT Sname FROM Student 的 WHERE 条件中的。上层的查询块称为外层查询或父查询，下层查询块称为内层查询或子查询。

SQL 语言允许多层嵌套查询，即一个子查询中还可以嵌套其他子查询。需要特别指出的是，子查询的 SELECT 语句中不能使用 ORDER BY 子句，该子句只能对最终查询结果排序。

嵌套查询使得用户可以用多个简单查询构成复杂的查询，从而增强 SQL 的查询能力。以层层嵌套的方式来构造程序正是 SQL 中“结构化”的含义所在。

1. 带有 IN 谓词的子查询

在嵌套查询中，子查询的结果往往是一个集合，所以谓词 IN 是嵌套查询中最经常使用的谓词。

[例 3.57] 查询与“刘晨”在同一个主修专业的学生学号、姓名和主修专业。

先分步来完成此查询，然后再构造嵌套查询。

① 确定“刘晨”的主修专业名。

```
SELECT Smajor
FROM Student
WHERE Sname='刘晨';
```

结果为计算机科学与技术。

② 查找所有主修计算机科学与技术专业的学生。

```
SELECT Sno,Sname,Smajor
FROM Student
WHERE Smajor='计算机科学与技术';
```

查询结果如图 3.12 所示。

Sno	Sname	Smajor
20180002	刘晨	计算机科学与技术
20180003	王敏	计算机科学与技术
20180004	张立	计算机科学与技术

图 3.12 例 3.57 查询结果

分步写查询毕竟比较麻烦，上述查询可以用嵌套查询来实现，即将第一步查询嵌入第二步查询的条件，构造嵌套查询语句如下：

```
SELECT Sno,Sname,Smajor /* 例 3.57 的解法一 */
FROM Student
WHERE Smajor IN
    (SELECT Smajor
```

```
FROM Student
WHERE Sname='刘晨');
```

本例中，子查询的查询条件不依赖于父查询，称为不相关子查询。

关系数据库管理系统求解该查询时，实际上也是由里向外分步去做的。即先处理子查询，确定“刘晨”的主修专业，得到结果“计算机科学与技术”，子查询的结果用于建立其父查询 WHERE 子句的查找条件，得到如下的语句：

```
SELECT Sno,Sname,Smajor
FROM Student
WHERE Smajor IN ('计算机科学与技术');
```

然后执行该语句。

本例中的查询也可以用自身连接来完成：

```
SELECT S1.Sno,S1.Sname,S1.Smajor /* 例 3.57 的解法二 */
FROM Student S1,Student S2
WHERE S1.Smajor=S2.Smajor AND S2.Sname='刘晨';
```

[例 3.58] 查询选修了课程名为“信息系统概论”的学生的学号和姓名。

本查询涉及学号、姓名和课程名三个属性。学号和姓名存放在 Student 表中，课程名存放在 Course 表中，但 Student 与 Course 两个表之间没有直接联系，必须通过 SC 表建立它们之间的联系。所以本查询实际上涉及三个关系。

SELECT Sno,Sname	③ 在 Student 关系中
FROM Student	取出 Sno 和 Sname
WHERE Sno IN	
(SELECT Sno	② 在 SC 关系中找到选修
FROM SC	81004 号课程的学生学号
WHERE Cno IN	
(SELECT Cno	① 在 Course 关系中找到
FROM Course	“信息系统概论”的课程号，
WHERE Cname='信息系统概论'	结果为 81004
)	
);	

本查询同样可以用连接查询实现：

```
SELECT Student.Sno,Sname
FROM Student,SC,Course
WHERE Student.Sno=SC.Sno AND
```

```
SC. Cno=Course. Cno AND
Course. Cname='信息系统概论';
```

有些嵌套查询可以用连接查询替代,有些是不能替代的。从例 3.57 和例 3.58 可以看到,查询涉及多个关系时,用嵌套查询逐步求解层次清楚,易于构造,具有结构化程序设计的优点。但是,有的查询使用连接查询可能效率更高些。所以在实际应用中,需要根据关系数据库管理系统的查询处理算法效率来选择使用哪种 SQL 语句。

2. 带有比较运算符的子查询

带有比较运算符的子查询是指父查询与子查询之间用比较运算符进行连接。当用户能确切知道内层查询返回的是单个值时,可以用>、<、=、>=、<=、!=或<>等比较运算符。

例如在例 3.57 中,由于一个学生只可能主修一个专业,也就是说内查询的结果是一个值,因此可以用“=”代替“IN”:

```
SELECT Sno,Sname,Smajor          /* 例 3.57 的解法三 */
FROM Student
WHERE Smajor =
      (SELECT Smajor
       FROM Student
       WHERE Sname='刘晨');
```

实现同一个查询请求可以有多种方法,就如上面的例 3.57 分别给出了三种不同的解法。不同的 SQL 语句执行效率会有差别,甚至差别会很大。这就是数据库编程人员应该掌握数据库性能调优技术的原因。

例 3.57 和例 3.58 中子查询的查询条件不依赖于父查询,这类子查询称为**不相关子查询**。不相关子查询是较简单的一类子查询。如果子查询的查询条件依赖于父查询,这类子查询称为**相关子查询**(correlated subquery),整个查询语句称为**相关嵌套查询**(correlated nested query)语句。

例 3.59 就是一个相关子查询的例子。

[例 3.59]找出每个学生超过他自己选修课程平均成绩的课程号。

```
SELECT Sno,Cno
FROM SC x
WHERE Grade >=(SELECT AVG(Grade)          /* 某学生的平均成绩 */
               FROM SC y
               WHERE y. Sno=x. Sno);
```

x、y 分别是表 SC 的别名,又称为元组变量,可以用来表示 SC 的一个元组。内层查询是求一个学生所有选修课程平均成绩的,至于是哪个学生的平均成绩要看参数 x. Sno 的值,而该值是与父查询相关的,因此这类查询称为**相关子查询**。

该语句的一种可能的执行过程采用以下三个步骤。

- ① 从外层查询中取出 SC 的一个元组 x，将元组 x 的 Sno 值(20180001)传送给内层查询。

```
SELECT AVG(Grade)
FROM SC y
WHERE y.Sno='20180001';
```

- ② 执行内层查询，得到值 89.3(近似值)，用该值代替内层查询，得到外层查询：

```
SELECT Sno, Cno
FROM SC x
WHERE Grade >= 89.3;
```

- ③ 执行这个查询，得到(20180001, 81002)

然后外层查询取出下一个元组重复做上述步骤①至③的处理，直到外层的 SC 元组全部处理完毕。结果如下：

```
(20180001,81002)
(20180002,81002)
(20180003,81001)
(20180004,81003)
(20180005,81003)
```

求解相关子查询不能像求解不相关子查询那样，先一次将子查询求解出来，然后求解父查询，由于内层查询与外层查询有关，因此必须反复求值。

3. 带有 ANY(SOME) 或 ALL 谓词的子查询

子查询返回单值时可以用比较运算符，但返回多值时要用 ANY(有的系统用 SOME)或 ALL 谓词修饰符。而使用 ANY 或 ALL 谓词时必须同时使用比较运算符。其语义如下所示：

> ANY	大于子查询结果中的某个值
> ALL	大于子查询结果中的所有值
< ANY	小于子查询结果中的某个值
< ALL	小于子查询结果中的所有值
>= ANY	大于或等于子查询结果中的某个值
>= ALL	大于或等于子查询结果中的所有值
<= ANY	小于或等于子查询结果中的某个值
<= ALL	小于或等于子查询结果中的所有值
= ANY	等于子查询结果中的某个值
= ALL	等于子查询结果中的所有值(通常没有实际意义)
!=(或<>) ANY	不等于子查询结果中的某个值
!=(或<>) ALL	不等于子查询结果中的任何一个值

[例 3.60] 查询非计算机科学与技术专业中比计算机科学与技术专业任意一个年龄小(出生日期晚)的学生的姓名、出生日期和主修专业。

```
SELECT Sname, Sbirthdate, Smajor
FROM Student
WHERE Sbirthdate > ANY ( SELECT Sbirthdate
                        FROM Student
                        WHERE Smajor = '计算机科学与技术')
AND Smajor <> '计算机科学与技术';      /* 注意这是父查询块中的条件 */
```

查询结果如图 3.13 所示。

Sname	Sbirthdate	Smajor
李勇	2000-3-8	信息安全
陈新奇	2001-11-1	信息管理与信息系统
赵明	2000-6-12	数据科学与大数据技术
王佳佳	2001-12-7	数据科学与大数据技术

图 3.13 例 3.60 查询结果

关系数据库管理系统执行此查询时, 首先处理子查询, 找出计算机科学与技术专业中所有学生的出生日期, 构成一个集合(1999-9-1, 2001-8-1, 2000-1-8)。然后处理父查询, 找所有不是计算机科学与技术专业且出生日期大于集合中任意一个的学生。

本查询也可以用聚集函数来实现, 首先用子查询找出计算机科学与技术专业学生中最早出生的日期(1999-9-1), 然后在父查询中查所有非计算机科学与技术专业且出生日期比 1999-9-1 晚的学生。SQL 语句如下:

```
SELECT Sname, Sbirthdate, Smajor
FROM Student
WHERE Sbirthdate >
      ( SELECT MIN( Sbirthdate)      /* 出生的日期最小, 就是年龄最大 */
        FROM Student
        WHERE Smajor = '计算机科学与技术')
AND Smajor <> '计算机科学与技术';
```

[例 3.61] 查询非计算机科学与技术专业中比计算机科学与技术专业所有学生年龄都小(出生日期晚)的学生的姓名及出生日期。

SQL 语句如下:

```
SELECT Sname, Sbirthdate
FROM Student
```

```

WHERE Sbirthdate > ALL
    ( SELECT Sbirthdate
      FROM Student
      WHERE Smajor='计算机科学与技术')
AND Smajor < > '计算机科学与技术';

```

关系数据库管理系统执行此查询时，首先处理子查询，找出计算机科学与技术专业中所有学生的年龄，构成一个集合(1999-9-1, 2001-8-1, 2000-1-8)。然后处理父查询，找所有不是计算机科学与技术专业且出生日期晚于集合中所有值的学生。

查询结果如图 3.14 所示。

name	birthdate
陈新奇	2001-11-1
王佳佳	2001-12-7

图 3.14 例 3.61 查询结果

本查询同样也可以用聚集函数实现。SQL 语句如下：

```

SELECT Sname, Sbirthdate
FROM Student
WHERE Sbirthdate >
    ( SELECT MAX( Sbirthdate)   /* 计算机科学与技术专业所有学生中最晚出生日期 */
      FROM Student
      WHERE Smajor='计算机科学与技术')
AND Smajor < > '计算机科学与技术';

```

关系数据库管理系统执行此查询时，首先处理子查询，找出计算机科学与技术专业所有学生中最晚的出生日期，结果是 2001-8-1，然后处理父查询，找所有不是计算机科学与技术专业且出生日期晚于 2001-8-1 的学生。

用聚集函数实现子查询通常比直接用 ANY 或 ALL 查询效率要高。ANY、ALL 与聚集函数的对应关系如表 3.6 所示。

表 3.6 ANY(或 SOME)、ALL 谓词与聚集函数、IN 谓词的等价转换关系

谓词	=	< >或!=	<	<=	>	>=
ANY	IN	--	<MAX	<= MAX	>MIN	>= MIN
ALL	--	NOT IN	<MIN	<= MIN	>MAX	>= MAX

表 3.6 中，=ANY 等价于 IN 谓词，<ANY 等价于 <MAX，<>ALL 等价于 NOT IN 谓词，<ALL 等价于 <MIN，等等。

4. 带有 EXISTS 谓词的子查询

EXISTS 代表存在量词 $\exists$ 。带有 EXISTS 谓词的子查询不返回任何数据，只产生逻辑真值“true”或逻辑假值“false”。

可以利用 EXISTS 来判断 $x \in S$ 、 $S \subseteq R$ 、 $S=R$ 、 $S \cap R$ 非空等是否成立。

[例 3.62] 查询所有选修了 81001 号课程的学生姓名。

本查询涉及 Student 和 SC 表。可以在 Student 中依次取每个元组的 Sno 值，用此值去检查 SC 表。若 SC 中存在这样的元组，其 Sno 值等于此 Student. Sno 值，并且其 Cno='81001'，则取此 Student. Sname 送入结果表。将此想法写成 SQL 语句如下：

```
SELECT Sname
FROM Student
WHERE EXISTS
    (SELECT *
     FROM SC
     WHERE Sno=Student. Sno AND Cno='81001');
```

使用存在量词 EXISTS 后，若内层查询结果非空，则外层的 WHERE 子句返回真值，否则返回假值。

由 EXISTS 引出的子查询，其目标列表表达式通常都用 *，因为带 EXISTS 的子查询只返回真值或假值，给出列名无实际意义。

本例中子查询的查询条件依赖于外层父查询的某个属性值(Student 的 Sno 值)，因此也是相关子查询。这个相关子查询的处理过程是：首先取外层查询中 Student 表的第一个元组，根据它与内层查询相关的属性值(Sno 值)处理内层查询，若 WHERE 子句返回值为真，则取外层查询中该元组的 Sname 放入结果表，然后再取 Student 表的下一个元组。重复这一过程，直至外层 Student 表全部检查完为止。

本例中的查询也可以用连接运算来实现，读者可以参照有关的例子自己给出相应的 SQL 语句。

与 EXISTS 谓词相对应的是 NOT EXISTS 谓词。使用存在量词 NOT EXISTS 后，若内层查询结果为空，则外层的 WHERE 子句返回真值，否则返回假值。

[例 3.63] 查询没有选修 81001 号课程的学生姓名。

```
SELECT Sname
FROM Student
WHERE NOT EXISTS
    (SELECT *
     FROM SC
     WHERE Sno=Student. Sno AND Cno='81001');
```

一些带 EXISTS 或 NOT EXISTS 谓词的子查询不能被其他形式的子查询等价替换,但所有带 IN 谓词、比较运算符、ANY 和 ALL 谓词的子查询都能用带 EXISTS 谓词的子查询等价替换。例如带有 IN 谓词的例 3.57 查询与“刘晨”在同一个主修专业的学生。可以用如下带 EXISTS 谓词的子查询替换:

```
SELECT Sno,Sname,Smajor                                /* 例 3.57 的解法 4 */
FROM Student S1
WHERE EXISTS
    (SELECT *
     FROM Student S2
     WHERE S2.Smajor=S1.Smajor AND S2.Sname='刘晨');
```

由于带 EXISTS 量词的相关子查询只关心内层查询是否有返回值,并不需要查具体值,因此其效率并不一定低于不相关子查询,有时是高效的方法。

[例 3.64] 查询选修了全部课程的学生姓名。

SQL 中没有全称量词(for all),但是可以把带有全称量词的谓词转换为等价的带有存在量词的谓词:

$$(\forall x)P \equiv \neg(\exists x(\neg P))$$

由于没有全称量词,可将题目的意思转换成等价的用存在量词的形式:查询这样的学生,没有一门课程是他不选修的。其 SQL 语句如下:

```
SELECT Sname
FROM Student
WHERE NOT EXISTS
    (SELECT *
     FROM Course
     WHERE NOT EXISTS
        (SELECT *
         FROM SC
         WHERE Sno=Student.Sno AND Cno=Course.Cno));
```

从而用 EXIST/NOT EXIST 来实现带全称量词的查询。

[例 3.65] 查询至少选修了学生 20180002 选修的全部课程的学生学号。

本查询可以用逻辑蕴涵来表达:查询学号为 x 的学生,对所有的课程 y,只要 20180002 学生选修了课程 y,则 x 也选修了 y。形式化表示如下:

用 p 表示谓词“学生 20180002 选修了课程 y”

用 q 表示谓词“学生 x 选修了课程 y”

则上述查询为

$$(\forall y)p \rightarrow q$$

SQL 语言中没有蕴涵(implication)逻辑运算,但是可以利用谓词演算将一个逻辑蕴涵的谓词等价转换为

$$p \rightarrow q \equiv \neg p \vee q$$

该查询可以转换为如下等价形式:

$$(\forall y)p \rightarrow q \equiv \neg(\exists y(\neg(p \rightarrow q))) \equiv \neg(\exists y(\neg(\neg p \vee q))) \equiv \neg \exists y(p \wedge \neg q)$$

它所表达的语义为:不存在这样的课程 y, 学生 20180002 选修了 y, 而学生 x 没有选修。用 SQL 语言表示如下:

```
SELECT Sno
FROM Student
WHERE NOT EXISTS
    (SELECT *
     FROM SC SCX
     WHERE SCX.Sno='20180002' AND
     NOT EXISTS
        (SELECT *
         FROM SC SCY
         WHERE SCY.Sno=Student.Sno AND/* 与子查询中的 SC 表区分开 */
         SCY.Cno=SCX.Cno));
```

3.3.4 集合查询

SELECT 语句的查询结果是元组的集合,所以对多个 SELECT 语句的结果可进行集合操作。集合操作主要包括并操作 UNION、交操作 INTERSECT 和差操作 EXCEPT。

注意:参加集合操作的各查询结果的列数必须相同,对应项的数据类型也必须相同。

[例 3.66]查询计算机科学与技术专业的学生及年龄不大于 19 岁(包括等于 19 岁)的学生。

```
SELECT *
FROM Student
WHERE Smajor='计算机科学与技术'
UNION
SELECT * FROM Student
WHERE (extract(year from current_date) - extract(year from Shirthdate)) <= 19;
```

本查询实际上是求计算机科学系的所有学生与年龄不大于 19 岁的学生的并集。使用 UNION 将多个查询结果合并起来时,系统会自动去掉重复元组。如果要保留重复元组则用 UNION ALL 操作符。

[例 3.67]查询 2020 年第 2 学期选修了课程 81001 或 81002 的学生。

本例即查询 2020 年第 2 学期选修课程 81001 的学生集合和选修课程 81002 的学生集合的

并集。

```
SELECT Sno
FROM SC
WHERE Semester='20202' AND Cno='81001'
UNION
SELECT Sno
FROM SC
WHERE Semester='20202' AND Cno='81002';
```

[例 3.68] 查询计算机科学与技术专业的学生与年龄不大于 19 岁的学生的交集。

```
SELECT *
FROM Student
WHERE Smajor='计算机科学与技术'
INTERSECT
SELECT *
FROM Student
WHERE( extract( year from current_date )-extract( year from Sbirthdate ) ) <= 19;
```

这实际上就是查询计算机科学与技术专业中年龄不大于 19 岁的学生。

```
SELECT *
FROM Student
WHERE Smajor='计算机科学与技术' AND
      ( extract( year from current_date )-extract( year from Sbirthdate ) ) <= 19;
```

[例 3.69] 查询既选修了课程 81001 又选修了课程 81002 的学生。就是查询选修课程 81001 的学生集合与选修课程 81002 的学生集合的交集。

```
SELECT Sno
FROM SC
WHERE Cno='81001'
INTERSECT
SELECT Sno
FROM SC
WHERE Cno='81002';
```

本例也可以表示为

```
SELECT Sno
FROM SC
WHERE Cno='81001' AND Sno IN
```

```
(SELECT Sno  
FROM SC  
WHERE Cno='81002');
```

[例 3.70] 查询计算机科学与技术专业的学生与年龄不大于 19 岁的学生的差集。

```
SELECT *  
FROM Student  
WHERE Smajor='计算机科学与技术'  
EXCEPT  
SELECT *  
FROM Student  
WHERE (extract(year from current_date)-extract(year from Sbirthdate)) <= 19;
```

也就是查询计算机科学与技术专业中年龄大于 19 岁的学生。

```
SELECT *  
FROM Student  
WHERE Smajor='计算机科学与技术' AND  
      (extract(year from current_date)-extract(year from Sbirthdate)) > 19;
```

3.3.5 基于派生表的查询

子查询不仅可以出现在 WHERE 子句中, 还可以出现在 FROM 子句中, 这时子查询生成的临时派生表 (derived table) 成为主查询的查询对象。例如, 例 3.59 也可以用如下的查询完成:

```
/* 找出每个学生超过他自己选修课程平均成绩的课程号 */  
SELECT Sno, Cno  
FROM SC, (SELECT Sno, Avg(Grade) FROM SC GROUP BY Sno)  
      AS Avg_SC(Avg_sno, Avg_grade)  
WHERE SC.Sno = Avg_SC.Avg_sno AND SC.Grade >= Avg_SC.Avg_grade;
```

这里 FROM 子句中的子查询将生成一个派生表 Avg_SC。该表由 Avg_sno 和 Avg_grade 两个属性组成, 记录了每个学生的学号及平均成绩。主查询将 SC 表与派生表 Avg_SC 按学号相等进行连接, 选出修课成绩大于其平均成绩的课程号。

如果子查询中没有聚集函数, 派生表可以不指定属性列, 子查询 SELECT 子句后面的列名为其默认属性。例如例 3.62 可以用如下查询完成:

```
/* 查询所有选修了 81001 号课程的学生姓名 */  
SELECT Sname  
FROM Student, (SELECT Sno FROM SC WHERE Cno='81001') AS SC1  
WHERE Student.Sno=SC1.Sno;
```

需要说明的是,通过 FROM 子句生成派生表时,AS 关键字可以省略,但必须为派生关系指定一个别名;而对于基本表,别名是可选择项。派生表是一个中间结果表,查询完成后派生表将被系统自动清除。

SELECT 语句是 SQL 语言的核心语句,从前面的例子可以看到其语句成分丰富多样。本书总结了 SELECT 语句的一般格式,读者可参见二维码内容。



扩展阅读: SELECT
语句的一般格式

3.4 数据更新

SQL 的数据更新包括插入数据、修改数据和删除数据三类语句。

3.4.1 插入数据

插入数据语句 INSERT 通常有两种形式,一种是插入一个元组,另一种是插入子查询结果。后者可以一次插入多个元组。

1. 插入一个元组

插入一个元组的 INSERT 语句的语法如下:

```
INSERT INTO <表名>[( <属性列 1>[, <属性列 2>]...)]  
VALUES (<常量 1>[, <常量 2>]...);
```

其功能是将一个新元组插入指定表中。新元组的属性列 1 的值为常量 1,属性列 2 的值为常量 2……INTO 子句中没有出现的属性列,新元组在这些列上将取空值。如果定义表时指定了相应属性列的默认值,则新元组在这些列上将取默认值。但必须注意的是,在表定义时说明了 NOT NULL 的属性列不能取空值,否则会出错。

如果 INTO 子句中没有指明任何属性列名,则新插入的元组必须在每个属性列上均有指定值。

[例 3.71] 将一个新学生元组(学号:20180009,姓名:陈冬,性别:男,出生日期:2000-5-22,主修专业:信息管理与信息系统)插入 Student 表。

```
INSERT INTO Student (Sno, Sname, Ssex, Smajor, Sbirthdate)  
VALUES ('20180009', '陈冬', '男', '信息管理与信息系统', '2000-5-22');
```

在 INTO 子句中指出了表名 Student,并指出了新增加的元组在哪些属性上要赋值,属性的顺序可以与 CREATE TABLE 中的顺序不一样,就如本例中 Sbirthdate 和 Smajor 次序交换了。VALUES 子句按照 INTO 子句指定的属性次序对新元组的各属性赋值。字符串常数要用单引号(英文符号)括起来。

[例 3.72] 将学生张成民的信息插入 Student 表。

```
INSERT INTO Student
```

```
VALUES ('20180008', '张成民', '男', '2000-4-15', '计算机科学与技术');
```

本例与例 3.71 的不同之处是在 INTO 子句中只指出了表名, 没有指出属性名。这表示 VALUES 子句要在表的所有属性列上都指定值, 属性列的次序与 CREATE TABLE 中的次序相同。

[例 3.73] 插入一条选课记录。

```
INSERT INTO SC(Sno, Cno, Semester, Teachingclass)
VALUES ('20180005', '81004', '20202', '81004-01');
```

数据库管理系统将在新插入记录的 Grade 列上自动地赋空值。

或者:

```
INSERT INTO SC
VALUES ('20180005', '81004', NULL, '20202', '81004-01');
```

因为 INTO 子句没有指出 SC 的属性名, 所以在 VALUES 子句中对应的 Grade 列上要明确给出空值。

2. 插入子查询结果

子查询不仅可以嵌套在 SELECT 语句中用以构造父查询的条件(如 3.3.3 小节所述), 也可以嵌套在 INSERT 语句中用以生成要插入的批量数据。

插入子查询结果的 INSERT 语句格式如下:

```
INSERT INTO <表名>[( <属性列 1>[, <属性列 2>...])
子查询;
```

[例 3.74] 对每一个专业, 求学生的平均年龄, 并把结果存入数据库。

首先在数据库中建立一个新表, 其中一列存放专业名, 另一列存放学生的平均年龄。

```
CREATE TABLE Smajor_age
(Smajor VARCHAR(20),
Avg_age SMALLINT);
```

然后对 Student 表按专业分组求平均年龄, 再把专业名和平均年龄存入新表中。

```
INSERT INTO Smajor_age(Smajor, Avg_age)
SELECT Smajor, AVG(extract(year from current_date) - extract(year from Sbirthdate))
FROM Student
GROUP BY Smajor;
```

3.4.2 修改数据

修改操作又称为更新操作, 其语句的一般语法如下:

```
UPDATE <表名>  
SET <列名>=<表达式>[,<列名>=<表达式>]...  
[WHERE <条件>];
```

其功能是修改指定表中满足 WHERE 子句条件的元组。其中 SET 子句中<表达式>的值用于取代相应的属性列值。如果省略 WHERE 子句,则表示要修改表中的所有元组。

1. 修改某一个元组的值

[例 3.75] 将学生 20180001 的出生日期改为 2001 年 3 月 18 日。

```
UPDATE Student  
SET Sbirthdate='2001-3-18'  
WHERE Sno='20180001';
```

2. 修改多个元组的值

[例 3.76] 将 2020 年第 1 学期选修 81002 课程所有学生的成绩减少 5 分。

```
UPDATE SC  
SET Grade= Grade-5  
WHERE Semester='20201' AND Cno='81002'
```

3. 带子查询的修改语句

子查询也可以嵌套在 UPDATE 语句中,用以构造修改的条件。

[例 3.77] 将计算机科学与技术专业学生的成绩置零。

```
UPDATE SC  
SET Grade=0  
WHERE Sno IN  
    (SELECT Sno  
     FROM Student  
     WHERE Smajor='计算机科学与技术');
```

3.4.3 删除数据

删除语句的一般格式如下:

```
DELETE FROM <表名>  
[WHERE <条件>];
```

其功能是从指定表中删除满足 WHERE 子句条件的所有元组。如果省略 WHERE 子句则表示删除表中全部元组,但表的定义仍在字典中。也就是说,DELETE 语句删除的是表中的数据,而不是关于表的定义。

1. 删除某一个元组的值

[例 3.78] 删除学号为 20180007 的学生记录。

```
DELETE FROM Student
WHERE Sno='20180007';
```

2. 删除多个元组的值

[例 3.79] 删除所有的学生选课记录。

```
DELETE FROM SC;
```

这条 DELETE 语句将使 SC 成为空表，它删除了 SC 的所有元组。

3. 带子查询的删除语句

子查询同样也可以嵌套在 DELETE 语句中，用以构造执行删除操作的条件。

[例 3.80] 删除计算机科学与技术专业所有学生的选课记录。

```
DELETE FROM SC
WHERE Sno IN
    (SELECT Sno
     FROM Student
     WHERE Smajor= '计算机科学与技术');
```

对某个基本表中数据的更新(增、删、改)操作有可能会破坏参照完整性，第 5 章 5.3 节参照完整性将详细讲解如何进行参照完整性检查和控制。

3.5 空值的处理

前面已经多处提到空值(NULL)的概念和处理，这里再系统讲解一下什么是空值，空值如何参与运算等问题。

所谓空值就是“不知道”“不存在”或“无意义”的值。SQL 语言中允许某些元组的某些属性在一定情况下取空值。一般有以下几种情况：

① 该属性应该有一个值，但目前还不知道它的具体值。例如，某学生在填学生登记表时漏了出生日期项，不知道该学生此信息，因此出生日期属性值取空值。

② 该属性不应该有值。例如，缺考学生的成绩为空，因为他没有参加考试。

③ 由于某种原因不便于填写。例如，一个人的电话号码不想让大家知道，则取空值。

因此，空值是一个很特殊的值，含有不确定性，对关系运算带来的特殊问题需要做特殊处理。

1. 空值的产生

[例 3.81] 向 SC 表中插入一个元组，学号为“20180006”，课程号为“81004”，选课学期为

2021 年第 1 学期,选课班没有确定,没有课程成绩。

```
INSERT INTO SC(Sno,Cno,Grade,Semester,Teachingclass)
VALUES('20180006','81004',NULL,'20211',NULL);
/* 在插入时该学生还没有选定教学班,没有考试成绩,都要取空值 */
```

或

```
INSERT INTO SC(Sno,Cno,Semester)
VALUES('20180006','81004','20211');
/* 在插入语句的 INTO 子句中没有指定的属性,系统自动置空值 */
```

[例 3.82] 将 Student 表中学号为 20180006 的学生的主修专业改为空值。

```
UPDATE Student
SET Smajor = NULL
WHERE Sno='20180006';
```

另外,外连接也会产生空值,参见 3.3.2 小节例 3.55。空值的关系运算也会产生空值。

2. 空值的判断

判断一个属性的值是否为空值,用 IS NULL 或 IS NOT NULL 来表示。

[例 3.83] 从 Student 表中找出漏填了数据的学生信息。

```
SELECT *
FROM Student
WHERE Sname IS NULL OR Ssex IS NULL OR Sbirthdate IS NULL OR Smajor IS NULL;
```

WHERE 子句中没有属性 Sno,是因为 Sno 是主码,不允许取空值,不能漏填。

3. 空值约束

在创建基本表时,如果属性定义(或者域定义)为 NOT NULL 约束,则该属性不能取空值。

主码的属性不能取空值。例如 SC 表的主码是(Sno,Cno),Sno 和 Cno 都不能取空值。

Student 表的主码是 Sno,不能取空值。

在创建基本表时,如果属性定义(或者域定义)为 UNIQUE 约束,那么该属性能不能取空值呢?对于具有 UNIQUE 约束的属性,是可以取空值的。进一步,是否可以有多个记录在此属性上取空值呢?很遗憾,SQL 标准中对此没有明确规定。各个数据库管理系统在实现时就不尽相同,Oracle、KingbaseES 和 PostgreSQL 允许有多个记录在具有 UNIQUE 约束的列上取空值,而 SQL Server 则不允许。读者需要了解所使用的数据库管理系统的具体规定,实验方法可参考二维码相关内容。

4. 空值的算术运算、比较运算和逻辑运算

空值与另一个值(包括另一个空值)的算术运算的结果为空值。



扩展阅读:加了
UNIQUE 约束条
件的属性能否取
空值

空值与另一个值(包括另一个空值)的比较运算的结果为 UNKNOWN。

有了 UNKNOWN 后,传统的逻辑运算中二值(TRUE,FALSE)逻辑就扩展成了三值逻辑。逻辑运算 AND、OR、NOT 的真值表如表 3.7 所示,其中 T 表示 TRUE, F 表示 FALSE, U 表示 UNKNOWN。

表 3.7 逻辑运算 AND、OR、NOT 真值表

x	y	$x \text{ AND } y$	$x \text{ OR } y$	NOT x
T	T	T	T	F
T	U	U	T	F
T	F	F	T	F
U	T	U	T	U
U	U	U	U	U
U	F	F	U	U
F	T	F	T	T
F	U	F	U	T
F	F	F	F	T

在查询语句中,只有使 WHERE 子句和 HAVING 短语中的选择条件为 TRUE 的元组才被选出作为输出结果。

[例 3.84] 找出选修 81001 号课程且成绩不及格的学生。

```
SELECT Sno
FROM SC
WHERE Grade < 60 AND Cno='81001';
```

选出的学生是那些参加了考试而成绩不及格(Grade 属性为非空值)的学生,不包括缺考(Grade 属性为空值)的学生。因为前者使条件 Grade<60 的值为 TRUE,后者使条件的值为 UNKNOWN。

[例 3.85] 选出选修 81001 号课程且成绩不及格的学生以及缺考的学生。

```
SELECT Sno
FROM SC
WHERE Grade < 60 AND Cno='81001'
UNION
SELECT Sno
FROM SC
WHERE Grade IS NULL AND Cno='81001';
```

或

```
SELECT Sno
FROM SC
WHERE Cno='81001' AND (Grade < 60 OR Grade IS NULL);
```

3.6 视图

视图是关系数据库系统提供给用户以多种角度观察数据库中数据的重要机制。

视图是从一个或几个基本表(或视图)导出的表。它与基本表不同,是一个虚表。数据库中只存放视图的定义,而不存放视图对应的数据,这些数据仍存放在原来的基本表中。所以一旦基本表中的数据发生变化,从视图中查询出的数据也就随之改变了。从这个意义上讲,视图就像一个窗口,透过它可以看到数据库中自己感兴趣的数据及其变化。

视图一经定义,就可以和基本表一样被查询、被删除。也可以在一个视图之上再定义新的视图,但对视图的更新(增、删、改)操作则有一定的限制。

本节讨论视图的定义、操作及作用。

3.6.1 定义视图

1. 建立视图

SQL 语言用 CREATE VIEW 命令建立视图,其一般语法如下:

```
CREATE VIEW <视图名>[( <列名>[, <列名>]...)]
AS <子查询>
[WITH CHECK OPTION];
```

其中,子查询可以是任意的 SELECT 语句,是否包含 ORDER BY 子句和 DISTINCT 短语取决于具体系统的实现。

WITH CHECK OPTION 表示对视图进行 UPDATE、INSERT 和 DELETE 操作时,要保证更新、插入或删除的行满足视图定义中的谓词条件(即子查询中的条件表达式)。

组成视图的属性列名或者全部省略,或者全部指定,没有第三种选择。如果省略了视图的各个属性列名,则隐含该视图由子查询中 SELECT 子句目标列中的诸字段组成。但在下列三种情况下必须明确指定组成视图的所有列名:

- ① 某个目标列不是单纯的属性名,而是聚集函数或列表表达式。
- ② 多表连接时选出了几个同名列作为视图的字段。
- ③ 需要在视图中为某个列启用新的更合适的名字。

[例 3.86] 建立信息管理与信息系统专业学生的视图。

```
CREATE VIEW IS_Student
AS
SELECT Sno,Sname,Ssex,Sbirthdate, Smajor
FROM Student
WHERE Smajor='信息管理与信息系统';
```

本例中省略了视图 IS_Student 的列名,表示隐含由子查询中 SELECT 子句的 5 个列名组成。

关系数据库管理系统执行 CREATE VIEW 语句的结果只是把视图的定义存入数据字典,并不执行其中的 SELECT 语句。只有在对视图查询时,才按视图的定义从基本表中将数据查出。

[例 3.87] 建立信息管理与信息系统专业学生的视图,并要求进行插入、修改和删除操作时,仍需保证该视图只有信息管理与信息系统专业的学生。

```
CREATE VIEW IS_Student
AS
SELECT Sno,Sname,Ssex,Sbirthdate, Smajor
FROM Student
WHERE Smajor='信息管理与信息系统'
WITH CHECK OPTION;
```

由于在定义 IS_Student 视图时加上了 WITH CHECK OPTION 子句,以后对该视图进行插入、修改和删除操作时,关系数据库管理系统会自动检查 Smajor='信息管理与信息系统'的条件。

例如:

```
INSERT INTO IS_Student values ('20180010','贾明','男','2001-11-1','信息管理与信息系统');
```

插入成功。

```
INSERT INTO IS_Student values ('2018011','王伟','男','2003-11-1','计算机科学与技术');
```

插入失败。原因是不符合 WITH CHECK OPTION 的条件。

若一个视图是从单个基本表导出的,并且只是去掉了基本表的某些行和某些列,但保留了主码,则称这类视图为行列子集视图。IS_Student 视图就是一个行列子集视图。

视图不仅可以建立在单个基本表上,也可以建立在多个基本表上。

[例 3.88] 建立信息管理与信息系统专业选修 81001 号课程的学生视图(包括学号、姓名、成绩属性)。

```
CREATE VIEW IS_C1(Sno,Sname,Grade)
AS
```

```

SELECT Student. Sno, Sname, Grade
FROM Student, SC
WHERE Smajor='信息管理与信息系统' AND Student. Sno=SC. Sno AND SC. Cno='81001';

```

由于视图 IS_C1 的属性列取自 Student 表与 SC 表的部分属性列, 所以必须在 CREATE VIEW IS_C1 后面明确说明视图的各个属性列名。

视图不仅可以建立在一个或多个基本表上, 也可以建立在一个或多个已定义好的视图上, 或建立在基本表与视图上。

[例 3.89] 建立信息管理与信息系统专业选修 81001 号课程且成绩在 90 分以上的学生的视图(包括学号、姓名、成绩属性)。

```

CREATE VIEW IS_C2
AS
SELECT Sno, Sname, Grade
FROM IS_C1
WHERE Grade>=90;

```

这里的视图 IS_C2 就是建立在视图 IS_C1 之上的。

定义基本表时, 为了减少数据库中的冗余数据, 表中只存放基本数据, 由基本数据经过各种计算派生出的数据一般是不存储的。由于视图中的数据并不实际存储, 所以定义视图时可以根据应用的需要设置一些派生属性列。这些派生属性由于在基本表中并不实际存在, 所以也称为虚拟列。带虚拟列的视图称为带表达式的视图。

[例 3.90] 将学生的学号、姓名、年龄定义为一个视图。

```

CREATE VIEW S_AGE( Sno, Sname, Sage)
AS
SELECT Sno, Sname, (extract(year from current_date)-extract(year from Sbirthdate))
FROM Student;

```

这里视图 S_AGE 是一个带表达式的视图。视图定义的 AS 子句中 SELECT 语句的学生年龄是通过计算得到的。

还可以用带有聚集函数和 GROUP BY 子句的查询来定义视图, 这种视图称为分组视图。

[例 3.91] 将学生的学号及平均成绩定义为一个视图。

```

CREATE VIEW S_GradeAVG( Sno, Gavg)
AS
SELECT Sno, AVG( Grade)
FROM SC
GROUP BY Sno;

```

由于 AS 子句中 SELECT 语句的目标列平均成绩是通过聚集函数得到的, 所以 CREATE

VIEW 中必须明确定义组成 S_GradeAVG 视图的各个属性列名。S_GradeAVG 是一个分组视图。

[例 3.92] 将 Student 表中所有的女生记录定义为一个视图。

```
CREATE VIEW F_Student(Fsno,Fname,Fsex,Fbirthdate,Fmajor)
AS
SELECT *
FROM Student
WHERE Ssex='女';
```

这里视图 F_Student 是由子查询“SELECT *”建立的。F_Student 视图的属性列与 Student 表的属性列一一对应。如果以后修改了基本表 Student 的结构, 则 Student 表与 F_Student 视图的映像关系可能会被破坏, 该视图就不能正常工作了。为避免出现这类问题, 最好在修改基本表之后删除由该基本表导出的视图, 然后重建这个视图。

2. 删除视图

该语句的格式如下:

```
DROP VIEW <视图名>[ CASCADE];
```

视图删除后视图的定义将从数据字典中删除。如果该视图上还导出了其他视图, 则使用 CASCADE 级联删除语句把该视图和由它导出的所有视图一起删除。

基本表删除后, 由该基本表导出的所有视图均无法使用了, 但是视图的定义没有从字典中清除。删除这些视图定义需要显式地使用 DROP VIEW 语句。

[例 3.93] 删除视图 S_AGE 和视图 IS_C1:

```
DROP VIEW S_AGE;          /* 成功执行 */
DROP VIEW IS_C1;          /* 报告错误 */
```

执行 DROP VIEW IS_C1 语句时, 由于 IS_C1 视图上还导出了 IS_C2 视图, 所以该语句执行时会报告错误, 提示视图 IS_C2 依赖于视图 IS_C1。

如果导出视图也确定可以删除, 则使用级联删除语句:

```
DROP VIEW IS_C1 CASCADE;
```

执行此语句不仅删除了 IS_C1 视图, 还级联删除了由它导出的 IS_C2 视图。

3.6.2 查询视图

视图定义后, 用户就可以像对基本表一样对视图进行查询了。

[例 3.94] 在信息管理与信息系统专业学生的视图中, 找出年龄小于或等于 20 岁的学生 (包括学生的学号和出生日期)。

```
SELECT Sno, Sbirthdate
```

```
FROM IS_Student
WHERE( extract( year from current_date )-extract( year from Sbirthdate) )<= 20;
```

关系数据库管理系统执行对视图的查询时，首先进行有效性检查，检查查询中涉及的表、视图等是否存在。如果存在，则从数据字典中取出视图的定义，把定义中的子查询和用户的查询结合起来，转换成等价的对基本表的查询，然后再执行修正了的查询。这一转换过程称为视图消解(view resolution)。

本例中，先找到视图 IS_Student 的定义如下：

```
CREATE VIEW IS_Student
AS
SELECT Sno,Sname,Ssex,Sbirthdate
FROM Student
WHERE Smajor='信息管理与信息系统'
WITH CHECK OPTION;
```

然后进行视图消解，转换后的查询语句如下：

```
SELECT Sno,Sbirthdate
FROM Student
WHERE Smajor='信息管理与信息系统' AND
      ( extract( year from current_date )-extract( year from Sbirthdate) )<= 20;
```

[例 3.95] 查询选修 81001 号课程的信息管理与信息系统专业学生。

```
SELECT IS_Student. Sno,Sname
FROM IS_Student,SC
WHERE IS_Student. Sno=SC. Sno AND SC. Cno='81001';
```

本查询涉及视图 IS_Student 和基本表 SC 的连接操作，在用户眼里视图如同基本表一样。

关系数据库管理系统先从数据字典中取出视图 IS_Student 的定义，然后进行视图消解，把上面的查询转换为：

```
SELECT Student. Sno,Sname
FROM Student,SC
WHERE Student. Sno=SC. Sno AND SC. Cno='81001' AND Smajor='信息管理与信息系统';
```

最后，通过这两个基本表的连接来完成用户请求。

在一般情况下，视图查询的转换是直截了当的。但有些情况下，这种转换不能直接进行，查询时就会出现问题，如例 3.96。

[例 3.96] 在例 3.91 中定义的视图 S_GradeAVG 视图中，查询平均成绩在 90 分以上的学生学号和平均成绩。

```
SELECT *  
FROM S_GradeAVG          /* FROM 后面是视图 S_GradeAVG */  
WHERE Gavg>=90;
```

将本例中的查询语句进行视图消解,与定义 S_GradeAVG 视图的子查询结合,转换成下列查询语句:

```
SELECT Sno,AVG(Grade)  
FROM SC  
WHERE AVG(Grade)>=90  
GROUP BY Sno;
```

因为 WHERE 子句中是不能用聚集函数作为条件表达式的,因此执行此修正后的查询将会出现语法错误。正确转换的查询语句如下:

```
SELECT Sno,AVG(Grade)  
FROM SC  
GROUP BY Sno  
HAVING AVG(Grade)>=90;
```

目前多数关系数据库管理系统对行列子集视图的查询均能进行正确转换。但对非行列子集视图的查询(如例 3.96)就不一定能正确转换了,因此这类查询应该直接对基本表进行。

例 3.96 也可以用如下 SQL 语句完成:

```
SELECT *  
FROM (SELECT Sno,AVG(Grade)    /* 子查询生成一个派生表 S_GradeAVG */  
      FROM SC  
      GROUP BY Sno ) AS S_GradeAVG(Sno, Gavg)  
WHERE Gavg>=90;
```

注意: 定义视图后对视图进行查询与基于派生表的查询是有区别的。视图一旦定义,该定义将保存在数据字典中,之后的所有查询都可以直接引用该视图。而派生表只是在语句执行时临时定义,语句执行后该派生表定义即被删除。

3.6.3 更新视图

更新视图是指通过视图来插入、删除和修改数据。由于视图是不实际存储数据的虚表,因此对视图的更新最终要转换为对基本表的更新。像查询视图那样,对视图的更新操作也是通过视图消解,转换为对基本表的更新操作。

为防止用户通过视图对数据进行更新操作时,有意无意地对不属于视图范围内的基本表数进行操作,可在定义视图时加上 WITH CHECK OPTION 子句。这样在视图上更新数据时,关系数据库管理系统会检查视图定义中的条件,若不满足条件则拒绝执行该操作。

[例 3.97] 将信息管理与信息系统专业学生视图 IS_Student 中 20180005 号学生姓名改为“刘新奇”。

```
UPDATE IS_Student
SET Sname='刘新奇'
WHERE Sno='20180005';
```

视图消解后，对视图 IS_Student 的更新语句就转换为对基本表 Student 的更新了：

```
UPDATE Student
SET Sname='刘新奇'
WHERE Sno='20180005' AND Smajor='信息管理与信息系统';
```

[例 3.98] 向信息管理与信息系统专业学生视图 IS_Student 中插入一个新的学生记录 (20180207, 赵新, 男, 2001-7-19)。

```
INSERT INTO IS_Student
VALUES('20180207','赵新','男','2001-7-19','信息管理与信息系统');
```

转换为对基本表的更新：

```
INSERT INTO Student(Sno,Sname,Ssex,Sbirthdate,Smajor)
VALUES('20180207','赵新','男','2001-7-19','信息管理与信息系统');
```

这里系统自动将‘信息管理与信息系统’放入 VALUES 子句。

[例 3.99] 删除信息管理与信息系统专业学生视图 IS_Student 中 20180207 号学生的记录。

```
DELETE FROM IS_Student
WHERE Sno='20180207';
```

转换为对基本表的更新：

```
DELETE FROM Student
WHERE Sno='20180207' AND Smajor='信息管理与信息系统';
```

在关系数据库中，并不是所有的视图都是可更新的，因为有些视图的更新不能唯一地有意义地转换成对相应基本表的更新。

例如，例 3.91 定义的视图 S_GradeAVG 是由学号和平均成绩两个属性列组成的，其中平均成绩一项是由 SC 表中对元组分组后计算平均值得来的。

如果想把视图 S_GradeAVG 中学号为“20180001”学生的平均成绩改成 90 分，SQL 语句如下：

```
UPDATE S_GradeAVG
SET Gavg=90
WHERE Sno='20180001';
```

这里对视图的更新无法转换成对基本表 SC 的更新,因为系统无法修改各科成绩以使 20180001 号学生的平均成绩成为 90。所以 S_GradeAVG 视图是不可更新的。

一般地,行列子集视图是可更新的。除行列子集视图外,有些视图理论上是可更新的,还有些视图从理论上就是不可更新的。

应该指出的是,不可更新的视图与不允许更新的视图是两个不同的概念。前者指理论上已证明其是不可更新的视图;后者指实际系统中不支持其更新,但它本身有可能是可更新的视图。

目前,各个关系数据库管理系统一般都只允许对一个表的行列子集视图进行更新,而且各个系统对视图的更新还有更进一步的规定。由于各系统实现方法上的差异,这些规定也不尽相同。读者一定要参考具体的产品手册,正确写出对视图的更新语句。

3.6.4 视图的作用

视图最终是定义在基本表之上的,对视图的一切操作最终也要转换为对基本表的操作,而且对于非行列子集视图进行查询或更新时还有可能出现问题。既然如此,为什么还要定义视图呢?这是因为合理使用视图能够带来许多好处,所以在实际应用开发中经常使用视图。

1. 视图能够对机密数据提供安全保护

有了视图机制,就可以在设计数据库应用系统时对不同的用户定义不同的视图,使机密数据不出现在不应看到这些数据的用户视图上。这样视图机制就自动提供了对机密数据的安全保护功能。假设 Student 表涉及全校 30 个院系的学生数据,可以在其上定义 30 个视图,每个视图只包含一个院系的学生数据,并只允许每个院系的主任查询和修改本院系的学生视图。

2. 视图对重构数据库提供了一定程度的逻辑独立性

第 1 章中已经介绍过数据的物理独立性与逻辑独立性的概念。数据的物理独立性是指用户的应用程序不依赖于数据库的物理结构。数据的逻辑独立性是指当数据库重构时,如增加新的关系或对原有关系增加新的字段等,用户的应用程序不会受影响。层次数据库和网状数据库一般能较好地支持数据的物理独立性,而对于逻辑独立性则不能完全地支持。

在关系数据库中,数据库的重构往往是不可避免的。重构数据库最常见的是将一个基本表“垂直”地分成多个基本表。例如:将学生关系 Student(Sno, Sname, Ssex, Sbirthdate, Smajor)分为 SX(Sno, Sname, Sbirthdate)和 SY(Sno, Ssex, Smajor)两个基本表。这时,原来的表 Student 为 SX 表和 SY 表自然连接的结果。如果建立一个视图 Student:

```
CREATE VIEW Student(Sno, Sname, Ssex, Sbirthdate, Smajor)
AS
SELECT SX. Sno, SX. Sname, SY. Ssex, SX. Sbirthdate, SY. Smajor
FROM SX, SY
WHERE SX. Sno=SY. Sno;
```

这样尽管数据库的逻辑结构改变了,变为 SX 和 SY 两个基本表,但不必修改应用程序。因为我们可以建立视图 Student,它和用户原来创建的 Student 表一模一样。

视图使用户的外模式保持不变,同时无须修改用户的应用程序就能和原来一样进行数据操作。

当然,视图只能在一定程度上提供数据的逻辑独立性,比如上面讲到,对视图的更新是有条件的,因此应用程序中修改数据的语句可能仍会因基本表结构的改变而需要做相应修改。

3. 视图能够简化用户的操作

视图机制使用户可以将注意力集中在所关心的数据上。通过定义视图使数据库看起来结构更加简单、清晰,并且可以简化用户的数据查询操作。

例如,那些定义了若干张表连接的视图就将表与表之间的连接操作对用户隐蔽起来了。换句话说,用户所做的只是对一个虚表的简单查询,而这个虚表是怎样得来的,用户无须了解。

适当利用视图可以更清晰地表达查询,例如,经常需要执行这样的查询“对每个学生找出其获得最高成绩的课程的课程号”。可以先定义一个视图,求出每个同学获得的最高成绩:

```
CREATE VIEW VMGrade
AS
SELECT Sno, MAX(Grade) Mgrade /* 每个学生获得的最高成绩 */
FROM SC
GROUP BY Sno;
```

然后用如下查询语句完成课程号查询:

```
SELECT SC. Sno, Cno
FROM SC, VMGrade
WHERE SC. Sno = VMGrade. Sno AND SC. Grade = VMGrade. Mgrade;
```

4. 视图使用户能以多种角度看待同一数据

视图机制能使不同的用户以不同的方式看待同一数据,当许多不同类型的用户共享同一个数据库时,这种灵活性是非常重要的。例如,有的教师希望了解学生的平均成绩,有的教师希望了解学生的最高成绩和最低成绩,他们都可以在基本表 SC 上定义自己感兴趣的视图,直接对这些视图查询。

由于上述诸多优点,视图被广泛用于实际应用开发中。

本章小结

关系数据库标准语言 SQL 是“数据库系统概论”课程学习和实验的重点。

SQL 可以分为数据定义、数据查询、数据更新、数据控制 4 大部分。人们有时把数据更新称为数据操纵，或把数据查询与数据更新合称为数据操纵。

本章系统而详尽地讲解了前面三部分的内容。数据控制中的数据安全性和完整性控制将在第 4 章和第 5 章中讲解。

本章在讲解 SQL 的同时，进一步讲解了关系数据库系统的基本概念，使关系数据库的许多概念更加具体、更加丰富。

SQL 的数据查询功能非常丰富，也是比较复杂的，读者一定要加强实践练习，举一反三，牢固掌握 SQL 语句。

习 题 3

- 试述 SQL 的特点。
- 说明在 DROP TABLE 时，RESTRICT 和 CASCADE 的区别。
- 有两个关系 $S(A, B, C, D)$ 和 $T(C, D, E, F)$ ，写出与下列查询等价的 SQL 表达式：
 - $\sigma_{A=10}(S)$ 。
 - $\Pi_{A,B}(S)$ 。
 - $S \bowtie T$ 。
 - $S \bowtie_{S.C=T.C} T$ 。
 - $S \bowtie_{A<E} T$ 。
 - $\Pi_{C,D}(S) \times T$ 。
- 用 SQL 语句建立第 2 章习题 2 第 6 题 SPJ 数据库的 4 个表(S、P、J 和 SPJ)，并针对建立的 4 个表用 SQL 语句完成该习题中的查询。
- 针对第 4 题中的 4 个表 S、P、J 及 SPJ，试用 SQL 完成以下各项操作：
 - 找出所有供应商的姓名和所在城市。
 - 找出所有零件的名称、颜色、重量。
 - 找出使用供应商 S1 所供应零件的工程代码。
 - 找出工程项目 J2 使用的各种零件的名称及其数量。
 - 找出上海厂商供应的所有零件代码。
 - 找出使用上海产的零件的工程名称。
 - 找出没有使用天津产的零件的工程代码。
 - 把全部红色零件的颜色改成蓝色。
 - 把由 S5 供给 J2 的零件 P6 改为由 S3 供应。
 - 从供应商关系中删除 S2 的记录，并从供应情况关系中删除相应的记录。
 - 请将 (S2, J6, P4, 200) 插入供应情况关系。
- 什么是基本表？什么是视图？两者之间的区别和联系是什么？
- 试述视图的优点。
- 哪类视图是可以更新的？哪类视图是不可更新的？各举一例说明。

9. 针对第4题中的4个表,为三建工程项目建立一个供应情况的视图,包括供应商代码(SNO)、零件代码(PNO)、供应数量(QTY)。针对该视图完成下列查询:

① 找出三建工程项目使用的各种零件代码及其数量。

② 找出供应商 S1 供应三建工程情况。

10. 什么是空值?举例说明。SQL 中如何表示空值?空值如何参加运算?

第3章实验 SQL 查询与操纵

实验 3.1 数据库定义。理解数据库对象及其作用,使用 SQL 数据定义语言(DDL)语句创建数据库模式和基本表。参考《概论辅导书》准备好实验数据,插入建立的基本表中。

实验 3.2 SQL 基本查询。掌握 SQL 的基本语法和基本用法,使用 SQL 查询语句完成单表查询和连接查询操作。

实验 3.3 SQL 高级查询。使用 SQL 嵌套查询和集合查询语句完成各类复杂查询操作。

实验 3.4 数据更新。使用 SQL 数据操纵语言(DML)语句完成各类更新操作(包括插入数据、修改数据、删除数据)。

实验 3.5 视图。理解视图的作用,掌握视图的创建、使用和删除等基本功能。

实验 3.6 索引。理解和掌握索引的设计、创建、使用和维护等功能,体验索引对于数据量大时查询效率提高的效果。

参考文献 3

[1] BOYCE R, CHAMBERLIN D D, KING W F, et al. Specifying queries as relational expressions[J]. CACM-Communications of the ACM, 1975, 18(11): 621-628.

[2] CHAMBERLIN D D, BOYCE R F. SEQUEL: A structured English query language[C]. Proceedings of ACM SIGMOD Workshop on Data Description, Access and Control, 1974: 249-264.

[3] CHAMBERLIN D D, ASTRAHAN M M, ESWARAN K P, et al. SEQUEL 2: a unified approach to data definition, manipulation and control[J]. IBM Journal of Research and Development, 1976, 20(6): 560-575.

SQL 是 Boyce 等人 1975 年在文献[1]提出的 SQUARE(Specifying Queries as Relational Expressions)语言的基础上发展起来的。文献[2]对 SQUARE 语言的语法进行了修改,形成了 SEQUEL。文献[3]对 SEQUEL 进行了进一步改进,最后形成了 SQL。SEQUEL 最早在关系数据库系统 SYSTEM R 上实现。

[4] REISNER P. Use of psychological experimentation as an aid to development of a query language[J]. TSE-IEEE Transactions on Software Engineering, 1977, 3(3): 218-229.

[5] DATE C J. A critique of the SQL database language[J]. ACM SIGMOD Record, 1984, 14(3): 8-54.

文献[4]和[5]分析了 SQL 的优缺点。

[6] X3H2(ANSI Database Committee). American National Standard Database Language SQL: Working Draft[R]. Document X3H2-85-1, 1984.

[7] ANSI. The Database Language SQL[R]. Document ANSI X3.135, 1986.

[8] GRIFFITHS P P, WADE B W. An authorization mechanism for a relational database system[J]. ACM Trans-

actions on Database Systems, 1976, 1(3): 242-255.

[9] CHAMBERLIN D D. A summary of user experience with the SQL data sublanguage[C]. Proceedings of the International Conference on Databases. Aberdeen, 1980.

[10] DATE C J. The Outer Join[C]. Proceedings of the 2nd International Conference on Database (ICOD-2). Cambridge, 1983.

[11] DAYAL U, BERNSTEIN P A. On the updatability of relational views[C]. Proceedings of the 4th International Conference on Very Large Data Base, 1978: 4368-377.

[12] SHASHA D, BONNET P. Database tuning: principles, experiments, and troubleshooting techniques (part I) [J]. Proceedings of the 2002 ACM SIGMOD International Conference on Management of Data, 2002.

[13] Information technology-database languages-SQL-part2 : foundation (SQL/Foundation): ISO/IEC 9075-2 CORR 1-2019[S]. 5th ed. ISO/IEC, 2019.

本章知识点讲解微视频:



SQL 数据定义



基本 SQL 查询



复杂 SQL 查询



空值

第4章

数据库安全性

在第1章中已经讲到,数据库的特点之一是由数据库管理系统提供统一的数据保护功能来保证数据的安全可靠和正确有效。数据库的数据保护主要包括数据库的安全性和完整性。本章讲解数据库的安全性,第5章将讨论数据库的完整性。

本章导读

数据库的安全性问题和计算机系统的安全性是紧密联系的,计算机系统的安全性问题可大致分为三大类,即技术安全类、管理安全类和政策法律类(详见二维码内容)。



扩展阅读:计算机系统的三类安全性问题

本章讨论数据库技术安全类问题,即从技术上如何保证数据库系统的安全性。

本章主要介绍有关计算机信息安全和数据库安全标准的内容。从4.2节起讲解数据库安全性控制的主要技术。希望读者理解和掌握通过SQL语句实现自主存取控制权限的授予和收回的方法;掌握创建用户、角色,以及分配权限给角色的方法。要加强实验练习掌握这些方法,还要进一步理解什么是强制存取控制机制,如何利用视图技术、审计技术、数据存储加密和传输加密等技术手段提高数据库的安全性。

本章较难理解的内容是强制存取控制机制中确定主体能否存取客体的存取规则,要理解制定该存取规则的原因。

4.1 数据库安全性概述

数据库的安全性是指保护数据库,以防不合法使用所造成的数据泄露、篡改或破坏。

安全性问题不是数据库系统所独有的,所有计算机系统都存在不安全因素,只是在数据库系统中由于大量数据集中存放,而且为众多用户直接共享,数据成为一个部门、企业乃至一个国家重要的资源,从而使安全性问题更为突出。系统安全保护措施是否有效是数据库系统的主要技术指标之一。

4.1.1 数据库的不安全因素

数据库的安全性事故频发。例如,2016年初某地多所医院连续遭受勒索软件的侵害,给医院和患者带来了极大的不便,对医院的工作造成了极大的负面影响。

综合来看,对数据库安全性产生威胁的因素主要有以下几方面。

1. 非授权用户对数据库的恶意存取和破坏

一些黑客(hacker)或不法分子在用户存取数据库时窃取用户名和用户口令等信息,然后假冒合法用户获取、修改甚至破坏用户数据。有的黑客还故意锁定并修改数据,进行勒索和破坏等犯罪活动。因此,必须阻止有损数据库安全的非法操作,以保证数据免受未经授权的访问和破坏。

2. 数据库中重要或敏感的数据被泄露

黑客或敌对分子千方百计地盗窃数据库中重要或敏感的机密数据,造成数据泄露。例如,攻击者利用SQL注入(SQL injection)技术,在应用程序中事先定义好的查询语句的结尾添加额外的SQL语句,在入侵检测不严的情况下欺骗数据库服务器执行非授权的查询和操作,进行SQL注入攻击,使得机密数据被泄露、篡改或锁定,数据库数据被破坏。

3. 安全环境的脆弱性

数据库的安全性与计算机系统(包括计算机硬件、操作系统、网络系统等)的安全性是紧密联系的。操作系统安全性脆弱、网络协议安全性保障不足等都会造成数据库安全性的破坏。因此,必须加强计算机系统整体的安全性保护。随着互联网、云计算和大数据技术的发展,计算机系统的安全性问题越来越突出。为此,必须建立一套可信(trusted)计算机系统的概念和标准。只有建立了完善的可信标准,即安全标准,才能规范和指导安全计算机系统部件的生产,较为准确地测定产品的安全性能指标,满足民用和军用的不同需要。

4.1.2 安全标准简介

计算机以及信息安全技术方面有一系列安全标准,最有影响的是可信计算机系统评价标准TCSEC和工厂安全通用准则CC标准。

1. 可信计算机系统评价标准 TCSEC

1985年美国国防部(Department of Defense, DoD)正式颁布了《可信计算机系统评价标准》(trusted computer system evaluation criteria,简称TCSEC或DoD85)。1991年的TCSEC/Trusted Database Interpretation(TCSEC/TDI)又把TCSEC扩展到了数据库系统。

TCSEC/TDI根据计算机系统对各项指标的支持情况将系统划分为4组7个等级,依次是D, C(C1、C2), B(B1、B2、B3), A(A1),其系统可靠或可信程度逐渐增高,如表4.1所示。

表 4.1 TCSEC/TDI 安全级别划分

安全级别	安全指标
A1	验证设计(verified design)
B3	安全域(security domain)
B2	结构化保护(structural protection)
B1	标记安全保护(labeled security protection)
C2	受控的存取保护(controlled access protection)
C1	自主安全保护(discretionary security protection)
D	最小保护(minimal protection)

D 级：该级是最低级别。保留 D 级的目的是将一切不符合更高标准的系统都归于 D 组。

C1 级：该级只提供了非常初级的自主安全保护，能够实现对用户和数据的分离，进行自主存取控制(discretionary access control, DAC)，保护或限制用户权限的传播。

C2 级：该级是安全产品的最低档，提供受控的存取保护，即将 C1 级的 DAC 进一步细化，以个人身份注册负责，并实施审计和资源隔离。

B1 级：标记安全保护。对系统的数据加以标记，并对标记的主体和客体实施强制存取控制(mandatory access control, MAC)以及审计等安全机制。B1 级别的产品才被认为是真正意义上的安全产品，满足此级别的产品前一般多冠以“安全”(security)或“可信的”字样，作为区别于普通产品的安全产品出售。

B2 级：结构化保护。建立形式化的安全策略模型，并对系统内的所有主体和客体实施 DAC 和 MAC。

B3 级：安全域。该级的可信计算基(trusted computing base, TCB)必须满足访问监控器的要求，审计跟踪能力更强，并提供系统恢复过程。

A1 级：验证设计，即在提供 B3 级保护的同时，给出系统的形式化设计说明和验证，以确信各安全保护真正实现。

表 4.2 列出了不同安全级别对安全指标的支持情况。随着安全级别的由低到高，系统对各项安全指标的支持也从无到有。其中，□表示该级不提供对该指标的支持，■表示该级新增的对该指标的支持，▒表示该级对该指标的支持与相邻低一级的等级一样，▨表示该级对该指标的支持较下一级有所增加或改动。

TCSEC/TDI 从 4 个方面来描述安全性级别划分的指标，即安全策略、责任、保证和文档。每个方面又细分为若干项，如表 4.2 中第一、二行所示。

表 4.2 不同安全级别对安全指标的支持情况

安全指标	安全策略							责任			保证								文档				
	自主存取控制	客体重用	标记完整性	标记信息的扩散	主体敏感度标记	设备标记	强制存取控制	标识与鉴别	可信路径	审计	系统体系结构	系统完整性	屏蔽信道分析	可信设施管理	可信恢复	可安全测试	设计规范和验证	配置管理	可信分配	安全特性用户指南	可信设施手册	测试文档	设计文档
C1	■	□	□	□	□	□	□	■	□	□	■	■	□	□	□	■	□	□	□	■	■	■	■
C2	▨	■	□	□	□	□	□	▨	□	■	■	■	□	□	□	▨	□	□	□	■	▨	■	■
B1	■	■	■	■	□	□	■	▨	□	▨	■	■	□	□	□	▨	■	□	□	■	▨	■	▨
B2	■	■	■	■	■	■	▨	■	■	▨	■	■	■	■	□	▨	▨	■	■	□	■	■	■
B3	▨	■	□	□	□	□	■	▨	▨	▨	■	■	▨	▨	■	▨	▨	■	□	■	▨	■	▨
A1	■	□	□	□	□	□	■	□	□	□	□	□	▨	■	■	▨	▨	▨	■	■	■	■	■

2. IT 安全通用准则 CC

自 1993 年起,多个国家组织了专门的委员会开发 IT 安全通用准则(common criteria, CC)。经过多次讨论和修订后,CC 2.1 版于 1999 年被 ISO 采用为国际标准,2001 年被我国采用作为国家标准。有关国际和我国信息安全标准发展简述,请参见二维码内容。



扩展阅读:信息安全标准发展简述

根据系统对安全保证要求的支持情况,CC 提出了评估保证级(evaluation assurance level, EAL),从 EAL1 至 EAL7 共分为 7 级,按保证程度逐渐增高。粗略而言,CC 的评估保证级与 TCSEC/TDI 的安全级别大致对应如表 4.3 所示。

表 4.3 CC 评估保证级(EAL)的划分及与 TCSEC/TDI 安全级别的对应

评估保证级	定义	TCSEC/TDI 安全级别 (近似相当)
EAL1	功能测试(functionally tested)	
EAL2	结构测试(structurally tested)	C1
EAL3	系统地测试和检查(methodically tested and checked)	C2
EAL4	系统地设计、测试和复查(methodically designed, tested and reviewed)	B1
EAL5	半形式化设计和测试(semiformally designed and tested)	B2
EAL6	半形式化验证的设计和测试(semiformally verified design and tested)	B3
EAL7	形式化验证的设计和测试(formally verified design and tested)	A1

4.2 数据库安全性控制

在计算机系统中,安全措施是一级一级层层设置的。例如,在图 4.1 所示的安全模型中,用户要求进入计算机系统时,系统首先根据输入的用户标识进行用户身份鉴别,只有合法的用户才准许进入计算机系统。对已进入系统的用户,数据库管理系统要执行安全保护,采取多种存取控制机制,目的是只允许用户执行合法操作,在用户退出系统后,根据情况还要进行安全审计。操作系统也会有自己的保护措施,读者可参考操作系统的有关书籍。对敏感数据,在传输过程中要进行加密保护,最后还应以密码形式存储到数据库中。

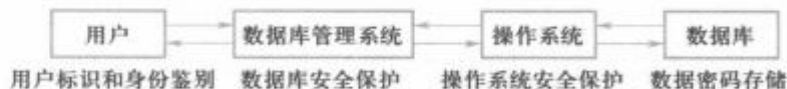


图 4.1 计算机系统的安全模型

本章讨论与数据库有关的安全性技术,对于强力逼迫透露口令、盗窃物理存储设备等行为而采取的保安措施,例如出入机房登记、加锁等,不在讨论之列。

图 4.2 是数据库管理系统的安全性控制模型示例。主要包含以下几部分:

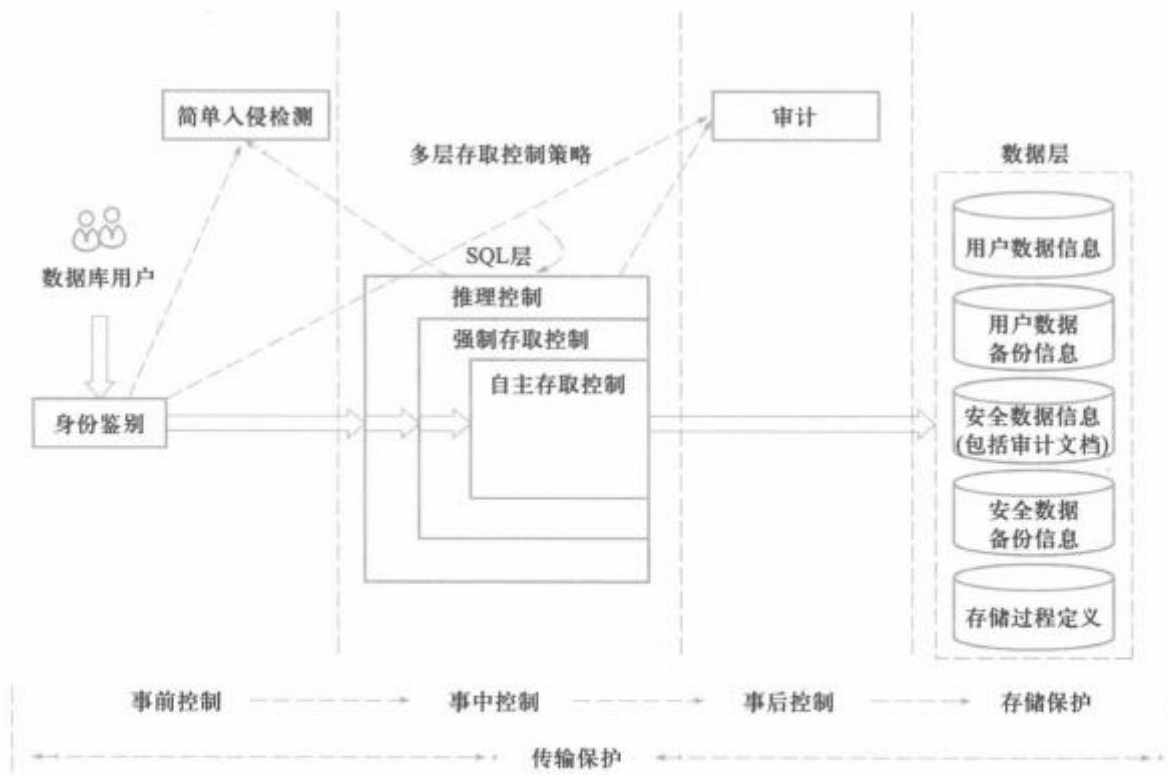


图 4.2 数据库管理系统的安全性控制模型

① 事前控制。通过身份鉴别和简单入侵检测共同实现事前的访问保护。

② 事中控制。数据库管理系统在 SQL 层提供多层存取控制策略,包括自主存取控制、强制存取控制、推理控制等。用户只能按照被授予的权限和安全规则存取合法数据。

③ 事后控制。为监控恶意访问,可根据具体安全需求配置审计规则,对用户访问行为和系统关键操作进行审计,及时发现可疑攻击者,把有关信息返回入侵检测子系统,对异常用户行为进行处理和阻拦。

④ 存储保护。在数据层,对敏感数据、重要的存储过程定义加密存储。数据库管理系统不仅存放用户数据,还存储与安全有关的审计文档,并进行加密存储。

⑤ 传输保护。用户身份信息、SQL 语句和数据等频繁地在客户端与服务器端进行传输,若采用明文方式传输很容易被截获或篡改,存在安全隐患。为了保证这些信息能够安全地进行交换和传输,数据库管理系统提供了传输加密功能。

4.2.1 用户身份鉴别

用户身份鉴别是数据库管理系统提供的最外层安全保护措施。每个用户在系统中都有一个用户标识。每个用户标识由用户名(user name)和用户标识号(user identification number, UID)两部分组成。UID 在系统的整个生命周期内是唯一的。系统内部记录着所有合法用户的标识,系统鉴别是指由系统提供一定的方式让用户标识自己的名字或身份。每次用户要求进入系统时,由系统进行核对,通过鉴定后才提供使用数据库管理系统的权限。

用户身份鉴别的方法有很多种,而且在一个系统中往往也是多种方法结合,以获得更强的安全性。常用的用户身份鉴别方法主要有以下几种。

1. 静态口令鉴别

静态口令一般由用户自己设定,鉴别时只要按要求输入正确的密码,系统即可允许用户使用数据库管理系统。这些密码是静态不变的,方式简单,但容易被攻击,安全性较低。因此通常采用双因子鉴别,即口令+数字证书。提供一套口令策略,包括密码复杂度、密码过期时限、登录失败用户锁定、密码历史保存等。在存储和传输过程中密码信息不可见,均以密文方式存在。

用户身份鉴别可以重复多次。

2. 动态口令鉴别

这种方式的口令是动态变化的,每次鉴别时均需使用动态产生的新口令登录数据库管理系统,即采用一次一密的方法。常用的方式如验证码和动态令牌方式,每次鉴别时要求用户使用验证码或动态令牌登录数据库管理系统。与静态口令鉴别相比,这种认证方式的安全性相对高一些。

3. 生物特征鉴别

它是一种通过生物特征进行认证的技术,其中,生物特征是指生物唯一具有的可测量、识别和验证的稳定生物特征,如指纹、虹膜、掌纹和人脸识别等。这种方式通过采用图像处理和

模式识别等技术实现了基于生物特征的认证,与传统的口令鉴别相比,其安全性无疑较高。

4. 智能卡鉴别

智能卡是一种不可复制的硬件,内置集成电路的芯片,具有硬件加密功能。智能卡由用户随身携带,登录数据库管理系统时将智能卡插入专用的读卡器进行身份验证。由于每次从智能卡中读取的数据是静态的,通过内存扫描或网络监听等技术还是可能截取到用户的身份验证信息,存在安全隐患。因此,实际应用中一般采用个人识别号(personal identification number, PIN)和智能卡相结合的方式。这样,即使PIN或智能卡中有一种被窃取,用户身份仍不会被冒充。

5. 入侵检测

检测前,管理员应按实际需求定义检测规则。在检测过程中,依据预先设置的检测规则实时进行入侵分析。若发现入侵情况,实时进行处理。

处理方式包括通过邮件报警和断开会话连接、锁定用户进行处罚等。

4.2.2 存取控制

数据库安全控制最重要的一点就是,确保只授权给有资格的用户访问数据库的权限,同时令所有未被授权的人员无法接近数据。这主要通过数据库系统的存取控制机制实现。

存取控制机制主要包括定义用户权限和合法权限检查两部分。

① 定义用户权限,并将用户权限登记到数据字典中。用户对某一数据对象的操作权力称为权限。某个用户应该具有何种权限是管理和政策方面的问题,而不是技术问题。数据库管理系统的功能是保证这些决定的执行。为此,数据库管理系统必须提供适当的语言来定义用户权限,这些定义经过编译后存储在数据字典中,被称作安全规则或授权规则。

② 合法权限检查。每当用户发出存取数据库的操作请求后(请求一般应包括操作类型、操作对象和操作用户等信息),数据库管理系统查找数据字典,根据安全规则进行合法权限检查,若用户的操作请求超出了定义的权限,系统将拒绝执行此操作。

定义用户权限和合法权限检查机制一起组成了数据库管理系统的存取控制子系统。

C2级的数据库管理系统支持自主存取控制,B1级的数据库管理系统支持强制存取控制。

这两类方法的简单定义是:

① 在自主存取控制方法中,用户对于不同的数据库对象有不同的存取权限,不同的用户对同一对象也有不同的权限,而且用户还可将其拥有的存取权限转授给其他用户。因此自主存取控制非常灵活。

② 在强制存取控制方法中,每一个数据库对象被标以一定的密级,每一个用户也被授予某一个级别的许可证。对于任意一个对象,只有具有合法许可证的用户才可以存取。强制存取控制因此相对比较严格。

下面介绍这两种存取控制方法。

4.2.3 自主存取控制方法

大型数据库管理系统都支持自主存取控制, SQL 标准也对自主存取控制提供支持。这主要通过 SQL 的 GRANT 语句和 REVOKE 语句来实现。

用户权限是由两个要素组成的: 数据库对象和操作类型。定义一个用户的存取权限就是要定义这个用户可以在哪些数据库对象上进行哪些类型的操作。在数据库系统中, 定义存取权限称为授权(authorization)。

在非关系数据库系统中, 用户只能对数据进行操作, 存取控制的数据库对象也仅限于数据本身。

在关系数据库系统中, 存取控制的对象不仅有数据本身(基本表和视图中的数据、属性列上的数据), 还有数据库与模式、基本表、视图和索引的创建等。表 4.4 列出了不同对象具有的主要存取权限。

表 4.4 关系数据库系统中不同对象具有的主要存取权限

对象类型	对象	操作类型
数据库和模式	数据库和模式	CREATE
	基本表	CREATE TABLE, ALTER TABLE
	视图	CREATE VIEW
	索引	CREATE INDEX
数据	基本表和视图	SELECT, INSERT, UPDATE, DELETE, REFERENCES, ALL PRIVILEGES
	属性列	SELECT, INSERT, UPDATE, REFERENCES, ALL PRIVILEGES

表 4.4 中, 创建数据库的权限在 SQL 标准中没有给出明确定义, 因此不同的关系数据库管理系统实现的语句和语法不同。4.2.4 小节将介绍 KingbaseES 创建数据库的语句。

具有创建数据库权限的用户可以在该数据库上创建模式。

具有创建模式权限的用户可以在该模式中创建表、视图、索引等数据库对象。

表 4.4 中, 属性列权限包括 SELECT、INSERT、UPDATE、REFERENCES, 其含义与表权限类似。需要说明的是, 对列的 UPDATE 权限指对于表中存在的某一列的值可以进行修改。当然, 有了这个权限之后, 在修改的过程中还要遵守表在创建时定义的主码及其他约束。列上的 INSERT 权限指用户可以插入一个元组, 对于插入的元组, 授权用户可以插入指定的值, 其他列或者为空, 或者为默认值。在给用户授予列 INSERT 权限时, 一定要包含主码的 INSERT 权限, 否则用户的插入动作会因为主码为空而被拒绝。

4.2.4 授予与收回对数据的操作权限

SQL 中使用 GRANT 和 REVOKE 语句向用户授予或收回对数据的操作权限。GRANT 语句向用户授予权限, REVOKE 语句收回已经授予用户的权限。

1. GRANT 语句

GRANT 语句的一般格式如下:

```
GRANT <权限> [, <权限>] ...  
ON <对象类型> <对象名> [, <对象类型> <对象名>] ...  
TO <用户> [, <用户>] ...  
[WITH GRANT OPTION];
```

其语义为:将对指定对象的指定操作权限授予指定的用户。发出该 GRANT 语句的可以是数据库管理员(DBA),也可以是该数据库的创建者(即数据库属主 owner),还可以是已经拥有该权限的用户。接受权限的用户可以是一个或多个具体用户,也可以是 PUBLIC,即全体用户。

如果指定了 **WITH GRANT OPTION** 子句,则获得某种权限的用户还可以把这种权限再授予其他的用户。如果没有指定该子句,则获得某种权限的用户只能使用该权限,不能传播。

SQL 标准允许具有 WITH GRANT OPTION 的用户把相应权限或其子集传递授予其他用户,但不允许循环授权,即被授权者不能把权限再授回给授权者或其祖先,如图 4.3 所示。



图 4.3 SQL 标准不允许循环授权

[例 4.1] 把查询 Student 表的权限授给用户 U1。

```
GRANT SELECT  
ON TABLE Student  
TO U1;
```

[例 4.2] 把对 Student 表和 Course 表的全部操作权限授予用户 U2 和 U3。

```
GRANT ALL PRIVILEGES  
ON TABLE Student, Course  
TO U2, U3;
```

[例 4.3] 把对表 SC 的查询权限授予所有用户。

```
GRANT SELECT
```

```
ON TABLE SC  
TO PUBLIC;
```

[例 4.4] 把查询 Student 表和修改学生学号的权限授予用户 U4。

```
GRANT UPDATE(Sno),SELECT  
ON TABLE Student  
TO U4;
```

这里, 实际上要授予 U4 用户的是对基本表 Student 的 SELECT 权限和对属性列 Sno 的 UPDATE 权限。对属性列授权时必须明确指出相应的属性列名。

[例 4.5] 把对表 SC 的 INSERT 权限授予 U5 用户, 并允许将此权限再授予其他用户。

```
GRANT INSERT  
ON TABLE SC  
TO U5  
WITH GRANT OPTION;
```

执行此 SQL 语句后, U5 不仅拥有了对表 SC 的 INSERT 权限, 还可以传播此权限, 即由 U5 用户发上述 GRANT 命令给其他用户。

[例 4.6] 用户 U5 将对表 SC 的 INSERT 权限授予用户 U6。

```
GRANT INSERT  
ON TABLE SC  
TO U6  
WITH GRANT OPTION;
```

同样, U6 还可以将此权限继续授予 U7。

[例 4.7] 用户 U6 将对表 SC 的 INSERT 权限授予用户 U7。

```
GRANT INSERT  
ON TABLE SC  
TO U7;
```

因为 U6 未给 U7 传播的权限, 因此 U7 不能再传播此权限。

由上面的例子可以看到, GRANT 语句可以一次向一个用户授权, 如例 4.1 所示, 这是最简单的一种授权操作; 也可以一次向多个用户授权, 如例 4.2、4.3 所示; 还可以一次传播多个同类对象的权限, 如例 4.2 所示; 甚至一次可以完成对基本表和属性列这些不同对象的授权, 如例 4.4 所示。表 4.5 是执行了例 4.1~4.7 的语句后“学生选课”数据库中的用户权限定义表。

表 4.5 执行了例 4.1~4.7 语句后“学生选课”数据库中的用户权限定义

授权用户名	被授权用户名	数据库对象名	允许的操作类型	能否转授权
DBA	U1	关系 Student	SELECT	不能
DBA	U2	关系 Student	ALL	不能
DBA	U2	关系 Course	ALL	不能
DBA	U3	关系 Student	ALL	不能
DBA	U3	关系 Course	ALL	不能
DBA	PUBLIC	关系 SC	SELECT	不能
DBA	U4	关系 Student	SELECT	不能
DBA	U4	属性列 Student. Sno	UPDATE	不能
DBA	U5	关系 SC	INSERT	能
U5	U6	关系 SC	INSERT	能
U6	U7	关系 SC	INSERT	不能

2. REVOKE 语句

授予用户的权限可以由数据库管理员或其他授权者用 REVOKE 语句收回。REVOKE 语句的一般格式如下：

```
REVOKE <权限> [, <权限>] ...
ON <对象类型> <对象名> [, <对象类型> <对象名>] ...
FROM <用户> [, <用户>] ... [ CASCADE | RESTRICT ] ;
```

若语句指定了 CASCADE，则级联收回授予的权限；若语句指定了 RESTRICT，则转授权限后不能收回。默认值为 RESTRICT。

[例 4.8] 把用户 U4 修改学生学号的权限收回。

```
REVOKE UPDATE(Sno)
ON TABLE Student
FROM U4;
```

[例 4.9] 收回所有用户对表 SC 的查询权限。

```
REVOKE SELECT
ON TABLE SC
FROM PUBLIC;
```

[例 4.10] 把用户 U5 对 SC 表的 INSERT 权限收回。

```
REVOKE INSERT
```

```
ON TABLE SC
FROM U5 CASCADE;
```

执行该语句后，收回了用户 U5 的 INSERT 权限，同时级联收回了 U6 和 U7 的 INSERT 权限。因为在例 4.6 和例 4.7 中，U5 将对 SC 表的 INSERT 权限授予了 U6，而 U6 又将其授予了 U7。

注意：如果 U6 或 U7 还从其他用户处获得了对 SC 表的 INSERT 权限，则他们仍具有此权限，系统只收回直接或间接从 U5 处获得的权限。

表 4.6 是执行了例 4.8~4.10 的语句后“学生选课”数据库的用户权限定义。

表 4.6 执行了例 4.8~4.10 语句后“学生选课”数据库的用户权限定义

授权用户名	被授权用户名	数据库对象名	允许的操作类型	能否转授权
DBA	U1	关系 Student	SELECT	不能
DBA	U2	关系 Student	ALL	不能
DBA	U2	关系 Course	ALL	不能
DBA	U3	关系 Student	ALL	不能
DBA	U3	关系 Course	ALL	不能
DBA	U4	关系 Student	SELECT	不能

SQL 提供了非常灵活的授权机制。数据库管理员拥有对数据库中所有对象的所有权限，并可以根据实际情况将不同的权限授予不同的用户。

用户对自己建立的基本表和视图拥有全部的操作权限，并且可以用 GRANT 语句把其中某些权限授予其他用户。被授权的用户如果获得 WITH GRANT OPTION，还可以把获得的权限再授予其他用户。

所有授予出去的权力在必要时又都可以用 REVOKE 语句收回。

可见，用户可以“自主”地决定将数据的存取权限授予何人，并决定是否也将“授权”的权限授予别人。因此，称这样的存取控制是自主存取控制。

3. 创建数据库的权限

上面介绍的 GRANT 和 REVOKE 语句是向用户授予或收回对数据库中数据的操作权限。这里简单介绍一下表 4.4 中关于创建数据库的语句。因为创建数据库也需要进行安全控制，也是要授权的。但是在 SQL 标准中没有创建数据库的标准定义，因此各个数据库系统实现的语句不同。下面仅以金仓数据库 KingbaseES 为例做简要说明。

CREATE USER 语句建立超级用户，再由超级用户创建具有 CREATE 数据库权限的用户。

CREATE USER 语句一般语法格式如下：

```
CREATE USER <username> [ WITH ]
[ SUPERUSER | CREATEDB ] [ PASSWORD 'password' ;
```

具有 SUPERUSER 权限的用户是系统的超级用户，在系统中跳过权限检查，可以执行任何操作。

具有 CREATEDB 权限的用户可以创建数据库，成为数据库的属主，具有在数据库上创建模式的权限，并可以把这些权限授予其他用户。

注意，超级用户是系统初始化时指定的。例如，在安装系统时可以指定一个名称为 system 的超级用户。

[例 4.11] 创建超级用户 system2。

首先以超级用户 system 登录，然后创建 system2：

```
CREATE USER system2 WITH SUPERUSER PASSWORD '123456';
```

[例 4.12] 创建具有 CREATEDB 权限的用户 U1 和普通用户 U2。

以超级用户 system 登录，创建用户 U1、U2 如下：

```
CREATE USER U1 WITH CREATEDB PASSWORD '123456';  
/* U1 具有创建数据库的权限了 */  
CREATE USER U2 PASSWORD '123456';      /* U2 是普通用户 */
```

[例 4.13] 创建数据库 U1DB。

以 U1 用户登录，创建数据库 U1DB：

```
CREATE DATABASE U1DB;      /* U1 创建了数据库 */
```

U1 成为数据库 U1DB 的属主，它可以在 U1DB 数据库上创建 SCHEMA。

4.2.5 数据库角色

数据库角色是被命名的一组与数据库操作相关的权限，角色是权限的集合。因此，可以为具有相同权限的用户创建一个角色。使用角色来管理数据库权限可以简化授权的过程。

在 SQL 中首先用 CREATE ROLE 语句创建角色，然后用 GRANT 语句给角色授权，用 REVOKE 语句收回授予角色的权限。

1. 角色的创建

创建角色的 SQL 语句格式如下：

```
CREATE ROLE <角色名>
```

刚刚创建的角色是空的，没有任何内容。可以用 GRANT 为角色授权。

2. 给角色授权

```
GRANT <权限> [, <权限>] ...  
ON <对象类型> 对象名  
TO <角色> [, <角色>] ...
```

数据库管理员和用户可以利用 GRANT 语句将权限授予某一个或几个角色。

3. 将一个角色授予其他的角色或用户

```
GRANT <角色 1> [, <角色 2>] ...  
TO <角色 3> [, <用户 1>] ...  
[ WITH ADMIN OPTION]
```

该语句把角色授予某用户, 或授予另一个角色。这样, 一个角色(例如角色 3)所拥有的权限就是授予它的全部角色(例如角色 1 和角色 2)所包含的权限的总和。

授予者可以是角色的创建者, 或者是在这个角色上的 WITH ADMIN OPTION 权限拥有者。

如果指定了 WITH ADMIN OPTION 子句, 则获得某种权限的角色或用户还可以把这种权限再授予其他的角色。

一个角色包含的权限包括直接授予这个角色的全部权限, 再加上其他角色授予这个角色的全部权限。

4. 角色权限的收回

```
REVOKE <权限> [, <权限>] ...  
ON <对象类型> <对象名>  
FROM <角色> [, <角色>] ...
```

用户可以收回角色的权限, 从而修改角色拥有的权限。

REVOKE 动作的执行者可以是角色的创建者, 或者是在这个(些)角色上的 WITH ADMIN OPTION 权限拥有者。

[例 4.14] 通过角色来实现将一组权限授予一个用户。

步骤如下:

① 创建一个角色 R1。

```
CREATE ROLE R1;
```

② 使用 GRANT 语句, 使角色 R1 拥有 Student 表的 SELECT、UPDATE、INSERT 权限。

```
GRANT SELECT, UPDATE, INSERT  
ON TABLE Student  
TO R1;
```

③ 将这个角色授予王平、张明、赵玲, 使他们具有角色 R1 所包含的全部权限。

```
GRANT R1  
TO 王平, 张明, 赵玲;
```

④ 一次性通过 R1 来收回王平的这三个权限。

```
REVOKE RI  
FROM 王平;
```

[例 4.15] 给角色增加新的权限。

```
GRANT DELETE  
ON TABLE Student  
TO RI
```

使角色 R1 在原来的基础上增加了 Student 表的 DELETE 权限。

[例 4.16] 减少角色的权限。

```
REVOKE SELECT  
ON TABLE Student  
FROM RI;
```

使 R1 减少了 SELECT 权限。

可以看出, 数据库角色是一组权限的集合。使用角色来管理数据库权限可以简化授权的过程, 使自主授权的执行更加灵活、方便。

4.2.6 强制存取控制方法

自主存取控制能够通过授权机制有效地控制对敏感数据的存取。但是由于用户对数据的存取权限是“自主”的, 用户可以自由地决定将数据的存取权限授予何人, 以及决定是否也将“授权”的权限授予别人。在这种授权机制下, 仍可能存在数据的无意泄露。比如, 甲将自己权限范围内的某些数据存取权限授权给乙, 甲的意图是仅允许乙本人操纵这些数据。但甲的这种安全性要求并不能得到保证, 因为乙一旦获得了对数据的权限, 就可以将数据备份, 获得自身权限内的副本, 并在不征得甲同意的前提下传播副本。造成这一问题的根本原因就在于这种机制仅仅通过对数据的存取权限来进行安全控制, 而数据本身并无安全性标记。要解决这一问题, 就需要对系统控制下的所有主客体实施强制存取控制策略。

所谓强制存取控制, 是指系统为保证更高层次的安全性, 按照 TCSEC/TDI 标准中安全策略的要求所采取的强制存取检查手段。它不是用户能直接感知或进行控制的。强制存取控制适用于那些对数据严格且固定密级分类的部门, 例如军事部门或政府部门。

在强制存取控制方法中, 数据库管理系统所管理的全部实体被分为主体和客体两大类。

主体是系统中的活动实体, 既包括数据库管理系统所管理的实际用户, 也包括代表用户的各进程。**客体**是系统中的被动实体, 是受主体操纵的, 包括文件、基本表、索引、视图等。对于主体和客体, 数据库管理系统为它们的每个实例(值)指派一个敏感度标记(label)。

敏感度标记被分成若干级别, 例如绝密级(top secret, TS)、机密级(secret, S)、秘密级(confidential, C)、公开(public, P)等。密级的次序是 $TS \geq S \geq C \geq P$ 。主体的敏感度标记称为许可证级别, 客体的敏感度标记称为密级。强制存取控制机制就是通过对主体和客体

的 label, 最终确定主体是否能够存取客体。

当某一用户(或主体)以标记 label 注册入系统时, 系统要求他对任何客体的存取必须遵循如下规则:

- ① 仅当主体的许可证级别大于或等于客体的密级时, 该主体才能读取相应的客体。
- ② 仅当主体的许可证级别小于或等于客体的密级时, 该主体才能写相应的客体。

这些系统规定: 仅当主体的许可证级别小于或等于客体的密级时, 该主体才能写相应的客体, 即用户可以为写入的数据对象赋予高于自己许可证级别的密级。这样一旦数据被写入, 该用户自己也不能再读该数据对象了。

规则①的含义是明显的, 而规则②需要解释一下。按照规则②, 用户可以为写入的数据对象赋予高于自己的许可证级别的密级。这样一旦数据被写入, 该用户自己也不能再读该数据对象了。如果违反了规则②, 就有可能把数据的密级从高流向低, 造成数据的泄漏。

例如, 某个绝密级的用户把一个绝密级别的数据恶意地降低为公开级别, 然后把它写回, 这样原来是绝密的数据大家都可以读到了, 造成了绝密级数据的泄漏。

这两种规则的共同点在于, 它们均禁止了拥有高许可证级别的主体更新低密级的数据对象, 从而防止了敏感数据的泄漏。

强制存取控制是对数据本身进行密级标记, 无论数据如何复制, 标记与数据是一个不可分的整体。只有符合密级标记要求的用户才可以操纵数据, 从而提供了更高级别的安全性。

前面已经提到, 较高安全性级别提供的安全保护包含较低级别的所有保护, 因此在实现强制存取控制时要首先实现自主存取控制, 即自主存取控制与强制存取控制共同构成数据库管理系统的安全机制。图 4.4 为数据库管理系统的安全检查示意图, 系统首先进行自主存取控制(DAC)检查, 对通过 DAC 检查的允许存取的数据对象, 再由系统自动进行强制存取控制(MAC)检查, 只有通过 MAC 检查的数据对象方可存取。



图 4.4 数据库管理系统的安全检查示意图

4.3 视图机制

前已提及, 视图具有安全保护的功能。为不同的用户定义不同的视图, 就可以把数据对象限制在一定的范围内, 通过视图机制把保密的数据对无权存取的用户隐藏起来, 从而自动对数据提供一定程度的安全保护。

视图机制还可以和授权机制相配合, 间接地实现支持存取谓词的用户权限定义。例如, 假定王平老师只能查询计算机科学与技术专业学生的信息, 专业负责人张明老师具有查询和增删改该专业学生信息的所有权限。这就要求系统能支持存取谓词的用户权限定义。在不直接支持存取谓词的系统中, 可以先建立计算机科学与技术专业学生的视图 CS_Student, 然后在视图上

进一步定义存取权限。

[例 4.17] 建立计算机科学与技术专业学生的视图, 把对该视图的 SELECT 权限授予王平, 把该视图上的所有操作权限授予张明。

```
CREATE VIEW CS_Student    /* 先建立视图 CS_Student */
AS
SELECT *
FROM Student
WHERE Smajor='计算机科学与技术'
WITH CHECK OPTION;      /* 对该视图进行增、删、改时, 必须满足 */
                          /* Smajor='计算机科学与技术'的条件 */
GRANT SELECT              /* 王平老师只能查询计算机科学与技术学生的信息 */
ON CS_Student
TO 王平;
GRANT ALL PRIVILEGES     /* 专业负责人张明具有查询和增、删、改该视图的所有权限 */
ON CS_Student
TO 张明;
```

4.4 审 计

数据库安全审计系统提供了一种事后检查的安全机制。前面讲的用户身份鉴别、存取控制是数据库安全保护的重要技术(安全策略方面), 但不是全部。审计(audit)功能就是数据库管理系统达到 C2 以上安全级别必不可少的一项指标。

任何系统的安全保护措施都不是完美无缺的, 蓄意盗窃、破坏数据的人总是想方设法打破控制。审计功能把用户对数据库的所有操作自动记录下来放入审计日志(audit log)。审计员可以利用审计日志监控数据库中的各种行为, 重现导致数据库现有状况的一系列事件, 找出非法存取数据的人、时间和内容等; 还可以通过对审计日志分析, 对潜在的威胁提前采取措施加以防范。

审计通常是很费时间和空间的, 所以数据库管理系统往往都将审计设置为可选特征, 允许数据库管理员根据具体应用对安全性要求灵活地打开或关闭审计功能。审计功能主要用于安全性要求较高的部门。它能对普通和特权用户行为、各种表操作、身份鉴别、自主和强制访问控制等操作进行审计。它既能审计成功操作, 也能审计失败操作。

1. 审计事件

审计事件一般有多种类别, 例如:

① 服务器事件。审计数据库服务器发生的事件, 包含数据库服务器的启动、停止、数据库服务器配置文件的重新加载等。

② 系统权限。审计对系统拥有的结构或模式对象进行的操作，要求该操作的权限是通过系统权限获得的。

③ 语句事件。审计 SQL 语句，如 DDL、DML 及 DCL 语句。

④ 模式对象事件。审计特定模式对象上进行的 SELECT 或 DML 操作。模式对象包括表、视图、存储过程、函数等，但不包括依附于表的索引、约束、触发器、分区表等。

2. 审计功能

审计功能主要包括以下几方面内容：

① 基本功能。提供多种审计查阅方式：基本、可选、有限等。

② 提供多套审计规则。审计规则一般在数据库初始化时设定，以方便审计员管理。

③ 提供审计分析和报表功能。

④ 提供审计日志管理功能。主要包括为防止审计员误删审计记录，审计日志必须先转储后删除；对转储的审计记录文件提供完整性和保密性保护；只允许审计员查阅和转储审计记录，不允许任何用户新增和修改审计记录等。

系统提供查询审计设置及审计记录信息的专门视图。对于系统权限级别、语句级别及模式对象级别的审计记录，也可通过相关的系统表直接查看。

3. AUDIT 语句和 NOAUDIT 语句

AUDIT 语句用来设置审计功能，NOAUDIT 语句则用来取消审计功能。

审计一般可以分为用户级审计和系统级审计。用户级审计是任何用户可设置的审计，主要是用户针对自己创建的数据库表或视图进行审计，记录所有用户对这些表或视图的一切成功和（或）不成功的访问要求，以及各种类型的 SQL 操作。

系统级审计只能由数据库管理员设置，用以监测成功或失败的登录要求、监测授权和收回操作，以及其他数据库级权限下的操作。

[例 4.18] 对修改 SC 表结构或修改 SC 表数据的操作进行审计。

① 先显示当前审计开关状态。

```
SHOW AUDIT_TRAIL;
```

② 打开审计开关。

```
SET AUDIT_TRAIL TO ON;
```

③ 对 SC 表设置审计。

```
AUDIT ALTER, UPDATE ON SC BY ACCESS;
```

BY ACCESS 审计方式表示系统对每个设置的审计操作都要进行记录。BY SESSION 审计方式则表示对于每次会话中涉及的同类审计操作，系统只记录最早的一次。

[例 4.19] 取消对 SC 表的 ALTER 和 UPDAE 操作审计。

NOAUDIT ALTER, UPDATE ON SC;

审计设置以及审计日志一般都存储在数据字典中。必须把审计开关打开,才可以在系统表 SYS_AUDITTRAIL 中查看到审计信息。

安全审计机制将特定用户或特定对象相关的操作记录到系统审计日志中,作为后续对操作的查询分析和追踪的依据。通过审计机制,可以约束用户可能的恶意操作。

4.5 数据加密

数据库的安全性归根结底是要保护数据的安全,特别是高度敏感性数据,例如财务数据、军事数据、国家机密数据等。除前面介绍的安全性措施外,还可以采用数据加密技术。数据加密是防止数据库数据在存储和传输中失密的有效手段。加密的基本思想是根据一定的算法将原始数据——明文 (plaintext) 经过一系列复杂的计算,变换为不可直接识别的格式——密文 (ciphertext),从而使不知道解密算法的人无法获知数据的内容。

数据加密主要包括存储加密和传输加密。

1. 存储加密

存储加密一般有透明和非透明两种方式。透明存储加密是内核级加密保护方式,对用户完全隐蔽;非透明存储加密则是通过多个加密函数实现的。

透明存储加密是数据在写到磁盘时对数据进行加密,授权用户读取数据时再对其进行解密。由于数据加密对用户隐蔽,即用户不知道数据是被加密的密文,数据库的应用程序不需要做任何修改,只需在创建表语句中说明需加密的字段即可。当对加密数据进行增加、删除、修改和查询操作时,数据库管理系统将自动对数据进行加密、解密工作。基于数据库内核的数据存储加密、解密方法性能较好,安全性较高。

2. 传输加密

数据库在使用过程中通过网络传输登录系统的信息、SQL 语句和数据,若采用明文方式传输,容易被网络中的恶意用户截获或篡改,存在安全隐患。为了保证这些信息能够安全地进行交换和传输,数据库管理系统提供了传输加密功能。常用的传输加密方式如链路加密和端到端加密。其中,链路加密对传输数据在链路层进行加密,它的传输信息由报头和报文两部分组成,前者是路由选择信息,后者则是传送的数据信息。这种方式对报文和报头均加密。相对地,端到端加密对传输数据在发送端加密、接收端解密。它只加密报文,不加密报头。与链路加密相比,它只在发送端和接收端需要密码设备,而中间节点不需要密码设备,因此它所需的密码设备数量相对较少。但这种方式不加密报头,从而容易被非法监听者发现并从中获取敏感信息。

图 4.5 是一种基于安全套接层协议 (secure socket layer protocol, SSL protocol) 的数据库管理系统可信传输方案。它采用的是一种端到端的传输加密方式。在这个方案中,通信双方协商建



图 4.5 基于 SSL protocol 的数据库管理系统可信传输示意图

立可信连接，一次会话采用一个密钥，传输数据在发送端加密、接收端解密，有效降低了重放攻击和恶意篡改的风险。此外，出于易用性考虑，这个方案的通信加密还对应用程序透明。它的实现思路包含以下三点。

① 确认通信双方端点的可靠性。数据库管理系统采用基于数字证书的服务器和客户端认证方式实现通信双方的可靠性确认。用户和服务器各自持有由外部数字证书认证中心 (certificate authority, CA) 或企业内建 CA 颁发的数字证书，双方在进行通信时均首先向对方提供己方证书，然后使用本地的 CA 信任列表和证书撤销列表 (certificate revocation list, CRL) 对接收到的对方证书进行验证，以确保证书的合法性和有效性，进而保证对方确系通信的端。

② 协商加密算法和密钥。确认双方端点的可靠性后，通信双方协商本次会话的加密算法与密钥。在此过程中，通信双方利用公钥基础设施 (public key infrastructure, PKI) 方式确保服务器端和客户端协商过程通信的安全可靠。

③ 可信数据传输。在加密算法和密钥协商完成后，通信双方开始进行业务数据交换。与普通通信路径不同的是，这些业务数据在被发送之前将被用某一组特定的密钥进行加密和消息摘要计算，以密文形式在网络上传输。当业务数据被接收时，需用相同的一组特定的密钥进行解密和摘要计算。所谓特定的密钥是由先前通信双方磋商决定的，为且仅为双方共享，通常称之为会话密钥。第三方即使窃取传输密文，因无会话密钥也无法识别密文信息。一旦第三方对密文进行任何篡改，就会被真实的接收方通过摘要算法识破。另外，会话密钥的生命周期仅限于本次通信，理论上每次通信所采用的会话密钥不同，因此避免了使用固定密钥而引起的密钥存储类问题。

数据加密使用已有的密码技术和算法对数据库中存储的数据和传输的数据进行保护。常见的数据加密算法可以分为对称加密算法 (如 AES 算法) 和非对称加密算法 (如 RSA 算法)。各数据库管理系统还常常自己设计和采用特定的加密算法，使加密后数据的安全性得到进一步提高。即使攻击者获取数据源文件，也很难获取原始数据。但是，数据的加密和解密过程增加了查询处理的复杂性，将使查询效率受到影响。加密数据的密钥管理和数据加密对应用程序的影响，都是数据加密过程中需要考虑和解决的问题。

4.6 其他安全性保护

为满足较高安全等级数据库管理系统的安全性保护要求,在自主存取控制和强制存取控制之外,还有推理控制以及数据库应用中的隐蔽信道和数据隐私等技术。

1. 推理控制

推理控制(inference control)处理的是强制存取控制未解决的问题。例如,利用属性之间的函数依赖关系,用户能从低安全等级信息推导出其无权访问的高安全等级信息,进而导致信息泄露。

数据库推理控制机制用来避免用户利用其能够访问的数据推知更高密级的数据,即用户利用其被允许的多次查询结果,结合相关领域背景知识以及数据之间的约束,推导出其不能访问的数据。在推理控制方面,常用的方法如基于函数依赖的推理控制和基于敏感关联的推理控制等。例如,某公司信息系统中假设姓名和职务属于低安全等级(如公开)信息,而工资属于高安全等级(如机密)信息。用户A的安全等级较低,他通过授权可以查询自己的工资、姓名、职务,及其他用户的姓名和职务。由于工资是机密信息,因此用户A不应知道其他用户的工资。但是,若用户B的职务和用户A相同,则利用函数依赖关系职务 $\rightarrow$ 工资,用户A可通过自己的工资信息(假设3000元),推出B的工资也是3000元,从而导致高安全等级的敏感信息泄露。

2. 隐蔽信道

隐蔽信道(covert channel)处理的内容也是强制存取控制未解决的问题。下面的例子就是利用未被强制存取控制的SQL语句执行后反馈的信息进行了间接信息传递。

一般情况下,如果INSERT语句对UNIQUE属性列写入重复值,则系统会报错且操作失败。那么,针对UNIQUE约束列,高安全等级用户(发送者)可先向该列插入(或不插入)数据,而低安全等级用户(接收者)后向该列插入相同数据。

如果插入失败,则表明发送者已向该列插入数据,此时二者约定发送者传输信息位为0;如果插入成功,则表明发送者未向该列插入数据,此时二者约定发送者传输信息位为1。通过这种方式,高安全等级用户按事先约定方式主动向低安全等级用户传输信息,使得信息流从高安全等级向低安全等级流动,从而导致高安全等级敏感信息泄露。

3. 数据隐私

随着人们对隐私的重视,数据隐私成为数据库应用中新的数据保护模式。

数据隐私是控制不愿被他人知道或他人不便知道的个人数据的能力。数据隐私范围很广,涉及数据管理中的数据收集、数据存储、数据发布等各个阶段。例如,在数据存储阶段应避免非授权的用户访问个人的隐私数据。通常可以使用数据库安全技术实现这一阶段的隐私保护。如使用自主访问控制、强制访问控制和基于角色的访问控制以及数据加密等。在数据处理阶段,需要考虑数据推理带来的隐私数据泄露。非授权用户可能通过分析多次查询的结

果,或基于完整性约束信息推导出其他用户的隐私数据。在数据发布阶段,应使包含隐私的数据发布结果满足特定的安全性标准。如发布的关系数据表首先不能包含原有表的候选码,同时还要考虑准标识符的影响。

准标识符是能够唯一确定大部分记录的属性集合。在现有安全性标准中, k 匿名(k -anonymization)标准要求每个具有相同准标识符的记录组至少包括 k 条记录,从而控制攻击者判别隐私数据所属个体的概率。还有 l 多样性(l -diversity)标准、 t 临近(t -closeness)标准等,从而使攻击者不能从发布数据中推导出额外的隐私数据。数据隐私也是当前研究的热点。

4. “三权分立”的安全管理机制

在传统的数据库安全模型中,数据库管理员通常是具有极大权限的超级用户。而不对数据库管理员进行任何权限检查,实际上为数据库中的数据安全带来了隐患。为了解决数据库管理员权限过于集中的问题,遵照国家标准《信息安全技术 数据库管理系统安全技术要求》(GB/T 20273—2019),引进了“三权分立”的安全管理机制。详细介绍可参见二维码内容。



扩展阅读:三权分立的
安全管理机制

要想万无一失地保证数据库安全,使之免于遭到任何蓄意的破坏,几乎是不可能的。但严密的安全措施将使蓄意的攻击者付出高昂的代价,从而迫使攻击者不得不放弃其破坏企图。

本章小结

随着数据库应用的日益广泛和深入,以及计算机网络的发展,数据的安全保密越来越重要。数据库管理系统是管理数据的核心,因而其自身必须具有一整套完整而有效的安全性机制。

实现数据库系统安全性的技术和方法有多种,数据库管理系统提供的安全措施主要包括用户身份鉴别、入侵检测、自主存取控制和强制存取控制技术、视图技术和审计技术、数据存储加密和传输加密等技术。本章全面讲解了这些技术。

当前,数据库新技术的发展和应用使数据库安全性面临新的挑战,数据库安全保护技术的研究、创新和产品研发没有止境。

习题 4

1. 什么是数据库的安全性?
2. 举例说明对数据库安全性产生威胁的因素。
3. 试述实现数据库安全性控制的常用方法和技术。
4. 什么是数据库中的自主存取控制方法和强制存取控制方法?
5. 对下列两个关系模式:

学生(学号,姓名,年龄,性别,家庭住址,班级号)

班级(班级号,班级名,班主任,班长)

请用 SQL 的 GRANT 语句完成下列授权功能:

- ① 授予用户 U1 对两个表的所有权限,并可给其他用户授权。
 - ② 授予用户 U2 对“学生”表具有查看权限,对“家庭住址”具有更新权限。
 - ③ 将对“班级”表查看权限授予所有用户。
 - ④ 将对“学生”表的查询、更新权限授予角色 R1。
 - ⑤ 将角色 R1 授予用户 U1,并且 U1 可继续授权给其他角色。
6. 今有以下两个关系模式:

职工(职工号,姓名,年龄,职务,工资,部门号)

部门(部门号,名称,经理名,地址,电话号)

请用 SQL 的 GRANT 语句和 REVOKE 语句(加上视图机制)实现以下授权定义或存取控制功能:

- ① 用户王明对两个表有 SELECT 权限。
 - ② 用户李勇对两个表有 INSERT 和 DELETE 权限。
 - ③ 每个职工只对自己的记录有 SELECT 权限。
 - ④ 用户刘星对职工表有 SELECT 权限,对“工资”字段具有更新权限。
 - ⑤ 用户张新具有修改这两个表的结构权限。
 - ⑥ 用户周平具有对两个表的所有权限,并具有给其他用户授权的权限。
 - ⑦ 用户杨兰具有从每个部门职工中 SELECT 最高工资、最低工资、平均工资的权限,但不能查看每个人的工资。
7. 针对第 6 题中①~⑦的每一种情况,撤销各用户所授予的权限。
8. 理解并解释强制存取控制机制中主体、客体、敏感度标记的含义。
9. 举例说明强制存取控制机制是如何确定主体能否存取客体的。
10. 什么是数据库的审计功能,为什么要提供审计功能?

第 4 章实验 安全性控制

实验 4.1 自主存取控制实验。理解和掌握自主存取控制权限的定义和维护方法,包括定义用户、定义角色、分配权限给用户、分配权限给角色和回收权限等基本功能。使用用户名登录数据库,验证权限分配是否正确。

***实验 4.2 审计实验。**掌握数据库审计的设置和管理方法,以便监控数据库操作,维护数据库安全,包括设置数据库审计开关、审计数据库对象、审计数据库语句、查看数据库审计记录,以及验证审计设置是否生效。

参考文献 4

- [1] US Department of Defense. Department of defense trusted computer system evaluation criteria [M]. The 'Or-

ange Book' Series, 1985.

[2] NCSC. Trusted database management system interpretation(TDI)[R], 1991.

[3] GRIFFITHS P P, WADE B W. An authorization mechanism for a relational database system[J]. ACM Transactions on Database Systems, 1976, 1(3): 242-255.

文献[3]讨论了 SYSTEM R 的授权机制。关系数据库系统原型 SYSTEM R 首先提出了授权和收回权力的概念。

[4] 刘启原, 刘怡. 数据库与信息系统的的核心[M]. 北京: 科学出版社, 2000.

文献[4]系统讲解了信息系统的的核心问题, 包括安全模型、与安全有关的标准、密码学与信息保密、攻击检测技术和理论、数据库的安全性、数据仓库的安全问题、网络安全问题、Java 语言及其安全性问题、信息安全和病毒问题等。

[5] 张孝. 可信 COBASE 的系统强制存取控制的设计与实现[D]. 北京: 中国人民大学, 1998.

文献[5]系统讨论了数据库系统的的核心问题, 给出了可信 COBASE 的系统设计和实现技术。可信 COBASE 是 COBASE 2.0 的安全版本, 它依据 TCSEC/TDI 对 B1 级的要求进行设计, 实现了强制存取控制和安全审计。

[6] 张俊, 彭朝晖, 肖艳芹, 等. DBMS 安全性评估保护轮廓 PP 的研究与开发[C]//中国计算机学会: 第 22 届全国数据库学术会议论文集(技术报告篇), 计算机科学, 2005: 72-75, 79.

文献[6]在深入研究 CC 以及国外主流 DBMS CC 安全性评估情况的基础上, 总结出 DBMS PP 的开发原则和方法, 并初步研究和开发了一个 EAL4 级 DBMS PP。根据该 PP 对 Kingbase ES 进行安全性开发、测试和内部评估, 取得了良好的效果。

[7] 张效祥, 徐家福. 计算机科学技术百科全书[M]. 3 版. 北京: 清华大学出版社, 2018.

[8] 国家市场监督管理总局, 中国国家标准化管理委员会. 信息安全技术数据库管理系统安全技术要求: GB/T 20273-2019[S]. 北京: 中国标准出版社, 2019.

本章知识点讲解微视频:



用户身份鉴别



权限的表达与检查



强制访问控制

数据库的完整性是指数据库数据的正确性 (correctness) 和相容性 (compatibility)。数据的正确性是指数据库数据符合现实世界语义且反映当前的实际状况。数据的相容性是指数据库同一对象在不同关系表中的数据是相同的, 一致的。

本章导读

本章系统地讲解关系数据库管理系统完整性实现的机制, 包括完整性约束的定义机制、检查方法和违约处理方法等。

实体完整性和参照完整性是关系系统的两个不变性, 即关系模型必须具备的完整性约束。其他完整性约束则可以归入用户定义的完整性, 由现实世界语义约束确定。

完整性约束作为数据库模式的一部分存入数据字典, 在数据库数据被修改时, 关系数据库管理系统的完整性约束检查机制将按照数据字典中定义的这些约束进行检查和处理。数据库完整性的定义可由 SQL 的数据定义语言 (DDL) 实现。

本章还讲解了触发器 (trigger), 触发器可以实施更为复杂的完整性定义、检查和违约操作, 具有更精细和更强大的数据控制能力。因为触发器规则中的动作体可以很复杂, 通常是一段过程化 SQL。

读者要通过上机实验掌握 SQL 的完整性约束定义功能、属性和元组完整性约束的定义语句, 能够验证完整性约束检查机制是否发挥作用, 掌握违约处理方法。进一步地, 要掌握触发器的概念, 并能使用触发器实现更复杂的完整性控制方法。

5.1 数据库完整性概述

数据库的完整性是指数据库数据的正确性和相容性。例如, 学生的学号必须唯一, 学生的性别只能是男或女, 百分制的课程成绩取值范围为 0~100, 学生所选的课程必须是学校开设的课程, 学生所在的院系必须是学校已成立的院系等。

数据库的完整性和数据库的安全性是两个既有联系又不尽相同的概念。

数据库的完整性是防止数据库中存在不符合语义的数据,也就是防止数据库中存在不正确的数据。数据的安全性是保护数据库防止恶意的破坏和非法的存取。因此,完整性检查和控制防范对象是不合语义的、不正确的数据,防止它们进入数据库。安全性控制的防范对象是非法用户和非法操作,防止他们对数据库数据的非法存取。

为维护数据库的完整性,关系数据库管理系统必须能够实现以下功能。

1. 提供定义完整性约束的机制

完整性约束也称为完整性规则,是数据库中的数据必须满足的语义约束。它表达了给定的数据模型中数据及其联系所具有的制约和依存规则,用以限定符合数据模型的数据库状态以及状态的变化,以保证数据的正确、有效和相容。SQL 标准使用了一系列概念来描述完整性约束,包括关系模型的实体完整性、参照完整性和用户定义的完整性。这些完整性约束一般由 SQL 的数据定义语言语句来实现。它们作为数据库模式的一部分存入数据字典。

2. 提供检查完整性约束的方法

关系数据库管理系统中检查数据是否满足完整性约束的机制称为完整性检查。一般在 INSERT、UPDATE、DELETE 语句执行后开始检查,也可以在事务提交时检查。检查这些操作执行后数据库中的数据是否违背了完整性约束。

3. 提供完整性的违约处理方法

关系数据库管理系统若发现用户的操作违背了完整性约束,就采取一定的动作,如拒绝 (NO ACTION) 执行该操作或级联 (CASCADE) 执行其他操作,进行违约处理以保证数据的完整性。

早期的关系数据库管理系统不支持完整性约束检查,因为该项检查费时费资源。现在商用的关系数据库管理系统产品都支持完整性控制,不必由应用程序来完成,从而减轻了应用程序员的负担。

更重要的是,完整性控制已成为关系数据库管理系统核心支持的功能,从而能够为所有用户和应用提供一致的数据库完整性。由应用程序来实现完整性控制是有漏洞的,有的应用程序定义的完整性约束可能会被其他应用程序破坏,因此数据库数据的正确性仍然无法保障。

本书第2章2.3节已讲解了关系数据库三类完整性约束的基本概念,下面将介绍用 SQL 实现这些完整性控制功能的方法。

5.2 实体完整性

5.2.1 定义实体完整性

关系模型的实体完整性在 CREATE TABLE 中用 PRIMARY KEY 定义。对单属性构成的码有两种说明方法,一种是定义为列级约束,另一种是定义为表级约束。对多个属性构成的码只

有一种说明方法,即定义为表级约束。

[例 5.1] 创建“学生”表 Student, 将 Sno 属性定义为主码。

```
CREATE TABLE Student
( Sno CHAR(8) PRIMARY KEY,           /* 在列级定义主码 */
  Sname CHAR(20) UNIQUE,
  Ssex CHAR(6),
  Sbirthdate Date,
  Smajor VARCHAR(40)
);
```

或者

```
CREATE TABLE Student
( Sno CHAR(8),
  Sname CHAR(20) UNIQUE,
  Ssex CHAR(6),
  Sbirthdate Date,
  Smajor VARCHAR(40),
  PRIMARY KEY(Sno)                   /* 在表级定义主码 */
);
```

[例 5.2] 创建“学生选课”表 SC, 将(Sno,Cno)属性组定义为主码。

```
CREATE TABLE SC
( Sno CHAR(8),
  Cno CHAR(5),
  Grade SMALLINT,
  Semester CHAR(5),                 /* 开课学期 */
  Teachingclass CHAR(8),           /* 学生选修某一门课所在的教学班 */
  PRIMARY KEY (Sno, Cno)           /* 主码由两个属性构成,必须在表级定义主码 */
);
```

5.2.2 实体完整性检查和违约处理

用 PRIMARY KEY 短语定义了关系的主码后,每当用户程序对基本表插入一条记录或对主码列进行更新操作时,关系数据库管理系统将按照实体完整性规则自动进行检查。包括:

- ① 检查主码值是否唯一,如果不唯一则拒绝插入或修改。
- ② 检查主码的各个属性是否为空,只要有一个为空就拒绝插入或修改。

从而保证了实体完整性。

检查记录中主码值是否唯一的一种方法是进行全表扫描,依次判断表中每一条记录的主码值与将插入记录的主码值(或修改的新主码值)是否相同,如图 5.1 所示。



图 5.1 用全表扫描方法检查主码唯一性

全表扫描是十分耗时的。为了避免对基本表进行全表扫描,关系数据库管理系统一般都在主码上自动建立一个索引,如图 5.2 所示的 $B+$ 树索引,通过索引查找基本表中是否已经存在新的主码值将大大提高效率。例如,如果新插入记录的主码值是 25,通过主码索引,从 $B+$ 树的根结点开始查找,只要读取三个结点就可以知道该主码值已经存在,即根结点(51)、中间结点(12 30)和叶结点(15 20 25)。所以不能插入这条记录。如果新插入记录的主码值是 86,也只要查找三个结点就可以知道该主码值不存在,所以可以插入该记录。

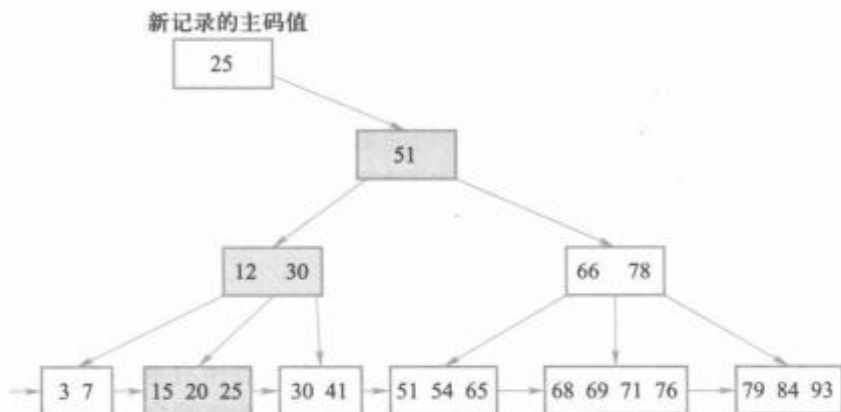


图 5.2 使用索引检查主码唯一性示例

5.3 参照完整性

5.3.1 定义参照完整性

关系模型的参照完整性在 CREATE TABLE 中用 FOREIGN KEY 短语定义哪些列为外码, 用 REFERENCES 短语指明这些外码参照哪些表的主码。

例如, 关系 SC 中一个元组表示一个学生选修某门课程的成绩, (Sno, Cno) 是主码。Sno、Cno 分别参照引用 Student 表的主码和 Course 表的主码。

[例 5.3] 定义 SC 中的参照完整性。

```
CREATE TABLE SC
(Sno CHAR(8),
 Cno CHAR(5),
 Grade SMALLINT,
 Semester CHAR(5),
 Teachingclass CHAR(8),
 PRIMARY KEY(Sno, Cno),      /* 在表级定义实体完整性 */
 FOREIGN KEY(Sno) REFERENCES Student(Sno),
                               /* 在表级定义参照完整性, Sno 是外码, 被参照表是 Student */
 FOREIGN KEY(Cno) REFERENCES Course(Cno)
                               /* 在表级定义参照完整性, Cno 是外码, 被参照表是 Course */
);
```

5.3.2 参照完整性检查和违约处理

参照完整性将两个表中的相应元组联系起来了。因此, 对被参照表和参照表进行更新(增、删、改)操作时有可能破坏参照完整性, 必须进行检查以保证这两个表的相容性。

例如, 对表 SC 和 Student 有 4 种可能破坏参照完整性的情况及违约处理, 如表 5.1 所示。

表 5.1 可能破坏参照完整性的情况及违约处理

被参照表 (如 Student)	参照表 (如 SC)	违约处理
可能破坏参照完整性	插入元组	拒绝
可能破坏参照完整性	修改外码值	拒绝
删除元组	可能破坏参照完整性	拒绝/级联删除/设置为空值
修改主码值	可能破坏参照完整性	拒绝/级联修改/设置为空值

① SC 表中增加一个元组, 该元组的 Sno 属性值在表 Student 中找不到一个元组, 其 Sno 属性值与之相等。

② 修改 SC 表中的一个元组, 修改后该元组的 Sno 属性值在表 Student 中找不到一个元组, 其 Sno 属性值与之相等。

③ 从 Student 表中删除一个元组, 造成 SC 表中某些元组的 Sno 属性值在表 Student 中找不到一个元组, 其 Sno 属性值与之相等。

④ 修改 Student 表中一个元组的 Sno 属性, 造成 SC 表中某些元组的 Sno 属性值在表 Student 中找不到一个元组, 其 Sno 属性值与之相等。

当上述不一致发生时, 系统可以采用以下策略加以处理。

① 拒绝执行。不允许该操作执行。该策略一般设置为默认策略。

② 级联操作。当删除或修改被参照表(Student)的一个元组导致与参照表(SC)的不一致时, 删除或修改参照表中所有导致不一致的元组。

例如, 删除 Student 表中 Sno 值为“20180001”的元组, 则要从 SC 表中级联删除“SC. Sno = '20180001'”的所有元组。

③ 设置为空值。当删除或修改被参照表的一个元组时造成不一致, 则将参照表中所有造成不一致的元组的对应属性设置为空值。例如, 下面两个关系:

学生(学号, 姓名, 性别, 出生日期, 主修专业)

专业(专业名, 专业编码)

其中“学生”关系的“主修专业”是外码, 参照“专业”关系的主码“专业名”。

假设“专业”表中“专业名”为计算机科学与技术的元组被删除, 按照表 5.1 的策略, 就要把“学生”表中“主修专业 = ‘计算机科学与技术’”的所有元组的主修专业设置为空值。这样处理的语义可以解释为: 某个专业删除了, 则选择该专业为主修专业的所有学生就要等待重新选择主修专业。

这里讲解一下外码能否接受空值的问题。

例如, “学生”表中“主修专业”是外码, 按照应用的实际情况可以取空值, 表示这个学生的主修专业尚未确定。

但在“学生选课”数据库中, 关系 SC 的 Sno 和 Cno 为外码, 它能否取空值呢? 答案是“不能”。因为 Sno 和 Cno 是关系 SC 的主属性, 按照实体完整性约束条件, 主属性不能为空值。若 SC 的 Sno 为空值, 则表明尚不存在的某个学生或者某个不知学号的学生选修了某门课程, 其成绩记录在 Grade 列中。这与学校的应用环境是不相符的, 因此 SC 的 Sno 列不能取空值。同样, SC 的 Cno 列也不能取空值。

因此对于参照完整性, 除了应该定义外码, 还应定义外码列是否允许空值。

一般地, 当对参照表和被参照表的操作违反了参照完整性时, 系统选用默认策略, 即拒绝执行。如果想让系统采用其他策略, 则必须在创建参照表时显式地加以说明。

[例 5.4] 显式说明参照完整性的违约处理示例。

```
CREATE TABLE SC
(
    Sno CHAR(8),
    Cno CHAR(5),
    Grade SMALLINT,      /* 成绩 */
    Semester CHAR(5),    /* 开课学期 */
    Teachingclass CHAR(8), /* 学生选修某一门课所在的教学班 */
    PRIMARY KEY(Sno, Cno), /* 在表级定义实体完整性, Sno、Cno 都不能取空值 */
    FOREIGN KEY(Sno) REFERENCES Student(Sno) /* 在表级定义参照完整性 */
    ON DELETE CASCADE /* 当删除 Student 表中的元组时, */
                        /* 级联删除 SC 表中 Sno 与 Student.Sno 相同的 SC 元组 */
    ON UPDATE CASCADE, /* 当更新 Student 表中的 Sno 时, */
                        /* 级联更新 SC 表中相应元组的 Sno */
    FOREIGN KEY(Cno) REFERENCES Course(Cno)
                        /* 在表级定义参照完整性 */
    ON DELETE NO ACTION /* 当删除 Course 表中的元组造成与 SC 表不一致, 即 SC 中存在
                        /* 这样的元组, 其 Cno 与 Course.Cno 相同, 则拒绝删除 Course
                        /* 表的这个元组 */
    ON UPDATE CASCADE /* 当更新 Course 表中的 Cno 时, */
                        /* 级联更新 SC 表中相应元组的 Cno */
);
```

可以对 DELETE 和 UPDATE 采用不同的策略。例如, 例 5.4 中当删除被参照表 Course 表中的元组, 造成与参照表(SC 表)不一致时, 拒绝删除被参照表的元组; 对更新操作则采用级联更新的策略。

从上面的讨论可以看到, 关系数据库管理系统在实现参照完整性时, 除了要提供定义主码、外码的机制外, 还需要提供不同的策略供用户选择。具体选择哪种策略, 要根据应用环境的要求确定。

5.4 用户定义的完整性

用户定义的完整性就是某一具体应用涉及的数据必须满足的语义要求。目前的关系数据库管理系统都提供了定义和检查这类完整性的机制, 使用和实体完整性、参照完整性相同的技术和方法来处理它们, 而不必由应用程序实现这一功能。

5.4.1 属性上的约束

1. 属性上约束的定义

在 CREATE TABLE 中定义属性的同时,可以根据应用要求定义属性上的约束,即属性值限制,包括:

- ① 列值非空(NOT NULL)。
- ② 列值唯一(UNIQUE)。
- ③ 检查列值是否满足一个条件表达式(CHECK 短语)。

2. 不允许取空值

[例 5.5]在定义 SC 表时,说明 Sno、Cno、Grade 属性不允许取空值。

```
CREATE TABLE SC
(
    Sno CHAR(8) NOT NULL,          /* Sno 属性不允许取空值 */
    Cno CHAR(5) NOT NULL,          /* Cno 属性不允许取空值 */
    Grade SMALLINT NOT NULL,       /* Grade 属性不允许取空值 */
    Semester CHAR(5),
    Teachingclass CHAR(8),
    PRIMARY KEY(Sno, Cno),         /* 在表级定义实体完整性,隐含 Sno、Cno 不允许 */
                                   /* 取空值,在列级不允许取空值的定义可不写 */
);
```

3. 列值唯一

[例 5.6]建立学院表 School,要求学院名称 SHname 列取值唯一,学院编号 SHno 列为主码。

```
CREATE TABLE School
(
    SHno CHAR(8) PRIMARY KEY,      /* SHno 列为主码 */
    SHname VARCHAR(40) UNIQUE,     /* 要求 SHname 值唯一 */
    SHfounddate Date               /* 学院创建日期 */
);
```

4. 用 CHECK 短语指定列值应该满足的条件

[例 5.7]Student 表的 Ssex 属性只允许取“男”或“女”。

```
CREATE TABLE Student
(
    Sno CHAR(8) PRIMARY KEY,        /* 在列级定义主码 */
    Sname CHAR(20) NOT NULL,        /* Sname 属性不允许取空值 */
    Ssex CHAR(6) CHECK(Ssex IN('男','女')), /* 性别属性 Ssex 只允许取'男'或'女' */
    Sbirthdate Date,
```

```
Smajor VARCHAR(40)
);
```

[例 5.8] SC 表 Grade 的值应该在 0~100。

```
CREATE TABLE SC
(Sno CHAR(8),
Cno CHAR(5),
Grade SMALLINT CHECK (Grade >= 0 AND Grade <= 100), /* Grade 取值范围 0~100 */
Semester CHAR(5),
Teachingclass CHAR(8),
PRIMARY KEY(Sno, Cno),
FOREIGN KEY(Sno) REFERENCES Student(Sno),
FOREIGN KEY(Cno) REFERENCES Course(Cno)
);
```

5. 属性上约束的检查和违约处理

当往表中插入元组或修改属性的值时，关系数据库管理系统将检查属性上的约束是否被满足，如果不满足则操作被拒绝执行。

5.4.2 元组上的约束

1. 元组上约束的定义

与属性上约束的定义类似，在 CREATE TABLE 语句中可以用 CHECK 短语定义元组上的约束，即元组级的限制。同属性值限制相比，元组级的限制可以设置不同属性之间取值的相互约束。

[例 5.9] 当学生的性别是“男”时，其姓名不能以 Ms. 打头。

```
CREATE TABLE Student
(Sno CHAR(8),
Sname CHAR(20) NOT NULL,
Ssex CHAR(6),
Sbirthdate Date,
Smajor VARCHAR(40),
PRIMARY KEY(Sno),
CHECK(Ssex='女' OR Sname NOT LIKE 'Ms. %')
); /* 定义了元组中 Sname 和 Ssex 两个属性值之间的约束条件 */
```

性别是女性的元组都能通过该项 CHECK 检查，因为 Ssex='女'成立；当性别是男性时，要通过检查则姓名一定不能以'Ms.'打头，因为 Ssex='男'时，条件要想为真值，Sname NOT LIKE 'Ms. %' 必须为真值。

2. 元组上约束的检查和违约处理

当往表中插入元组或修改属性的值时, 关系数据库管理系统将检查元组上的约束是否被满足, 如果不满足则操作被拒绝执行。

5.5 完整性约束命名子句

以上讲解的完整性约束都是在 CREATE TABLE 语句中定义的。SQL 还在 CREATE TABLE 语句中提供了完整性约束命名子句 CONSTRAINT, 用来对完整性约束命名, 从而可以灵活地增加、删除一个完整性约束。

1. 完整性约束命名子句格式

CONSTRAINT <完整性约束名> <完整性约束>

<完整性约束>包括 NOT NULL、UNIQUE、PRIMARY KEY、FOREIGN KEY、CHECK 短语等。

[例 5.10] 建立“学生”表 Student, 要求学号在 10000000 到 29999999 之间, 姓名不能取空值, 出生日期在 1980 年之后, 性别只能是“男”或“女”。

```
CREATE TABLE Student
( Sno CHAR(8)
  CONSTRAINT C1 CHECK(Sno BETWEEN '10000000' AND '29999999'),
  Sname CHAR(20)
  CONSTRAINT C2 NOT NULL,
  Sbirthdate Date
  CONSTRAINT C3 CHECK(Sbirthdate >'1980-1-1'),
  Ssex CHAR(6)
  CONSTRAINT C4 CHECK(Ssex IN('男','女')),
  Smajor VARCHAR(40),
  CONSTRAINT StudentKey PRIMARY KEY(Sno)
);
```

在 Student 表上建立了 5 个约束条件, 包括主码约束(命名为“StudentKey”)和 C1、C2、C3、C4 这 4 个列级约束。

[例 5.11] 建立“教师”表 Teacher, 要求每个教师的应发工资(每月)不低于 3 000 元。应发工资是工资列 Sal 与扣除项 Deduct 之和。

```
CREATE TABLE Teacher
( Eno CHAR(8) PRIMARY KEY,    /* 在列级定义主码 */
  Ename VARCHAR(20),
```

```

Job CHAR(8),
Sal NUMERIC(7,2),           /* 每月工资 */
Deduct NUMERIC(7,2),        * 每月扣除项 */
Schoolno CHAR(8),           /* 教师所在的学院编号 */
CONSTRAINT TeacherFKey FOREIGN KEY( Schoolno) REFERENCES School( Schoolno),
                               /* 外码约束(命名为 TeacherFKey) */
CONSTRAINT C1 CHECK( Sal+Deduct>=3000) /* 应发工资的约束条件 C1 */
);

```

2. 修改表中的完整性限制

可以使用 ALTER TABLE 语句修改表中的完整性约束。

[例 5.12] 去掉例 5.10 Student 表中对出生日期的限制。

```

ALTER TABLE Student
DROP CONSTRAINT C3;

```

[例 5.13] 修改表 Student 中的约束条件, 要求学号改为在 900000 到 999999 之间, 出生日期改为 1985 年之后。

可以先删除原来的约束条件, 再增加新的约束条件。

```

ALTER TABLE Student
DROP CONSTRAINT C1;
ALTER TABLE Student
ADD CONSTRAINT C1 CHECK( Sno BETWEEN '900000' AND '999999' );
ALTER TABLE Student
DROP CONSTRAINT C3;
ALTER TABLE Student
ADD CONSTRAINT C3 CHECK( Sbirthdate >'1985-1-1' );

```

* 5.6 域的完整性限制

在第 1、2 章中已经讲到, 域是数据库中一个重要概念。一般地, 域是一组具有相同数据类型的值的集合。SQL 支持域的概念, 可以用 CREATE DOMAIN 语句建立一个域并指明域应该满足的完整性约束, 然后就可以用域来定义属性。这样定义的优点是, 数据库中不同的属性可以来自同一个域, 当域的完整性约束改变时只要修改域的定义即可, 不必一一修改域上的各个属性。

[例 5.14] 建立一个性别域, 并声明性别域的取值范围。

```

CREATE DOMAIN GenderDomain CHAR(6)
CHECK( VALUE IN ('男','女') );

```

这样例 5.10 中对 Ssex 的说明可以改写为：

```
Ssex GenderDomain
```

[例 5.15] 建立一个性别域 GenderDomain，并对其中的限制命名。

```
CREATE DOMAIN GenderDomain CHAR(6)
CONSTRAINT GD CHECK(VALUE IN ('男','女'));
```

[例 5.16] 删除域 GenderDomain 的限制条件 GD。

```
ALTER DOMAIN GenderDomain
DROP CONSTRAINT GD;
```

[例 5.17] 在域 GenderDomain 上增加性别的限制条件 GDD。

```
ALTER DOMAIN GenderDomain
ADD CONSTRAINT GDD CHECK (VALUE IN('1','0'));
```

这样，通过例 5.16 和例 5.17，就把性别的取值范围由('男','女')改为('1','0')。

SQL 标准支持数据库完整性断言 (ASSERTION) 功能。用户可以使用 DDL 中的 CREATE ASSERTION 语句，通过声明性断言 (declarative assertion) 来指定更具一般性的完整性约束，可以定义涉及多个表或聚集操作的比较复杂的完整性约束 (详细内容可扫描二维码阅读)。



扩展阅读：SQL
断言

5.7 触 发 器

触发器是用户定义在关系表上的一类由事件驱动的特殊过程。触发器类似于约束，但是比约束更加灵活，可以实施更为复杂的检查和违约操作。它不是只提供一种选择，即中止导致违约的事务/操作，因此具有更精细和更强大的数据控制能力。

触发器一经定义就被保存在数据库服务器中。任何用户对表的更新操作均由服务器自动激活相应的触发器，在关系数据库管理系统核心层进行集中的完整性控制。

触发器在 SQL 99 之后才写入 SQL 标准，但是不少关系数据库管理系统早就支持触发器，因此不同的关系数据库管理系统实现的触发器语法各不相同、互不兼容。本节的例子是按照 SQL 标准写的，不同的关系数据库管理系统产品会有差别，请读者在上机实验时注意阅读所安装系统的使用说明。

5.7.1 定义触发器

触发器又叫作事件-条件-动作规则 (event-condition-action rule, ECA rule)。当特定的系

统事件(如对一个表的更新操作,事务的结束等)发生时,对规则的条件进行检查,如果条件成立则执行规则中的动作,否则不执行该动作。规则中的动作体可以很复杂,涉及其他表和其他数据库对象,通常是一段 SQL 存储过程。

SQL 使用 CREATE TRIGGER 命令建立触发器,其一般格式如下:

```
CREATE TRIGGER <触发器名>                                /* 每当触发事件发生时,该触发器被激活 */
| BEFORE | AFTER | <触发事件> ON <表名>                  /* 指明触发器激活的时间是在执行
                                                            /* 触发事件前或后 */
REFERENCING NEW|OLD AS<变量>                             /* REFERENCING 指出引用的变量 */
FOR EACH|ROW | STATEMENT|                                /* 定义触发器的类型,指明动作体执行的频率 */
[ WHEN <触发条件> ] <触发动作体>                        /* 仅当触发条件为真时才执行触发动作体 */
```

下面对定义触发器的各部分语法进行详细说明。

① 创建触发器。只有表的拥有者,即创建表的用户才可以在表上创建触发器,并且一个表上只能创建一定数量的触发器。触发器的具体数量由具体的关系数据库管理系统产品在设计时确定。

② 触发器名。触发器名可以包含模式名,也可以不包含模式名。同一模式下的触发器名必须是唯一的,并且触发器名和表名必须在同一模式下。

③ 表名。触发器只能定义在基本表上,不能定义在视图上。当基本表的数据发生变化时,将激活定义在该表上相应触发事件的触发器,因此该表也称为触发器的目标表。

④ 触发事件。触发事件可以是 INSERT、DELETE 或 UPDATE,也可以是这几个事件的组合,如 INSERT OR DELETE、INSERT AND DELETE 等,还可以是 UPDATE OF <触发列, ...>,即进一步指明修改哪些列时激活触发器。

AFTER/BEFORE 是触发的时机。AFTER 表示在触发事件的操作执行之后激活触发器, BEFORE 则表示在触发事件的操作执行之前激活触发器。

⑤ 触发器类型。触发器按照所触发动作的间隔尺寸可以分为行级触发器(FOR EACH ROW)和语句级触发器(FOR EACH STATEMENT)。

如果定义的触发器为语句级触发器,那么执行了此 UPDATE 语句后触发动作体将执行一次。如果是行级触发器,那么 UPDATE 语句影响多少行,就触发多少次。

⑥ 触发条件。触发器被激活时,只有当触发条件为真时触发动作体才执行,否则触发动作体不执行。如果省略 WHEN 触发条件,则触发动作体在触发器激活后立即执行。

⑦ 触发动作体。触发动作体既可以是一个匿名 PL/SQL 过程块,也可以是对已创建存储过程的调用。如果是行级触发器,用户可以在过程体中使用 NEW 或 OLD 引用 UPDATE/INSERT 事件之后(前)的新(旧)值;如果是语句级触发器,则不能在触发动作体中使用 NEW 或 OLD 进行引用。

如果触发动作体执行失败,激活触发器的事件(即对数据库的更新操作)就会终止执行,触发器的目标表或触发器可能影响的其他对象不发生任何变化。

[例 5.18] 当对表 SC 的 Grade 属性进行修改时, 若分数增加了 10%, 则将此次操作记录到另一个表 SC_U(Sno CHAR(8), Cno CHAR(5), Oldgrade SMALLINT, Newgrade SMALLINT) 中。其中, Oldgrade 是修改前的分数, Newgrade 是修改后的分数。运行触发器之前需要创建 SC_U 表。

```
CREATE TRIGGER SC_T          /* SC_T 是触发器的名字 */
AFTER UPDATE ON SC          /* UPDATE ON SC 是触发事件, */
    /* AFTER 是触发的时机, 表示当对 SC 的 Grade 属性修改完后再触发下面的规则 */
REFERENCING
    OLD AS OldTuple
    NEW AS NewTuple
FOR EACH ROW /* 行级触发器, 即每执行一次 Grade 的更新, 下面的规则就执行一次 */
WHEN (NewTuple. Grade >= 1.1 * OldTuple. Grade)
    /* 触发条件, 只有该条件为真时才执行下面的 insert 操作 */
BEGIN
    INSERT INTO SC_U (Sno, Cno, OldGrade, NewGrade) /* 触发动作体 */
    VALUES( OldTuple. Sno, OldTuple. Cno, OldTuple. Grade, NewTuple. Grade)
END
```

在本例中 REFERENCING 指出引用的变量, 如果触发事件是 UPDATE 操作并且有 FOR EACH ROW 子句, 则可以引用的变量有 OLD 和 NEW, 分别表示修改之前的元组和修改之后的元组。若没有 FOR EACH ROW 子句, 则可以引用的变量有 OLD TABLE 和 NEW TABLE, OLD TABLE 表示表中原来的内容, NEW TABLE 表示表中变化后的部分。

[例 5.19] 将每次对表 Student 的插入操作所增加的学生个数记录到表 Student InsertLog (numbers INT) 中, 运行触发器之前需要创建此表。

```
CREATE TRIGGER Student_Count
AFTER INSERT ON Student /* 指明触发器激活的时间是在执行 INSERT 后 */
REFERENCING
    NEW TABLE AS Delta
FOR EACH STATEMENT /* 语句级触发器, 即执行完 INSERT 语句后 */
    /* 下面的触发动作体才执行一次 */
BEGIN
    INSERT INTO StudentInsertLog (Numbers)
    SELECT COUNT(*) FROM Delta
END
```

在本例中出现的“FOR EACH STATEMENT”表示触发事件 INSERT 语句执行完成后才执行一次触发器中的动作, 这种触发器叫作语句级触发器。而例 5.18 中的触发器是行级触发器。默认的触发器是语句级触发器。“Delta”是一个关系名, 其模式与 Student 相同, 包含的元组是

INSERT 语句增加的元组。

[例 5.20] 定义一个 BEFORE 行级触发器, 为教师表 Teacher 定义完整性规则: 教授的工资不得低于 4 000 元, 如果低于 4 000 元, 自动改为 4 000 元。

```
CREATE TRIGGER Update_Sal      /* 对教师表插入或更新时激活触发器 */
BEFORE UPDATE ON Teacher      /* BEFORE 触发事件 */
REFERENCING NEW AS newTuple
FOR EACH ROW                  /* 这是行级触发器 */
BEGIN                          /* 定义触发动作体, 这是一个 PL/SQL 过程块 */
    IF (newTuple.job='教授') AND (newTuple.sal < 4000)
        /* 因为是行级触发器, 可在过程体中 */
        THEN newTuple.sal := 4000; /* 使用插入或更新操作后的新值 */
    END IF;
END;                            /* 触发动作体结束 */
```

因为定义的是 BEFORE 触发器, 在插入和更新教师记录前就可以按照触发器的规则调整教授的工资, 不必等插入后再检查调整。

5.7.2 执行触发器

触发器的执行是由触发事件激活, 并由数据库服务器自动执行的。一个数据表上可能定义了多个触发器, 如多个 BEFORE 触发器、多个 AFTER 触发器等, 同一个表上的多个触发器激活时遵循如下的执行顺序:

- ① 执行该表上的 BEFORE 触发器。
- ② 激活触发器的 SQL 语句。
- ③ 执行该表上的 AFTER 触发器。

对于同一个表上的多个 BEFORE/AFTER 触发器, 遵循“谁先创建谁先执行”的原则, 即按照触发器创建的时间先后顺序执行。有些关系数据库管理系统是按照触发器名称的字母排序顺序执行触发器。

5.7.3 删除触发器

删除触发器的 SQL 语句格式如下:

DROP TRIGGER <触发器名> ON <表名>;

触发器必须是一个已经创建的触发器, 并且只能由具有相应权限的用户删除。

触发器是一种功能强大的工具, 但在使用时要慎重, 因为在每次访问一个表时都可能触发一个触发器, 这样会影响系统性能。

本章小结

本章系统讲解了数据库完整性的概念,介绍了使用 SQL 语句实现数据库完整性定义的方法。关系数据库管理系统将负责检查完整性约束,当发现违约时进行处理。对于违反参照完整性的操作,本书讲解了不同的处理策略。用户要根据应用语义来定义合适的处理策略,以保证数据库的正确性。

本章还介绍了实现数据库完整性的方法之一——触发器。

触发器与各种完整性约束方法相比,其违约处理的功能更加灵活,更加强大。当违约时,触发器的动作体可以是一段 SQL 存储过程或 PL/SQL 过程块。触发器不仅可以用于数据库完整性检查,也可以用来实现数据库系统的其他功能,包括数据库安全性、应用系统的一些业务流程和控制流程等。不过也要注意,一个触发器的动作可能激活另一个触发器,最坏的情况是导致激活一个触发链,从而有可能造成难以预见的错误。

习题 5

1. 什么是数据库的完整性?
2. 数据库的完整性概念与数据库的安全性概念有什么区别和联系?
3. 什么是数据库的完整性约束?
4. 关系数据库管理系统的完整性控制机制应具备哪三方面的功能?
5. 关系数据库管理系统在实现参照完整性时需要考虑哪些方面?
6. 假设下面两个关系模式:

职工(职工号,姓名,出生日期,职务,工资,部门号),其中职工号为主码;

部门(部门号,名称,经理姓名,电话),其中部门号为主码。

用 SQL 定义这两个关系模式,要求在模式中完成以下完整性约束的定义:

- ① 定义每个模式的主码。
- ② 定义参照完整性约束。
- ③ 定义职工年龄不超过 65 岁。

7. 在关系数据库管理系统中,当操作违反实体完整性、参照完整性和用户定义的完整性约束时,一般是如何分别进行处理的?

*8. 某单位想举行一个小型联谊会,关系 Male 记录注册的男宾信息,关系 Female 记录注册的女宾信息。建立一个断言,将来宾的人数限制在 50 人以内(提示:先创建关系 Female 和关系 Male)。

第5章实验 完整性控制

实验 5.1 实体完整性实验。掌握实体完整性的定义和维护方法。定义实体完整性,删除实体完整性。能够写出两种方式定义实体完整性的 SQL 语句:创建表时定义实体完整性、创建表后使用 ALTER TABLE 语句定义实体完整性。设计 SQL 语句验证完整性约束是否起作用。

实验 5.2 参照完整性实验。掌握参照完整性的定义和维护方法。定义参照完整性,定义参照完整性的违

约处理，删除参照完整性。能够写出两种方式定义参照完整性的 SQL 语句：创建表时定义参照完整性、创建表后使用 ALTER TABLE 语句定义参照完整性。

实验 5.3 用户自定义完整性实验。掌握用户自定义完整性的定义和维护方法。针对具体应用语义，选择 NULL/NOT NULL、DEFAULT、UNIQUE、CHECK 等定义属性上的约束条件。

实验 5.4 触发器实验。掌握数据库触发器的设计和使用方法。定义 BEFORE 触发器和 AFTER 触发器、行级触发器或语句级触发器。能够理解不同类型触发器的作用和执行原理，设计数据操纵语言(DML)语句验证触发器的有效性。

参考文献 5

[1] HAMMER M M, MCLEOD D J. Semantic integrity in a relational data base system[C]. Proceedings of the 1st International Conference on Very Large Data Bases, 1975: 25-47.

文献[1]讨论数据库管理系统中语义完整性的概念。

[2] CODD E F. Extending the data base relational model to capture more meaning[J]. ACM Transactions on Database Systems, 1979, 4(4): 397-434.

文献[2]扩展了关系模型的语义表达能力。

[3] STONEBRAKER M. Implementation of integrity constraints and views by query modification[C]. Proceedings of International Conference on ACM SIGMOD, 1975.

文献[3]讨论 INGRES 数据库管理系统的完整性约束机制。

[4] CHAMBERLIN D D, ASTRAHAN M M, ESWARAN K P, et al. Sequel 2: a unified approach to data definition, manipulation, and control[J]. IBM Journal of Research and Development, 1976, 20(6): 560-575.

文献[4]提出了 SYSTEM R 中 SQL 的 Integrity Assert(完整性断言)和触发器语句。

[5] BERNSTEIN P A, BLAUSTEIN B T, CLARKE E M. Fast maintenance of semantic integrity assertions using redundant aggregate data[C]. Proceedings of the 6th International Conference on Very Large Data Bases, 1980, 6: 126-136.

[6] HSU A, IMIELINSKI T. Integrity checking for multiple updates[J]. ACM SIGMOD Record, 1985, 14(4): 152-168.

本章知识点讲解微视频:



模型固有的完整性



用户定义的完整性



5-3 通用工具——

触发器

第二篇

设计与应用开发篇

本篇讲解在开发应用系统时如何设计数据库，以及如何基于已经选定的数据库管理系统进行编程。

本篇包括三章。

第6章关系数据理论，详细讲解关系规范化理论，它既是关系数据库的重要理论基础，也是数据库设计的有力工具。规范化理论为数据库设计提供了理论指南和工具。

第7章数据库设计，讨论数据库设计的方法和步骤，详细讲解数据库设计各阶段的目标、方法和应注意的问题；重点讲解概念结构和逻辑结构的设计。以“高校本科教务管理”信息系统为示例，抽象出“学生选课管理”“学生学籍管理”“教师教学管理”三个应用场景，阐述如何在理论的指导下完成分E-R图的设计、集成和优化。

第8章数据库编程，从两方面讲解数据库编程技术和方法：一是使用扩展的SQL功能，包括引入新的SQL子句、新的数据库内置函数、PL/SQL与存储过程/自定义函数；二是在高级语言程序中使用SQL，实现高级语言与数据库之间的交互，开发应用需求中复杂业务逻辑的技术和方法。本章还介绍了JDBC编程的原理、方法和步骤。

关系数据理论的主要内容是关系规范化理论,用来指导数据库逻辑设计,具体包括两部分:一是给出判断数据库逻辑设计“好坏程度”的准则;二是如果逻辑设计中存在“不好”的关系模式,如何将其修改为“好”的关系模式。

本章导读

在关系数据库模式设计中,我们面临两个问题,第一,什么是一个“好”的关系数据库模式,其标准是什么。如果没有判断的标准,就很难对一个具体的设计进行评价。第二,能否开发数据库设计辅助工具来减轻设计者的负担。本章6.1和6.2节介绍关系规范化理论,用于回答第一个问题;6.3和6.4节介绍函数依赖公理系统以及模式分解算法,探索自动进行模式分解的方法,用于回答第二个问题。

本章首先从数据库逻辑设计中如何构造一个好的数据库模式这一问题出发,阐明关系规范化理论研究的实际背景,然后讨论两类最重要的数据依赖——函数依赖和多值依赖。读者要牢固掌握数据依赖的基本概念,如函数依赖、部分函数依赖、完全函数依赖、传递函数依赖等,掌握1NF到4NF的概念、规范化的含义和作用;能够了解各种范式以及在插入、删除和更新数据时可能出现的问题,并直观地描述解决办法。

本章进一步讨论函数依赖的推理规则,介绍函数依赖公理系统,讲解保持函数依赖的模式等价准则及相应的模式分解算法。要掌握这些算法并在数据库模式设计中应用。6.5节是选读内容,介绍无损连接的模式分解概念及相应算法,并解释其与保持函数依赖的模式分解之间的关系。

读者学习本章后应能够根据应用语义完整地写出关系模式的数据依赖集合,并能根据数据依赖分析某一个关系模式属于第几范式,能够运用介绍的有关算法辅助设计和优化数据库模式。

6.1 问题的提出

前面已经讨论了数据库系统的一般概念,介绍了关系数据库的基本概念、关系模型的三要

素以及关系数据库的标准语言 SQL。但是还有一个很基本的问题尚未涉及：针对一个具体问题，应该如何构造一个适合它的数据库模式，即应该构造几个关系模式，每个关系模式由哪些属性组成等。如何判断这个模式是好的，也就是设计的标准如何，是关系数据库逻辑设计要解决的问题。

首先回顾一下关系模式的形式化定义。

在第2章关系数据库中已经讲过，一个关系模式可以表示为

$$R(U, D, \text{DOM}, F)$$

这里：

- ① 关系名 R 是符号化的元组语义。
- ② U 为组成该关系模式的属性名集合。
- ③ D 为 U 中的属性所来自的域。
- ④ DOM 为属性向域的映射。
- ⑤ F 为 U 上的一组数据依赖。

由于 D 、 DOM 与模式设计关系不大，因此在本章中把关系模式表示为 $R(U, F)$ 。当且仅当 U 上的一个关系 r 满足 F 时，称 r 为关系模式 $R(U, F)$ 的一个关系。

作为二维表，关系要符合一个最基本的条件是每一个分量必须是不可分的数据项。满足了这个条件的关系模式就属于第一范式(1NF)。

在模式设计中，给定一个模式 S_d ，它仅由单个关系模式组成，问题是要设计一个模式 S_b ，它与 S_d 等价(等价的确切含义后面再展开)，但在某些指定的方面更好一些。这里通过一个例子来说明一个不好的模式会有些什么问题，分析它们产生的原因，并从中找出设计一个好的关系模式的办法。

在举例之前，先非形式地讨论一下数据依赖的概念。

数据依赖是一个关系属性与属性之间的一种约束关系。这种约束关系是通过属性间值的相等与否体现出来的数据间的相关联系。它是现实世界属性间相互联系的抽象，是数据内在的性质和语义的体现。

人们已经提出了许多种类型的数据依赖，其中最重要的是函数依赖(functional dependency, FD)和多值依赖(multivalued dependency, MVD)。

函数依赖极为普遍地存在于现实生活中。比如描述一个学生的关系，可以有学号(Sno)、姓名(Sname)、所在学院(School)等属性。由于通常一个学号只对应一个学生，一个学生只能在一个学院学习，因而当“学号”值确定之后，学生的姓名及其他属性值也就被唯一地确定了。属性间的这种依赖关系类似于数学中的函数 $y=f(x)$ ，自变量 x 确定之后，相应的函数值 y 也就唯一地确定了。

类似的有 $\text{Sname}=f(\text{Sno})$ ， $\text{School}=f(\text{Sno})$ ，即 Sno 函数决定 Sname ， Sno 函数决定 School ，或者说 Sname 和 School 函数依赖于 Sno ，记作 $\text{Sno} \rightarrow \text{Sname}$ ， $\text{Sno} \rightarrow \text{School}$ 。

[例 6.1] 建立一个描述学校教务的数据库，该数据库涉及的对象包括学生的学号(Sno)、

所在学院(School)、所在学院院长姓名(Mname)、选修课的课程号(Cno)和成绩(Grade)。假设用一个单一的关系模式 Student 来表示,则该关系模式 Student(U, F)的属性组为

$$U = \{ \text{Sno}, \text{School}, \text{Mname}, \text{Cno}, \text{Grade} \}$$

现实世界的已知事实(语义)告诉我们:

- ① 一个学院有若干学生,但一个学生只属于一个学院。
- ② 一个学院只有一名(正职)院长。
- ③ 一个学生可以选修多门课程,每门课程有若干学生选修。
- ④ 每个学生学习每一门课程有一个成绩。

于是得到 U 上的一组函数依赖 F (如图 6.1 所示)。

$$F = \{ \text{Sno} \rightarrow \text{School}, \text{School} \rightarrow \text{Mname}, (\text{Sno}, \text{Cno}) \rightarrow \text{Grade} \}$$

如果只考虑函数依赖这一种数据依赖,可以得到一个描述学生的关系模式 Student(U, F)。

表 6.1 是某一时刻关系模式 Student 的一个实例,即数据表。

表 6.1 Student 表

Sno	School	Mname	Cno	Grade
S1	信息学院	张明	C1	95
S2	信息学院	张明	C1	90
S3	信息学院	张明	C1	88
S4	信息学院	张明	C1	70
S5	信息学院	张明	C1	78
⋮	⋮	⋮	⋮	⋮

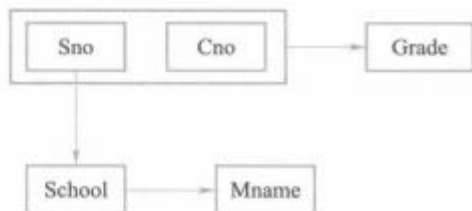


图 6.1 Student 上的一组函数依赖

但是,这个关系模式存在以下问题:

① 数据冗余(data redundancy)。比如,院长姓名重复出现,重复次数与该学院所有学生的所有课程成绩出现次数相同,如表 6.1 所示。这将浪费大量的存储空间。

② 更新异常(update anomaly)。由于数据冗余,当更新数据库中的数据时,系统要付出很大的代价来维护数据库的完整性,否则会面临数据不一致的危险。比如,某学院更换院长后,必须修改与该学院学生有关的每一个元组。

③ 插入异常(insertion anomaly)。如果一个学院刚成立,尚无学生,则无法把这个学院及其院长的信息存入数据库。

④ 删除异常(deletion anomaly)。如果某个学院的学生全部毕业了,则在删除该学院学生信息的同时,这个学院及其院长的信息也丢掉了。

鉴于存在以上种种问题,可以得出这样的结论:例 6.1 的 Student 关系模式不是一个好的模式。一个好的模式应当不会发生插入异常、删除异常和更新异常,数据冗余应尽可能

能少。

为什么会发生这些问题呢？

这是因为这个模式中的函数依赖存在某些不好的性质。这正是本章要讨论的问题。假如把这个单一的模式改造一下，分成以下三个关系模式：

$S(\{Sno, School\}, \{Sno \rightarrow School\})$;
 $SC(\{Sno, Cno, Grade\}, \{(Sno, Cno) \rightarrow Grade\})$;
 $SCHOOL(\{School, Mname\}, \{School \rightarrow Mname\})$

则这三个模式都不会发生插入异常、删除异常和更新异常的问题，数据的冗余也得到了控制。

一个模式的数据依赖会有哪些不好的性质，如何改造一个不好的模式，这就是规范化要讨论的内容。

6.2 规范化

本节首先讨论一个关系的属性间不同的依赖情况，以及如何根据属性间依赖情况判定关系是否具有某些不合适的性质。通常按属性间依赖情况区分关系的规范化程度为第一范式、第二范式、第三范式和第四范式等。然后直观地描述如何将具有不合适性质的关系转换为更合适的形式。

6.2.1 函数依赖

定义 6.1 给定关系模式 $R(U, F)$ ， X, Y 是 U 的子集。若对于 $R(U, F)$ 的任意一个可能的关系 r ， r 中不可能存在两个元组在 X 上的属性值相等，而在 Y 上的属性值不等，则称 X 函数确定 Y 或 Y 函数依赖于 X ，记作 $X \rightarrow Y$ 。

函数依赖和别的数据依赖一样是语义范畴的概念，只能根据语义来确定一个函数依赖。例如，“姓名 $\rightarrow$ 年龄”这个函数依赖只有在该部门没有同名人的条件下成立。如果允许有同名人，则年龄就不再函数依赖于姓名了。

设计者也可以对现实世界做强制性规定，例如规定不允许同名人出现，因而使“姓名 $\rightarrow$ 年龄”函数依赖成立。这样当插入某个元组时该元组上的属性值必须满足规定的函数依赖，若发现有同名人存在，则拒绝插入该元组。

注意：函数依赖不是指关系模式 R 的某个或某些关系满足的约束条件，而是指 R 在不同时刻的关系实例均要满足的约束条件。

例如，6.1 节关系模式 $Student(U, F)$ 中有 $Sno \rightarrow School$ 成立，也就是说在任何时刻该模式的关系实例（即 $Student$ 数据表）中，不可能存在两个元组在 Sno 上的值相等，而在 $School$ 上的值不等。因此，表 6.2 的 $Student$ 表是错误的。因为表中有两个元组在 Sno 上都等于“S1”，而